마이크로
파이썬으로
만드는
드론과
사물인터넷

마이크로 파이썬으로 만드는 드론과 사물인터넷

장문철 | 지음

발행 | 2025년 01월 01일
지은이 | 장문철
펴낸이 | 안재민
펴낸곳 | 먼슬리북스
출판신고 | 경기도 시흥시 은행로 169 203호
전화 | 070-7704-5662
이메일 | munjjac@naver.com

ISBN | 1234-5678-0000
값 22,000원

※잘못된 책은 구입하신 서점에서 교환해 드립니다.
※이 책은 저작권법에 따라 보호를 받는 저작물이므로 전재와 무단 복제를 금지합니다.

작가의 말

기술이 발전하며 하늘을 나는 드론은 더 이상 상상 속의 산물이 아니라, 우리의 일상에 점점 깊숙이 자리 잡아가고 있습니다. 드론은 이제 단순한 취미용 기기에서 벗어나 물류, 농업, 영상 촬영 등 다양한 산업 분야에서 활발히 활용되고 있습니다. 이러한 드론의 가능성을 더욱 확장하는 데 있어 프로그래밍의 중요성은 아무리 강조해도 지나치지 않습니다.

이 책은 드론과 마이크로파이썬을 접목하여 드론 제어와 사물인터넷 기술을 함께 배울 수 있는 실용적인 가이드입니다. 처음 드론을 조립하고 프로그래밍을 설정하는 기본 단계부터, IoT와 결합하여 데이터를 처리하고 응용하는 고급 기능까지, 실습을 통해 체계적으로 학습할 수 있도록 구성하였습니다.

저는 이 책을 통해 독자 여러분이 단순히 기술을 배우는 것에서 그치지 않고, 상상력을 더해 새로운 아이디어를 실현할 수 있는 계기가 되었으면 합니다. 무엇보다, 프로그래밍을 통해 여러분만의 드론과 IoT 솔루션을 만들어 가는 즐거움을 느낄 수 있기를 바랍니다.

끝으로, 이 책이 나오기까지 도움을 주신 모든 분들께 감사의 마음을 전하며, 독자 여러분의 창의적인 프로젝트와 성장을 진심으로 응원합니다.

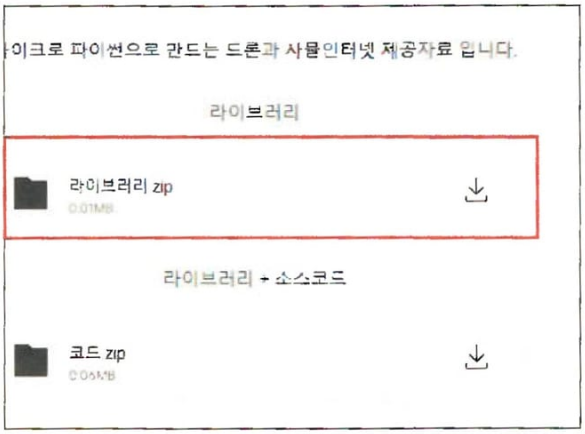
저자 장문철

독자지원센터

저자블로그

아래 저자가 운영하는 블로그주소에 접속하여 라이브러리 또는 소스코드를 다운로드 할 수 있습니다.

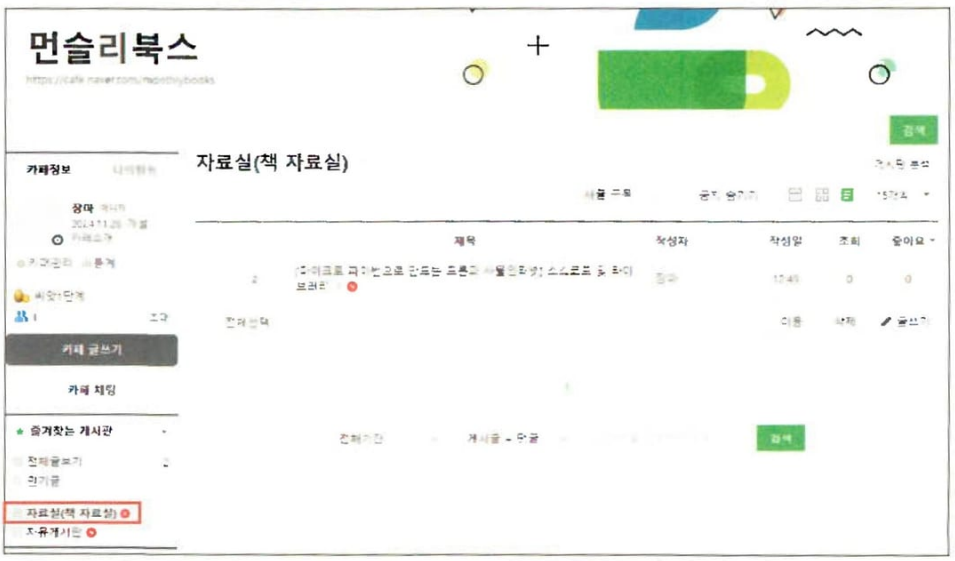
https://munjjac.tistory.com/15



출판사 네이버카페

출판사에서 운영하는 네이버카페 [먼슬리북스] 사이트에 접속하여 [자료실(책 자료실)]에 접속하여 관련된 책 자료의 다운로드를 할 수 있습니다.

https://cafe.naver.com/monthlybooks



강의문의

저자가 운영하는 [다두이노 스쿨] 사이트를 통해 파이썬, 아두이노, 라즈베리파이, 자율주행자동차 등 다양한 강의를 의뢰 할 수 있습니다. 아래 사이트에 접속 후 [강의요청] 을 통해 강의의 의뢰가 가능합니다.

https://daduino.modoo.at/

제품구매 방법

이책에서 사용하는 키트는 www.daduino.co.kr 사이트에 접속 합니다.

다두이노 사이트에서 **"파이썬 드론"** 을 검색 후

[마이크로 파이썬으로 만드는 드론과 사물인터넷 키트] 제품의 구매가 가능합니다.

Chapter1 시작하기

드론과 마이크로파이썬 ---------- 15

드론이란? ---------- 15

파이썬이란? ---------- 17

드론 조립 ---------- 19

마이크로 파이썬 ---------- 30

개발환경 구성 ---------- 30

Thonny IDE 설치 ---------- 30

드론 연결 ---------- 36

Thonny 유용한 기능 설정 ---------- 39

pyDrone에 라이브러리 설치 ---------- 41

마이크로파이썬 인터프리터 업로드 방법 ---------- 44

파이썬 기초문법 ---------- 49

출력과 입력 ---------- 49

변수와 자료형 ---------- 52

연산자와 제어문 ---------- 60

오류 및 예외처리 ---------- 72

함수, 클래스 ---------- 75

import ---------- 83

Chapter2 드론의 기본기능

LED제어하기 ----------87

녹색 LED제어하기----------87

안전하게 코드 종료하기 ----------89

on(), off()메서드 사용해서 제어하기 ----------90

버튼입력받기 ----------92

버튼 상태확인하기----------92

버튼의 상태가 변경될때만 값 출력하기 ----------94

버튼눌릴때만 값 출력하기----------95

배터리 전압 ----------97

측정하기 ----------97

ADC측정하기----------98

배터리전압으로 환산하기 ----------99

배터리의 잔량을 %로 표시하기 ----------100

모터제어하기 ----------102

한 개의 모터 구동하기 ----------103

4개의 모터 구동하기 ----------105

Chapter3 드론 비행

조종데이터 분석하기 ---------- 109
드론 조종데이터 수신받기 ---------- 109
드론 조종데이터 분리하여 출력하기 ---------- 116
드론제어코드 작성하기 ---------- 119
수신데이터에서 조종신호 분리하기 ---------- 119
조종신호 및 버튼신호 분리하여 출력하기 ---------- 121
드론 자세 보정하기 ---------- 123
드론 조종코드 작성하기 ---------- 126
드론 비행 ---------- 129
드론 경로주행 ---------- 136
코딩하기 ---------- 136
버튼을 확인하는 코드 만들기 ---------- 136
이륙 및 착륙하기 ---------- 138
경로주행 코딩하기 ---------- 142

Chapter4 기본기능 다루기

RGB LED ----------147

제어하기 ----------147

사물인터넷드론 독과 드론을 연결----------147

빨간색 LED켜기 ----------150

빨간색, 녹색, 파란색 LED켜기 ----------151

빨간색 LED의 밝기 조정하기 ----------152

무지개 색 표현하기 ----------153

버튼 입력받기 ----------155

버튼입력 ----------155

버튼을 누를때만 출력 ----------157

버튼을 확인하는 코드를 함수처리----------158

여러개의 버튼을 함수처리----------159

버튼을 이용하여 RGB LED의 색상 조절하기----------161

부저 출력하기 ----------164

도레미파솔라시도 출력하기----------164

학교종 출력하기 ----------165

버튼2개를 이용하여 도레미 출력하기 ----------166

조도센서 측정 ----------168

조도센서로 밝기 확인하기----------168

어두워지면 자동으로 켜지는 전등 만들기----------170

Chapter5 응용기능 다루기

OLED 디스플레이 장치 ---------- 173

라이브러리 업로드하기 ---------- 173

글자출력하기 ---------- 174

글자 위치변경하여 출력하기 ---------- 175

숫자 출력하기 ---------- 176

선 및 네모 그리기 ---------- 177

DHT11 온도습도 ---------- 179

센서값 읽기 ---------- 179

DHT11온습도센서 값 읽어 출력하기 ---------- 179

예외처리하기 ---------- 180

OLED에 온습도 값 표시 ---------- 182

초음파센서 값 읽기 ---------- 184

초음파센서로 거리값 측정하기 ---------- 184

함수로 만들어서 편리하게 사용하기 ---------- 187

가까이 다가오면 부저 출력 ---------- 188

초음파센서 피아노 만들기 ---------- 191

Chapter6 블루투스 사물인터넷

블루투스 통신으로 ----------197

데이터 주고받기 ----------197

라이브러리 설치 ----------197

블루투스 통신으로 데이터 주고받기 ----------198

온도습도 데이터 측정하여 블루투스 통신으로 데이터 전송하기----------202

time.sleep를 사용하지 않고 2초마다 데이터 전송하기----------204

버튼의 상태 블루투스 통신으로 데이터 전송하기 ----------206

블루투스 통신을 활용한 ----------209

RGB LED제어 ----------209

프로토콜 만들어 데이터 수신받기----------209

RGB 데이터 적용하기 ----------211

RGB LED제어하기 ----------213

Chapter7 WIFI 사물인터넷

WiFi 통신 ---------- 217

WIFI 스캔을 통해 주변 WIFI 찾기 ---------- 218

WIFI 연결하기 ---------- 220

WIFI 고정 IP 설정하기 ---------- 221

WIFI AP모드로 변경하기 ---------- 223

OLED 인터넷시간 ---------- 225

표시하기 ---------- 225

라이브러리 업로드하기 ---------- 226

NTP 서버에 접속하여 시간받기 ---------- 226

시간을 표시하기 ---------- 228

OLED에 NTP 시간 표시하기 ---------- 232

OLED 기상청 ---------- 235

날씨 출력하기 ---------- 235

라이브러리 업로드하기 ---------- 235

기상청 RSS 접속 ---------- 236

기상청 RSS 접속하여 RSS데이터 받기 ---------- 238

온도 습도 데이터만 출력하기 ---------- 240

주기적으로 접속하여 온도 습도 데이터 출력하기 ---------- 242

OLED에 온도와 습도 표시하기 ---------- 245

OLED 공공데이터 ---------- 248

미세먼지 출력하기 ---------- 248

공공데이터 포탈에서 활용신청 및 데이터 확인하기 ----------248
라이브러리 업로드하기 ----------253
공공데이터 미세먼지 데이터 출력하기----------254
json 데이터 분리하여 출력하기 ----------256
강남대로의 미세먼지, 초미세먼지만 출력하기 ----------258
OLED에 미세먼지 표시하기 ----------260
OLED 주식가격 ----------263
출력하기 ----------263
네이버 주식데이터 접속하여 값확인하기 ----------263
라이브러리 업로드하기----------264
삼성전자 주식 시세 확인하기----------264
온도,습도,조도 측정하여 ----------270
이메일 보내기 ----------270
구글이메일 설정하기 ----------270
라이브러리 설치----------275
내 구글 계정으로 이메일보내기 ----------275
이메일보내는 부분 함수화 하기 ----------278
온도, 습도, 조도를 측정하여 이메일 보내기 ----------280

CHAPTER 01

시작하기

드론의 기본 개념과 마이크로파이썬에 대해 알아보고, 드론 조립을 통해 기본 구조를 이해합니다. 마지막으로 개발 환경을 구축하여 드론 제어를 위한 첫걸음을 내딛습니다.

1-1

드론과 마이크로파이션

드론이란?

드론은 원격으로 조종되거나 자율적으로 비행할 수 있는 무인 항공기를 뜻합니다. 처음에는 군사 목적으로 개발되었지만, 현재는 촬영, 물류, 농업, 구조 활동, 취미 생활 등 다양한 분야에서 활용되고 있습니다.

[일반적인 촬영용 드론]

드론의 기본 구조는 크게 네 가지 주요 구성 요소로 나눌 수 있습니다. 첫째, 프레임은 드론의 뼈대 역할을 하며, 모터와 프로펠러를 지지합니다. 프레임의 크기와 강도는 드론의 크기와 용도에 따라 다양합니다. 둘째, 모터와 프로펠러는 드론의 비행을 위한 추진력을 제공합니다. 일반적으로 쿼드콥터는 4개의 모터와 프로펠러를 사용하며, 각 모터는 비행 중 균형을 유지하기 위해 정교하게 제어됩니다. 셋째, 배터리는 드론의 전원을 공급하며, 비행 시간과 무게의 균형을 고려해 선택됩니다.

마지막으로, 비행 컨트롤러는 드론의 두뇌로, 센서를 통해 수집한 데이터를 처리하여 안정적으로 비행할 수 있도록 합니다.

드론이 안정적으로 비행하기 위해서는 여러 기술이 결합됩니다. IMU(관성측정장치)는 드론의 움직임과 자세를 감지하고, GPS는 위치를 파악해 주행 경로를 계획합니다. 또한, 원격 조종기나 소프트웨어를 통해 사용자가 드론을 제어할 수 있습니다.

[농약을 살포중인 드론]

최근에는 드론의 제어 기술로 프로그래밍 언어가 중요한 역할을 하고 있습니다. 특히 마이크로파이썬(MicroPython)과 같은 경량화된 프로그래밍 언어는 드론 제어를 위한 소형 컴퓨터에서 효율적으로 작동할 수 있어, 드론의 가능성을 더욱 확장시키고 있습니다. 이러한 기술을 이용하면 사용자 정의 비행 패턴, 자동 비행, 데이터 수집 등 다양한 응용이 가능합니다.

이 책에서는 드론의 기본 개념과 조립 과정을 다루며, 마이크로파이썬을 활용한 드론 제어 방법을 소개할 것입니다. 드론의 하드웨어와 소프트웨어가 어떻게 결합되어 비행이 이루어지는지 이해하면, 더 깊이 있는 드론 개발과 활용이 가능할 것입니다.

파이썬이란?

파이썬 개요 및 특징:

파이썬은 1991년에 Guido van Rossum이 개발한 고급 프로그래밍 언어로, 간결하고 읽기 쉬운 문법을 가지고 있으며, 다양한 운영 체제에서 사용할 수 있습니다.

파이썬의 주요 특징은 다음과 같습니다:

- 간결한 문법: 파이썬은 들여쓰기를 통해 코드 블록을 정의하며, 이로 인해 가독성이 좋습니다.
- 다양한 라이브러리와 모듈: 파이썬은 다양한 라이브러리와 모듈을 포함하고 있어, 다양한 작업을 수행하기 위한 도구가 풍부하게 제공됩니다.
- 크로스 플랫폼 지원: 파이썬은 다양한 운영 체제에서 동작하므로 이식성이 뛰어납니다.
- 동적 타이핑: 변수의 데이터 타입을 런타임에 결정하므로 유연한 프로그래밍이 가능합니다.

마이크로파이썬 개요 및 특징:

마이크로파이썬(MicroPython)은 파이썬 프로그래밍 언어를 기반으로 한 작은 규모의 파이썬 인터프리터입니다.

주요 특징은 다음과 같습니다:

- 임베디드 시스템 지원: 마이크로파이썬은 주로 임베디드 시스템, 마이크로컨트롤러, IoT 디바이스 등의 작고 제한된 자원을 가진 환경에서 사용됩니다.
- 소형 크기: 마이크로파이썬은 작고 경량화된 인터프리터로, 메모리와 저장 공간을 효율적으로 사용합니다.
- 하드웨어 제어: 마이크로파이썬을 사용하여 하드웨어를 제어하고 센서와 액추에이터를 다룰 수 있으며, 이로 인해 IoT 개발에 유용합니다.
- 파이썬 호환성: 마이크로파이썬은 파이썬 3.x 버전의 문법과 라이브러리를 기반으로 하며, 파이썬 개발자들이 비교적 쉽게 사용할 수 있습니다.

파이썬과 마이크로파이썬 차이:

- 용도: 파이썬은 범용 프로그래밍 언어로 다양한 분야에서 사용됩니다. 반면에 마이크로파이썬은 주로 임베디드 시스템과 IoT 디바이스에서 사용되며, 하드웨어 제어에 특화되어 있습니다.

- 크기와 성능: 파이썬은 상대적으로 큰 인터프리터이고, 메모리와 저장 공간을 많이 사용합니다. 마이크로파이썬은 소형 크기로 최적화되어 자원 제약이 있는 환경에서 사용하기에 적합합니다.

- 라이브러리: 파이썬은 다양한 라이브러리와 모듈을 제공하며, 범용적으로 활용 가능합니다. 마이크로파이썬은 하드웨어 관련 라이브러리와 모듈이 강조됩니다.

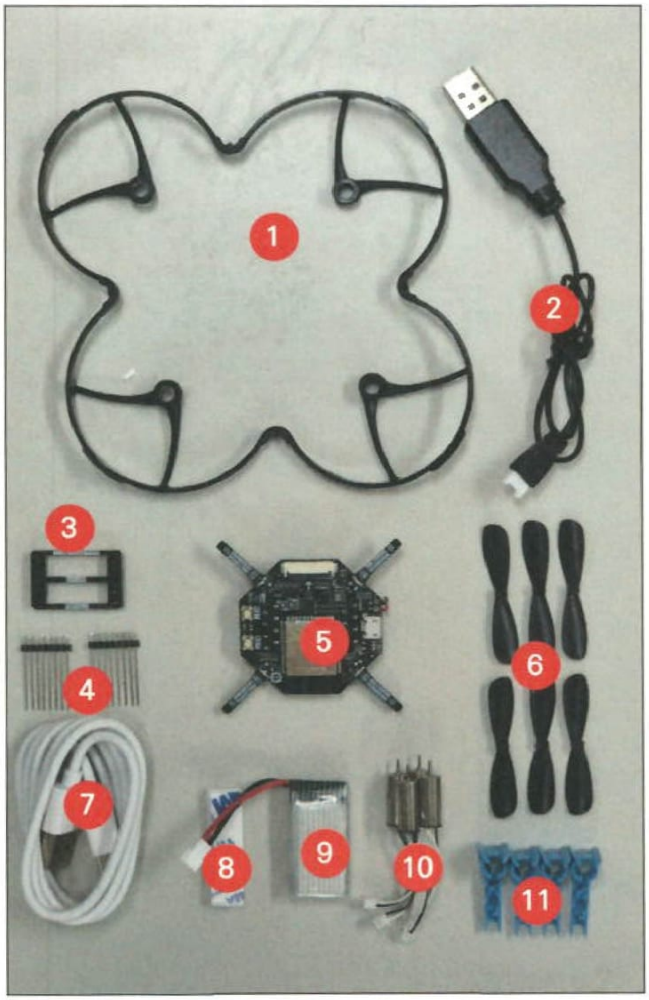
- 생태계: 파이썬은 넓은 개발자 커뮤니티와 풍부한 문서화, 지원을 가지고 있습니다. 마이크로파이썬은 상대적으로 작은 생태계를 가지고 있지만, IoT 개발자들 사이에서는 인기가 있습니다.

이러한 차이로 인해 파이썬과 마이크로파이썬은 서로 다른 용도와 환경에서 사용됩니다. 파이썬은 범용 프로그래밍 언어로 다양한 분야에서 활용되는 반면, 마이크로파이썬은 제한된 자원을 가진 임베디드 시스템과 IoT 디바이스에서 하드웨어 제어를 위해 사용됩니다.

1-2

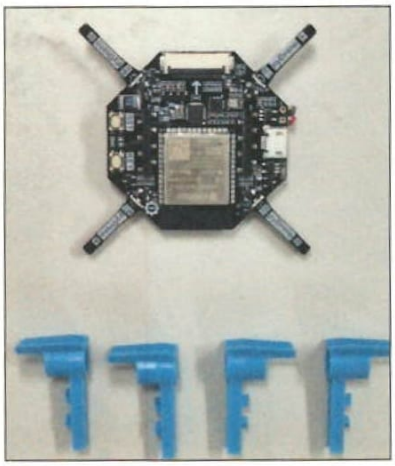
드론 조립

01 드론을 조립하여 비행을 하기위한 준비를 합니다. 아래의 물품을 확인합니다.

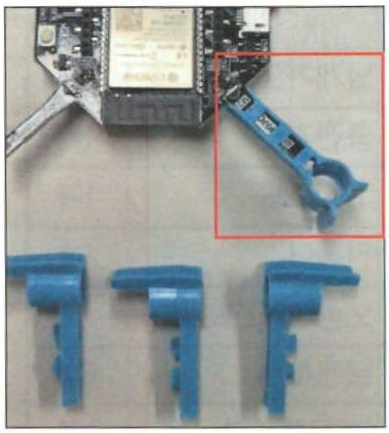


번호	이름	수량
1	날개가드	1개
2	충전기	1개
3	고정보드	1개
4	핀헤더	1개
5	드론베이스보드	1개
6	날개A,B	6개
7	USB케이블	1개
8	밸크로테이프	1쌍
9	배터리	1개
10	모터	4개
11	모터지지대	4개

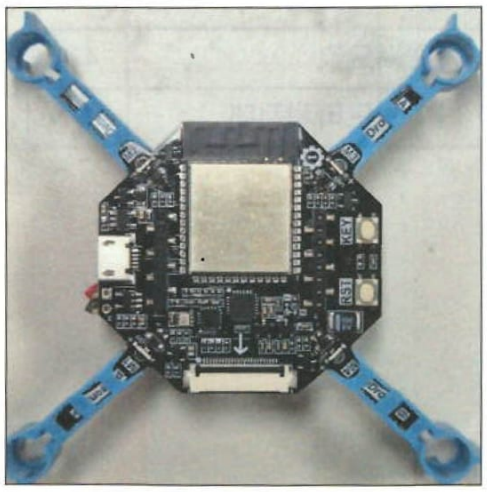
02 파이드론의 베이스보드와 지지대 4개를 준비합니다.



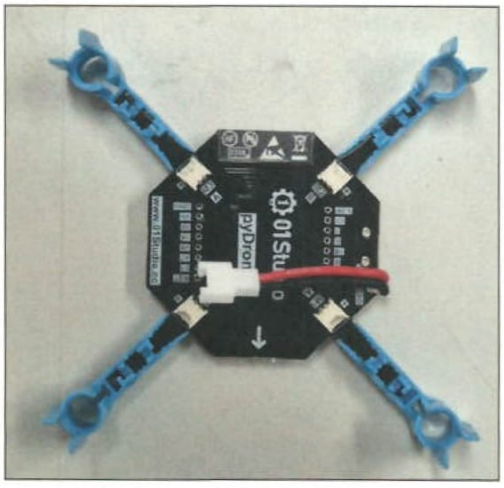
03 지지대를 드론의 날개부분에 넣어 조립합니다.



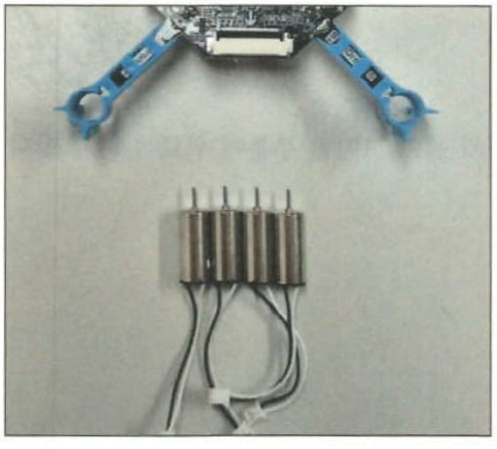
04 4개의 지지대를 모두 조립하였습니다.



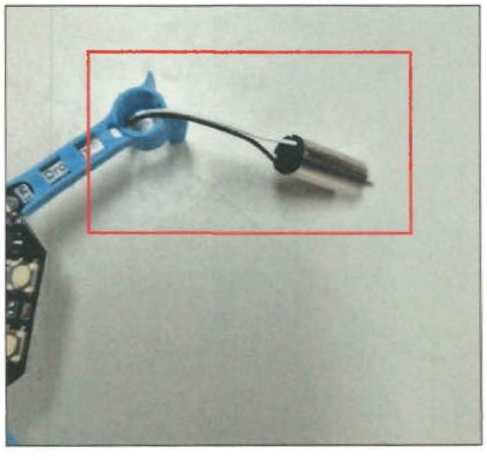
05 지지대를 조립한 아래에서 바라본 모습입니다.



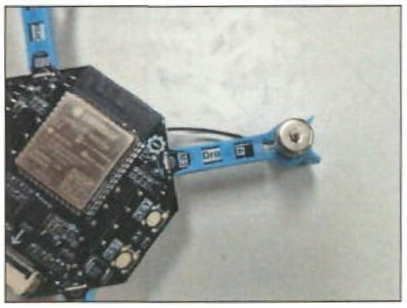
06 모터4개를 준비합니다. 모터는 4개의 사양모두 동일합니다.



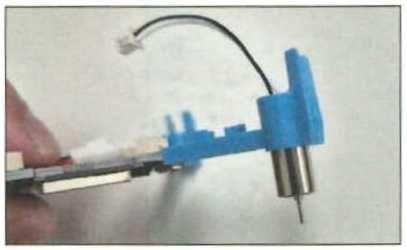
07 모터의 케이블을 위에서 아래로 넣은다음 모터를 끼워넣습니다.



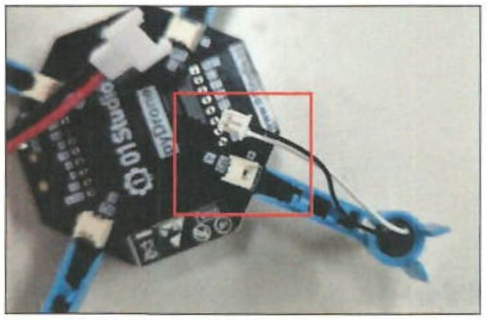
08 모터가 조립되었습니다.



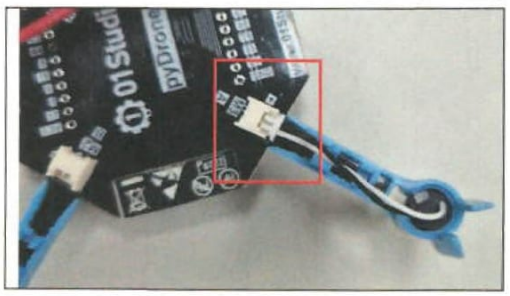
09 옆에서 바라본 모터가 조립된 모습입니다.



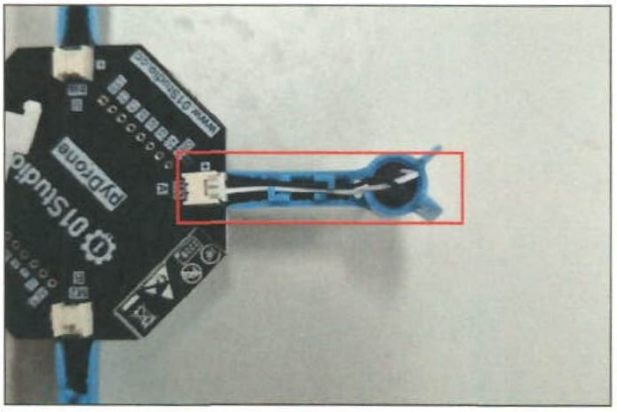
10 모터의 케이블을 커넥터와 연결합니다. 모터 커넥터의 핀이 보이는 부분이 위로 향하게 합니다.



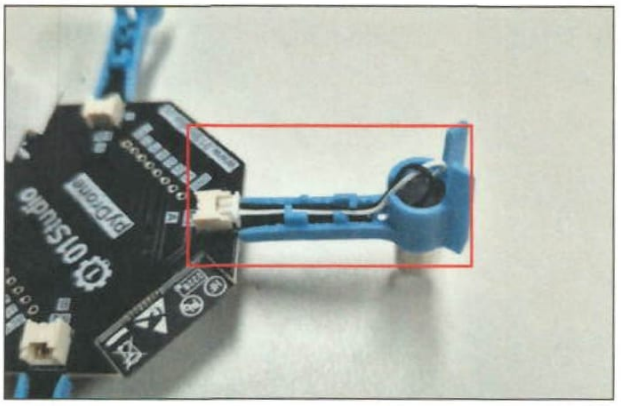
11 모터커넥터를 연결합니다.



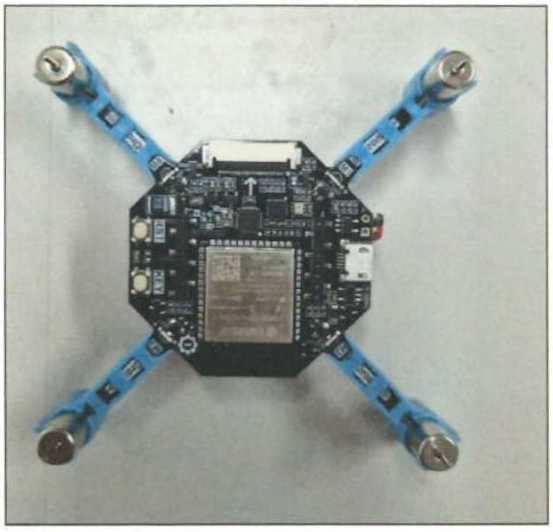
12 모터의 케이블을 지지대 안으로 정리합니다.



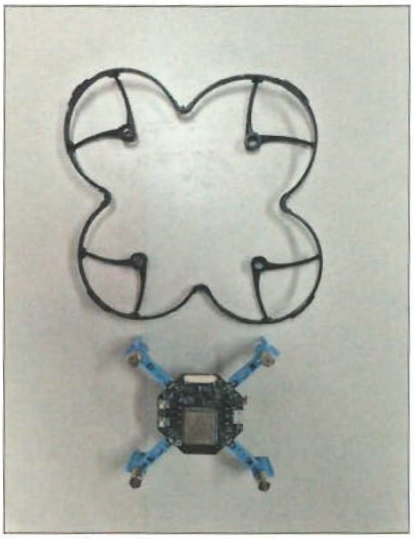
13 모터의 케이블을 지지대 안으로 정리합니다.



14 4대의 모터 모두 조립완료하였습니다.

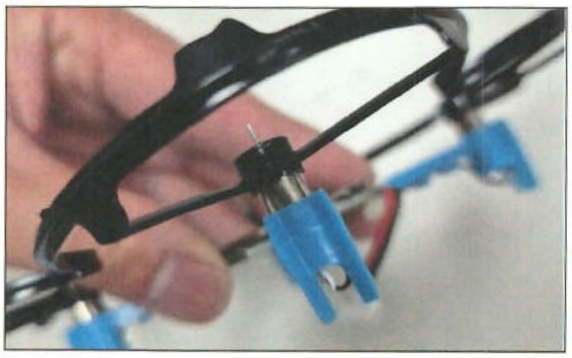


15 날개가드를 준비합니다.

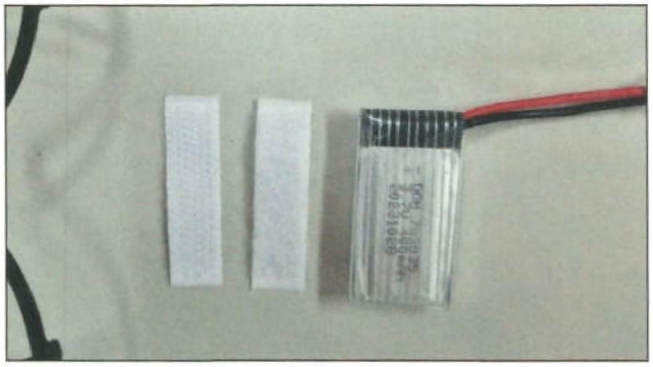


16 날개가드를 모터에 끼워넣습니다.

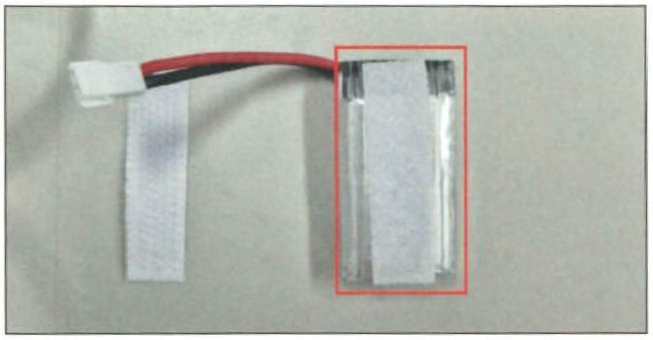
17 옆에서 본 모습입니다.



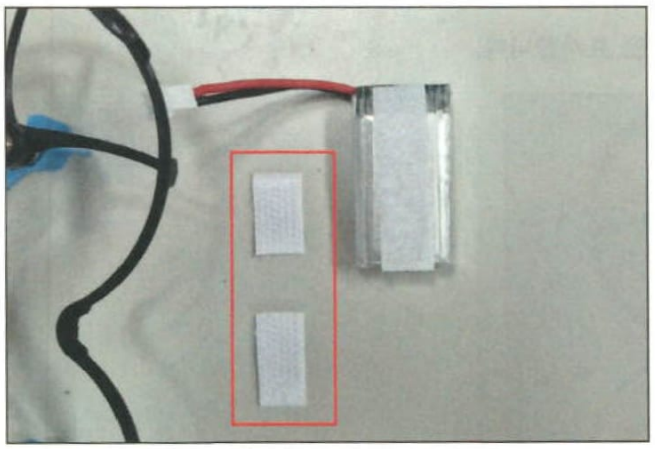
18 배터리와 벨크로테이프를 준비합니다.



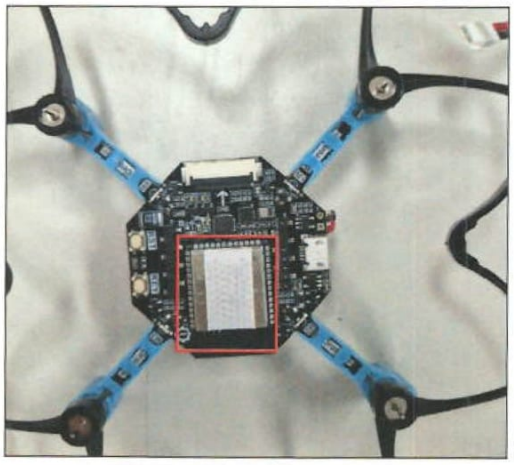
19 벨크로테이프의 부드러운 부분을 배터리에 부착합니다.



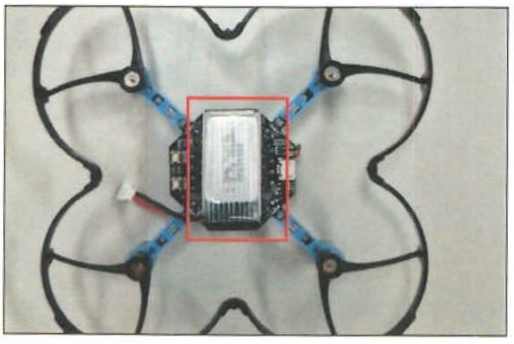
20 까끌한 벨트로테이프를 반으로 잘라줍니다.



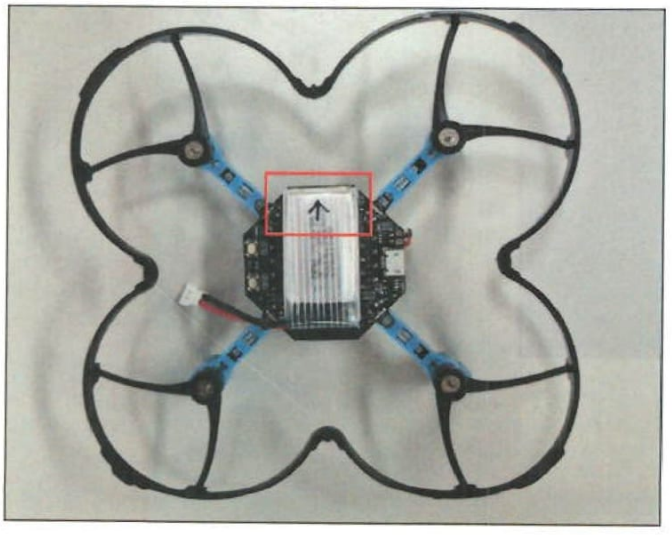
21 파이드론 베이스보드에 아래와 같이 부착합니다.



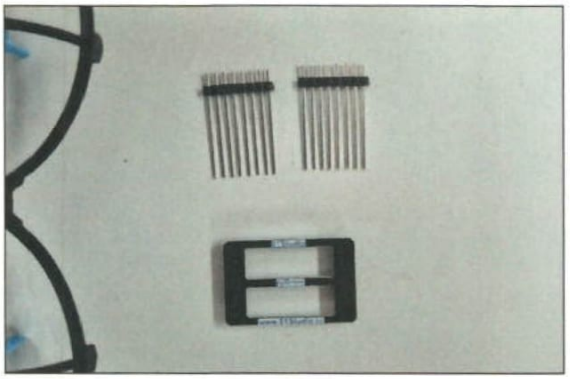
22 배터리를 부착합니다.



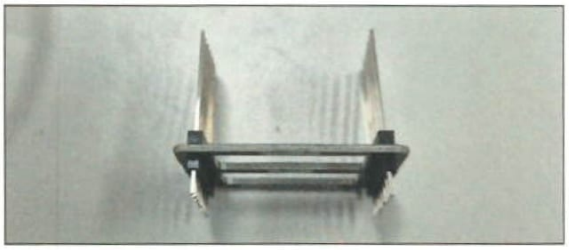
23 네임펜등을 이용하여 앞방향을 화살표로 표시합니다.



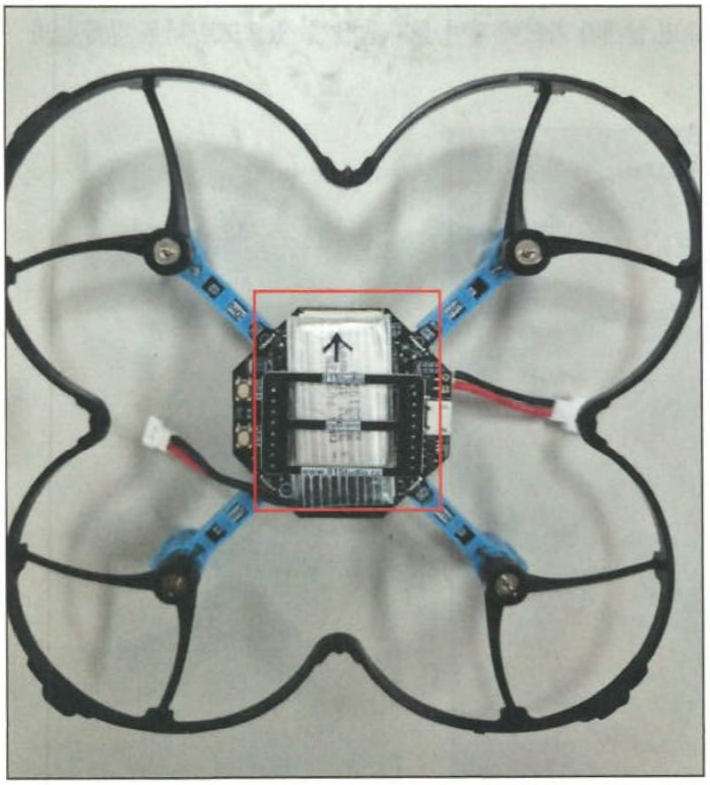
24 배터리 고정을 위한 핀헤더와 고정보드를 준비합니다.



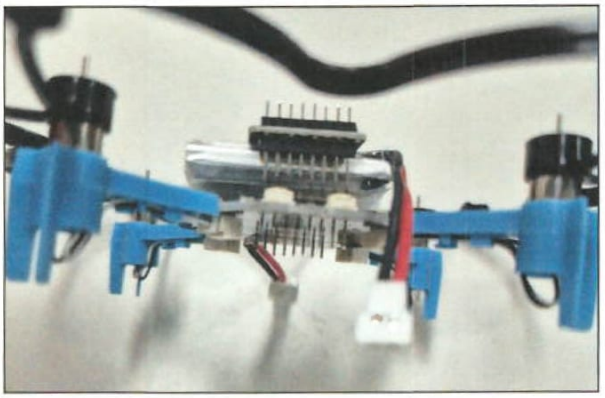
25 핀헤더와 고정보드를 다음과 같이 연결합니다.



26 고정보드를 이용해서 배터리를 고정합니다.

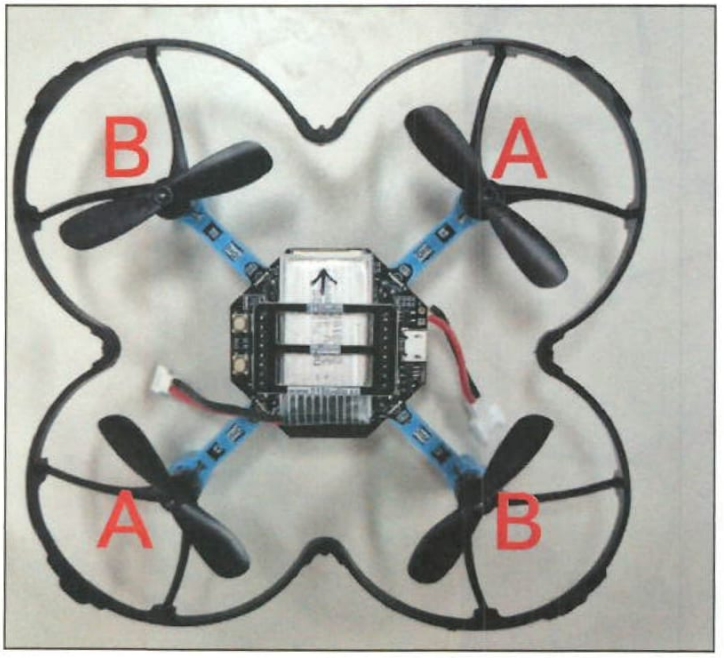


27 옆에서 본 모습입니다.

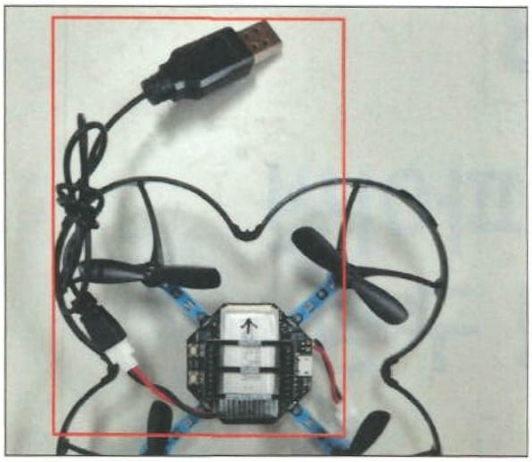


28 A날개 2개와, B날개 2개를 준비합니다.

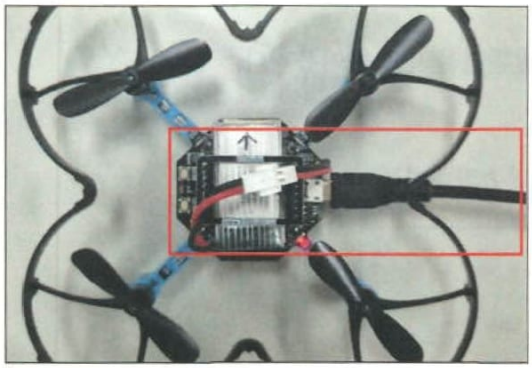
29 아래와 같이 A날개와 B날개를 조립합니다. 또한 PCB에 A,B 표시가 있어 PCB에 표시된 대로 A날개와 B날개를 조립합니다. 너무 깊게 눌러 넣으면 날개가 가드에 닿아 돌지 않을 수 있으므로 너무 깊에 눌러 넣지 않습니다.



30 배터리충전은 제공된 충전기를 이용해서 충전이 가능합니다.



31 배터리를 드론보드와 연결한 상태라면 마이크로 USB를 이용하여 배터리의 충전이 가능합니다.



32 조립을 완료하였습니다.

1-3

마이크로 파이썬 개발환경 구성

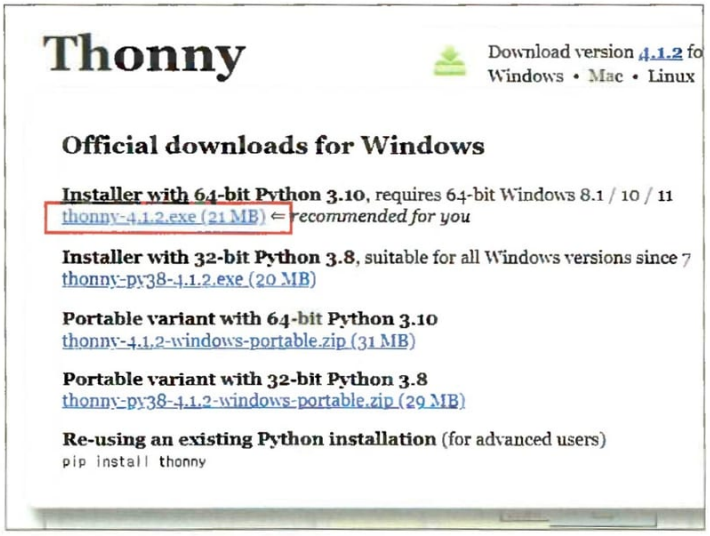
Thonny IDE 설치

pyDrone에 프로그램 하기위해서 마이크로파이썬 개발환경을 구축하여봅니다.

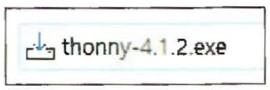
01 구글에서 thonny를 검색 후 Thonny 사이트에 접속합니다.

02 [Windows] 부분에 마우스를 이동합니다.

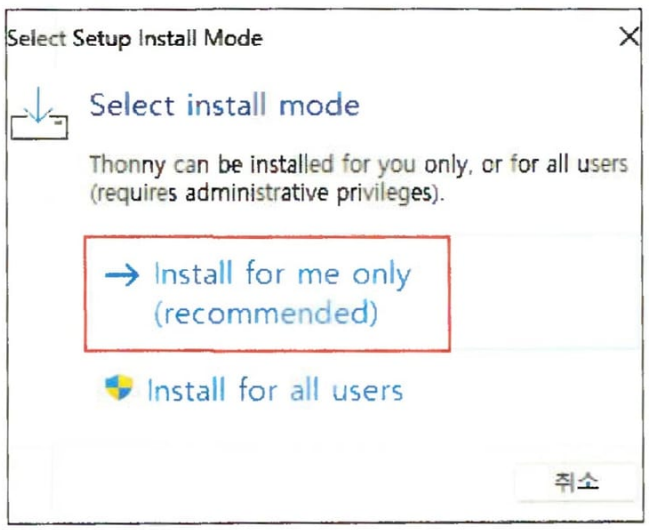
03 Thonny의 설치파일을 다운로드 받습니다. 64bit 설치버전으로 다운로드 받아 진행합니다. 버전은 다운로드 시점의 최신버전을 다운로드받습니다.



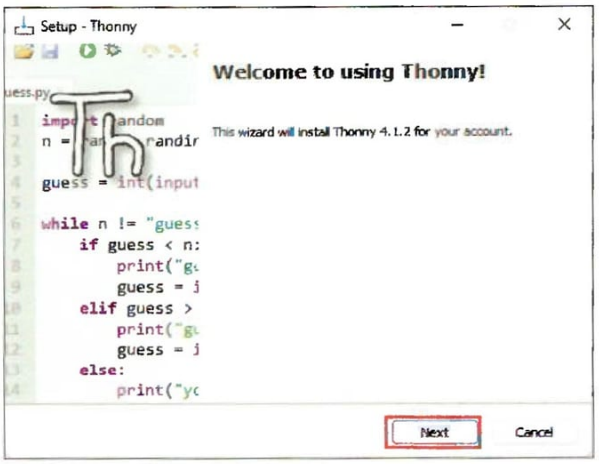
04 다운로드된 파일을 더블클릭하여 설치를 진행합니다. 버전은 다운로드 시점의 최신버전을 다운로드받습니다.



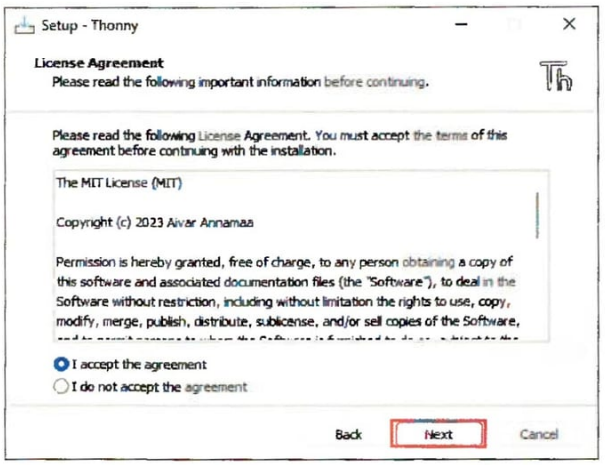
05 Install for me only를 클릭하여 설치를 진행합니다.



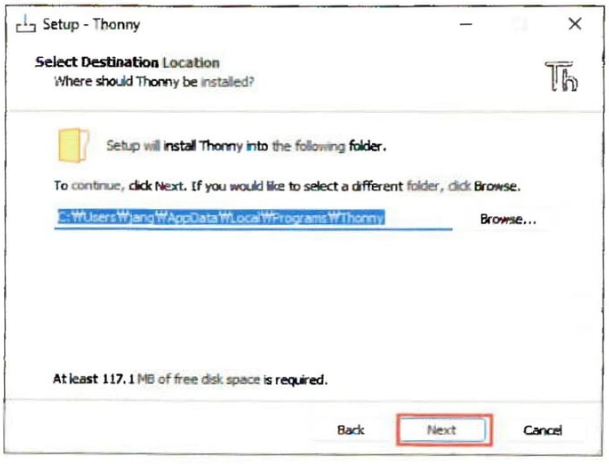
06 [Next]를 클릭하여 계속진행합니다.



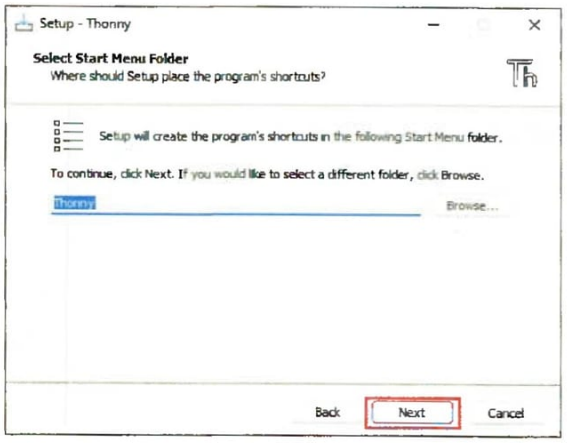
07 [Next]를 클릭하여 계속진행합니다.



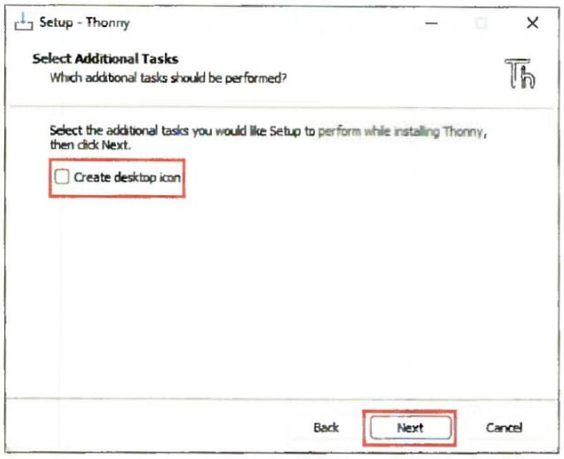
08 [Next]를 클릭하여 계속진행합니다. 설치되는 폴더의 위치입니다. 변경하지 않습니다.



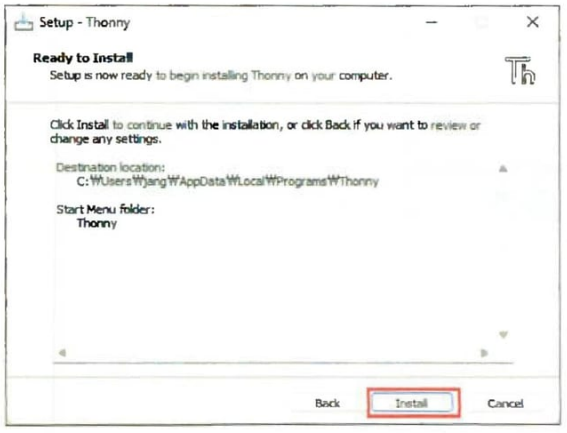
09 [Next]를 클릭하여 계속진행합니다.



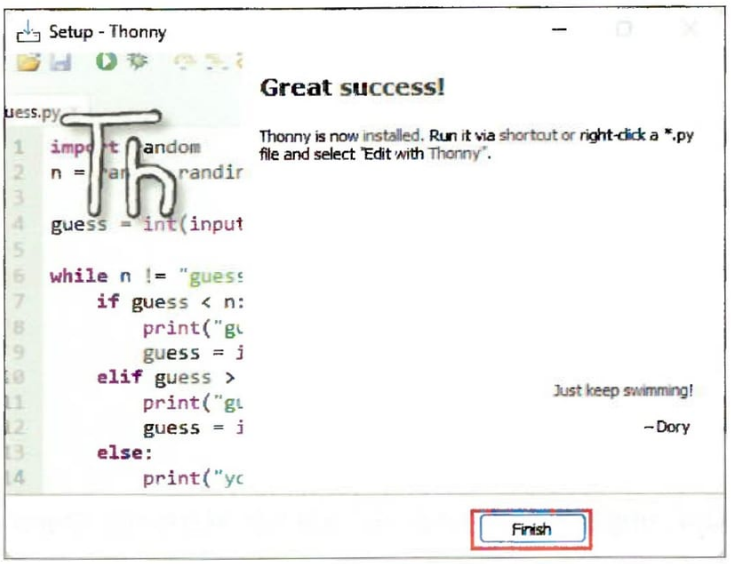
10 [Next]를 클릭하여 계속진행합니다. 바탕화면에 아이콘을 만드옵션으로 체크하여도 무방합니다.



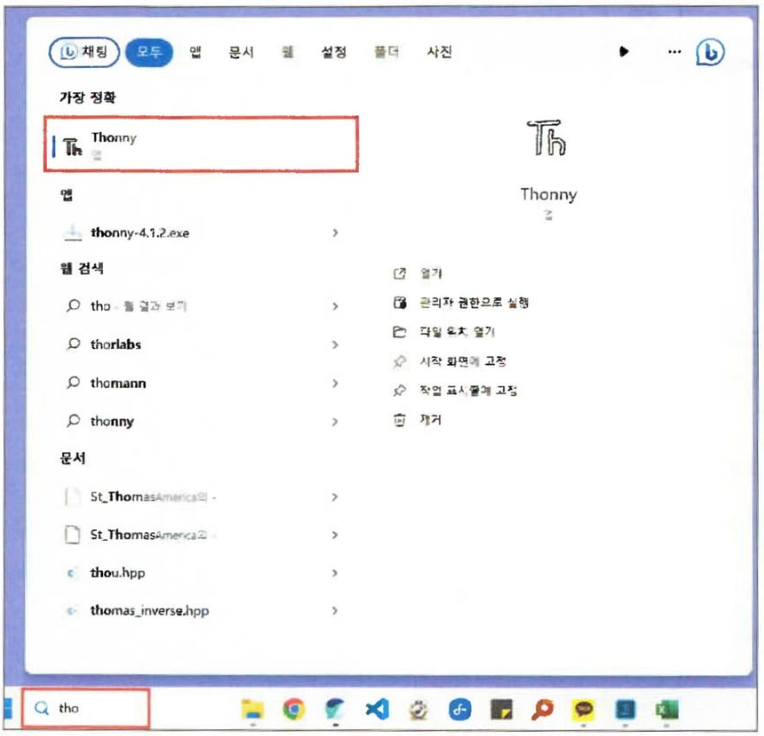
11 [Install]를 클릭하여 설치합니다.



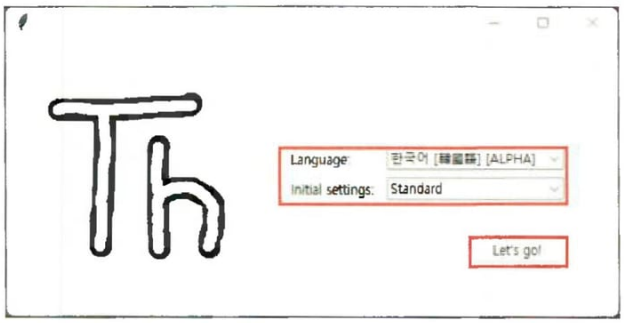
12 [Finish]를 클릭하여 설치를 완료합니다.



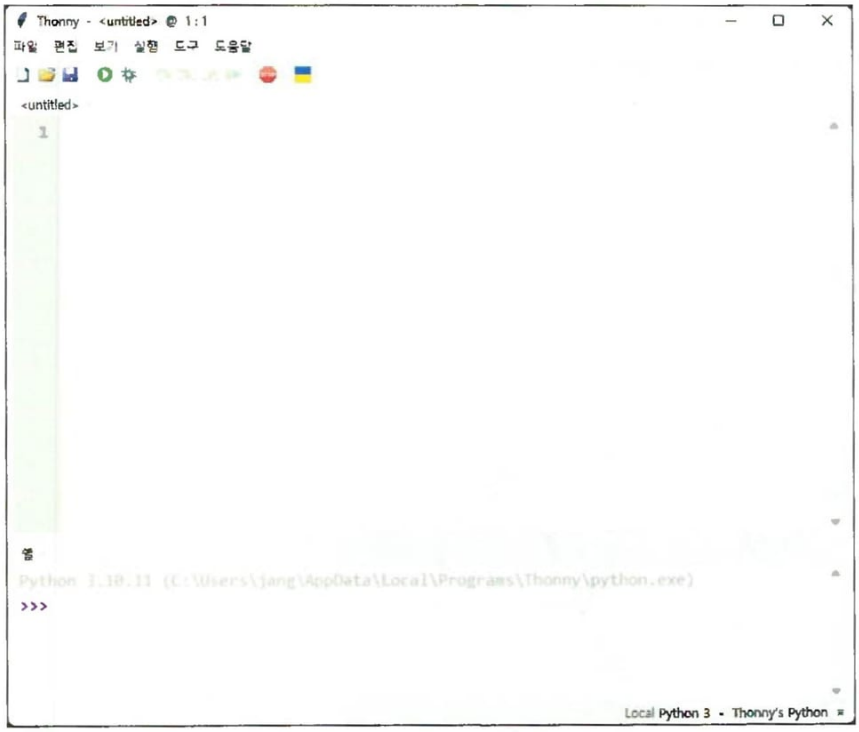
13 바탕화면에 아이콘이 없다면 [검색]에서 thonny를 검색 후 앱을 실행합니다.



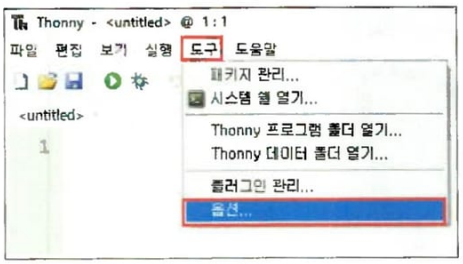
14 언어를 [한국어]로 변경 후 [Lets go]를 클릭하여 Thonny를 실행합니다.



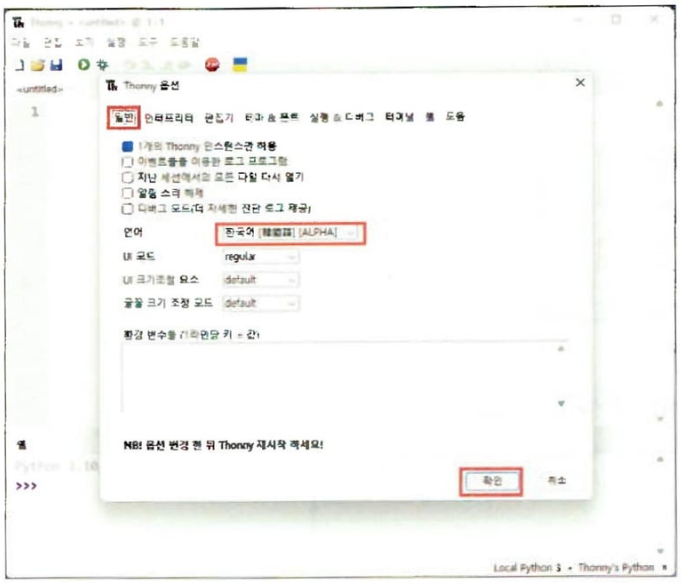
15 파이썬 코드를 작성할 수 있는 Thonny가 실행되었습니다.



16 영어로 설정하였다면 아래의 [도구Tool] -> [옵션option] 에 접속합니다.



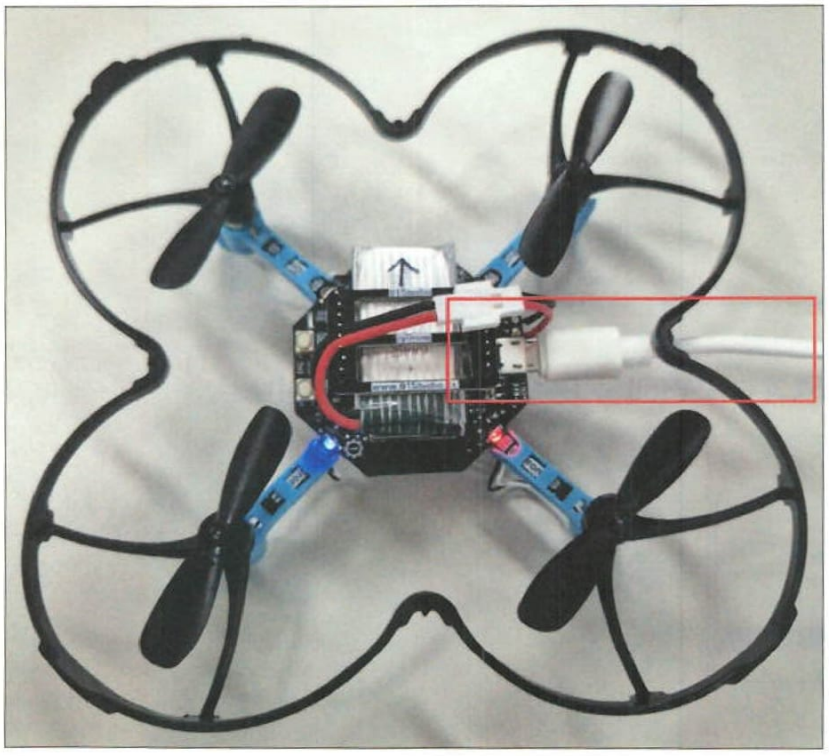
17 [언어 Language]를 한국어로 변경 후 [확인 OK]를 눌러 언어를 한국어로 변경합니다.



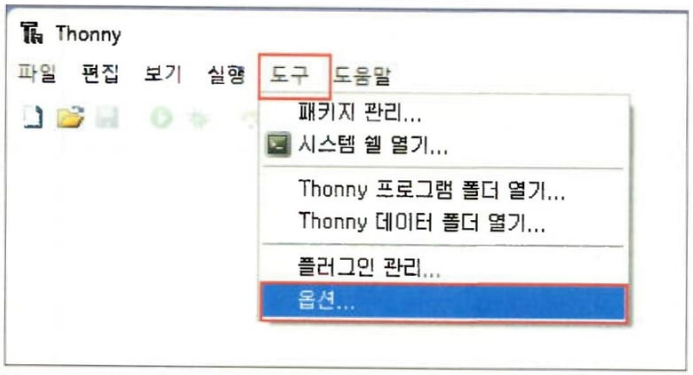
드론 연결

Thonny IDE와 드론을 연결합니다.

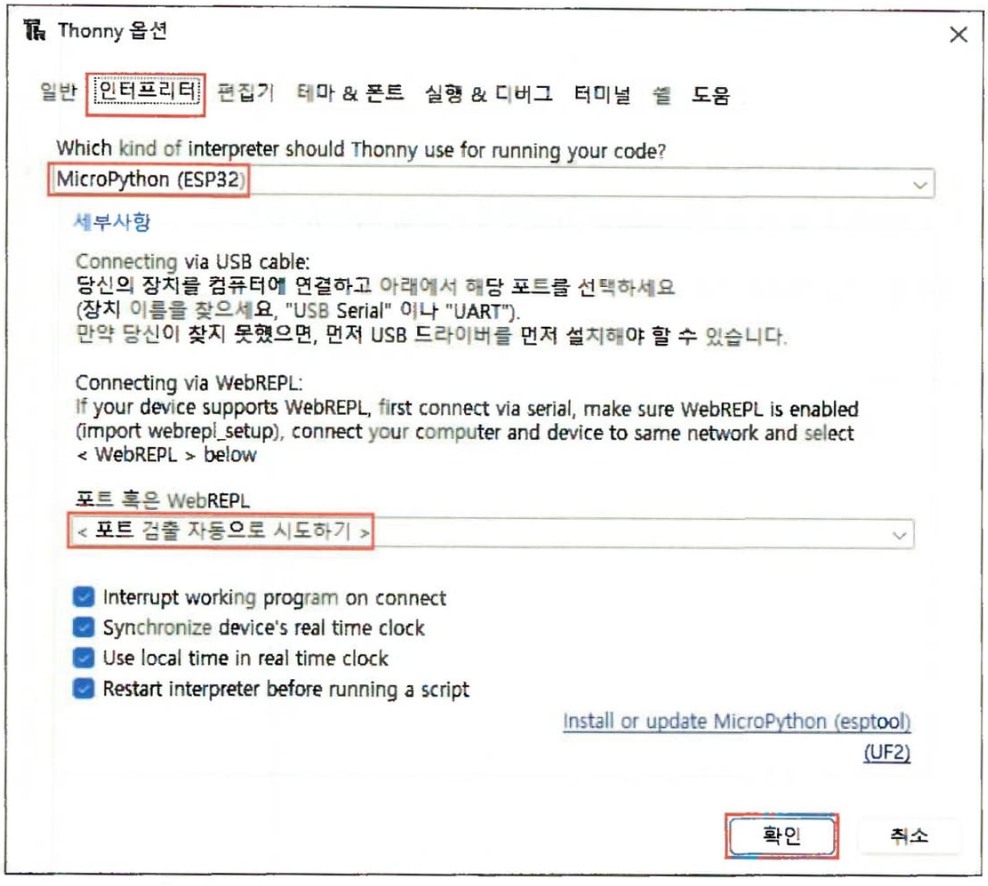
01 USB케이블을 이용하여 드론과 PC를 연결합니다.



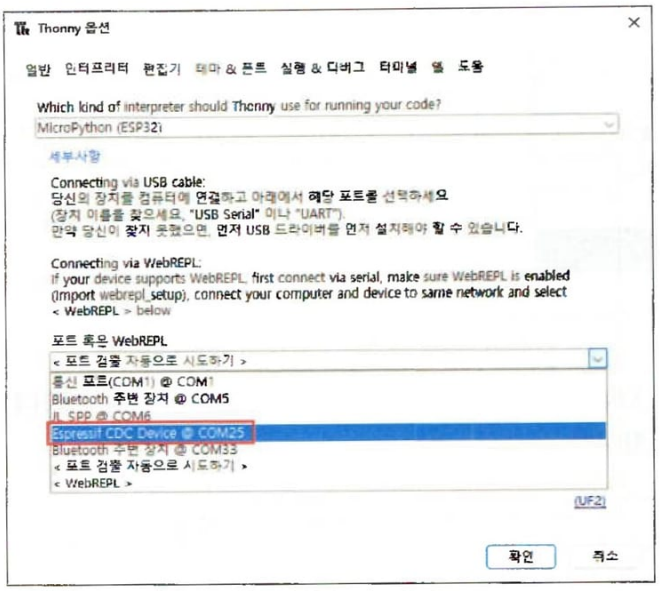
02 Thonny에서 드론의 보드를 선택하기 위해서 [도구] -> [옵션]을 클릭합니다.



03 [인터프리터] 탭으로 이동하여 장치는 MicroPython (ESP32)를 선택합니다. 포트는 자동검출로 선택 후 [확인]을 눌러 드론보드를 설정합니다.

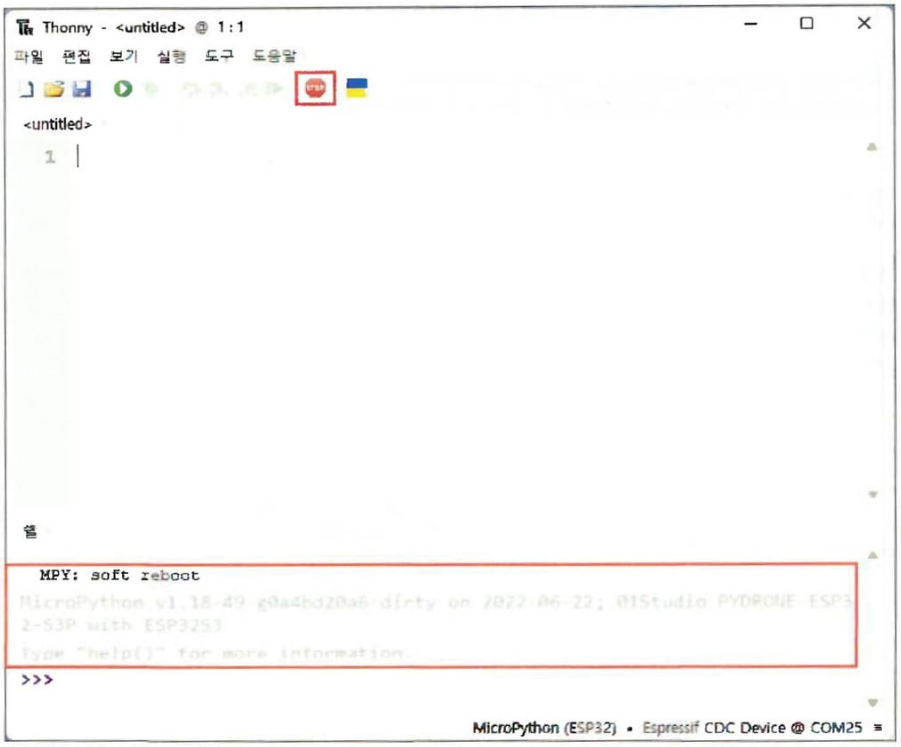


04 자동으로 포트가 선택되지 않는다면 Espressif CDC Device로 선택된 포트를 수동으로 선택합니다.

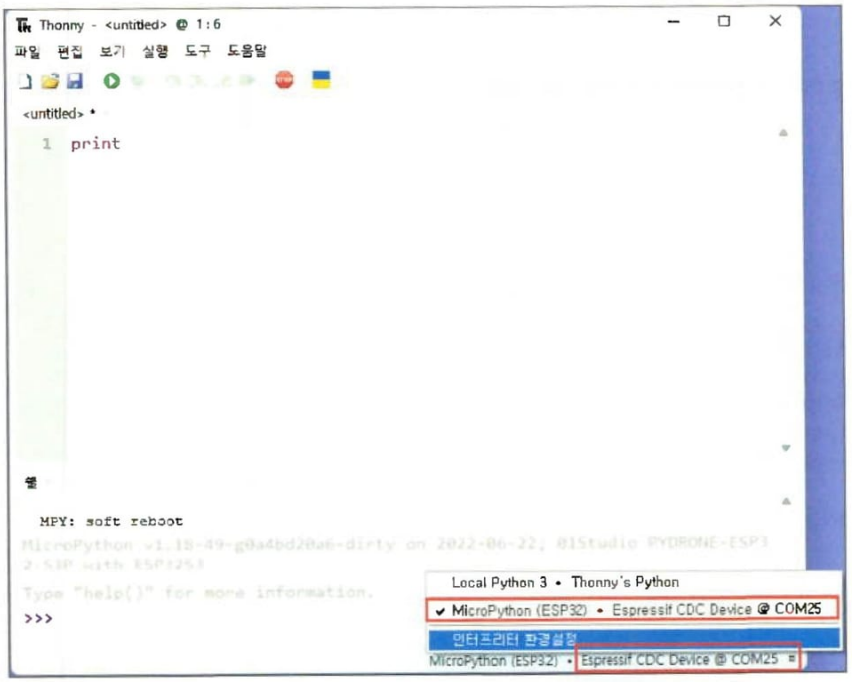


05 드론 Thonny가 잘 연결되었다면 아래 [쉘] 영역에 >>> 명령대기 부분이 출력됩니다. 연결되지 않는다면 [STOP]아이콘을 눌러 ESP32보드를 리셋하여 연결을 확인합니다.

잘 연결되었다면 설치를 완료하였습니다.



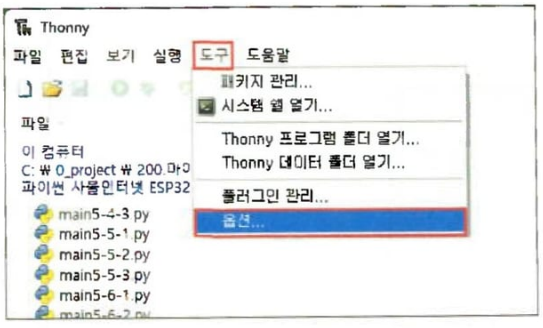
06 Thonny 편집기에서 오른쪽 아래 부분을 클릭하여 연결된 파이썬 환경의 확인이 가능하며 다른 환경으로 변경도 가능합니다.



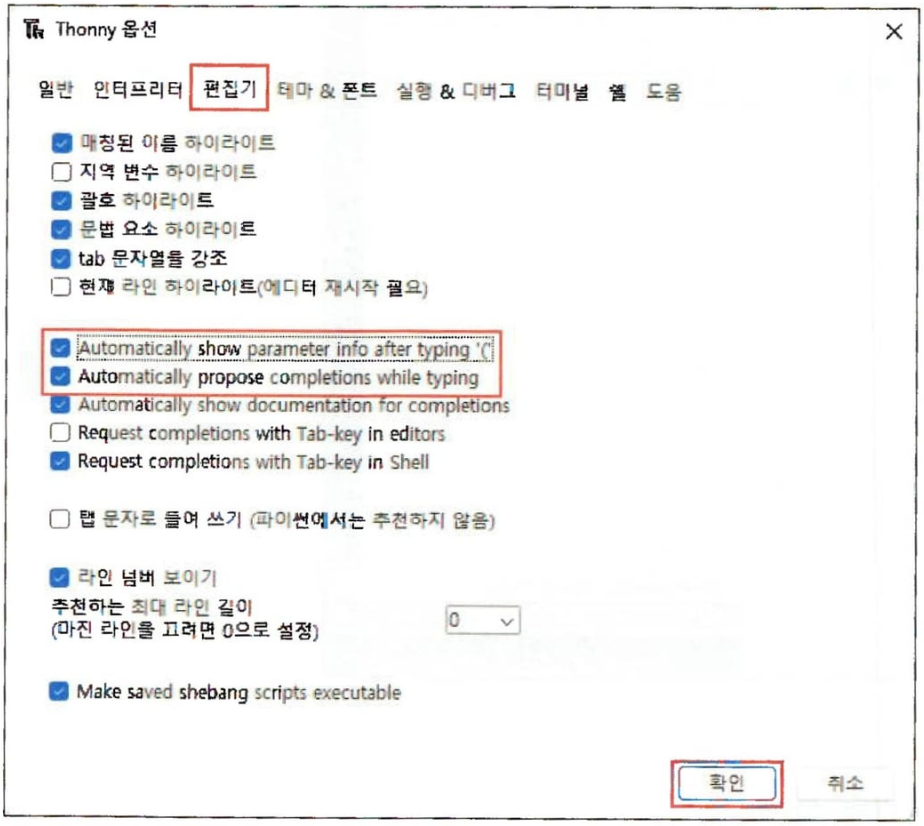
Thonny 유용한 기능 설정

01 Thonny에서 파이썬을 조금더 편하게 사용하기 위해서 자동완성 기능을 활성화 합니다.

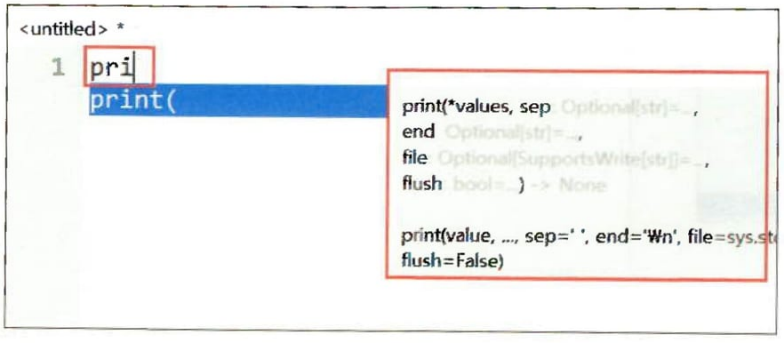
01 [도구] -> [옵션]을 클릭합니다.



02 편집기 탭에서 Automatically로 시작하는 옵션 두 개를 체크한다음 [확인]을 눌러 저장합니다. Thonny에서 자동완성을 사용하기 위한 옵션입니다.



03 print를 입력한다고 하면 print 모든글자를 입력하지 않고 몇글자만 입력하더라도 thonny에서 완성된 함수등을 추천하여 중간에 엔터를 입력하면 글자를 완성합니다. 자동완성기능으로 코드를 작성할 때 매우 편리한 기능입니다.



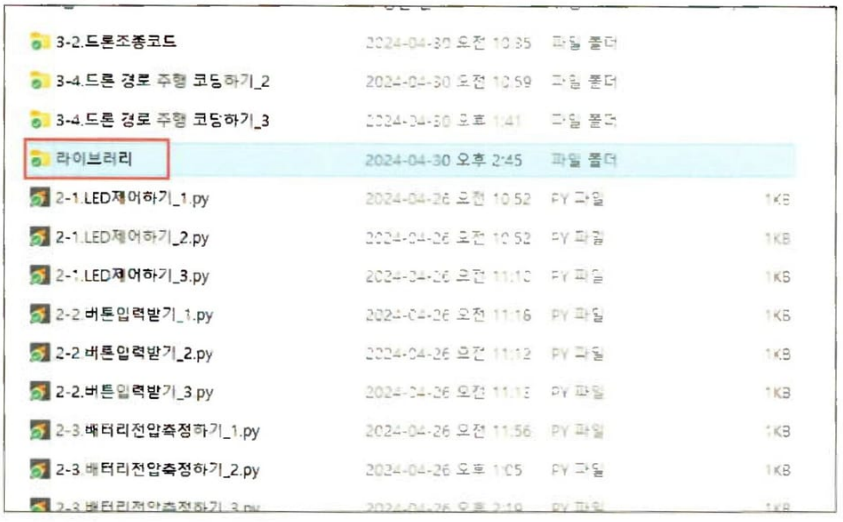
pyDrone에 라이브러리 설치

pyDrone의 다양한 추가기능을 다루기 위해서 pyDrone 장치에 라이브러리가 설치되어야 합니다.
pyDrone에 라이브러리를 업로드하는 방법에 대해서 알아봅니다.

01 아래 주소에 접속한다음 [라이브러리.zip]을 다운로드 받습니다.

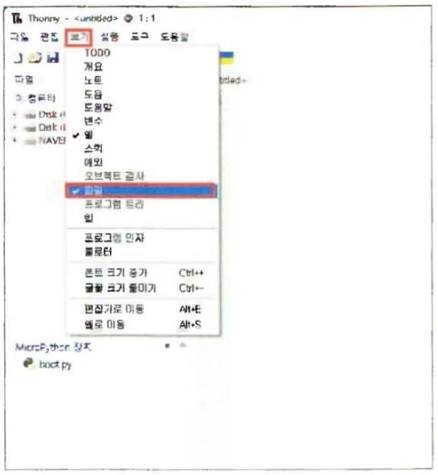
https://munjjac.tistory.com/15

02 다운로드 받은 라이브러리 파일의 압축을 풀어 준비합니다.

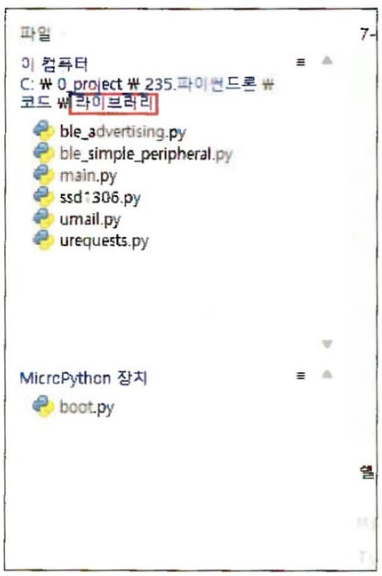


03 드론조종을 위한 블루투스 및 OLED라이브러리가 있습니다. main.py파일은 드론 조종 코드입니다. 앱을 이용해서 드론의 조종이 가능합니다.

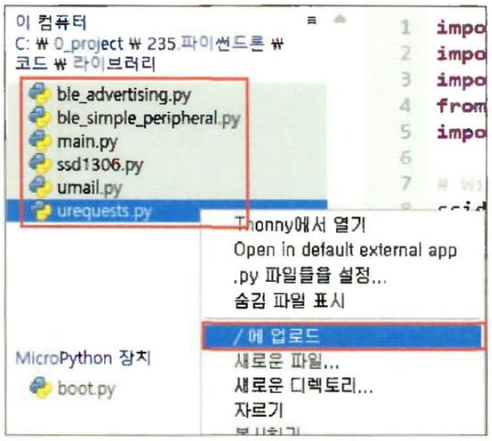
04 마이크로파이썬 장치의 파일 내용을 확인하기 위해 [보기] -> [파일]을 체크합니다.



05 압축을 푼 [라이브러리] 폴더로 이동합니다.



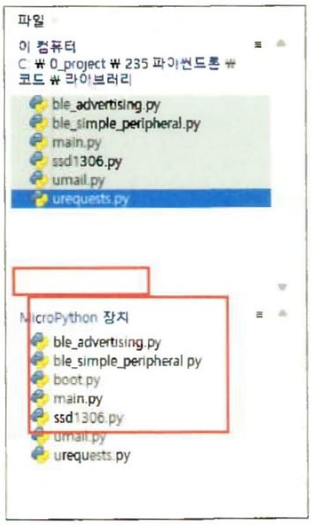
06 처음파일을 선택 후 [쉬프트+마지막 파일]을 선택하면 모든 파일의 선택이 가능합니다. 모든 파일을 선택 후 마우스 오른쪽을 클릭한다음 [/에 업로드]를 클릭하여 마이크로파이썬 장치에 라이브러리를 업로드합니다.



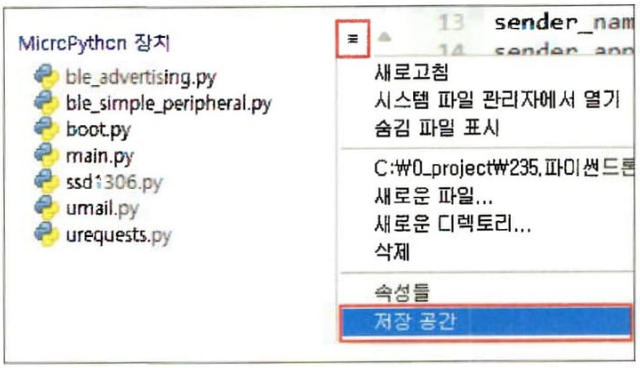
07 업로드 중입니다.

08 업로드가 완료되면 마이크로파이썬 장치에 라이브러리가 모두 업로드 되었습니다.

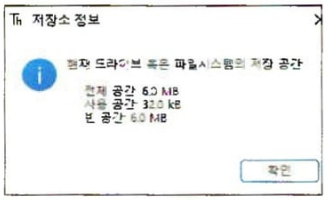
책에서 사용하는 모든 라이브러리의 업로드가 완료되었습니다. 블루투스, OLED, 웹접속, 이메일 보내기의 예제 에서는 라이브러리가 필수로 업로드 되어야 코드가 동작합니다.



09 마이크로파이썬 장치의 저장공간의 확인도 가능합니다.



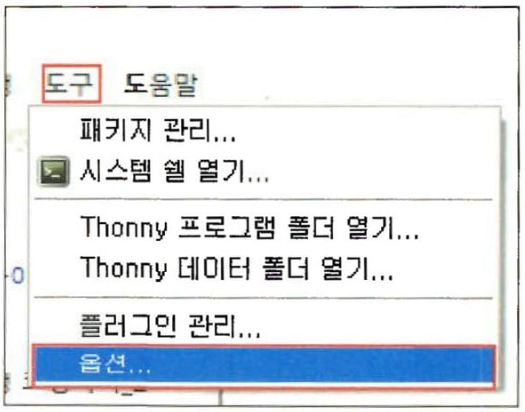
10 전체공간 6.0MB 중에 32KB를 사용하였습니다.



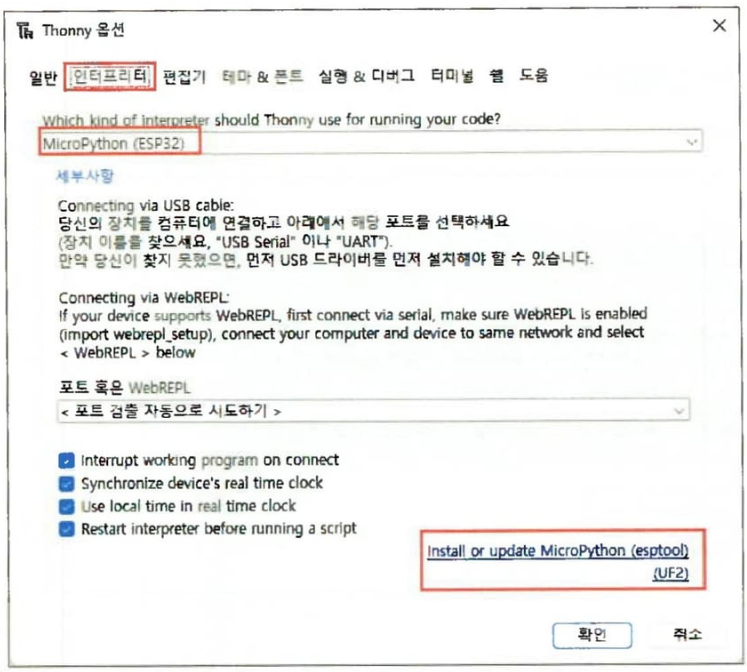
마이크로파이썬 인터프리터 업로드 방법

드론을 제어하는 마이크로파이썬 인터프리터는 기본적으로 드론에 업로드 되어 있습니다. 다만 실수로나 다른 인터프리터를 업로드 하였거나 아두이노를 사용하여 코드를 업로드 하였다면 드론을 제어하는 인터프리터를 다시 업로드 해야합니다. pyDrone에 마이크로파이썬 인터프리터를 업로드 하는 방법에 대해서 알아봅니다.

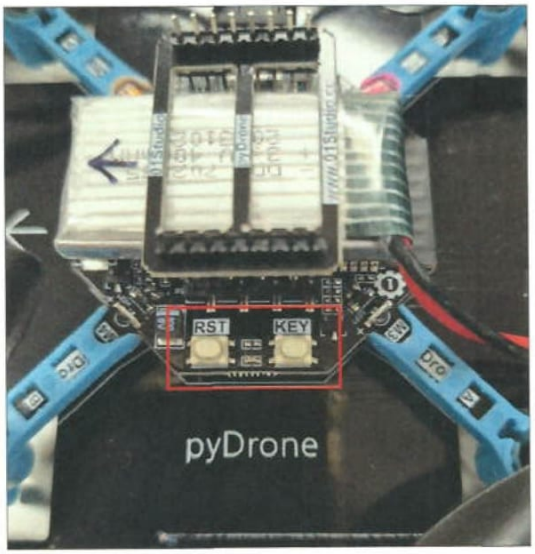
01 thonny에서 [도구] -> [옵션]을 클릭합니다.



02 [인터프리터] 탭에서 [Install or update Micropython...] 부분을 클릭합니다.



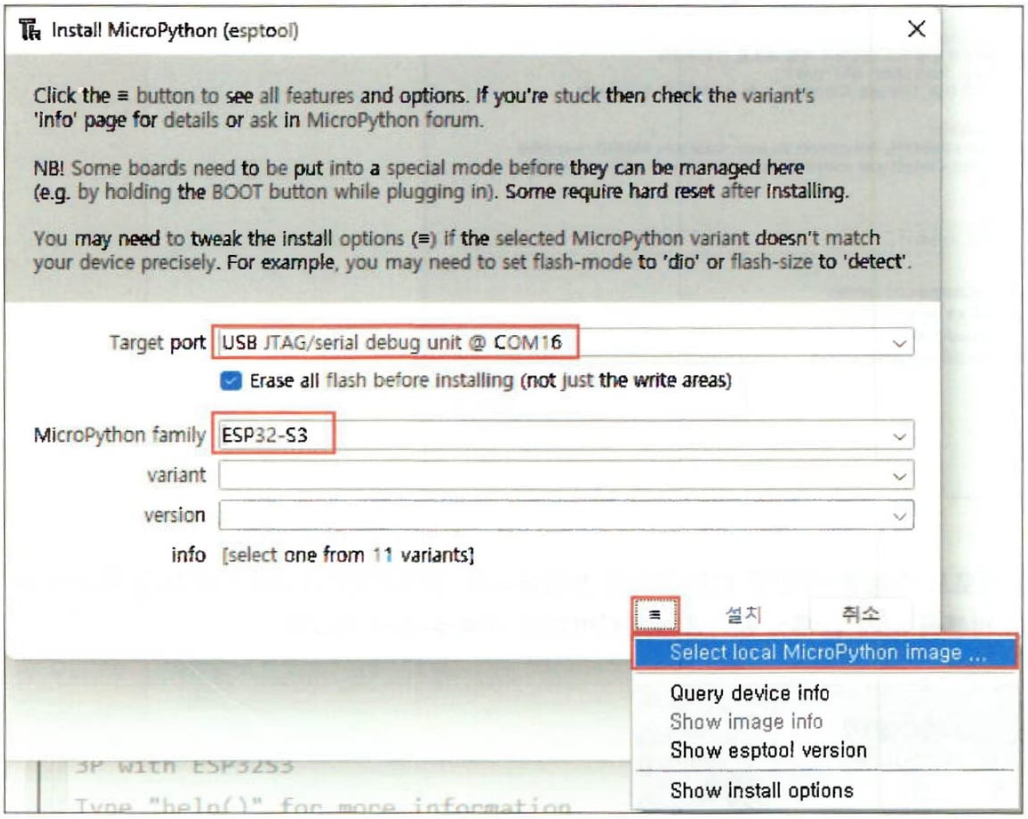
03 드론을 업로드 가능한 상태인 부트모드로 진입합니다. [RST]버튼과 [KEY]버튼을 동시에 누른다음 [RST] 버튼을 먼저 손에서 뗀다음 [KEY]버튼을 마저 손에서 뗍니다.



04 Target port 부분에서 [UST JTAG/serial debug]으로된 포트로 선택합니다. 부트모드시 기존의 포트번호가 변경됩니다.

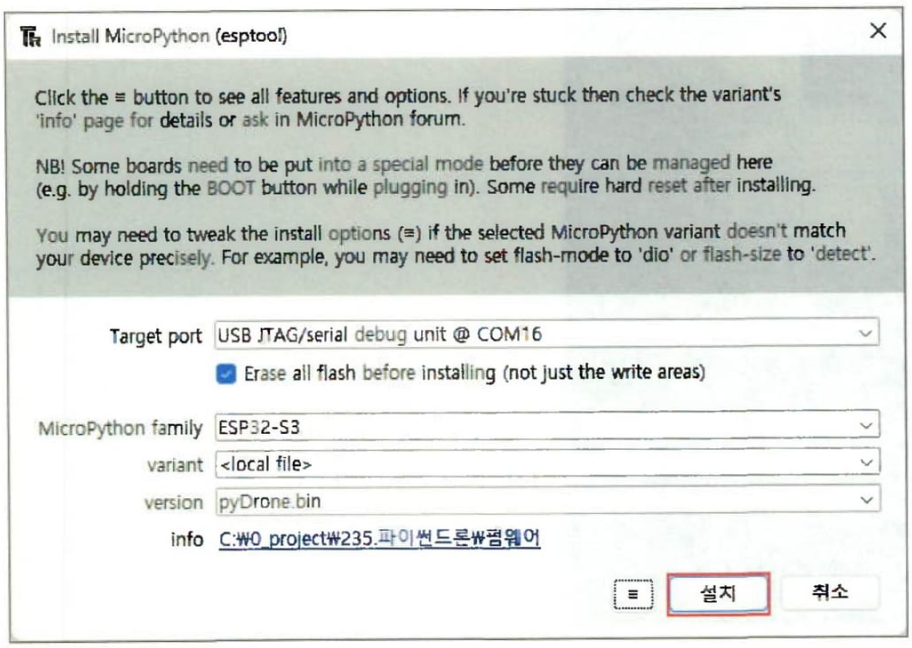
MicroPython family 는 ESP32-S3로 선택합니다.

더보기에서 [Select local Micropython Image]를 클릭합니다.

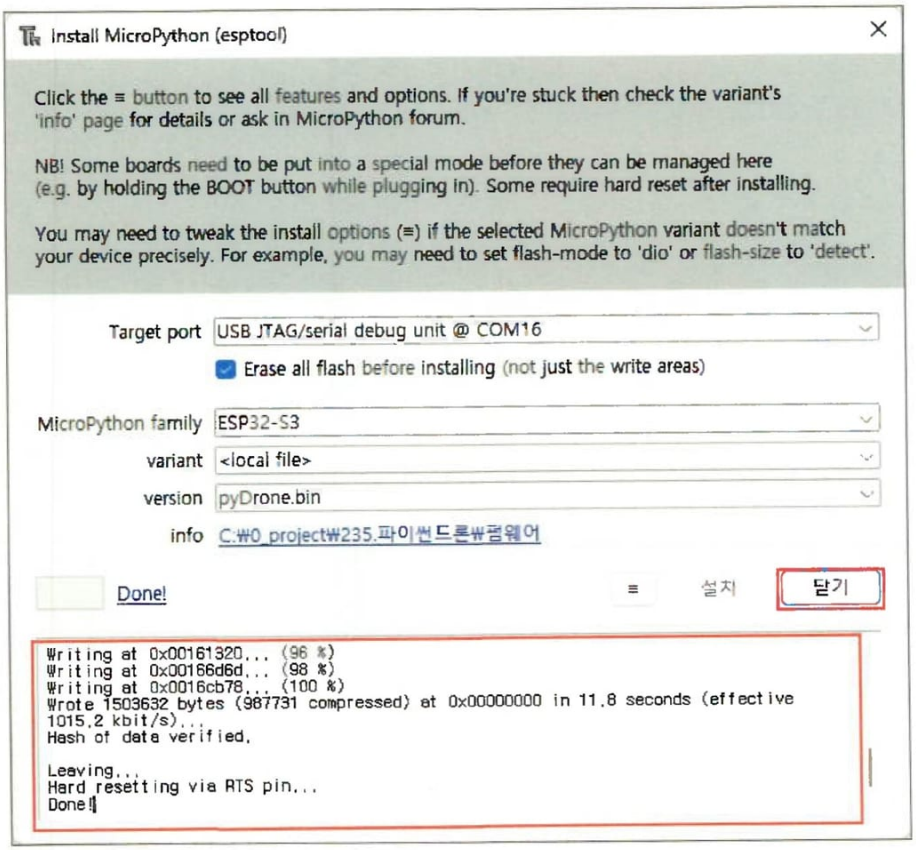


05 제공자료의 [펌웨어] 폴더에서 pyDrone.bin파일을 선택합니다.

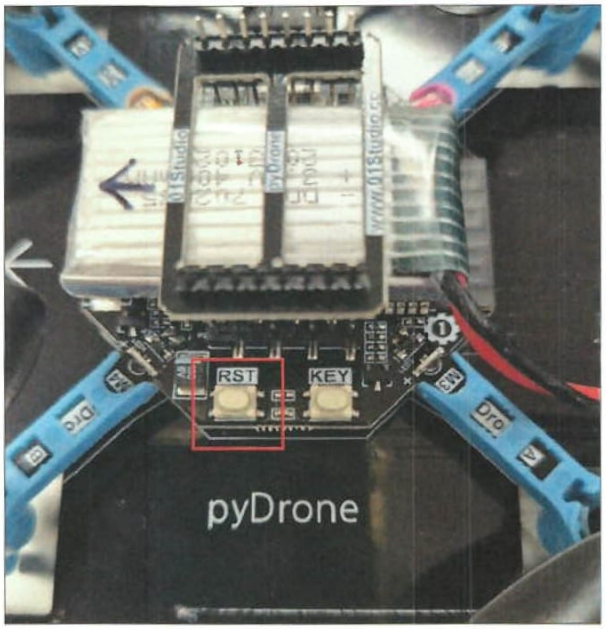
06 [설치]를 눌러 설치를 진행합니다.



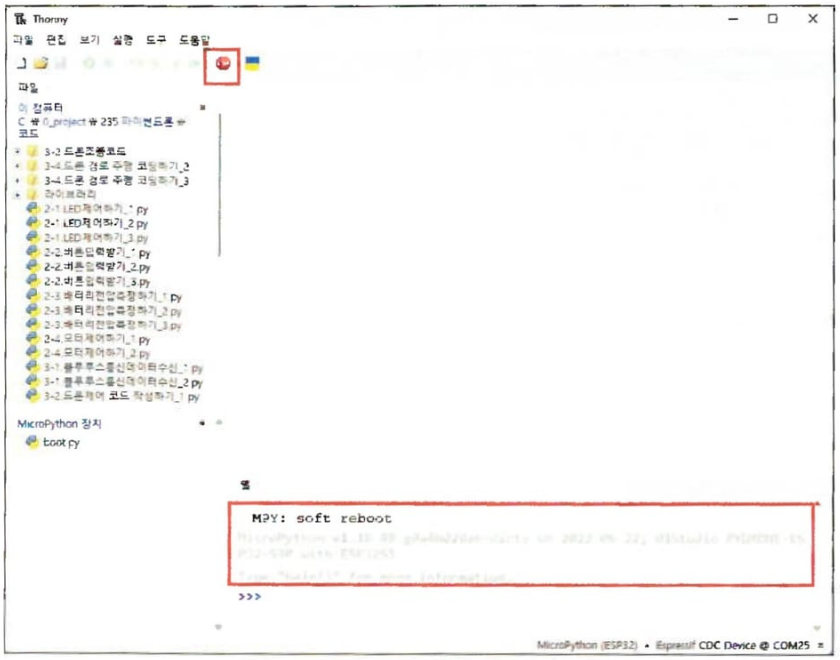
07 Done! 이 출력되었다면 설치가 잘 완료되었습니다. [닫기]를 눌러 설치 화면을 닫습니다.



08 [RST] 버튼을 눌러 드론을 리셋 후 사용합니다.



09 Thonny에서 보드를 다시 선택 후 사용합니다. 보드를 다시 선택 후 [STOP]을 누르면 pyDrone과 연결됩니다.



1-4

파이썬 기초문법

파이썬을 사용하여 드론을 제어합니다. 파이썬의 기초문법에 대해서 실습을 통해 배워보도록 합니다.

출력과 입력

1-4-1.출력

print()함수를 이용한 출력방법을 소개합니다. 아래의 코드를 Thonny IDE에 입력 합니다.

main1-4-1.py

```
01    print("1 hello")
02    print("2 hello","hi")
03    print("3 hello","hi","happy")
04    print("4 hello"+"hi")
05    print("5 hello",end="")
06    print("6 hi")
07    print('7 hello', "hi", 'good')
08    print("8 'say hello'")
09    print('9 "say happy"')
```

코드설명

1: `print("1 hello")`: "1 hello"라는 문자열을 출력하는 명령입니다.

2: `print("2 hello","hi")`: "2 hello"와 "hi"라는 두 개의 문자열을 출력합니다. 여러 개의 인자를 `,`로 구분하여 출력할 수 있습니다.

3: `print("3 hello","hi","happy")`: "3 hello", "hi", "happy"라는 세 개의 문자열을 출력합니다. 역시 `,`로 구분되

어 출력됩니다.

4: `print("4 hello"+"hi")`: "4 hellohi"라는 문자열을 출력합니다. 문자열 연결 연산자 `+`를 사용하여 두 문자열을 이어붙입니다.

5: `print("5 hello",end="")`: "5 hello"를 출력하고, `end` 매개변수를 빈 문자열("")로 설정하여 다음 `print` 문이 줄바꿈 없이 이어서 출력됩니다.

6: `print("6 hi")`: "6 hi"라는 문자열을 출력하며, 앞선 `print` 문과 이어서 출력됩니다.

7: `print('7 hello', "hi", 'good')`: '7 hello', "hi", 'good'라는 세 개의 문자열을 출력합니다. 작은 따옴표나 큰 따옴표로 감싸진 문자열은 모두 문자열로 처리됩니다.

8: `print("8 'say hello'")`: "8 'say hello'"라는 문자열을 출력합니다. 작은 따옴표 안에 있는 문자열은 큰 따옴표로 둘러싸여 있기 때문에 출력됩니다.

9: `print('9 "say happy"')`: '9 "say happy"'라는 문자열을 출력합니다: 큰 따옴표 안에 있는 문자열은 작은 따옴표로 둘러싸여 있기 때문에 출력됩니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

각 결과값이 쉘 영역에 출력되었습니다.

쉘

```
MPY: soft reboot
1 hello
2 hello hi
3 hello hi happy
4 hellohi
5 hello6 hi
7 hello hi good
8 'say hello'
9 "say happy"
```

1-4-2.입력

input()함수를 사용하여 사용자 값을 입력받아 저장하고 그값을 더하는 방법을 알아봅니다.

main1-4-2.py

```
1   a = input("첫 번째 값을 입력하세요:")
2   print("첫 번째 값 출력",a)
3
4   b = input("두 번째 값을 입력하세요:")
5   print("두 번째 값 출력",b)
6
7   print("더한 값:",a+b)
```

코드설명

1: `a = input("첫 번째 값을 입력하세요:")`: 사용자로부터 입력을 받기 위해 `input` 함수를 사용합니다. 이 함수는 사용자로부터 문자열을 입력받고, 그 값을 변수 `a`에 저장합니다. 입력 시에 "첫 번째 값을 입력하세요:"라는 메시지가 사용자에게 표시됩니다.

2: `print("첫 번째 값 출력",a)`: 변수 `a`에 저장된 값을 출력하는 명령입니다. "첫 번째 값 출력" 라는 문자열과 `a` 변수에 저장된 값이 함께 출력됩니다.

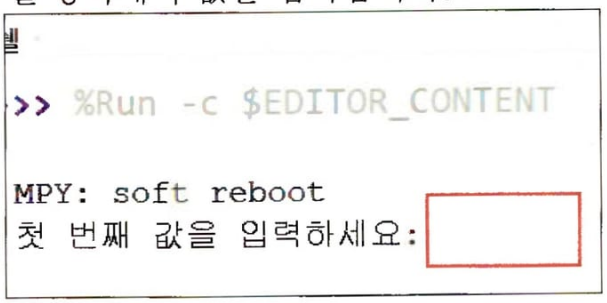
4: `b = input("두 번째 값을 입력하세요:")`: 두 번째 값을 입력받기 위해 `input` 함수를 다시 사용합니다. 사용자로부터 입력받은 값을 변수 `b`에 저장합니다. 입력 시에 "두 번째 값을 입력하세요:"라는 메시지가 표시됩니다.

5: `print("두 번째 값 출력",b)`: 변수 `b`에 저장된 값을 출력하는 명령입니다. "두 번째 값 출력" 라는 문자열과 `b` 변수에 저장된 값이 함께 출력됩니다.

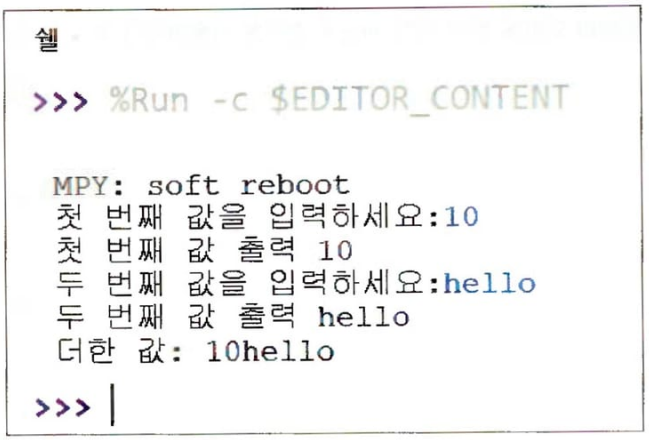
7: `print("더한 값:",a+b)`: 변수 `a`와 `b`에 저장된 값을 더하는 명령입니다. 그러나 주의해야 할 점은 `input` 함수로 입력받은 값은 문자열로 저장되기 때문에, 이 코드는 두 문자열을 단순히 이어붙이는 연산을 수행합니다. 즉, 두 문자열을 연결하고 "더한 값:" 문자열과 함께 출력합니다. 정수나 실수로 더하기를 원하면 입력 값을 정수나 실수로 변환해야 합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

쉘 영역에서 값을 입력합니다.



첫 번째 값은 10, 두 번째 값은 hello를 입력하였습니다. input()으로 입력한 값은 모두 문자열로 10과 hello를 더해 문자열을 이어붙인 형태로 출력되었습니다. 문자열의 덧셈은 문자열을 이어붙인 형태입니다.



1-4-3.숫자형,문자형,불형

숫자형 - 정수형 (int):

정수를 나타냅니다. 예를 들어, 1, 100, -5 등의 값을 저장할 수 있습니다.

숫자형 - 부동소수점형 (float):

실수를 나타냅니다. 소수점을 가진 숫자를 표현할 때 사용합니다.

예를 들어, 3.14, -0.5, 2.0 등이 있습니다.

문자열 (str):

문자의 시퀀스를 나타냅니다. 작은 따옴표(')나 큰 따옴표(")로 둘러싸인 문자들의 집합입니다. 예를 들어, "Hello, World!"나 'Python'과 같은 문자열이 있습니다. 문자열은 인덱싱, 슬라이싱 및 다양한 문자열 연산을 지원합니다.

불형 (Boolean):

불리언 (bool): 참(True) 또는 거짓(False)을 나타냅니다. 주로 조건문과 논리 연산에 사용됩니다. 예를 들어, 조건문에서 어떤 조건이 참이면 True를 반환하고, 그렇지 않으면 False를 반환합니다.

숫자형, 문자형, 불형을 확인하는 코드를 작성합니다.

main1-4-3.py

```
01    a =10
02    b =3.14
03    c ="abc"
04    d ='20'
05    e =True
06
07    print("타입비교:",type(a),type(b),type(c),type(d),type(e))
08
09    print("a+b:",a+b)
10    print("c+d:",c+d)
11    print("b+d:",str(b)+d)
12    print("b+d:",b+int(d))
```

코드설명

1. `a = 10`: 정수형 변수 `a`를 선언하고 값을 10으로 초기화합니다.

2. `b = 3.14`: 부동소수점형 변수 `b`를 선언하고 값을 3.14로 초기화합니다.

3. `c = "abc"`: 문자열 변수 `c`를 선언하고 값을 "abc"로 초기화합니다.

4. `d = '20'`: 문자열 변수 `d`를 선언하고 값을 '20'으로 초기화합니다. 따옴표로 감싸져 있기 때문에 문자열로 처리됩니다.

5. `e = True`: 불리언 변수 `e`를 선언하고 값을 True로 초기화합니다.

7. `print("타입비교:", type(a), type(b), type(c), type(d), type(e))`: 변수들의 타입을 비교하여 출력합니다. `type()` 함수를 사용하여 각 변수의 데이터 타입을 확인하고, 이를 문자열과 함께 출력합니다.

9. `print("a+b:", a + b)`: 변수 `a`와 `b`의 값을 더한 결과를 출력합니다. `a`는 정수, `b`는 부동소수점이므로 정수와 부동소수점을 더한 결과가 출력됩니다.

10. `print("c+d:", c + d)`: 문자열 변수 `c`와 `d`의 값을 이어붙여 출력합니다. 문자열 연결이므로 "abcd"가 출력됩니다.

11. `print("b+d:", str(b) + d)`: 부동소수점 변수 `b`를 문자열로 변환한 후 문자열 변수 `d`와 이어붙여 출력합니다. 결과적으로 "3.1420"이 출력됩니다.

12. `print("b+d:", b + int(d))`: 부동소수점 변수 `b`와 정수로 변환한 문자열 변수 `d`를 더하고 결과를 출력합니다. `int(d)`는 문자열 '20'을 정수 20으로 변환하며, 따라서 부동소수점과 정수를 더한 결과가 출력됩니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

각각 변수의 타입과 값이 출력되었습니다.

```
셸
>>> %Run -c $EDITOR_CONTENT

 MPY: soft reboot
 타입비교: <class 'int'> <class 'float'> <class 'str'> <class 'str'> <class 'bool'
 a+b: 13.14
 c+d: abc20
 b+d: 3.1420
 b+d: 23.14

>>>
```

1-4-4.리스트

리스트(list)는 파이썬에서 가장 기본적이고 유용한 데이터 구조 중 하나입니다. 리스트는 여러 항목을 담을 수 있는 순서가 있는 배열 형태의 데이터 구조를 제공합니다. 각 항목은 콤마로 구분되며, 대괄호 `[ ]`로 묶여 있습니다. 리스트는 다양한 데이터 유형을 혼합하여 저장할 수 있습니다. 리스트를 사용하면 데이터를 쉽게 추가, 삭제, 수정하거나 검색할 수 있습니다.

리스트를 생성하고 값을 확인하는 코드를 작성합니다.

main1-4-4.py

```
01  a_list = [1,2,3.14,"hello"]
02  print("a_list:",a_list)
03
04  b_list = [1,2,3.14,"hello",['hi',500]]
05  print("b_list:",b_list)
06
07  #리스트 인덱스
08  c_list = [1,2,3,4,5,6]
09  print("c_list:",c_list)
10  print("c_list[0]:",c_list[0])
11  print("c_list[2]:",c_list[2])
12  print("c_list[-1]:",c_list[-1])
13  print("c_list[-3]:",c_list[-3])
14
15  #리스트 슬라이스
16  d_list = [1,2,3,4,5,6]
17  print("d_list:",d_list)
18  print("d_list[0:2]:",d_list[0:2])
19  print("d_list[:2]:",d_list[:2])
20  print("d_list[2:]:",d_list[2:])
```

코드설명

1. `a_list = [1, 2, 3.14, "hello"]`: 리스트 `a_list`를 생성하고, 정수, 부동소수점, 문자열로 구성된 요소들을 포함합니다.

2. `print("a_list:", a_list)`: 리스트 `a_list`의 내용을 출력합니다.

4. `b_list = [1, 2, 3.14, "hello", ['hi', 500]]`: 리스트 `b_list`를 생성하고, 리스트 안에 또 다른 리스트를 포함합니다. 이러한 중첩된 리스트를 만들 수 있습니다.

5. `print("b_list:", b_list)`: 리스트 `b_list`의 내용을 출력합니다.

8. `c_list = [1, 2, 3, 4, 5, 6]`: 정수로만 구성된 리스트 `c_list`를 생성합니다.

10. `print("c_list[0]:", c_list[0])`: 리스트 `c_list`의 첫 번째 요소(인덱스 0)를 출력합니다.

11. `print("c_list[2]:", c_list[2])`: 리스트 `c_list`의 세 번째 요소(인덱스 2)를 출력합니다.

12. `print("c_list[-1]:", c_list[-1])`: 리스트 `c_list`의 마지막 요소(음수 인덱스 -1을 사용하여)를 출력합니다.

13. `print("c_list[-3]:", c_list[-3])`: 리스트 `c_list`의 뒤에서 세 번째 요소(음수 인덱스 -3을 사용하여)를 출력합니다.

15. `d_list = [1, 2, 3, 4, 5, 6]`: 정수로만 구성된 또 다른 리스트 `d_list`를 생성합니다.

17. `print("d_list:", d_list)`: 리스트 `d_list`의 내용을 출력합니다.

18. `print("d_list[0:2]:", d_list[0:2])`: 리스트 `d_list`의 처음부터 두 번째 요소(인덱스 0부터 1)까지 슬라이싱하여 출력합니다.

19. `print("d_list[:2]:", d_list[:2])`: 리스트 `d_list`의 처음부터 두 번째 요소(인덱스 0부터 1)까지 슬라이싱하여 출력합니다. 슬라이스 시작 인덱스를 지정하지 않으면 자동으로 0부터 시작합니다.

20. `print("d_list[2:]:", d_list[2:])`: 리스트 `d_list`의 세 번째 요소(인덱스 2)부터 마지막 요소까지 슬라이싱하여 출력합니다. 슬라이스 종료 인덱스를 지정하지 않으면 자동으로 리스트의 끝까지 포함합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

셸

```
MPY: soft reboot
a_list: [1, 2, 3.14, 'hello']
b_list: [1, 2, 3.14, 'hello', ['hi', 500]]
c_list: [1, 2, 3, 4, 5, 6]
c_list[0]: 1
c_list[2]: 3
c_list[-1]: 6
c_list[-3]: 4
d_list: [1, 2, 3, 4, 5, 6]
d_list[0:2]: [1, 2]
d_list[:2]: [1, 2]
d_list[2:]: [3, 4, 5, 6]
```

1-4-5.튜플

튜플(tuple)은 리스트와 유사한 데이터 구조로, 여러 항목을 저장할 수 있는 순서가 있는 컬렉션입니다. 하지만 튜플과 리스트의 가장 큰 차이점은 튜플은 불변(immutable)하다는 점입니다. 즉, 한 번 생성된 튜플은 그 내용을 변경할 수 없습니다.

튜플은 괄호 `( )`로 묶거나, 괄호 없이 콤마 `,`로 구분된 항목을 나열하여 생성할 수 있습니다. 주로 데이터를 변경하면 안 되는 상황에서 사용하며, 리스트보다 메모리 효율적인 경우가 있습니다.

main1-4-5.py

```
01   a_tuple = (1,2,3,"hello")
02   print("a_tuple:",a_tuple)
03   print(type(a_tuple))
04
05   b_tuple =2,3,4,"hello"
06   print("b_tuple:",b_tuple)
07   print(type(b_tuple))
08
09   #튜플 인덱싱 및 슬라이싱
10   print("b_tuple[0]:",b_tuple[0])
11   print("b_tuple[:2]:",b_tuple[:2])
12
13   #튜플 값 변경
14   b_tuple[0] =10
```

코드설명

1. `a_tuple = (1, 2, 3, "hello")`: 튜플 `a_tuple`을 생성하고, 정수와 문자열로 구성된 요소들을 포함합니다. 튜플은 소괄호 `()`를 사용하여 생성합니다.

2. `print("a_tuple:", a_tuple)`: 튜플 `a_tuple`의 내용을 출력합니다.

3. `print(type(a_tuple))`: 튜플 `a_tuple`의 데이터 타입을 출력합니다. 이 코드는 `type()` 함수를 사용하여 튜플의 데이터 타입을 확인하는 예제입니다.

5. `b_tuple = 2, 3, 4, "hello"`: 괄호 없이 튜플 `b_tuple`을 생성합니다. 일반적으로 튜플은 괄호로 둘러싸지만, 괄호 없이도 여러 값들을 쉼표로 구분하여 튜플을 생성할 수 있습니다.

6. `print("b_tuple:", b_tuple)`: 튜플 `b_tuple`의 내용을 출력합니다.

7. `print(type(b_tuple))`: 튜플 `b_tuple`의 데이터 타입을 출력합니다.

10. `print("b_tuple[0]:", b_tuple[0])`: 튜플 `b_tuple`의 첫 번째 요소(인덱스 0)를 출력합니다.

11. `print("b_tuple[:2]:", b_tuple[:2])`: 튜플 `b_tuple`의 처음부터 두 번째 요소(인덱스 0부터 1)까지 슬라이싱하여 출력합니다.

13. `b_tuple[0] = 10`: 이 코드는 튜플의 값을 변경하려고 시도하는 부분입니다. 그러나 튜플은 변경 불가능한 자료형이므로 에러가 발생할 것입니다. 튜플 내부의 값을 수정하려면 새로운 튜플을 생성해야 합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

튜플은 리스트와 마찬가지로 인덱싱과 슬라이싱을 이용하여 값을 가져올수 있습니다. 다만 튜플의 요소를 변경하면 에러가 발생합니다.

```
쉘
>>> %Run -c $EDITOR_CONTENT

 MPY: soft reboot
 a_tuple: (1, 2, 3, 'hello')
 <class 'tuple'>
 b_tuple: (2, 3, 4, 'hello')
 <class 'tuple'>
 b_tuple[0]: 2
 b_tuple[:2]: (2, 3)
 Traceback (most recent call last):
   File "<stdin>", line 14, in <module>
 TypeError: 'tuple' object doesn't support item assignmen

>>>
```

1-4-6.딕셔너리

딕셔너리(dictionary)는 파이썬에서 사용되는 키-값 쌍(key-value pair) 데이터 구조입니다. 딕셔너리는 중괄호 `{}`로 묶여 있으며, 각 항목은 키(key)와 그에 해당하는 값(value)으로 구성됩니다. 키는 고유하며 변경되지 않는 값이며, 값은 키에 연결된 데이터를 나타냅니다.

딕셔너리는 데이터를 검색하고 저장하는 데 효율적이며, 키를 사용하여 값을 빠르게 찾을 수 있습니다. 딕셔너리는 다양한 데이터 형식을 값으로 포함할 수 있으며, 유연하게 데이터를 구조화하는 데 사용됩니다.

main1-4-6.py

```
1   a_dict = {"name":'dain', "age": 7, "like": "mom"}
2   print("a_dict:",a_dict)
3   print('a_dict["name"]:',a_dict["name"])
4
5   #값추가
6   a_dict["height"] =120
7   print("a_dict:",a_dict)
```

코드설명

1. `a_dict = {"name": 'dain', "age": 7, "like": "mom"}`: 딕셔너리 `a_dict`를 생성합니다. 이 딕셔너리에는 "name", "age", "like"라는 키(key)와 이에 해당하는 값(value)들이 포함되어 있습니다. 딕셔너리는 중괄호 `{}`를 사용하여 생성하며, 각 항목은 콜론(`:`)으로 키와 값이 구분됩니다.
2. `print("a_dict:", a_dict)`: 딕셔너리 `a_dict`의 내용을 출력합니다.
3. `print('a_dict["name"]:', a_dict["name"])`: 딕셔너리 `a_dict`에서 "name" 키의 값을 출력합니다. 대괄호를 사용하여 딕셔너리에서 특정 키에 해당하는 값을 얻을 수 있습니다.
5. `a_dict["height"] = 120`: 딕셔너리 `a_dict`에 "height"라는 키와 값 120을 추가합니다. 새로운 키와 값을 추가할 때는 대괄호를 사용하여 키를 지정하고 등호(`=`)를 통해 값을 할당합니다.
6. `print("a_dict:", a_dict)`: 딕셔너리 `a_dict`의 내용을 출력합니다. 이번에는 "height" 키와 값이 추가되어 출력됩니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

딕셔너리를 생성하고 그 값이 출력되었습니다.

```
쉘
>>> %Run -c $EDITOR_CONTENT

 MPY: soft reboot
 a_dict: {'name': 'dain', 'age': 7, 'like': 'mom'}
 a_dict["name"]: dain
 a_dict: {'like': 'mom', 'height': 120, 'name': 'dain', 'age': 7}
>>>
```

연산자와 제어문

1-4-7.산술연산자

산술 연산자(arithmetic operators)는 숫자형 데이터를 사용하여 수학적 연산을 수행하는 데 사용되는 연산자들입니다. 주요 산술 연산자는 다음과 같습니다:

1. 덧셈 연산자 (+): 두 숫자를 더합니다.
2. 뺄셈 연산자 (-): 첫 번째 숫자에서 두 번째 숫자를 뺍니다.
3. 곱셈 연산자 (*): 두 숫자를 곱합니다.
4. 나눗셈 연산자 (/): 첫 번째 숫자를 두 번째 숫자로 나눕니다. 결과는 부동소수점 형태일 수 있습니다.
5. 정수 나눗셈 연산자 (//): 첫 번째 숫자를 두 번째 숫자로 나눈 뒤 소수점 이하를 버린 정수값을 반환합니다.
6. 나머지 연산자 (%): 첫 번째 숫자를 두 번째 숫자로 나눈 나머지를 반환합니다.
7. 거듭제곱 연산자 (**): 첫 번째 숫자를 두 번째 숫자의 거듭제곱으로 계산합니다.

산술 연산자는 숫자형 데이터를 다루는 데 사용되며, 수학적 계산을 프로그램에서 수행할 때 필수적입니다.

main1-4-7.py

```
01  a =50
02  b =20
03
04  print("a+b:", a+b) #더하기
05  print("a-b:", a-b) #빼기
06  print("a*b:", a*b) #곱하기
07  print("a/b:", a/b) #나누기
08  print("a%b:", a%b) #나머지
09  print("a//b:", a//b) #몫
10  print("a**b:", a**b) #거듭제곱
```

코드설명

1. `a = 50`: 정수 변수 `a`를 선언하고 값을 50으로 초기화합니다.

2. `b = 20`: 정수 변수 `b`를 선언하고 값을 20으로 초기화합니다.

4. `print("a+b:", a+b)`: 변수 `a`와 `b`를 더한 결과를 출력합니다. 이 경우 50 + 20 = 70이므로 "a+b: 70"이 출력됩니다.

5. `print("a-b:", a-b)`: 변수 `a`에서 `b`를 뺀 결과를 출력합니다. 50 - 20 = 30이므로 "a-b: 30"이 출력됩니다.

6. `print("a*b:", a*b)`: 변수 `a`와 `b`를 곱한 결과를 출력합니다. 50 * 20 = 1000이므로 "a*b: 1000"이 출력됩니다.

7. `print("a/b:", a/b)`: 변수 `a`를 변수 `b`로 나눈 결과를 출력합니다. 50 / 20 = 2.5이므로 "a/b: 2.5"가 출력됩니다.

8. `print("a%b:", a%b)`: 변수 `a`를 변수 `b`로 나눈 나머지를 출력합니다. 50 % 20 = 10이므로 "a%b: 10"이 출력됩니다.

9. `print("a//b:", a//b)`: 변수 `a`를 변수 `b`로 나눈 몫을 출력합니다. 50 // 20 = 2이므로 "a//b: 2"가 출력됩니다.

10. `print("a**b:", a**b)`: 변수 `a`를 변수 `b`만큼 거듭제곱한 결과를 출력합니다. 50^20은 매우 큰 수이므로 정확한 결과를 표현하기 어렵지만, "a**b: 큰 수"와 같이 출력될 것입니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

산술연산의 결과가 출력되었습니다.

```
쉘
>>> %Run -c $EDITOR_CONTENT

 MPY: soft reboot
 a+b: 70
 a-b: 30
 a*b: 1000
 a/b: 2.5
 a%b: 10
 a//b: 2
 a**b: 95367431640625000000000000000000000

>>>
```

1-4-8.비교연산자

비교 연산자(comparison operators)는 두 개의 값을 비교하여 논리적인 참(True) 또는 거짓(False)을 반환하는 연산자들입니다. 주로 조건문과 함께 사용되어 프로그램에서 조건을 평가하는 데 사용됩니다. 다음은 주요 비교 연산자입니다:

1. 동등 비교 연산자 (==): 두 값이 서로 같으면 참(True)을 반환합니다.
2. 부등 비교 연산자 (!=): 두 값이 서로 다르면 참(True)을 반환합니다.
3. 크다 비교 연산자 (>): 왼쪽 값이 오른쪽 값보다 크면 참(True)을 반환합니다.
4. 작다 비교 연산자 (<): 왼쪽 값이 오른쪽 값보다 작으면 참(True)을 반환합니다.
5. 크거나 같다 비교 연산자 (>=): 왼쪽 값이 오른쪽 값보다 크거나 같으면 참(True)을 반환합니다.
6. 작거나 같다 비교 연산자 (<=): 왼쪽 값이 오른쪽 값보다 작거나 같으면 참(True)을 반환합니다.

비교 연산자는 조건문에서 사용되어 프로그램이 특정 조건을 만족하는지 여부를 판단할 때 중요하게 활용됩니다.

비교연산 코드를 작성합니다.

main1-4-8.py

```
01    a =10
02    b =20
03
04    print("a==b:", a==b) #같음
05    print("a!=b:", a!=b) #다름
06    print("a<b:", a<b) #a가 작음
07    print("a<=b:", a<=b) #a가 작거나 같음
08    print("a>b:", a>b) #a가 큼
09    print("a>=b:", a>=b) #a가 크거나 같
```

코드설명

1. `a = 10`: 정수 변수 `a`를 선언하고 값을 10으로 초기화합니다.

2. `b = 20`: 정수 변수 `b`를 선언하고 값을 20으로 초기화합니다.

4. `print("a==b:", a==b)`: 변수 `a`와 `b`의 값이 같은지를 비교한 결과를 출력합니다. `a`와 `b`가 다르므로 "a==b: False"가 출력됩니다.

5. `print("a!=b:", a!=b)`: 변수 `a`와 `b`의 값이 다른지를 비교한 결과를 출력합니다. `a`와 `b`가 다르므로 "a!=b: True"가 출력됩니다.

6. `print("a<b:", a<b)`: 변수 `a`가 변수 `b`보다 작은지를 비교한 결과를 출력합니다. `a`가 `b`보다 작으므로 "a<b: True"가 출력됩니다.

7. `print("a<=b:", a<=b)`: 변수 `a`가 변수 `b`보다 작거나 같은지를 비교한 결과를 출력합니다. `a`가 `b`보다 작으므로 "a<=b: True"가 출력됩니다.

8. `print("a>b:", a>b)`: 변수 `a`가 변수 `b`보다 큰지를 비교한 결과를 출력합니다. `a`가 `b`보다 작으므로 "a>b: False"가 출력됩니다.

9. `print("a>=b:", a>=b)`: 변수 `a`가 변수 `b`보다 크거나 같은지를 비교한 결과를 출력합니다. `a`가 `b`보다 작으므로 "a>=b: False"가 출력됩니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

비교연산의 결과값이 출력되었습니다.

쉘

```
>>> %Run -c $EDITOR_CONTENT

 MPY: soft reboot
 a==b: False
 a!=b: True
 a<b: True
 a<=b: True
 a>b: False
 a>=b: False

>>>
```

1-4-9.논리연산자

논리 연산자(logical operators)는 불리언 데이터를 다룰 때 사용되며, 여러 개의 불리언 표현식을 조합하여 새로운 불리언 결과를 얻는 데 사용됩니다. 주요 논리 연산자는 다음과 같습니다:

1. 논리 AND (and): 모든 피연산자가 참(True)일 때만 참(True)을 반환하며, 하나라도 거짓(False)이면 거짓(False)을 반환합니다.
2. 논리 OR (or): 최소한 하나의 피연산자가 참(True)이면 참(True)을 반환하며, 모든 피연산자가 거짓(False)이면 거짓(False)을 반환합니다.
3. 논리 NOT (not): 피연산자의 불리언 값을 반전시킵니다. 참(True)은 거짓(False)으로, 거짓(False)은 참(True)으로 변환됩니다.

main1-4-9.py

```
01    a =10
02    b =20
03
04    #and
05    print("True and True:", True and True)
06    print("False and True:", False and True)
07    print("a > 5 and b > 10:", a >5 and b >10)
08    print("a > 15 and b > 10:", a >15 and b >10)
09
10    #or
11    print("True or True:", True or True)
```

```
print("False or True:", False or True)
print("a > 5 and b > 10:", a >5 or b >10)
print("a > 15 and b > 10:", a >15 or b >30)

#not
print("not True:", not True)
print("not False:", not False)
```

코드설명

1. `a = 10`: 정수 변수 `a`를 선언하고 값을 10으로 초기화합니다.

2. `b = 20`: 정수 변수 `b`를 선언하고 값을 20으로 초기화합니다.

5. `print("True and True:", True and True)`: `and` 논리 연산자를 사용하여 `True`와 `True`를 비교한 결과를 출력합니다. `and` 연산자는 두 개의 조건이 모두 `True`일 때만 `True`를 반환하므로 "True and True: True"가 출력됩니다.

6. `print("False and True:", False and True)`: `and` 논리 연산자를 사용하여 `False`와 `True`를 비교한 결과를 출력합니다. 하나 이상의 조건이 `False`이면 전체 표현식은 `False`가 되므로 "False and True: False"가 출력됩니다.

7. `print("a > 5 and b > 10:", a > 5 and b > 10)`: 변수 `a`가 5보다 크고 변수 `b`가 10보다 큰지를 비교한 결과를 출력합니다. `a`는 10이고 `b`는 20이므로 둘 다 조건을 만족하므로 "a > 5 and b > 10: True"가 출력됩니다.

8. `print("a > 15 and b > 10:", a > 15 and b > 10)`: 변수 `a`가 15보다 크고 변수 `b`가 10보다 큰지를 비교한 결과를 출력합니다. `a`는 10이므로 첫 번째 조건을 만족하지 않으므로 "a > 15 and b > 10: False"가 출력됩니다.

10. `print("True or True:", True or True)`: `or` 논리 연산자를 사용하여 `True`와 `True`를 비교한 결과를 출력합니다. `or` 연산자는 두 개의 조건 중 하나라도 `True`이면 전체 표현식은 `True`가 되므로 "True or True: True"가 출력됩니다.

11. `print("False or True:", False or True)`: `or` 논리 연산자를 사용하여 `False`와 `True`를 비교한 결과를 출력합니다. 둘 중 하나가 `True`이므로 "False or True: True"가 출력됩니다.

12. `print("a > 5 or b > 10:", a > 5 or b > 10)`: 변수 `a`가 5보다 크거나 변수 `b`가 10보다 큰지를 비교한 결과를 출력합니다. `a`는 10이므로 첫 번째 조건을 만족하므로 "a > 5 or b > 10: True"가 출력됩니다.

13. `print("a 〉 15 or b 〉 10:", a 〉 15 or b 〉 30)`: 변수 `a`가 15보다 크거나 변수 `b`가 30보다 큰지를 비교한 결과를 출력합니다. 두 조건 모두 만족하지 않으므로 "a 〉 15 or b 〉 10: False"가 출력됩니다.

16. `print("not True:", not True)`: `not` 논리 연산자를 사용하여 `True`를 부정한 결과를 출력합니다. `not` 연산자는 주어진 조건을 반대로 뒤집습니다. 따라서 "not True: False"가 출력됩니다.

17. `print("not False:", not False)`: `not` 논리 연산자를 사용하여 `False`를 부정한 결과를 출력합니다. "not False: True"가 출력됩니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

논리연산자의 결과가 출력되었습니다.

```
쉘

MPY: soft reboot
True and True: True
False and True: False
a > 5 and b > 10: True
a > 15 and b > 10: False
True or True: True
False or True: True
a > 5 and b > 10: True
a > 15 and b > 10: False
not True: False
not False: True

>>>
```

1-4-10.조건문

조건문은 프로그래밍에서 조건에 따라 코드의 실행 흐름을 제어하는 제어 구조입니다. 조건문은 주어진 조건을 평가하고 그 결과에 따라 다른 코드 블록을 실행하거나 스킵합니다. 주로 "만약(if)" 특정 조건이 참(True)이라면 무엇을 실행하라, "그렇지 않다면 (else)" 다른 것을 실행하라와 같은 로직을 구현하는 데 사용됩니다.

조건문의 중요한 특징 중 하나는 들여쓰기(indentation)입니다. 파이썬과 같은 프로그래밍 언어에서 들여쓰기는 코드 블록의 시작과 끝을 나타내는 방법으로 사용됩니다. 일반적으로 들여쓰기는 공백 문자 또는 탭 문자를 사용하여 구현되며, 코드 블록은 같은 수준의 들여쓰기로 표시됩니다.

주요 조건문 구조는 다음과 같습니다:

1. if 문: 주어진 조건이 참(True)인 경우에만 특정 코드 블록을 실행합니다.
2. if-else 문: 주어진 조건이 참(True)이면 첫 번째 코드 블록을 실행하고, 그렇지 않으면 두 번째 코드 블록을 실행합니다.
3. if-elif-else 문: 여러 개의 조건을 순차적으로 평가하며, 처음으로 참(True)인 조건을 만나면 해당 코드 블록을 실행합니다. 그렇지 않으면 else 블록을 실행합니다.

조건문을 사용하여 프로그램의 실행 흐름을 제어할 때, 들여쓰기를 정확하게 사용해야 합니다. 들여쓰기 오류는 코드의 의도를 혼동시키고 프로그램 오류를 발생시킬 수 있으므로 주의가 필요합니다.

main1-4-10.py

```
01    a =10
02
03    #if
04    if a >5:
05        print("a는 5보다 큼")
06
07    if a <5:
08        print("a는 5보다 작음")
09
10    #if else
11    if a >20:
12        print("a는 20보다 큼")
13    else:
14        print("a는 20보다 작음")
15
16    #if elif else
17    if a >0 and a <9:
18        print("a는 0보다 크고 9보다 작음")
19    elif a >=10 and a <20:
20        print("a는 10이상이고 20보다 작음")
21    else:
22        print("a는 20보다 큼")
```

코드설명

1. `a = 10`: 정수 변수 `a`를 선언하고 값을 10으로 초기화합니다.

4. `if a > 5:`: `if` 조건문을 사용하여 변수 `a`가 5보다 큰지를 검사합니다. 조건이 참인 경우에만 아래의 코드 블록을 실행합니다.

5. `print("a는 5보다 큼")`: 조건이 참일 때 실행되는 코드 블록입니다. 변수 `a`가 5보다 크므로 "a는 5보다 큼"이 출력됩니다.

7. `if a < 5:`: `if` 조건문을 사용하여 변수 `a`가 5보다 작은지를 검사합니다. 조건이 거짓이므로 아래의 코드 블록은 실행되지 않습니다.

10. `if a > 20:`: `if` 조건문을 사용하여 변수 `a`가 20보다 큰지를 검사합니다.

11. `print("a는 20보다 큼")`: 조건이 참일 때 실행되는 코드 블록입니다. 변수 `a`가 20보다 크지 않으므로 실행되지 않고, 대신 `else` 블록이 실행됩니다.

13. `else:`: 이 부분은 `if` 조건문이 거짓인 경우 실행되는 코드 블록입니다.

14. `print("a는 20보다 작음")`: `else` 블록에 있는 코드가 실행되므로 "a는 20보다 작음"이 출력됩니다.

16. `if a > 0 and a < 9:`: `if` 조건문을 사용하여 변수 `a`가 0보다 크고 9보다 작은지를 검사합니다. 조건이 참이면 아래의 코드 블록을 실행합니다.

18. `print("a는 0보다 크고 9보다 작음")`: 조건이 참일 때 실행되는 코드 블록입니다. 변수 `a`가 0보다 크고 9보다 작으므로 "a는 0보다 크고 9보다 작음"이 출력됩니다.

19. `elif a >= 10 and a < 20:`: `elif` 조건문을 사용하여 변수 `a`가 10 이상이고 20보다 작은지를 검사합니다. 이전의 조건문이 거짓인 경우에만 실행됩니다.

20. `print("a는 10이상이고 20보다 작음")`: `elif` 블록에 있는 코드가 실행되므로 "a는 10이상이고 20보다 작음"이 출력됩니다.

21. `else:`: 이 부분은 이전의 모든 조건문이 거짓인 경우 실행되는 코드 블록입니다.

22. `print("a는 20보다 큼")`: `else` 블록에 있는 코드가 실행되므로 "a는 20보다 큼"이 출력됩니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

조건문의 결과가 출력되었습니다.

```
쉘
>>> %Run -c $EDITOR_CONTENT

 MPY: soft reboot
 a는 5보다 큼
 a는 20보다 작음
 a는 10이상이고 20보다 작음

>>>
```

1-4-11.반복문

반복문(loop)은 프로그래밍에서 동일한 코드 블록을 여러 번 실행하는 데 사용되는 제어 구조입니다. 반복문은 특정 조건을 만족하는 동안 코드를 반복적으로 실행하거나, 컬렉션(예: 리스트 또는 배열)의 각 항목을 처리하는 데 유용하게 사용됩니다.

주요 반복문 유형은 다음과 같습니다:

1. for 반복문: 주로 컬렉션(예: 리스트, 튜플, 문자열)의 각 항목을 순회하며 코드 블록을 실행합니다. 반복 횟수는 컬렉션의 크기에 의해 결정됩니다.

2. while 반복문: 주어진 조건이 참(True)인 동안 코드 블록을 반복해서 실행합니다. 반복 횟수는 조건이 거짓(False)이 될 때까지 지속됩니다.

main1-4-11.py

```
#for, range
for i in range(5):
    print("hello",end=",")
print("")

#for 리스트 반복
fruits_list = ["banana","apple","orange"]
for fruit in fruits_list:
    print(fruit,end=",")
print("")


#while
i =0
while i <5:
    print(i,end=",")
    i +=1
print("")

#while break
i =0
while True:
    print(i,end=",")
    i +=1
    if i >=5:
        break
print("")
```

코드설명

1. `for, range`: `for` 루프와 `range` 함수를 사용하여 반복 작업을 수행하는 코드입니다.

2. `for i in range(5):`: `range(5)`는 0부터 4까지의 숫자를 생성하는 범위를 나타내며, `for` 루프를 사용하여 이 범위의 숫자를 변수 `i`에 하나씩 대입합니다.

3. `print("hello", end=",")`: "hello"를 출력하고 줄 바꿈 대신 콤마로 끝납니다.

6. `for 리스트 반복`: 리스트 요소를 반복하는 `for` 루프를 사용하는 코드입니다.

7. `fruits_list = ["banana", "apple", "orange"]`: 과일 이름을 포함하는 리스트 `fruits_list`를 생성합니다.

8. `for fruit in fruits_list:`: `fruits_list`의 각 요소를 변수 `fruit`에 대입하여 반복합니다.

9. `print(fruit, end=",")`: 리스트의 각 과일 이름을 출력하고 콤마로 끝납니다.

12. `while`: `while` 루프를 사용하여 반복 작업을 수행하는 코드입니다.

13. `i = 0`: 정수 변수 `i`를 초기화합니다.

14. `while i < 5:`: `i`가 5보다 작은 동안 다음 루프를 계속 반복합니다.

15. `print(i, end=",")`: 변수 `i`의 값을 출력하고 콤마로 끝납니다.

17. `i += 1`: `i`의 값을 1씩 증가시킵니다.

19. `while break`: `while` 루프와 `break` 문을 사용하여 반복 작업을 수행하는 코드입니다.

20. `i = 0`: 정수 변수 `i`를 초기화합니다.

22. `while True:`: 무한 루프를 시작합니다.

23. `print(i, end=",")`: 변수 `i`의 값을 출력하고 콤마로 끝납니다.

24. `i += 1`: `i`의 값을 1씩 증가시킵니다.

25. `if i >= 5:`: `i`가 5 이상인 경우에 `break` 문을 사용하여 루프를 종료합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

반복문의 결과가 출력되었습니다.

쉘

```
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  hello,hello,hello,hello,hello,
  banana,apple,orange,
  0,1,2,3,4,
  0,1,2,3,4,

>>>
```

오류 및 예외처리

1-4-12.오류

다음의 코드를 작성하여 의도적으로 에러를 발생시킵니다.

main1-4-12.py

```
1   a =10
2
3   print(a/0)
```

코드설명

0으로 나누는 연산은 수학적으로 정의되지 않으며, 파이썬에서는 이런 경우를 처리하면 예외(에러)가 발생합니다. 구체적으로 말하면 "ZeroDivisionError"라는 예외가 발생하며 프로그램이 중단됩니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

오류가 발생하며 프로그램이 종료되었습니다.

쉘

```
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  Traceback (most recent call last):
    File "<stdin>", line 3, in <module>
  ZeroDivisionError: divide by zero

>>>
```

1-4-13.예외처리

예외 처리(exception handling)는 프로그램이 실행 중에 예상치 못한 상황 또는 오류가 발생했을 때, 이러한 상황을 처리하고 프로그램이 비정상적으로 종료되는 것을 방지하기 위한 프로그래밍 기법입니다. 예외 처리는 코드를 보다 견고하게 만들어 프로그램이 예기치 않은 오류에도 안정적으로 작동할 수 있도록 도와줍니다.

주요 개념은 다음과 같습니다:

1. 예외(Exception): 예상치 못한 상황이나 오류를 나타내는 객체로, 예외는 프로그램의 실행 중에 발생할 수 있습니다. 예를 들어, 0으로 나누기와 같은 수학적 오류는 예외를 발생시킬 수 있습니다.

2. 예외 처리 구문: 일반적으로 `try`, `except` 블록을 사용하여 예외 처리를 구현합니다.
   - `try` 블록: 예외가 발생할 수 있는 코드를 포함합니다.
   - `except` 블록: `try` 블록에서 예외가 발생하면 실행되는 블록으로, 예외를 처리하거나 로깅할 수 있습니다.

예외 처리는 프로그램 안정성을 향상시키며, 예외가 발생해도 프로그램이 계속 실행되도록 보장합니다. 좋은 예외 처리는 예외를 적절하게 처리하고 사용자에게 명확한 오류 메시지를 제공하여 디버깅을 돕는 역할을 합니다.

main1-4-3.py

```
01    a =10
02
03    #try except
04    try:
05        print(a/0)
06    except:
07        print("에러")
08
09    #에러원인
10    try:
11        print(a/0)
12    except Exception as e:
13        print("에러원인:",e)
```

코드설명

1. `a = 10`: 정수 변수 `a`를 선언하고 값을 10으로 초기화합니다.

4. `try except`: 예외 처리를 위한 `try`와 `except` 블록을 사용하는 코드입니다. 이 블록을 사용하여 예외가 발생할 가능성이 있는 코드를 실행하고, 예외가 발생하면 그에 따른 처리를 수행합니다.

5. `print(a/0)`: 예외가 발생할 가능성이 있는 코드입니다. 0으로 나누는 작업은 예외를 발생시킵니다.

6. `except:`: `try` 블록에서 예외가 발생하면 이 블록의 코드가 실행됩니다. 여기서는 간단히 "에러"를 출력합니다.

10. `try:`: 또 다른 예외 처리 블록을 시작합니다.

11. `print(a/0)`: 예외가 발생할 가능성이 있는 코드입니다. 여전히 0으로 나누는 작업을 시도하고 있습니다.

12. `except Exception as e:`: `try` 블록에서 예외가 발생하면 이 블록의 코드가 실행됩니다. `Exception`은 모든 예외의 기본 클래스이며, `as e`는 발생한 예외를 변수 `e`에 할당하는 것을 의미합니다.

13. `print("에러원인:", e)`: 예외가 발생한 원인을 출력합니다. 이 경우 "에러원인: division by zero"와 같이 0으로 나누려고 했기 때문에 "division by zero"라는 에러 메시지가 출력됩니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

에러가 발생해도 프로그램이 종료하지 않고 에러의 원인을 출력하였습니다.

```
쉘
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  에러
  에러원인: divide by zero

>>>
```

1-4-14. 함수

함수(function)는 프로그램에서 재사용 가능한 코드 블록을 나타내며, 특정 작업을 수행하거나 값을 반환하는 데 사용됩니다. 함수는 코드를 모듈화하고, 가독성을 높이며, 유지보수를 쉽게 만드는 데 도움이 됩니다.

주요 개념은 다음과 같습니다:

1. 함수 정의: 함수를 정의하려면 함수 이름, 매개변수(parameter), 함수 본문을 포함하는 코드 블록이 필요합니다. 함수 이름은 호출할 때 사용되며, 매개변수는 함수에 전달되는 값의 이름이나 변수입니다.

2. 함수 호출: 함수를 호출하려면 함수 이름과 필요한 매개변수 값을 전달합니다. 함수는 호출될 때 정의된 코드 블록을 실행하고 결과를 반환할 수 있습니다.

3. 매개변수: 함수가 입력으로 받는 값을 나타냅니다. 매개변수는 함수 정의에서 선언하며, 함수가 호출될 때 실제 값이 전달됩니다.

4. 반환값: 함수가 작업을 수행한 후 결과 값을 반환할 수 있습니다. 반환값은 함수 호출자에게 전달되며, 함수 정의에서 `return` 키워드를 사용하여 지정됩니다.

함수는 프로그램의 모듈성과 재사용성을 높이는 데 중요한 역할을 합니다. 코드를 함수로 나누고 필요할 때 호출하여 사용하면 코드의 가독성이 향상되고 유지보수가 용이해집니다.

main1-4-14.py

```
01  #함수1
02  def func_1():
03      print("hello")
04
05  func_1()
06
07  #함수2
08  def func_2(name):
09      print("이름:",name)
10
11  func_2("dain")
12
13  #함수3
14  def func_3():
15      a =10
16      b =20
17      return a + b
18
19  c = func_3()
20  print(c)
21
22  #함수4
23  def func_4(a,b):
24      return a * b
25
26  d = func_4(10,4)
27  print(d)
28
29  #함수5
30  def func_5(a,b):
31      return a + b, a * b
32
33  e,f = func_5(10,4)
34  print(e,f)
```

코드설명

1. `#함수1`: 함수를 정의하고 호출하는 예제입니다.

2. `def func_1():`: `func_1`이라는 이름의 함수를 정의합니다.

3. `print("hello")`: 함수 `func_1` 내에서 "hello"를 출력하는 코드입니다.

5. `func_1()`: `func_1` 함수를 호출합니다. 이로 인해 "hello"가 출력됩니다.

7. `#함수2`: 매개변수를 가지고 있는 함수를 정의하고 호출하는 예제입니다.

8. `def func_2(name):`: `name`이라는 매개변수를 받는 `func_2` 함수를 정의합니다.

9. `print("이름:", name)`: `name` 매개변수의 값을 출력하는 코드입니다.

11. `func_2("dain")`: `func_2` 함수를 호출하고 "dain"을 인자로 전달합니다. 함수 내에서는 "이름: dain"이 출력됩니다.

13. `#함수3`: 함수 내에서 값을 계산하고 반환하는 예제입니다.

14. `def func_3():`: 매개변수 없이 `func_3` 함수를 정의합니다.

15. `a = 10`: 변수 `a`에 10을 할당합니다.

16. `b = 20`: 변수 `b`에 20을 할당합니다.

17. `return a + b`: 변수 `a`와 `b`의 합계를 반환합니다.

19. `c = func_3()`: `func_3` 함수를 호출하고 그 결과를 변수 `c`에 저장합니다. 함수 내에서 변수 `a`와 `b`를 더한 결과인 30이 반환됩니다.

20. `print(c)`: 변수 `c`의 값을 출력합니다. 이로 인해 30이 출력됩니다.

22. `#함수4`: 매개변수를 받아서 값을 반환하는 예제입니다.

23. `def func_4(a, b):`: `a`와 `b`라는 두 개의 매개변수를 받는 `func_4` 함수를 정의합니다.

24. `return a * b`: 매개변수 `a`와 `b`를 곱한 결과를 반환합니다.

26. `d = func_4(10, 4)`: `func_4` 함수를 호출하고 10과 4를 인자로 전달합니다. 함수 내에서는 10과 4를 곱한 결과인 40이 반환됩니다.

27. `print(d)`: 변수 `d`의 값을 출력합니다. 이로 인해 40이 출력됩니다.

29. `#함수5`: 여러 개의 값을 반환하는 함수를 정의하고 호출하는 예제입니다.

30. `def func_5(a, b):`: `a`와 `b`라는 두 개의 매개변수를 받는 `func_5` 함수를 정의합니다.

31. `return a + b, a * b`: 매개변수 `a`와 `b`를 더한 결과와 매개변수 `a`와 `b`를 곱한 결과를 튜플로 반환합니다.

33. `e, f = func_5(10, 4)`: `func_5` 함수를 호출하고 10과 4를 인자로 전달합니다. 함수 내에서는 두 결과를 튜플로 묶어 반환하므로 `e`에는 덧셈 결과인 14가, `f`에는 곱셈 결과인 40이 할당됩니다.

34. `print(e, f)`: 변수 `e`와 `f`의 값을 출력합니다. 이로 인해 "14 40"이 출력됩니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

함수를 생성하고 호출하였습니다.

```
쉘
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  hello
  이름: dain
  30
  40
  14 40

>>>
```

1-4-15.클래스

클래스(class)는 객체지향 프로그래밍(OOP)에서 중요한 개념으로, 비슷한 특징을 가진 객체들을 추상화하고 그룹화하기 위한 틀 또는 설계도 역할을 합니다. 클래스는 객체의 속성(attribute)과 동작(method)을 정의하며, 이러한 클래스를 사용하여 여러 개의 객체를 생성할 수 있습니다.

주요 개념은 다음과 같습니다:

1. 클래스 정의: 클래스를 정의하려면 클래스의 이름과 클래스의 속성(변수)과 동작(메서드)을 정의합니다. 속성은 클래스의 특징을 나타내는 데이터를 저장하고, 메서드는 클래스가 수행하는 동작을 정의합니다.

2. 객체(Object): 클래스를 기반으로 생성된 실제 인스턴스를 객체라고 합니다. 클래스의 정의를 토대로 여러 객체를 생성할 수 있으며, 각 객체는 고유한 상태를 가질 수 있습니다.

클래스는 객체지향 프로그래밍에서 코드의 구조와 재사용성을 향상시키는 데 사용되며, 현실 세계의 개념을 프로그램으로 모델링하는 데 유용합니다. 클래스는 속성과 메서드를 포함하여 객체의 특성과 동작을 정의하고, 이를 통해 프로그램을 더 모듈화하고 관리하기 쉽게 만듭니다.

main1-4-15.py

```
01  class Human1():
02      def set_name_like(self, age, like):
03          self.age = age
04          self.like = like
05
06      def info(self):
07          print(f"나이:{self.age}  좋아하는것:{self.like}")
08
09  jang = Human1()
10  lee = Human1()
11
12  jang.set_name_like(15,"apple")
13  jang.info()
14
15  lee.set_name_like(10,"banana")
16  lee.info()
17
18
19  class Human2():
20      def __init__(self, age, like):
21          self.age = age
22          self.like = like
23
24      def info(self):
25          print(f"나이:{self.age}  좋아하는것:{self.like}")
26
27
28  dain = Human2(7,"시나모롤")
29  kim = Human2(7,"축구")
30
31  dain.info()
32  kim.info()
```

코드설명

1. `class Human1():`: `Human1` 클래스를 정의합니다.

2. `def set_name_like(self, age, like):`: `set_name_like` 메서드를 정의합니다. 이 메서드는 `self`, `age`, `like` 매개변수를 받습니다.

3. `self.age = age`: 객체의 `age` 속성을 입력된 `age` 값으로 설정합니다.

4. `self.like = like`: 객체의 `like` 속성을 입력된 `like` 값으로 설정합니다.

6. `def info(self):`: `info` 메서드를 정의합니다. 이 메서드는 객체의 정보를 출력하는 역할을 합니다.

7. `print(f"나이:{self.age} 좋아하는것:{self.like}")`: 객체의 `age`와 `like` 속성을 출력합니다.

9-10. `jang`과 `lee`라는 두 개의 `Human1` 클래스의 객체를 생성합니다.

12-13. `jang` 객체의 `set_name_like` 메서드를 호출하여 나이를 15로, 좋아하는 것을 "apple"로 설정한 후, `info` 메서드를 호출하여 정보를 출력합니다.

15-16. `lee` 객체의 `set_name_like` 메서드를 호출하여 나이를 10으로, 좋아하는 것을 "banana"로 설정한 후, `info` 메서드를 호출하여 정보를 출력합니다.

Human2 클래스

19. `class Human2():`: `Human2` 클래스를 정의합니다.

20. `def __init__(self, age, like):`: `__init__` 메서드를 정의합니다. 이 메서드는 객체를 초기화하는 역할을 합니다. `self`, `age`, `like` 매개변수를 받습니다.

21. `self.age = age`: 객체의 `age` 속성을 입력된 `age` 값으로 설정합니다.

22. `self.like = like`: 객체의 `like` 속성을 입력된 `like` 값으로 설정합니다.

24-25. `def info(self):`: `info` 메서드를 정의합니다. 이 메서드는 객체의 정보를 출력하는 역할을 합니다.

26. `print(f"나이:{self.age} 좋아하는것:{self.like}")`: 객체의 `age`와 `like` 속성을 출력합니다.

28-29. `dain`과 `kim`라는 두 개의 `Human2` 클래스의 객체를 생성하면서 각각의 나이와 좋아하는 것을 초기화합니다.

31-32. `dain`과 `kim` 객체의 `info` 메서드를 호출하여 정보를 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

클래스를 생성하고 객체를 생성하여 활용하였습니다.

쉘

```
>>> %Run -c $EDITOR_CONTENT

 MPY: soft reboot
 나이:15  좋아하는것:apple
 나이:10  좋아하는것:banana
 나이:7  좋아하는것:시나모롤
 나이:7  좋아하는것:축구

>>>
```

1-4-16. import

`import`는 파이썬에서 외부 모듈 또는 패키지에서 코드를 가져와 사용하는 데 사용되는 키워드입니다. 파이썬은 기본 라이브러리와 외부 라이브러리를 모듈 형태로 제공하며, 이러한 모듈을 `import`를 사용하여 현재 스크립트 또는 프로젝트에 가져올 수 있습니다.

간단히 말하면, `import`를 사용하면 다음과 같은 일을 할 수 있습니다:

1. 외부 모듈 또는 패키지의 함수, 클래스 또는 변수를 현재 스크립트에서 사용할 수 있도록 가져옵니다.
2. 코드의 재사용을 촉진하며, 기능을 확장하거나 개선하는 데 도움이 됩니다.
3. 다른 프로그래머가 작성한 코드를 활용할 수 있습니다.

`import`를 사용하면 파이썬 생태계에서 다양한 기능과 라이브러리를 활용하여 프로그램을 개발할 수 있습니다.

main1-4-16.py

```
#import1
import random

print(random.randint(10, 50))

#import2
from random import randint

print(randint(10, 50))

#import3
from random import randint,randrange

print(randint(10, 50))
print(randrange(50))

#import4
import random as rd

print(rd.randint(10, 50))
```

코드설명

1. `import random`: 모듈 `random`을 가져옵니다.

4. `print(random.randint(10, 50))`: `random` 모듈에서 `randint` 함수를 사용하여 10과 50 사이의 난수를 생성하고 출력합니다.

import2

6. `from random import randint`: `random` 모듈에서 `randint` 함수만 가져옵니다.

9. `print(randint(10, 50))`: `randint` 함수를 직접 사용하여 10과 50 사이의 난수를 생성하고 출력합니다.

import3

12. `from random import randint, randrange`: `random` 모듈에서 `randint`와 `randrange` 함수를 가져옵니다.

14. `print(randint(10, 50))`: `randint` 함수를 사용하여 10과 50 사이의 난수를 생성하고 출력합니다.

15. `print(randrange(50))`: `randrange` 함수를 사용하여 0부터 49 사이의 난수를 생성하고 출력합니다.

import4

17. `import random as rd`: 모듈 `random`을 가져오되, 모듈의 별명을 `rd`로 지정합니다.

20. `print(rd.randint(10, 50))`: `rd`라는 별명으로 `random` 모듈의 `randint` 함수를 사용하여 10과 50 사이의 난수를 생성하고 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

import를 사용하여 다른사람이 만든 라이브러리를 불러와 사용하였습니다.

쉘

```
>>> %Run -c $EDITOR_CONTENT

 MPY: soft reboot
 20
 26
 30
 24
 10

>>>
```

CHAPTER 02

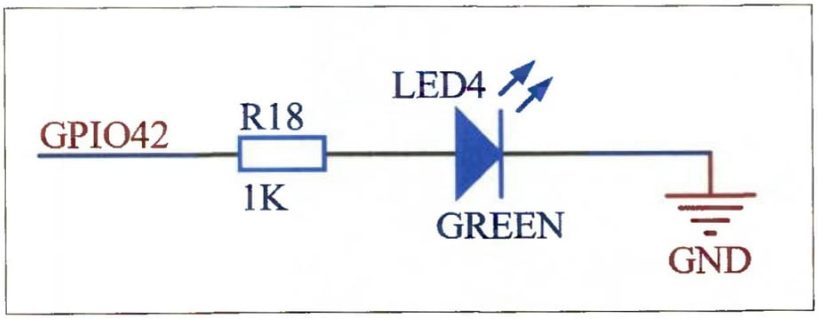
드론의 기본기능

LED, 버튼, 모터, 전압측정등 드론의 기본기능을 마이크로파이썬을 이용하여 다루어 보도록 합니다.

2-1

LED제어하기

드론에는 4개의 LED가 있지만 실제로 제어가능한 LED는 42번핀에 연결된 [M4 LED]로 녹색 LED입니다. 다른 LED는 내부적으로 전원의 상태, 충전여부, 동작여부로 내부적으로 사용중에 있습니다. 마이크로파이썬 코드를 이용하여 42번핀에 연결된 [M4 LED]를 제어하는 방법에 대해서 알아봅니다.



녹색 LED제어하기

42번핀에 연결된 파란색 LED를 깜빡이는 코드를 만들어봅니다.

2-1.LED제어하기_1.py

```
01    from machine import Pin
02    import time
03
04    green_led = Pin(42,Pin.OUT)
05
06    while True:
07        green_led.value(1)
08        time.sleep(2.0)
09        green_led.value(0)
10        time.sleep(2.0)
```

코드설명

01~02: 라이브러리를 불러옵니다. `machine` 라이브러리에서 `Pin` 클래스를 사용하기 위함이고, `time` 라이브러리는 시간 관련 기능을 사용하기 위해 불러옵니다.

04: 42번 핀을 출력 모드로 설정합니다. `green_led`라는 변수에 이 핀 객체를 저장하여 LED 제어에 사용합니다.

06: 무한 루프를 시작합니다. 이 루프 안에서 LED의 상태를 반복적으로 변경합니다.

07: `green_led`의 값을 1로 설정하여 LED를 켭니다.

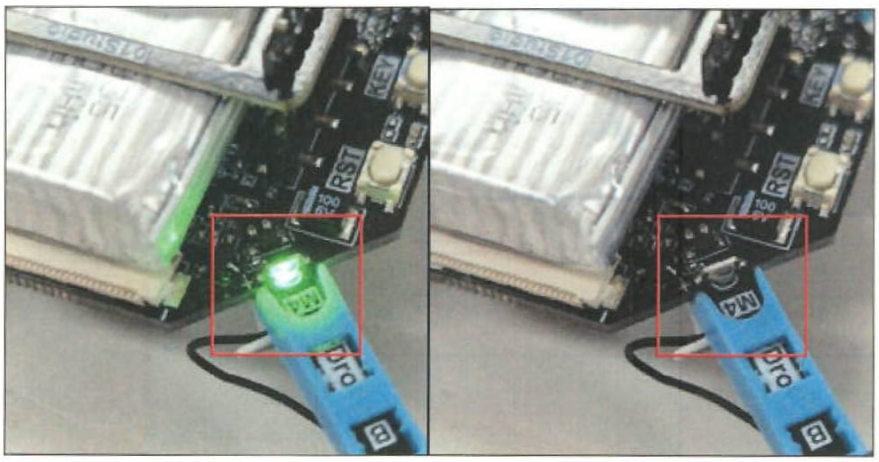
08: 프로그램을 2초간 일시 정지합니다. 이 동안 LED는 켜진 상태를 유지합니다.

09: `green_led`의 값을 0으로 설정하여 LED를 끕니다.

10: 프로그램을 다시 2초간 일시 정지합니다. 이 동안 LED는 꺼진 상태를 유지합니다. 이러한 과정이 반복되면서 LED가 2초 간격으로 깜빡이게 됩니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

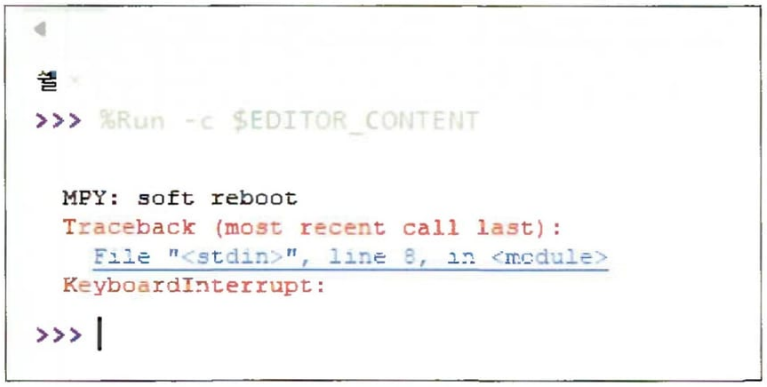
M4모터쪽에 위치한 녹색 LED가 1초마다 켜졌다 꺼졌다를 반복하며 깜빡입니다.



코드를 종료하는 방법으로는 두가지 방법이 있습니다.

첫번째. [STOP] STOP아이콘을 클릭하여 강제로 종료합니다.

두번째. 쉘 영역에서 [컨트롤 + C]를 눌러 키보드인터럽트를 발생시켜 종료합니다.



LED가 켜져있을 때 코드를 종료하면 LED가 켜진상태로 계속 유지됩니다. 코드를 종료하면 LED를 끄는 방법에 대해서 알아봅니다.

코드를 안전하게 종료하기 위해서는 두 번째방법인 키보드인터럽트가 발생하면 사용하고 있는 자원을 종료한다음에 코드를 종료합니다. LED를 예를 들면 LED를 끄고 코드를 안전하게 종료합니다. 다음은 키보드 인터럽트가 발생시켜 LED를 끄고 종료하는 방법에 대해서 알아봅니다.

안전하게 코드 종료하기

키보드인터럽트가 발생하면 LED를 끄고 코드를 종료하는 코드를 작성해봅니다.

2-1.LED제어하기_2.py

```
01    from machine import Pin
02    import time
03
04    green_led = Pin(42,Pin.OUT)
05
06    try:
07        while True:
08            green_led.value(0)
09            time.sleep(1.0)
10            green_led.value(1)
11            time.sleep(1.0)
12
13    except KeyboardInterrupt:
14        green_led.value(0)
15        print("코드를 종료합니다.")
```

코드설명

06: 예외 처리 구문의 시작으로, 프로그램 실행 중에 특정 예외가 발생할 경우 대응하기 위한 구조를 설정합니다.

13: KeyboardInterrupt 예외를 처리합니다. 이는 주로 사용자가 프로그램 실행을 중단시킬 때 발생하는 예외입니다 (예를 들어 Ctrl+C를 누를 때).

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

코드를 종료하기 위해서 쉘 영역에서 [컨트롤 + C]를 눌러 키보드인터럽트를 발생시켜 종료합니다.

```
쉘 ×

>>> %Run -c $EDITOR_CONTENT

 MPY: soft reboot
 코드를 종료합니다.
```

on(), off()메서드 사용해서 제어하기

on(), off()메서드를 사용해서 LED를 제어하는 방법에 대해서 알아봅니다.

2-1.LED제어하기_3.py

```
01  from machine import Pin
02  import time
03
04  green_led = Pin(42,Pin.OUT)
05
06  try:
07      while True:
08          green_led.off()
09          time.sleep(1.0)
10          green_led.on()
11          time.sleep(1.0)
12
13  except KeyboardInterrupt:
14      green_led.off()
15      print("코드를 종료합니다.")
```

코드설명

08: green_led.off() 메소드를 호출하여 LED를 끕니다.

10: green_led.on() 메소드를 호출하여 LED를 켭니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

on(), off()메서드를 사용해서 value(0), value(1)과 동일하게 LED를 켜고 끌 수 있습니다.

2-2

버튼입력받기

드론에 장착된 버튼값을 입력받아봅니다.

버튼 상태확인하기

버튼의 상태를 확인하는 코드를 작성합니다.

2-2.버튼입력받기_1.py

```
01    from machine import Pin
02    import time
03
04    key = Pin(0, Pin.IN, Pin.PULL_UP)
05
06    try:
07        while True:
08            key_value = key.value()
09            print(key_value)
10            time.sleep(0.1)
11
12    except KeyboardInterrupt:
13        print("코드를 종료합니다.")
```

코드설명

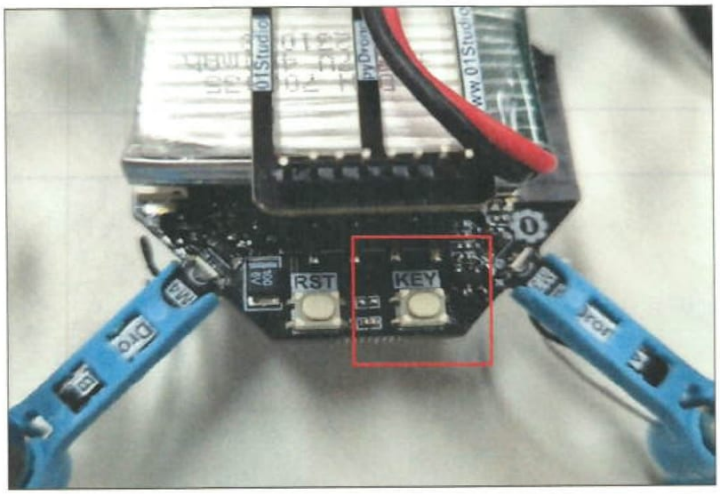
04: 0번 핀을 입력 모드로 설정하고, 내부 풀업 저항을 활성화합니다. 이 설정은 key라는 변수에 저장되어, 버튼 입력을 읽는 데 사용됩니다.

08: key.value()를 호출하여 현재 버튼의 상태를 읽고, 이 값을 key_value 변수에 저장합니다.

09: 읽은 버튼의 값(key_value)를 콘솔에 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

드론의 KEY 버튼을 눌러 값을 확인합니다.



버튼을 누르지 않을때는 0, 버튼을 누를때는 1이 출력되었습니다. 내부 풀업저항으로 인해 누르지 않을때는 1, 눌렀을 때 0이 출력됩니다. 누르지 않을때는 풀업저항이 VCC와 연결되어 있어 1이 출력되고 버튼을 누르면 GND와 연결되어 0이 출력됩니다.

```
셸
-
1
1
1
0
0
0
0
```

버튼의 상태가 변경될때만 값 출력하기

값이 계속출력되는게 아닌 버튼을 눌렀을때만 즉 버튼의 상태가 변경될때만 값이 출력되도록 코드를 작성해봅니다.

2-2.버튼입력받기_2.py

```
01    from machine import Pin
02    import time
03
04    key = Pin(0, Pin.IN, Pin.PULL_UP)
05    prev_key = key.value()
06
07    try:
08        while True:
09            curr_key = key.value()
10            if curr_key != prev_key:
11                prev_key = curr_key
12                print("버튼 눌림",curr_key)
13                time.sleep(0.1)
14
15    except KeyboardInterrupt:
16        print("코드를 종료합니다.")
```

코드설명

05: `prev_key` 변수를 초기화하여 처음에 `key`의 현재 값을 저장합니다. 이는 버튼의 상태 변경을 감지하기 위한 기준점으로 사용됩니다.

09: `key.value()`를 호출하여 현재 버튼의 상태를 읽고, 이 값을 `curr_key` 변수에 저장합니다.

10: 현재 버튼의 값(`curr_key`)이 이전 값(`prev_key`)과 다른지 검사합니다. 이는 버튼의 상태가 변화했는지 확인하기 위함입니다.

11: 상태 변화가 있을 경우, `prev_key`를 현재 값으로 업데이트합니다.

12: 버튼의 상태 변화가 감지되면 "버튼 눌림"과 현재 버튼의 값(`curr_key`)을 콘솔에 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

버튼이 눌렸을때와 0의 값일때와, 버튼이 눌렀다 떼었을 때 1의 값일때만 값이 출력되었습니다. 버튼의 상태가 변경될 때 값이 출력되었습니다.

```
쉘
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  버튼 눌림 0
  버튼 눌림 1
  버튼 눌림 0
  버튼 눌림 1
```

버튼눌릴때만 값 출력하기

버튼이 눌렸을때만 값을 출력하도록 코드를 작성해봅니다.

2-2.버튼입력받기_3.py

```
01  from machine import Pin
02  import time
03
04  key = Pin(0, Pin.IN, Pin.PULL_UP)
05  prev_key = key.value()
06
07  try:
08      while True:
09          curr_key = key.value()
10          if curr_key != prev_key:
11              prev_key = curr_key
12              if curr_key ==0:
13                  print("버튼 눌림",curr_key)
14              time.sleep(0.1)
15
16  except KeyboardInterrupt:
17      print("코드를 종료합니다.")
```

코드설명

12: 만약 curr_key가 0이면, 즉 버튼이 눌린 상태라면,

13: "버튼 눌림"과 현재 버튼의 값(curr_key)을 콘솔에 출력합니다. 이는 버튼이 눌려 있는 상태를 사용자에게 알리는 것입니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

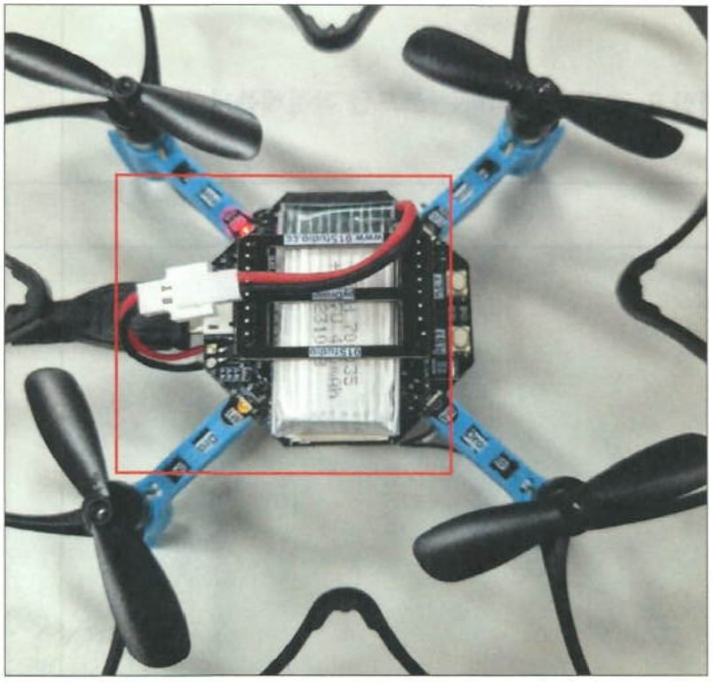
KEY 버튼이 눌렸을때만 값이 출력되었습니다. 조건문을 추가하여 버튼이 값이 0일때만 동작하여 눌렸을때만 출력됩니다. 눌렀다가 떼었을때는 값이 1이 되므로 값이 출력되지 않습니다.

```
쉘 ×

>>> %Run -c $EDITOR_CONTENT


  MPY: soft reboot
  버튼 눌림 0
  버튼 눌림 0
  버튼 눌림 0
  버튼 눌림 0
```

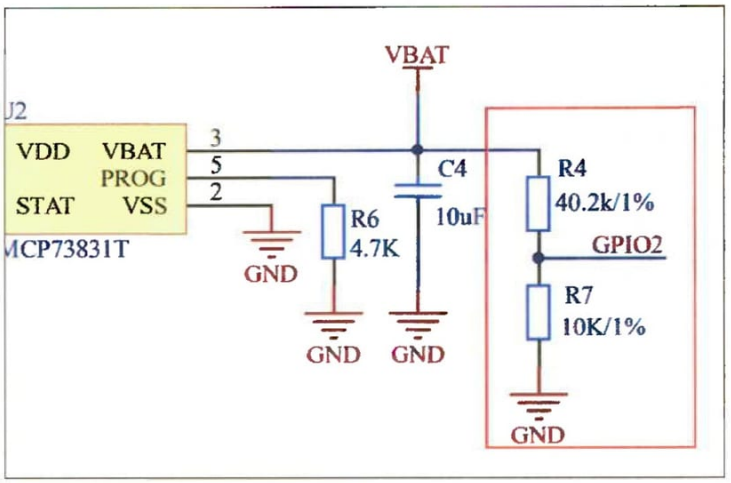
2-3

배터리 전압 측정하기

배터리의 전압을 측정하기 위해서는 배터리를 드론과 연결한다음 진행합니다.



배터리의 전압을 측정하기 위한 회로입니다. ESP32-S3의 ADC측정전압은 0~1.1V로 배터리의 최대전압인 약 4.3V의 전압을 측정하기 위해서 저항을 이용하여 전압분배 회로를 구성하였습니다. 들어오는 전압을 약 1/5로 줄여서 ADC에 입력하여 측정합니다.



ADC측정하기

배터리의 전압을 ADC를 이용하여 측정하고 그 값을 출력하는 코드를 작성해봅니다.

2-3.배터리전압측정하기_1.py

```
01    from machine import Pin,ADC
02    import time
03
04    adc = ADC(Pin(2))
05
06    try:
07        while True:
08            adc_value = adc.read()
09
10            print(adc_value)
11
12            time.sleep(1.0)
13
14    except KeyboardInterrupt:
15        print("코드를 종료합니다.")
```

코드설명

01~02: 라이브러리를 불러옵니다. `machine`에서 `Pin` 클래스와 `ADC` (Analog to Digital Converter) 모듈은 핀을 제어하고 아날로그 신호를 디지털 값으로 변환하는 데 사용되며, `time` 라이브러리는 프로그램 실행 중 일시 정지를 구현하기 위해 필요합니다.

04: 2번 핀을 아날로그 입력으로 설정하고, 이를 `adc`라는 변수에 ADC 객체로 초기화하여 저장합니다. 이 변수는 아날로그 신호를 읽는 데 사용됩니다.

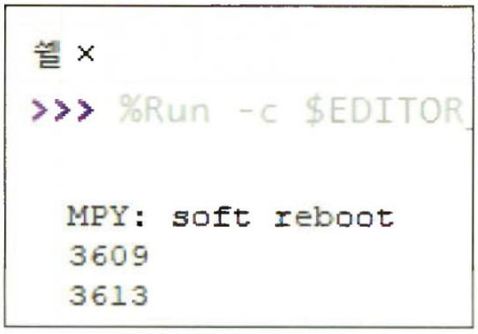
08: `adc.read()`를 호출하여 현재 연결된 아날로그 핀에서 신호를 읽고, 이 값을 `adc_value` 변수에 저장합니다.

10: 읽은 아날로그 값(`adc_value`)를 콘솔에 출력합니다. 이는 센서나 다른 아날로그 입력 장치로부터의 실시간 데이터를 모니터링하는 데 유용합니다.

12: 프로그램을 1.0초 동안 일시 정지합니다. 이는 아날로그 값을 너무 자주 읽는 것을 방지하고 데이터의 변화를 쉽게 관찰할 수 있도록 합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

ADC로 값을 측정하여 0~4095 범위의 디지털값으로 출력되었습니다.



배터리전압으로 환산하기

디지털값을 전압으로 환산하여 배터리의 전압을 측정해보도록 합니다.

2-3.배터리전압측정하기_2.py

```
01  from machine import Pin,ADC
02  import time
03
04  adc = ADC(Pin(2))
05
06  try:
07      while True:
08          voltage = adc.read()/4095*5.02*0.96
```

```
09
10            print(voltage)
11
12            time.sleep(1.0)
13
14    except KeyboardInterrupt:
15        print("코드를 종료합니다.")
```

코드설명

08: adc.read() 함수를 호출하여 아날로그 핀에서 신호를 읽고, 이 값을 4095로 나누어 상대적인 아날로그 값으로 변환한 후, 실제 전압 값으로 환산하기 위해 5.02와 0.96을 곱합니다. 이는 ADC 값이 0에서 4095 사이의 범위를 가지고 최대 5.02V (계산 상의 최대 전압)를 나타내며, 실제 회로의 특성을 반영하여 0.96을 곱하는 보정 과정입니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

디지털값을 전압으로 환산하여 출력하였습니다.

```
쉘 ×

>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  4.118974
  4.259019
  4.227245
```

배터리의 잔량을 %로 표시하기

배터리의 잔량을 %로 표시해도록 합니다. 우리가 사용하여 리튬폴리머 배터리는 완충시 4.25V가량 완전방전시 3.0V가량으로 4.25V와 3.0V사이의 값이 0~100%로 환산하여봅니다.

2-3.배터리전압측정하기_3.py

```
01  from machine import Pin,ADC
02  import time
03
04  adc = ADC(Pin(2))
05
06  try:
07      while True:
08          voltage = adc.read()/4095*5.02*0.96
09          percent = (voltage -3.0) / (4.25 -3.0) *100
10          print(percent)
11          time.sleep(1.0)
12
13  except KeyboardInterrupt:
14      print("코드를 종료합니다.")
```

코드설명

09: 계산된 전압 값(voltage)을 사용하여 특정 범위 내에서의 백분율을 계산합니다. 이 예에서는 전압이 3.0V에서 4.25V 사이의 범위에서 어디에 위치하는지를 백분율로 표현합니다. 이는 특정 센서의 출력 전압을 이용해 센서의 상태를 백분율로 나타내기 위한 계산입니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

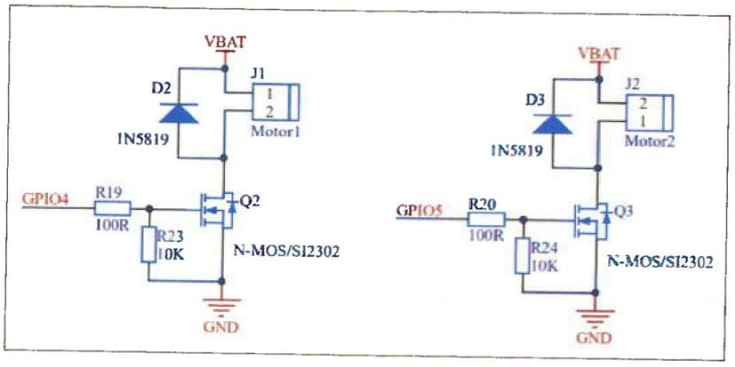
배터리의 전압이 %로 환산되어 출력되었습니다.

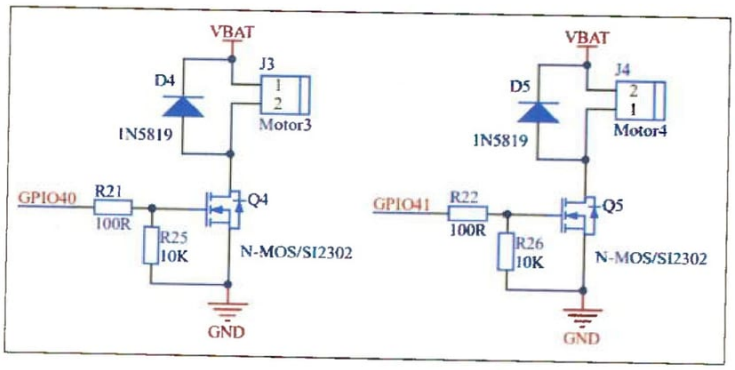
```
쉘

>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  86.12862
  96.9556
  98.46199
  95.7317
  89.42379
  88.10574
```

2-4

모터제어하기

드론에는 총 4개의 모터드라이버로 구성되어있습니다. M1,M2,M3,M4의 4개의 모터가 있고 각각 GPIO4, GPIO5, GPIO40, GPOIO41번핀에 연결되어 있습니다. M2,M4모터는 반시계방향으로 회전하고 M1,M3모터는 시계방향으로 회전합니다. FET를 이용한 드라이버는 모터의 방향제어는 불가능하고 속도제어만 가능합니다. 모터에 연결되는 커넥터의 극성을 반대로하여 모터의 방향을 변경하였습니다.





한 개의 모터 구동하기

M1모터를 구동하는 코드를 작성해봅니다.

2-4.모터제어하기_1.py

```
01  from machine import Pin, PWM
02  import time
03
04  M1_motor = PWM(Pin(4), freq =10000 ,duty=0)
05
06  try:
07      while True:
08          M1_motor.duty(0)
09          print("0")
10          time.sleep(2)
11
12          M1_motor.duty(100)
13          print("100")
14          time.sleep(1)
15
16          M1_motor.duty(200)
17          print("200")
18          time.sleep(2)
19
20  except KeyboardInterrupt:
21      M1_motor.deinit()
22      print("코드를 종료합니다.")
```

코드설명

01~02: 라이브러리를 불러옵니다. `machine` 모듈에서 `Pin`과 `PWM` (Pulse Width Modulation) 클래스는 핀을 제어하고 PWM 신호를 생성하는 데 사용되며, `time` 라이브러리는 프로그램 실행 중 일시 정지를 구현하기 위해 필요합니다.

04: 4번 핀에 PWM 신호를 설정하고, `M1_motor`라는 변수에 PWM 객체로 초기화하여 저장합니다. 이 변수는 모터의 속도를 제어하는 데 사용됩니다. `freq`는 PWM 신호의 주파수를 10000 Hz로 설정하며, `duty`는 초기 듀티 사이클을 0%로 설정하여 모터를 멈춘 상태로 초기화합니다.

08: `M1_motor.duty(0)`를 호출하여 PWM 듀티 사이클을 0%로 설정합니다. 이는 모터를 정지시키는 설정입니다.

09: "0"을 콘솔에 출력하여 현재 모터의 듀티 사이클이 0%임을 나타냅니다.

10: 프로그램을 2초 동안 일시 정지합니다. 이 기간 동안 모터는 멈춘 상태를 유지합니다.

12: `M1_motor.duty(100)`을 호출하여 PWM 듀티 사이클을 약 10%로 설정합니다. 이는 모터의 속도를 늘리는 설정입니다.

13: "100"을 콘솔에 출력하여 현재 모터의 듀티 사이클이 약 10%임을 나타냅니다.

14: 프로그램을 1초 동안 일시 정지합니다. 이 기간 동안 모터는 증가된 속도로 동작합니다.

16: `M1_motor.duty(200)`을 호출하여 PWM 듀티 사이클을 약 20%로 설정합니다. 이는 모터의 속도를 더욱 늘리는 설정입니다.

17: "200"을 콘솔에 출력하여 현재 모터의 듀티 사이클이 약 20%임을 나타냅니다.

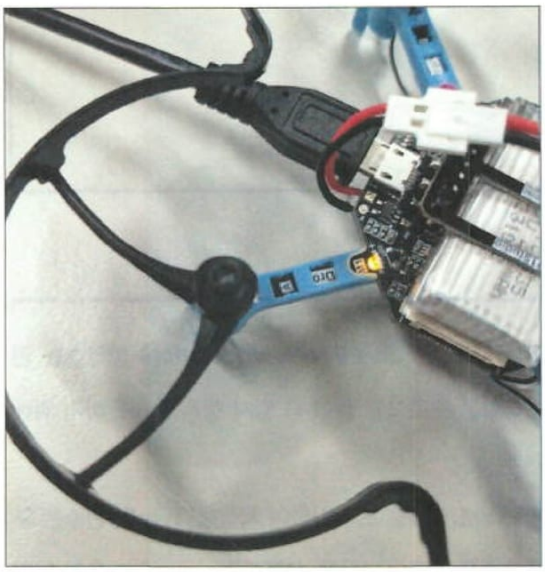
18: 프로그램을 2초 동안 일시 정지합니다. 이 기간 동안 모터는 더 높은 속도로 동작합니다.

21: `M1_motor.deinit()`을 호출하여 PWM 객체를 해제하고, 연결된 모터의 제어를 중지합니다. 이는 자원을 안전하게 해제하고 하드웨어에 대한 제어를 정상적으로 종료합니다.

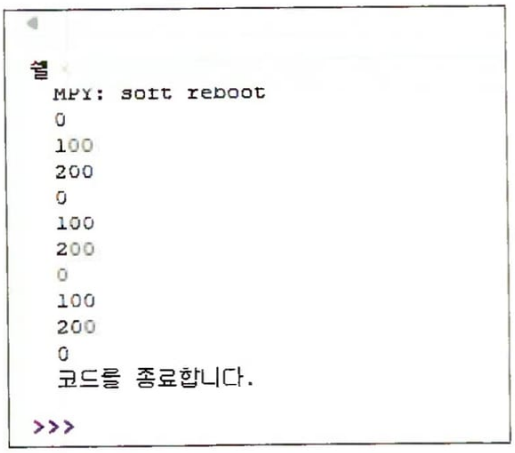
22: "코드를 종료합니다."라는 메시지를 출력하여 사용자에게 프로그램이 종료되었음을 알립니다. 이 메시지는 사용자에게 명확한 피드백을 제공하며, 프로그램이 정상적으로 종료 처리를 완료했음을 알립니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

M1 모터가 0,100,200의 PWM출력만큼 회전합니다. 0일때는 돌지 않고 100보다 200일 때 더 빨리 회전합니다. PWM신호는 0~1023까지 제어가 가능하나 200보다 큰값은 너무빨리 돌아 위험하기 때문에 코드에 적용하지 않았습니다.



코드를 종료할때는 쉘 영역에서 [컨트롤 + C]를 입력하여 키보드 인터럽트를 발생시켜 안전하게 종료합니다.



4개의 모터 구동하기

4개의 모터를 모두 구동하는 코드를 작성합니다. 모터의 회전방향을 확인해봅니다.

2-4.모터제어하기_2.py

```
01  from machine import Pin, PWM
02  import time
03
04  M1_motor = PWM(Pin(4), freq =10000 ,duty=0)
05  M2_motor = PWM(Pin(5), freq =10000 ,duty=0)
06  M3_motor = PWM(Pin(40), freq =10000 ,duty=0)
07  M4_motor = PWM(Pin(41), freq =10000 ,duty=0)
08
09  try:
10      while True:
11          M1_motor.duty(0)
12          M2_motor.duty(0)
13          M3_motor.duty(0)
14          M4_motor.duty(0)
15          print("0")
16          time.sleep(3)
17
18          M1_motor.duty(100)
19          M2_motor.duty(100)
20          M3_motor.duty(100)
21          M4_motor.duty(100)
22          print("100")
23          time.sleep(1)
24
25
26  except KeyboardInterrupt:
27      M1_motor.deinit()
28      M2_motor.deinit()
29      M3_motor.deinit()
30      M4_motor.deinit()
31      print("코드를 종료합니다.")
```

코드설명

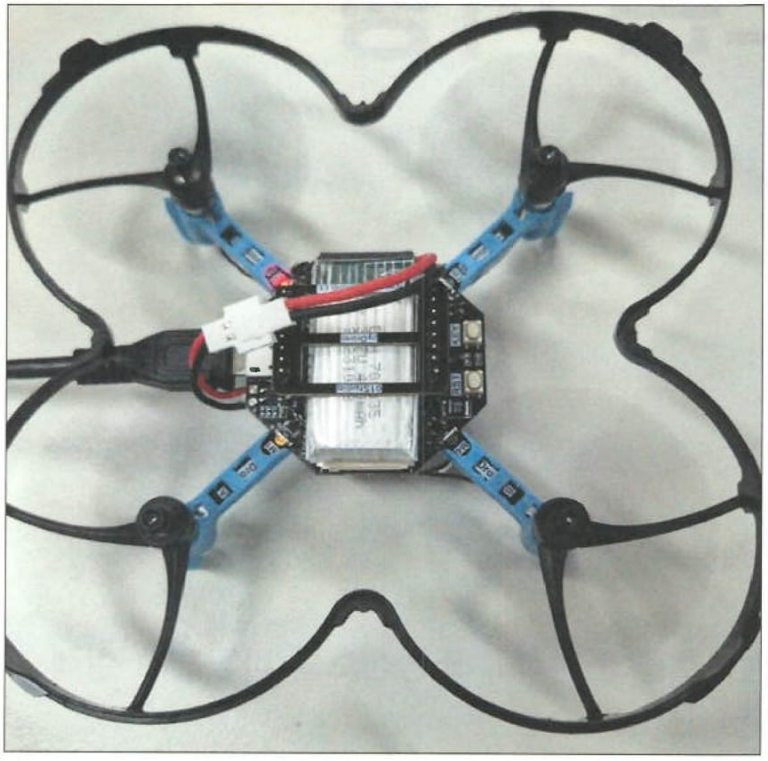
04~07: 4개의 모터(M1, M2, M3, M4)를 각각 다른 핀(4, 5, 40, 41번)에 PWM 신호로 연결하고 초기화합니다. 모든 모터의 PWM 주파수는 10000 Hz로 설정되고, 초기 듀티 사이클은 0%로 설정하여 모터를 멈춘 상태로 시작합니다.

11~14: 모든 모터(M1, M2, M3, M4)의 듀티 사이클을 0%로 설정합니다. 이는 모든 모터를 정지시키는 설정입니다.

18~21: 모든 모터(M1, M2, M3, M4)의 듀티 사이클을 약 10%로 설정합니다. 이는 모터들의 속도를 늘리는 설정입니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

모터4개가 모두 동작합니다. 모터가 멈출때쯤 회전방향을 확인합니다.



CHAPTER 03

드론 비행

드론비행에 필요한 조종신호를 분석하고 조종신호를 바탕으로 실제 드론을 비행해보도록 합니다.

3-1

조종데이터 분석하기

블루투스 통신으로 수신받은 드론 조종신호를 분석하고 조종데이터만 분리하는 방법에 대해서 알아봅니다.

드론 조종데이터 수신받기

스마트폰과 블루투스로 연결하여 드론 조종데이터를 수신받는 코드를 작성합니다.

3-1.블루투스통신데이터수신_1.py

```
01  import bluetooth,ble_simple_peripheral,time
02
03  ble = bluetooth.BLE()
04  p = ble_simple_peripheral.BLESimplePeripheral(ble,name='pyD_1234')
05
06  def on_rx(read):
07      print("RX", read)
08
09  p.on_write(on_rx)
```

코드설명

01: 필요한 라이브러리와 모듈을 불러옵니다. 여기서 `bluetooth`는 블루투스 기능을 활용하기 위한 모듈이며, `ble_simple_peripheral`은 간단한 블루투스 저에너지(BLE) 주변 장치를 설정하고 관리하기 위한 유틸리티 모듈입니다.

03: `bluetooth.BLE()`를 호출하여 블루투스 저에너지 객체를 생성하고, 이를 `ble` 변수에 저장합니다. 이 객체는 BLE 기능을 제어하는 데 사용됩니다.

04: `ble_simple_peripheral.BLESimplePeripheral` 클래스의 인스턴스를 생성합니다. 이 클래스는 블루투스 저에너지 주변 장치를 구현하며, 생성 시 `ble` 객체와 주변 장치의 이름을 인자로 받습니다. 여기서 주변 장치의 이름은 'pyD_1234'로 설정되어 있습니다. 이 인스턴스는 `p` 변수에 저장됩니다.

06: `on_rx` 함수를 정의합니다. 이 함수는 BLE 장치에서 데이터를 수신했을 때 호출되며, 수신된 데이터는 `read` 매개변수를 통해 전달됩니다.

07: `print("RX", read)`를 사용하여 수신된 데이터를 콘솔에 출력합니다. "RX"는 수신을 나타내는 레이블이며, `read`는 수신된 실제 데이터입니다.

09: `p.on_write(on_rx)`는 `p` 객체의 `on_write` 이벤트 핸들러를 설정합니다. 이 코드는 `on_rx` 함수를 해당 이벤트의 콜백으로 지정하여, BLE 장치로부터 데이터를 쓰기(write)할 때마다 `on_rx` 함수가 호출되도록 합니다. 이를 통해 주변 장치로부터 데이터가 전송될 때마다 해당 데이터를 처리하고 응답할 수 있습니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

안드로이드 스마트폰의 [플레이스토어]에서 "다두이노 파이"를 검색 한다음 [다두이노 파이드론]을 설치합니다.

* 블루투스 4.0이상의 기기에서 지원이되며 안드로이드 버전이 오래된 경우 동작하지 않을 수 있습니다.

* 삼성 S24 스마트폰으로 테스트하였습니다.

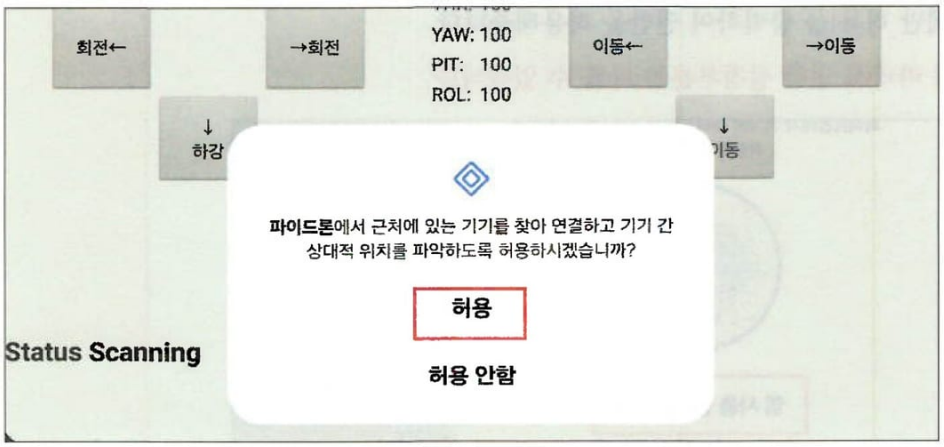
앱을 처음 실행시 블루투스 사용을 위한 권한이 필요로 합니다. 블루투스를 사용하기 위해서 위치 권한을 [앱 사용중에만 허용]을 클릭하여 권한을 허용해줍니다.

*안드로이드 버전에 따라서 권한 설정부분은 다를 수 있습니다.

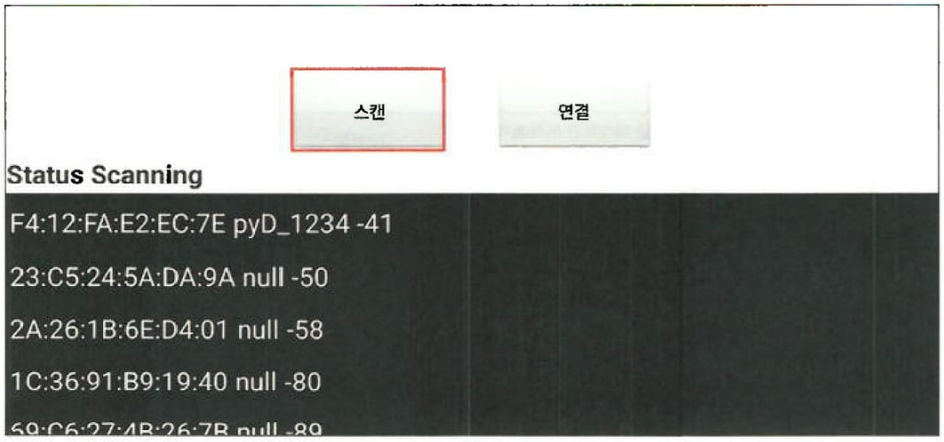
[정확한 위치로 변경]을 선택합니다.

스크롤을 아래로 내려 [스캔]을 클릭하여 주변 블루투스를 스캔합니다.

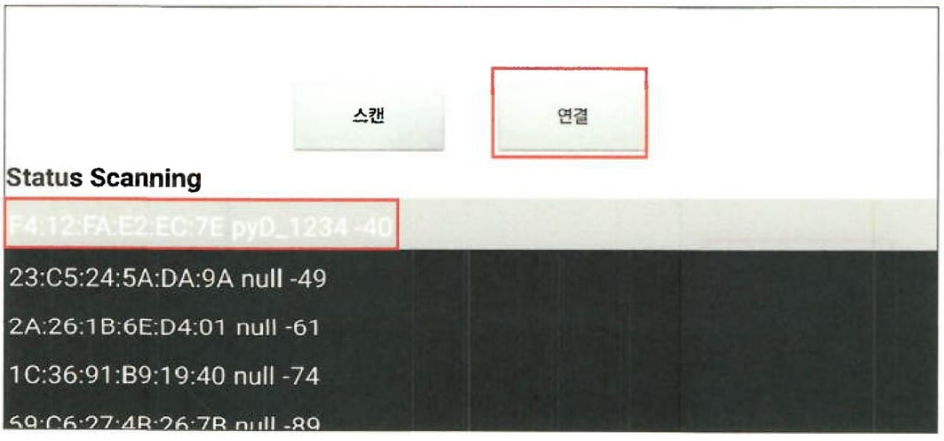
앱을 설치 후 처음 실행시 근처에 있는 기기를 찾아 연결하는 권한을 [허용]으로 설정합니다.



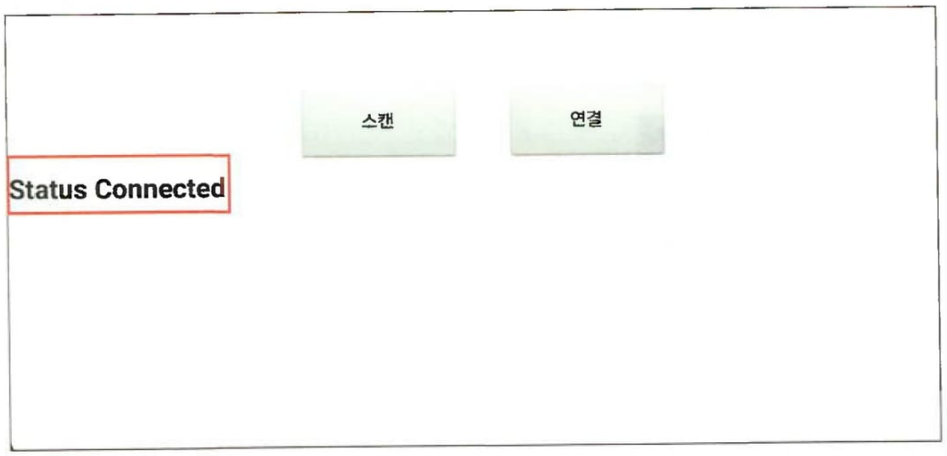
[스캔]을 클릭하면 주변 블루투스 기기를 스캔합니다.



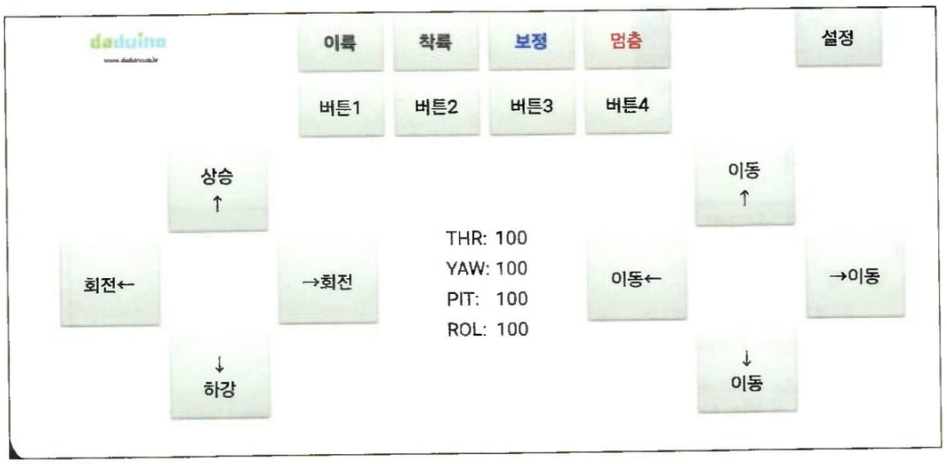
내가 코드에서 설정한 이름을 선택 후 [연결]을 눌러 기기를 연결합니다. 코드에서 pyD_1234의 이름으로 설정하였습니다. 앞에 F4:12..는 기기의 mac주소입니다. -40은 신호강도입니다. mac 주소와 신호강도는 동일하지 않을 수 있습니다.



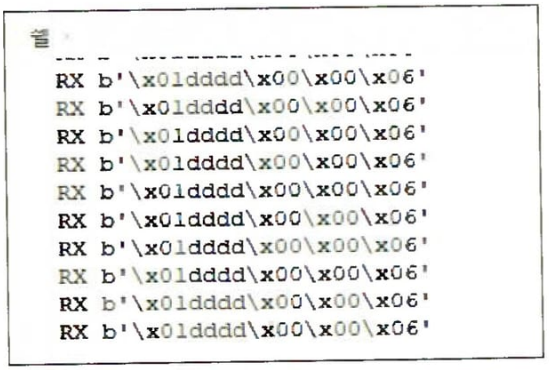
연결이되면 Status가 Connected로 변경됩니다.



스크롤을 위로 올려 데이터를 드론으로 전송이 가능합니다.



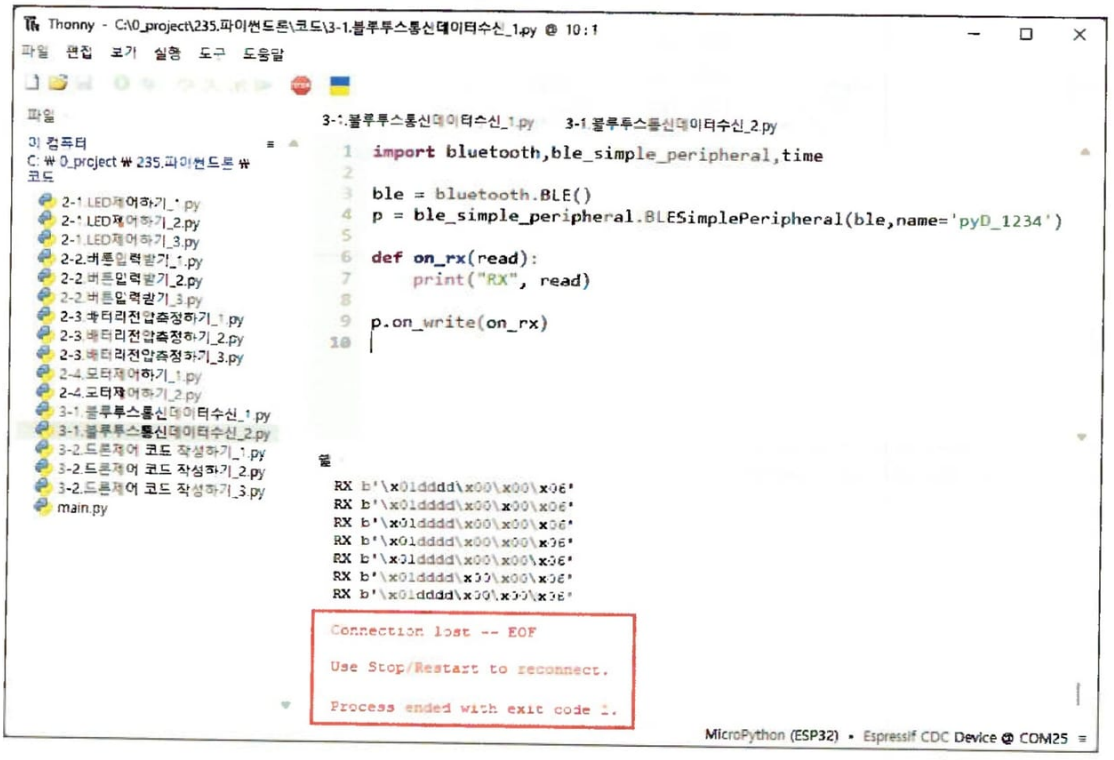
전송된 데이터는 쉘영역에서 확인이 가능합니다.



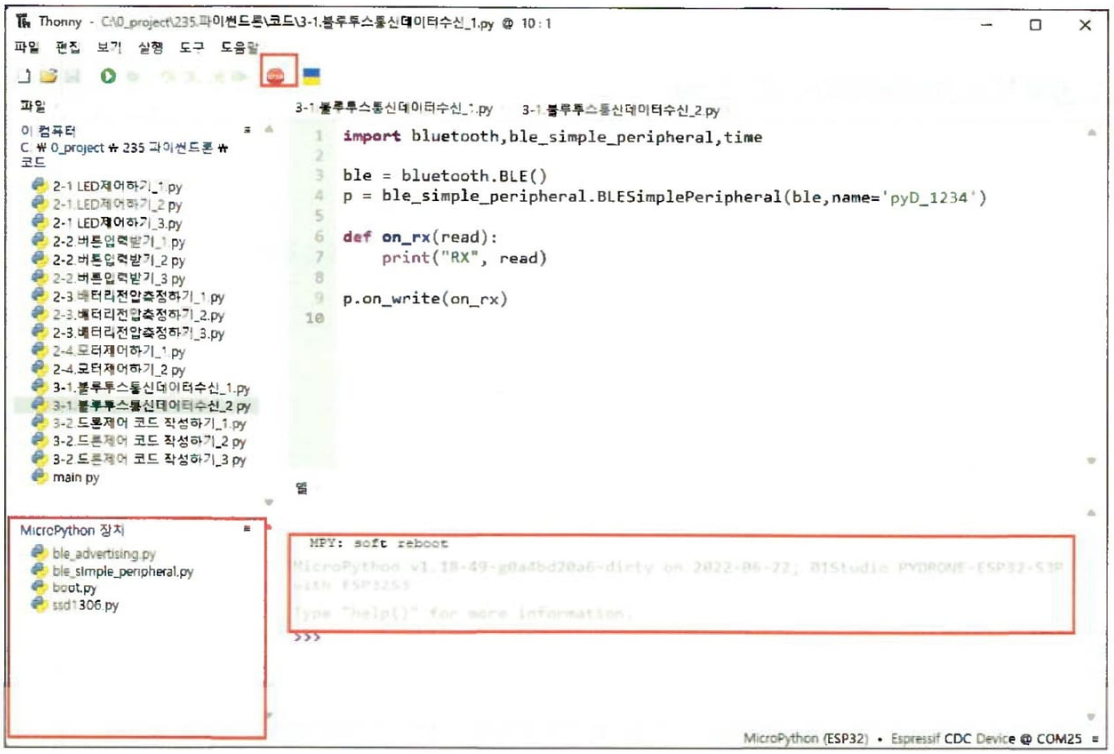
블루투스 연결의 경우 코드를 변경하고 다시 실행하면 블루투스를 끊고난 다음 다시 연결해야 합니다.

드론의 리셋버튼(RST)을 눌러 드론을 리셋합니다.

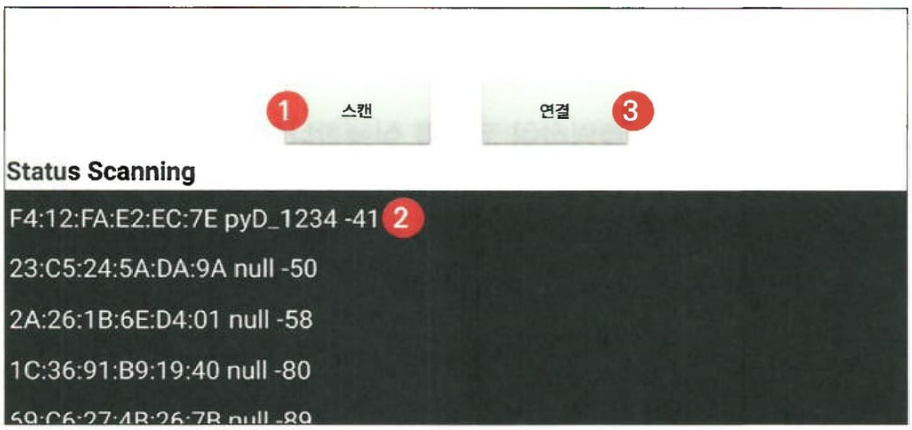
Thonny에서 연결이 끊겼습니다.



[STOP]을 눌러 다시 드론과 연결합니다. 드론이 연결되었다면 쉘 영역에 >>> 가 보이고 MicroPython장치의 파일이 표시됩니다.



블루투스 [스캔]을 눌러 [장치를 선택] 하고 다시 드론과 [연결]합니다.



블루투스연결의 경우 위의 과정인 드론리셋 -> 앱다시연결 과정을 코드가 변경될때마다 진행한다음 테스트 합니다.

드론 조종데이터 분리하여 출력하기

드론의 조종데이터를 분리하여 출력해보도록 합니다.

3-1.블루투스통신데이터수신_2.py

```
01  import bluetooth,ble_simple_peripheral,time
02
03  ble = bluetooth.BLE()
04  p = ble_simple_peripheral.BLESimplePeripheral(ble,name='pyD_1234')
05
06  def on_rx(read):
07
08      print("="*50)
09      for i in range(len(read)):
10          print(i,read[i])
11      print("="*50)
12
13  p.on_write(on_rx)
```

코드설명

08: 콘솔에 등호(=)를 50번 반복하여 출력합니다. 이는 출력을 구분하기 위한 시각적 구분선 역할을 합니다.

09~10: 수신된 데이터(read)를 바이트 단위로 반복 처리하면서 각 바이트의 인덱스와 값을 출력합니다. 이는 데이터의 구조와 내용을 이해하는 데 도움을 줍니다.

11: 마지막으로 등호를 다시 50번 반복하여 출력하여 데이터 출력의 끝을 나타냅니다.

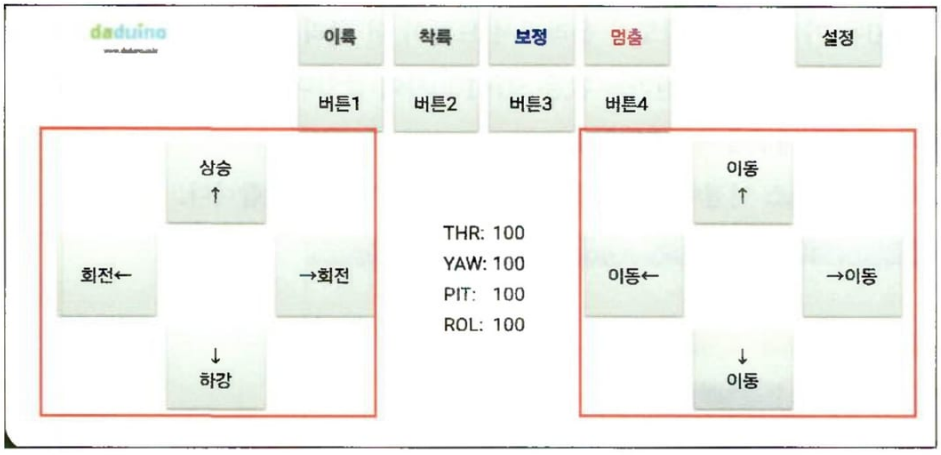
[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

0~7번까지 총 8개의 데이터가 50mS마다 수신되어 값이 출력되었습니다. 상승, 회전, 이동 버튼을 눌러 출력되는 값을 확인해봅니다. [설정] 버튼은 감도설정 버튼으로 버튼을 눌렀을 때 전송되는 값의 크기를 수정가능합니다. 실제 드론에 입력되는 값은 -100~100입니다. 수신데이터의 경우 -(음수)를 처리하기 힘들어 0~200까지의 값을 보낸다음 드론에 입력될 때 -100을 하여 -100~+100까지의 범위로 사용합니다.

지금은 순수하게 들어오는 값을 확인해보고 있으므로 0~200사이의 값이 출력됩니다.

```
셀
 6 0
 7 6
==========================================================
==========================================================
 0 1
 1 100
 2 100
 3 100
 4 100
 5 0
 6 0
 7 6
==========================================================
```

조종기의 조종버튼을 눌러 값을 확인해봅니다.



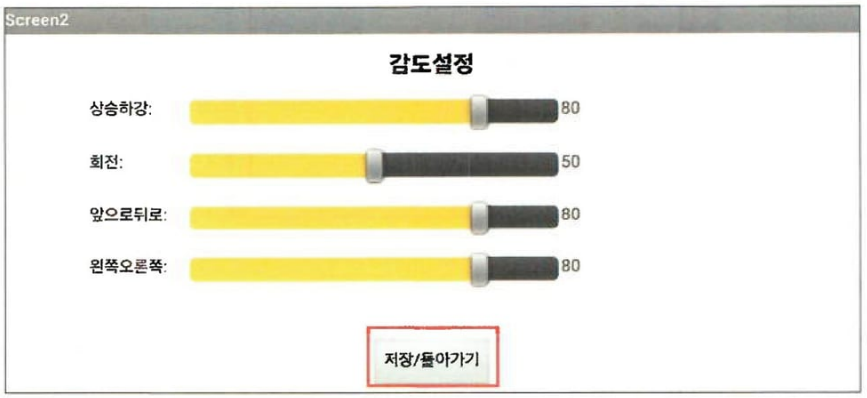
아래는 각 번지마다 연결된 버튼의 수신값입니다.

0: 1 항상1로 사용하지 않음
1: ROLL 왼쪽오른쪽 이동 100~200 기본값 100
2: PITCH 앞으로뒤로 이동 100~200 기본값 100
3: YAW 왼쪽오른쪽 회전 100~200 기본값 100
4: THROTTLE 상승 하강 100~200 기본값 100
5: 이륙(24), 착륙(72), 보정(40), 멈춤(136), 버튼1(1), 버튼2(2), 버튼3(3), 버튼4(4)
6: 0 항상0으로 사용하지 않음
7: 7 항상6으로 사용하지 않음
상승, 회전, 이동의 감도는 [설정]을 통해 조절이 가능합니다.

드론의 조종감도를 [설정] 버튼을 눌러 변경이 가능합니다.

기본값은 50입니다. 100에 가까울수로 감도가 올라가서 드론이 더 빨리 이동합니다. 0에 가까울수록 감도가 낮아 천천히 움직입니다. 추천하는 값은 50~100사이 값입니다. 설정 완료 후 [저장/돌아가기] 버튼을 눌러 설정을 저장할 수 있습니다.

*감도설정으로 이동하면 블루투스 연결이 끊어짐으로 다시 연결한 다음 진행합니다.



버튼을 눌러 변경된 값의 확인이 가능합니다.

3-2

드론제어코드 작성하기

수신데이터에서 조종신호 분리하기

블루투스로 수신받은 데이터에서 조종신호를 분리하여 출력하는 코드를 작성합니다.

3-2.드론제어 코드 작성하기_1.py

```
01  import bluetooth,ble_simple_peripheral,time
02
03  ble = bluetooth.BLE()
04  p = ble_simple_peripheral.BLESimplePeripheral(ble,name='pyD_1234')
05
06  def on_rx(read):
07
08      control_data = [None] *4
09
10      print("="*50)
11      for i in range(len(read)):
12          print(i,read[i])
13      print("="*50)
14
15      for i in range(4):
16          control_data[i] = read[i+1] -100
17
18      print('control:',control_data)
19
20  p.on_write(on_rx)
```

코드설명

15~16: control_data 리스트의 각 요소에 대해, read 리스트에서 해당 요소의 값을 100을 뺀 결과를 저장합니다. 이는 수신된 데이터를 특정 범위 내의 값으로 조정하는 과정입니다.

18: control_data 리스트의 내용을 출력하여, 조정된 제어 데이터를 확인할 수 있게 합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

[다두이노 파이드론] 앱에서 상승, 회전, 이동 버튼을 클릭합니다.

control 데이터의 값이 변경되는 것을 확인합니다. -100~100 사이 값으로 변경됩니다. 앱에서 들어오는 데이터는 0~200사이의 값이지만 실제 드론에 입력되는 값은 -100~100으로 코드에서 -100을 하여 조종데이터에 넣었고 그 값을 출력하였습니다.

```
쓸
  control: [0, 0, 0, 0]
  ==================================================
  0 1
  1 100
  2 100
  3 100
  4 100
  5 0
  6 0
  7 6
  ==================================================
  control: [0, 0, 0, 0]
```

조종신호 및 버튼신호 분리하여 출력하기

조종신호 및 버튼의 값을 분리하여 출력하는 코드를 작성해봅니다.

3-2.드론제어 코드 작성하기_2.py

```
01  import bluetooth,ble_simple_peripheral,time
02
03  ble = bluetooth.BLE()
04  p = ble_simple_peripheral.BLESimplePeripheral(ble,name='pyD_1234')
05
06  def on_rx(read):
07
08      control_data = [None] *4
09      for i in range(4):
10          control_data[i] = read[i+1] -100
11      print("="*50)
12      print("rol:",control_data[0])
13      print("pit:",control_data[1])
14      print("yaw:",control_data[2])
15      print("thr:",control_data[3])
16
17      if read[5] ==136:
18          print('stop')
19      elif read[5] ==24:
20          print('take_off')
21      elif read[5] ==72:
22          print('landing')
23      elif read[5] ==40:
24          print('calibration')
25      elif read[5] ==1:
26          print('button1')
27      elif read[5] ==2:
28          print('button2')
29      elif read[5] ==3:
30          print('button3')
31      elif read[5] ==4:
32          print('button4')
33      print("="*50)
34
35  p.on_write(on_rx)
```

코드설명

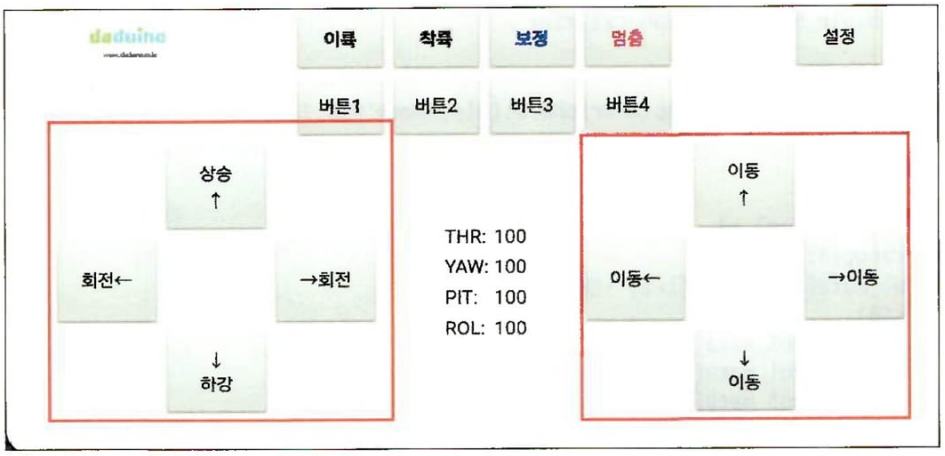
09-10: 반복문을 통해 read 배열에서 첫 번째 바이트를 건너뛰고, 다음 4바이트의 데이터를 control_data 리스트에 저장합니다. 각 값에서 100을 빼서 저장하는데, 이는 수신된 데이터의 범위 조정을 위한 것입니다.

11-33: 수신된 제어 데이터와 명령을 출력하는 부분입니다. 각 데이터에 대한 설명(롤, 피치, 요, 스로틀)과 특정 명령(정지, 이륙, 착륙 등)에 따른 행동을 출력합니다.

17-32: 추가적으로, read[5]의 값에 따라 다양한 상태나 명령('stop', 'take_off', 'landing' 등)을 출력합니다. 이는 특정 바이트를 통해 드론이나 다른 장치의 상태를 제어하거나 피드백을 주기 위한 명령어들을 구분합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

[다두이노 파이드론] 앱에서 상승, 회전, 이동 버튼을 클릭합니다.



쉘영역에서 출력되는 값을 확인합니다.

왼쪽오른쪽 이동 roll

위아래이동 pitch

회전 yaw

상승하강 throttle 값이 변경되는지 확인합니다.

```
쉘
  ==================================================
  rol: 0
  pit: 80
  yaw: 0
  thr: 0
  ==================================================
  ==================================================
  rol: 0
  pit: 80
  yaw: 0
  thr: 0
  ==================================================
```

이륙, 착륙, 보정, 멈춤, 버튼1~버튼4를 눌러 각각의 조건문에 맞게 값이 출력되는지 확인합니다.

버튼을 눌렀을때 조종신호 아래 출력되는 값을 확인합니다.

```
셸
  pit: 0
  yaw: 0
  thr: 0
  take_off
  ----------------------------------------------------------------
  ================================================================
  rol: 0
  pit: 0
  yaw: 0
  thr: 0
  take_off
  ================================================================
```

드론 자세 보정하기

드론의 전원을 켜면 자세보정을 하는 코드를 작성해봅니다.

3-2.드론제어 코드 작성하기_3.py

```
01    import bluetooth,ble_simple_peripheral,time
02    from machine import Pin
03    import drone
04
05    ble = bluetooth.BLE()
06    p = ble_simple_peripheral.BLESimplePeripheral(ble,name='pyD_1234')
07
08    d = drone.DRONE(flightmode =1,debug =0)
09
10    def do_calibration():
11        green_led = Pin(42,Pin.OUT)
12        while True:
```

```
13            print(d.read_cal_data())
14            if d.read_calibrated():
15                print(d.read_cal_data())
16                green_led.off()
17                break
18            green_led.on()
19            time.sleep_ms(50)
20            green_led.off()
21            time.sleep_ms(50)
22
23    def on_rx(read):
24
25        control_data = [None] *4
26        for i in range(4):
27            control_data[i] = read[i+1] -100
28        print("="*50)
29        print("rol:",control_data[0])
30        print("pit:",control_data[1])
31        print("yaw:",control_data[2])
32        print("thr:",control_data[3])
33
34        if read[5] ==136:
35            print('stop')
36        elif read[5] ==24:
37            print('take_off')
38        elif read[5] ==72:
```

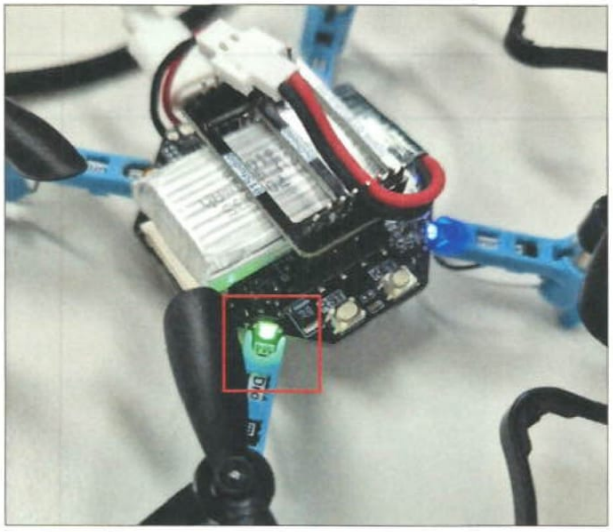
코드설명

10~21: do_calibration 함수는 드론의 캘리브레이션 과정을 수행합니다. 이 함수는 GPIO 핀 42번을 출력 모드로 설정한 후 LED를 켜고 끄면서 드론의 캘리브레이션 상태를 사용자에게 시각적으로 표시합니다. 캘리브레이션이 완료되면 LED를 끄고 함수를 종료합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

처음실행시 M4의 녹색 LED가 깜박이면서 보정을 합니다. 보정을 완료하면 깜박임이 멈춥니다. 계속 깜박일 경우 드론의 칩등이 파손되어 정상적인 동작을 하지 않습니다.

*M4녹색 LED는 드론이 연결된 다음 항상 켜져 있습니다. 이는 드론에서 제어권을 가지고 있으므로 처음시작시에만 M4녹색 LED로 보정여부를 확인할 수 있습니다. 코드가 실행중에 보정버튼을 눌러도 M4 녹색 LED는 깜빡이지 않습니다.



터미널에 보정된 값이 출력되고 완료되었습니다.

```
OK
MPY: soft reboot
MAC:  f4:12:fa:e2:ec:7e
Starting advertising
(0, 0, 0)
(249602, 1313, 2452)
(249580, 1328, 2461)
(249602, 1304, 2475)
(228169, 1260, 2312)
(212885, 1272, 2005)
(210488, 1291, 1886)
(172838, 1302, 1646)
(172732, 1262, 1592)
(4850, 1281, 1476)
(4850, 1281, 1476)
>>>
```

드론 조종코드 작성하기

실제 드론을 조종하는 코드를 입력하여 앱을 이용하여 드론을 조종해봅니다. [3-3.드론조종코드] 폴더를 생성하고 파일의 이름은 main.py로 생성합니다.

3-3.드론조종코드/main.py

```
01    import bluetooth,ble_simple_peripheral,time
02    from machine import Pin
03    import drone
04
05    ble = bluetooth.BLE()
06    p = ble_simple_peripheral.BLESimplePeripheral(ble,name='pyD_1234')
07
08    d = drone.DRONE(flightmode =1,debug =0)
09
10    def do_calibration():
11        green_led = Pin(42,Pin.OUT)
12        while True:
13            print(d.read_cal_data())
14            if d.read_calibrated():
15                print(d.read_cal_data())
16                green_led.off()
17                break
18            green_led.on()
19            time.sleep_ms(50)
20            green_led.off()
21            time.sleep_ms(50)
22
23    def on_rx(read):
24
25        control_data = [None] *4
26        for i in range(4):
27            control_data[i] = read[i+1] -100
28
29         d.control(rol = control_data[0], pit = control_data[1], yaw = control_data[2],
thr = control_data[3])
30
31        if read[5] ==136:
32            print('stop')
33            d.stop()
34        elif read[5] ==24:
35            print('take_off')
36            d.take_off(distance =120)
37        elif read[5] ==72:
38            print('landing')
39            d.landing()
40        elif read[5] ==40:
41            print('calibration')
42            do_calibration()
43        elif read[5] ==1:
44            print('button1')
45        elif read[5] ==2:
46            print('button2')
47        elif read[5] ==3:
```

```
48              print('button3')
49          elif read[5] ==4:
50              print('button4')
51
52      do_calibration()
53      p.on_write(on_rx)
```

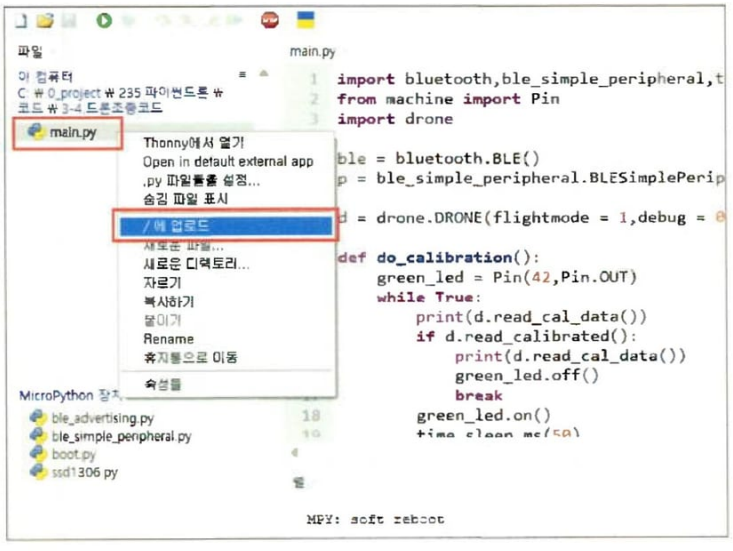
코드설명

29: 변환된 제어 데이터를 사용하여 d.control() 메서드를 호출함으로써 드론을 실시간으로 제어합니다.

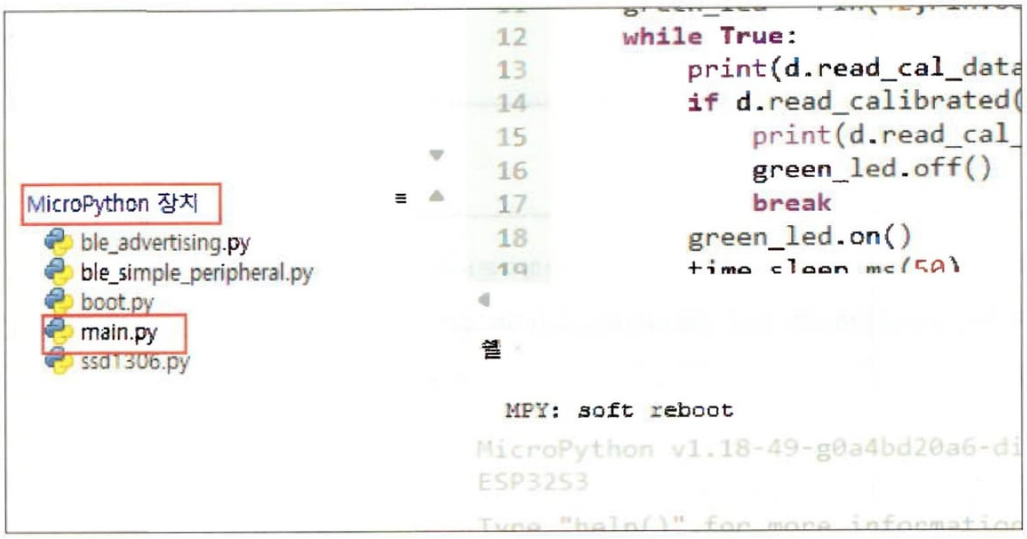
31-50: 추가적으로, 수신된 데이터의 여섯 번째 바이트(read[5])를 통해 특정 동작을 제어합니다. 예를 들어 '정지(stop)', '이륙(take_off)', '착륙(landing)', '캘리브레이션(calibration)' 등의 명령을 처리합니다.

main.py파일을 드론에 업로드합니다.

main.py를 드론에 업로드하면 드론의 전원이 켜진 후 main.py파일을 자동으로 실행합니다. main.py 파일에 마우스 오른쪽을 클릭후 [/에 업로드]를 클릭하여 드론에 main.py파일을 업로드 합니다.



드론에 main.py파일이 업로드 되었습니다. USB케이블을 분리하고 리셋버튼을 누르면 main.py 코드를 자동으로 실행합니다.



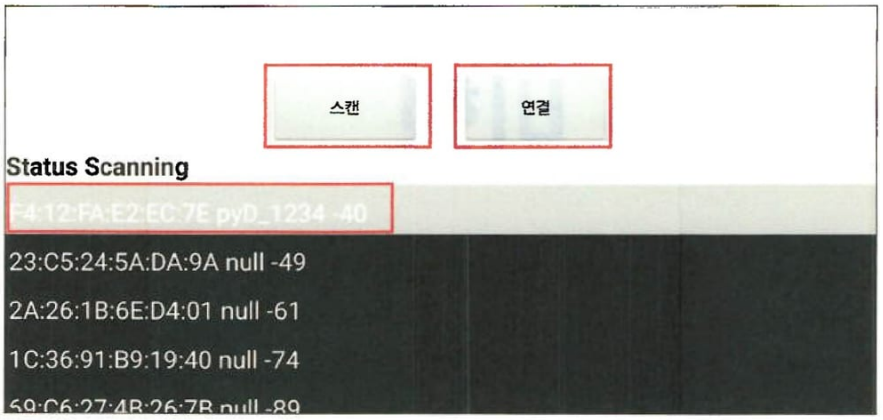
3-3

드론 비행

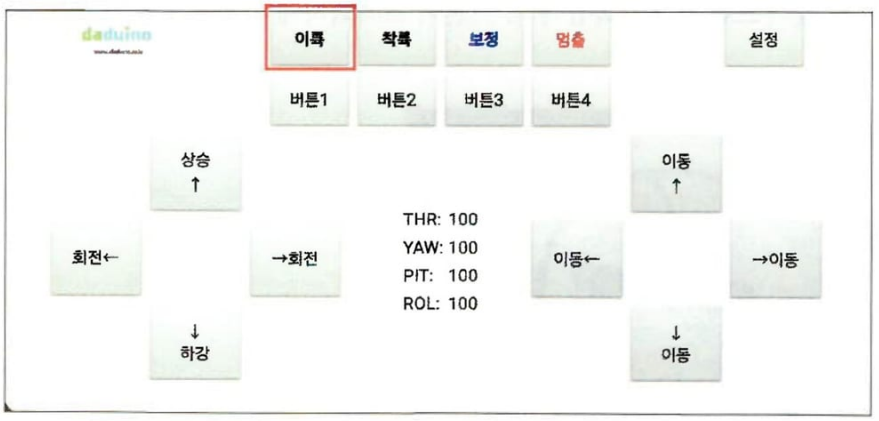
[3-3.드론조종코드/main.py] 파일을 드론에 업로드 한 상태에서 진행합니다.

드론에서 PC와 연결된 USB케이블을 분리합니다. 배터리는 드론과 연결합니다. 드론을 평평한 곳에 둔다음 [RST]리셋 버튼을 눌러 드론을 초기화합니다. 드론이 초기화시에 자세보정을 하므로 평평한곳에서 시작합니다.

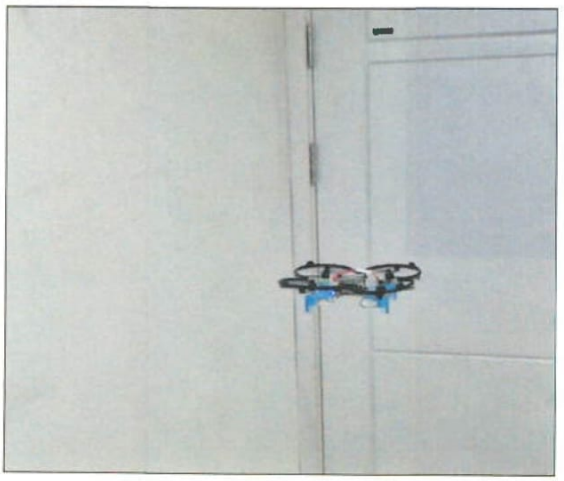
[다두이노 파이드론] 앱에서 [스캔]을 눌러 드론을 찾은 다음 클릭하여 [연결] 합니다. pyD_1234는 드론 코드에서 설정한 이름으로 원하는 이름으로 변경하여 사용합니다. 이름이 8자가 초과될 경우 오류가 발생하므로 8자 이내로 설정합니다.



드론이 스마트폰과 페어링되었다면 [이륙] 버튼을 눌러 이체를 이륙합니다,

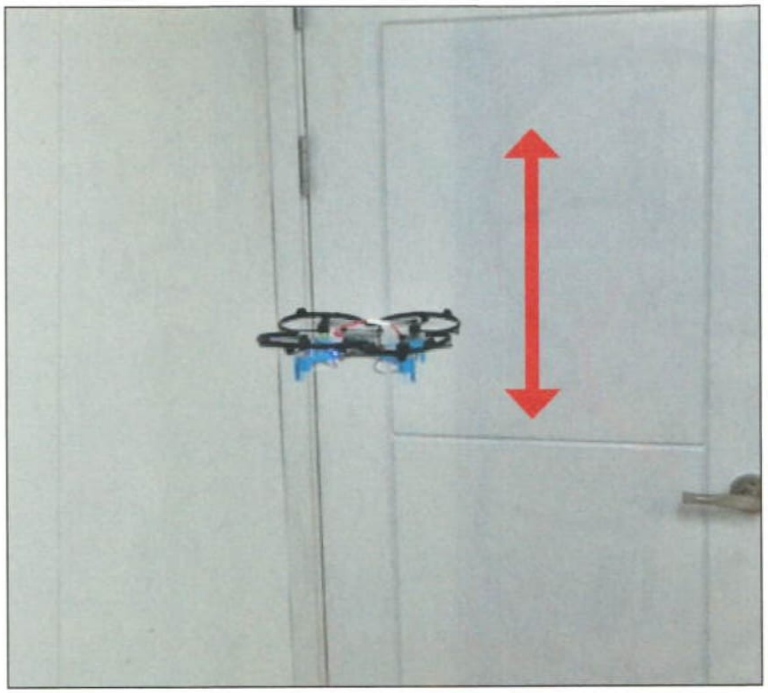


코드에서 설정된 120cm의 높이로 자동으로 이륙합니다.



[상승], [하강] 버튼은 드론을 상승 또는 하강 합니다.

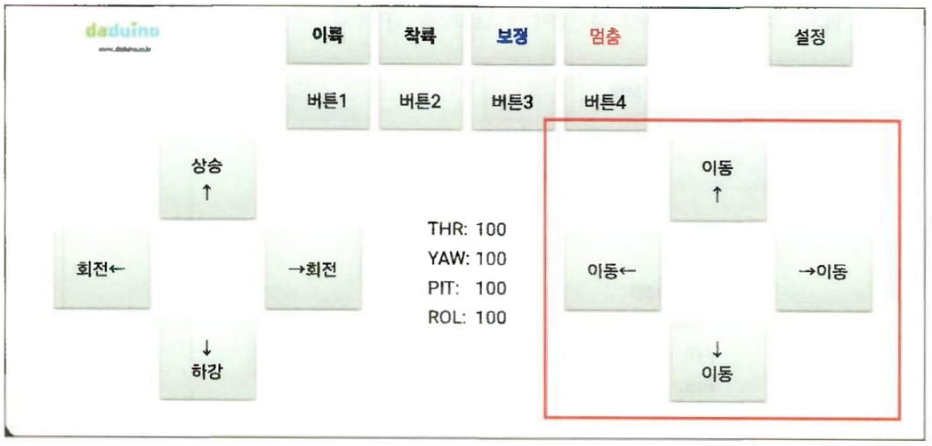
드론이 상승, 하강 합니다.



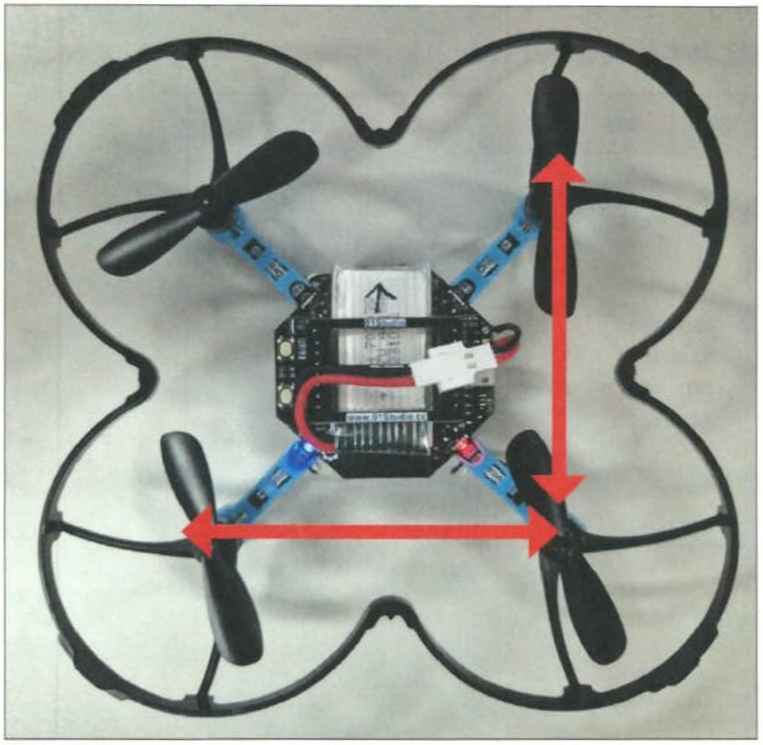
[회전] 버튼은 드론을 왼쪽 또는 오른쪽으로 회전시킵니다.

드론이 회전 합니다. 비행중에만 회전이 가능합니다.

[이동] 버튼은 드론을 앞으로뒤로, 왼쪽오른쪽으로 이동합니다.



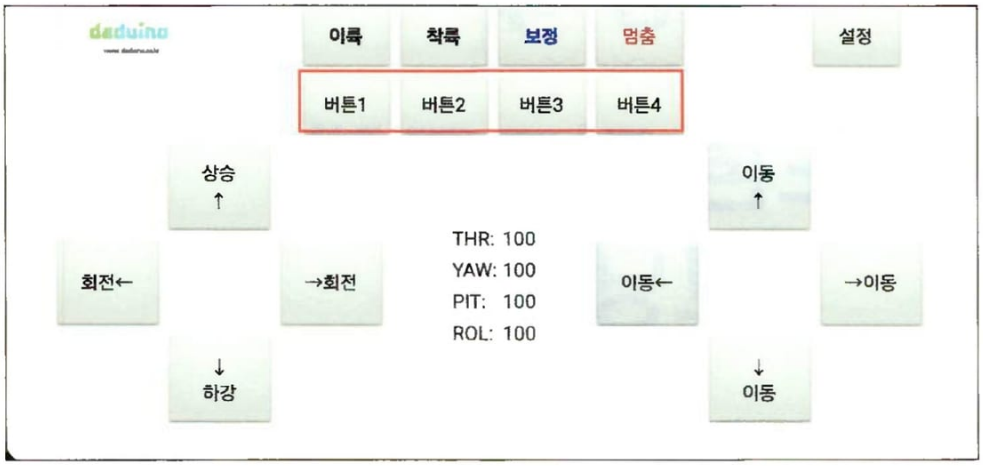
드론의 이동방향에 따라 움직입니다.



[착륙] 버튼은 드론을 서서히 착륙시킵니다. [멈춤] 버튼은 날개를 멈춰 드론을 추락시킵니다. 긴급 상황일 경우 [멈춤] 버튼을 눌러 드론의 날개가 동작하지 않도록 합니다.

[보정] 버튼은 드론의 센서를 보정합니다. 드론을 보정할때는 평평한곳에 위치 한 후 사용합니다. 드론 기체가 리셋될 때 보정을 하므로 특별히 사용하지 않습니다.

[버튼1~버튼4] 버튼은 사용자가 코드를 이용해서 특별한 기능을 넣을 수 있도록 준비해둔 버튼으로 사용자가 코드를 작성해서 사용합니다.



드론의 조종순서는

01 드론 리셋

02 드론 연결

03 [이륙] 버튼을 클릭하여 드론 이륙

04 드론 조종

05 [착륙] 버튼을 이용해서 드론 착륙 긴급상황일 경우 [멈춤] 버튼을 눌러 드론 추락

순서대로 진행합니다. 처음 드론 조종이 서툴다면 드론 이륙 후 [이동] 버튼만으로 드론을 조종한 다음 익숙해지면 상승하강, 회전을 통해 드론을 익숙하게 조종합니다.

3-4

드론 경로주행 코딩하기

버튼을 확인하는 코드 만들기

드론이 바로 동작하지 않고 버튼을 누를 때 동작하기 위해서 버튼을 확인하는 코드를 작성해봅니다.

3-4.드론 경로 주행 코딩하기_1.py

```
01  from machine import Pin
02  import time
03
04  key = Pin(0, Pin.IN, Pin.PULL_UP)
05  prev_key = key.value()
06
07  def get_key():
08      global  prev_key
09      curr_key = key.value()
10      if curr_key != prev_key:
11          prev_key = curr_key
12          if curr_key ==0:
13              return True
14          time.sleep(0.1)
15
16      return False
17
18
19  try:
20      while True:
21          if get_key():
22              time.sleep(5.0)
23              print("버튼이 눌렸습니다")
24
25  except KeyboardInterrupt:
```

```
26        print("코드를 종료합니다.")
```

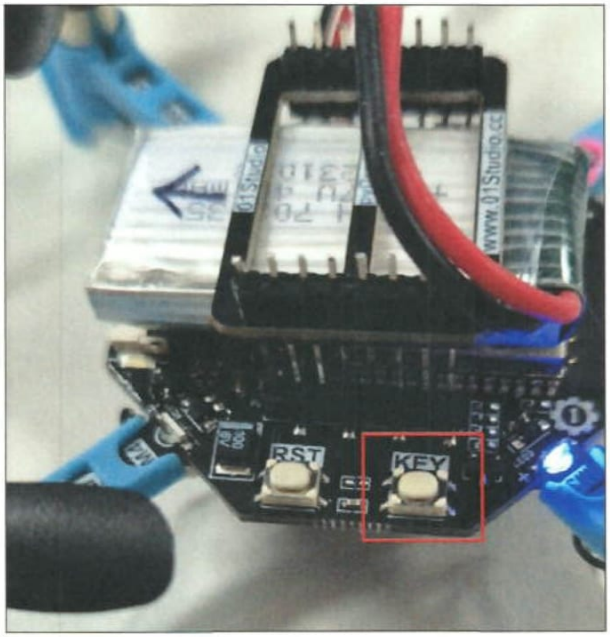
코드설명

07~16: `get_key` 함수는 버튼의 상태를 확인하고, 상태 변화가 있을 때 버튼이 눌렸는지 여부를 반환합니다.

19~25: 무한 루프를 통해 `get_key` 함수의 반환 값이 True인 경우 (버튼이 눌림)에만 "버튼이 눌렸습니다"를 출력하고, 5초 동안 대기합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

[KEY]버튼을 누르면 버튼이 눌렸습니다를 출력합니다.



[KEY]버튼을 눌러 쉘 영역에서 버튼의 눌림을 확인합니다.

```
쉘
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
버튼이 눌렸습니다
```

이륙 및 착륙하기

버튼을 누르면 3초뒤에 5초동안 이륙하고 5초뒤에 착륙하는 코드를 작성해봅니다.

[3-4.드론 경로 주행 코딩하기_2] 폴더를 생성 후 main.py파일을 생성후 진행합니다.

3-4.드론 경로 주행 코딩하기_2/main.py

```
import bluetooth,ble_simple_peripheral,time
from machine import Pin
import drone

ble = bluetooth.BLE()
p = ble_simple_peripheral.BLESimplePeripheral(ble,name='pyD_1234')

d = drone.DRONE(flightmode =1,debug =0)

key = Pin(0, Pin.IN, Pin.PULL_UP)
prev_key = key.value()

def do_calibration():
    green_led = Pin(42,Pin.OUT)
    while True:
        print(d.read_cal_data())
        if d.read_calibrated():
            print(d.read_cal_data())
            green_led.off()
            break
        green_led.on()
        time.sleep_ms(50)
        green_led.off()
        time.sleep_ms(50)

def get_key():
    global  prev_key
    curr_key = key.value()
    if curr_key != prev_key:
        prev_key = curr_key
        if curr_key ==0:
            return True
        time.sleep(0.1)

    return False

do_calibration()

try:
    while True:
        if get_key() ==True:
            #버튼을 누르면 3초동안 대기
            for i in range(3 *20):
                d.control(rol =0, pit =0, yaw =0, thr =0)
                time.sleep_ms(50)

            #이륙 5초동안
            for i in range(5 *20):
```

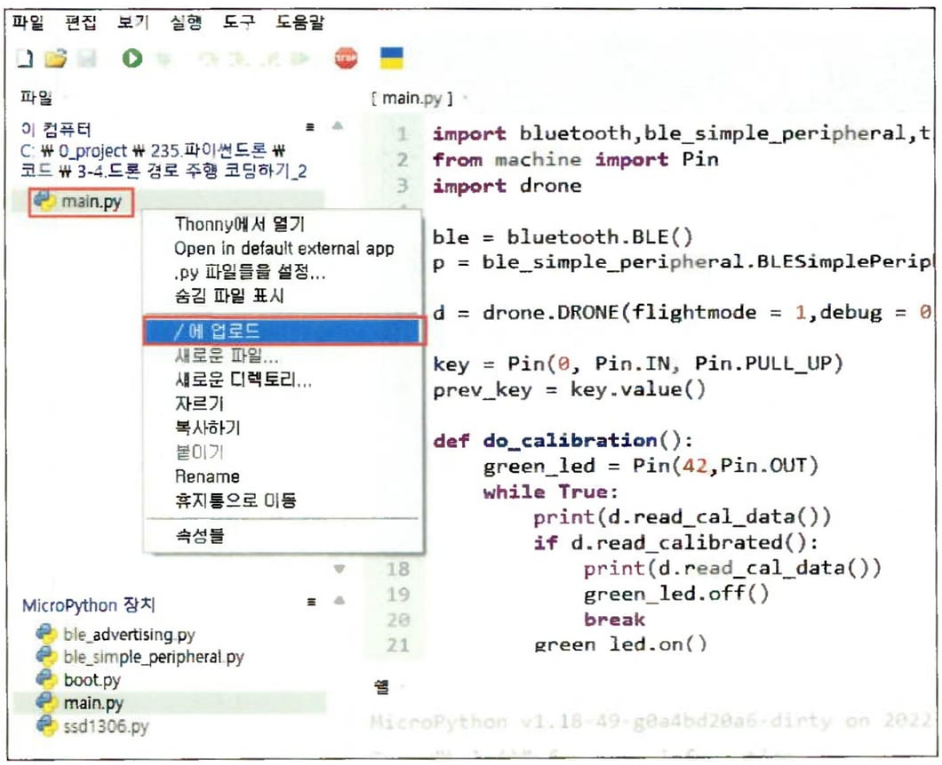
```
49                  d.control(rol =20, pit =-20, yaw =0, thr =0)
50                  d.take_off(distance =120)
51                  time.sleep_ms(50)
52
53              #착륙하기 5초동안
54              for i in range(5 *20):
55                  d.control(rol =20, pit =-20, yaw =0, thr =0)
56                  time.sleep_ms(50)
57
58  except KeyboardInterrupt:
59      print("코드를 종료합니다.")
```

코드설명

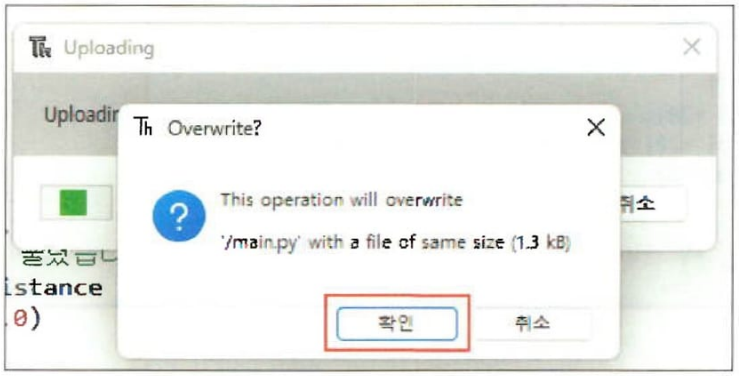
39-58: 무한 루프 내에서 `get_key` 함수를 통해 버튼 입력을 확인하고, 버튼이 눌린 경우 드론의 동작을 제어합니다. 먼저 드론을 3초 동안 정지 상태로 유지하고, 이후 5초 동안 이륙하며, 다시 5초 동안 착륙 절차를 진행합니다. 각 동작은 50밀리초 간격으로 제어 명령을 보냅니다.

main.py파일을 드론에 업로드합니다.

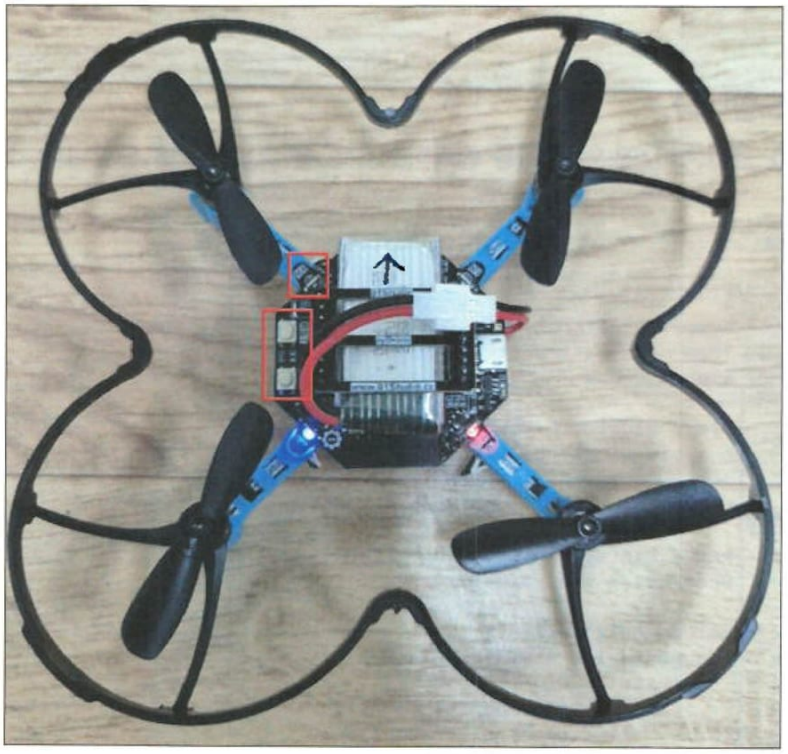
main.py 파일을 마우스 오른쪽을 클릭 후 [/에 업로드]를 클릭하여 코드를 드론에 업로드합니다.



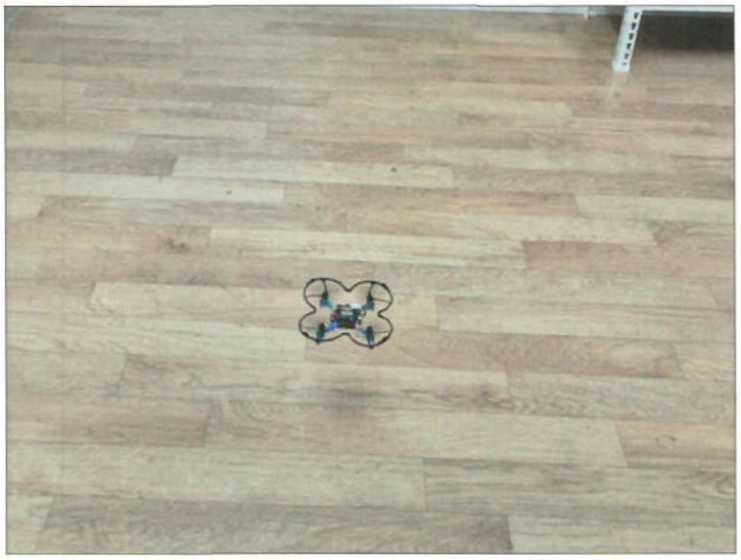
main.py파일이 드론에 이미 있다면 [확인]을 눌러 덮어쓰기 하여 업로드 합니다.



USB를 분리한다음 드론의 기체를 평평한곳에 위치한다음 [RST]버튼을 눌러 기체를 리셋합니다. M4의 녹색LED가 깜빡이면 보정중으로 보정이 완료되면 [KEY]버튼을 눌러 드론의 비행을 시작합니다.



3초후에 드론이 상승하기 시작합니다.



설정한 120cm의 높이까지 상승합니다.

드론이 똑바로 올라갔다 내려오지 않을 경우 코드에서 rol, pit 값을 조절하여야 합니다.

```
기본값
d.control(rol = 0, pit = 0, yaw = 0, thr = 0)

왼쪽으로 기체가 이동시 rol값을 -> 방향으로 조금 입력합니다. 값은 10~30정도로 테스트한다음
입력합니다.
d.control(rol = 10, pit = 0, yaw = 0, thr = 0)

오른쪽으로 기체가 이동시 rol값을 <- 방향으로 조금 입력합니다. 값은 -10~-30정도로 테스트한
다음 입력합니다.
d.control(rol = -10, pit = 0, yaw = 0, thr = 0)

앞으로 기체가 이동시 pit값을 뒤로 조금 입력합니다. 값은 -10~-30정도로 테스트한다음 입력합
니다.
d.control(rol = 0, pit = -10, yaw = 0, thr = 0)

뒤로 기체가 이동시 pit값을 앞으로 조금 입력합니다. 값은 10~30정도로 테스트한다음 입력합니
다.
d.control(rol = 0, pit = 10, yaw = 0, thr = 0)


두 개의 값을 동시에 입력해도 됩니다. 예를 들어 왼쪽 앞으로 기체가 흐를시 roll은 오른쪽 pit
는 뒤쪽방향의 값을 조금 입력합니다.
d.control(rol = 20, pit = -20, yaw = 0, thr = 0)
```

경로주행 코딩하기

버튼을 누르면 이륙-〉 앞으로 -〉 뒤로 -〉 착륙 하는 과정의 코드를 작성합니다.

[3-4.드론 경로 주행 코딩하기_3] 폴더를 생성 후 main.py파일을 생성후 진행합니다.

3-4.드론 경로 주행 코딩하기_3/main.py

```
01    import bluetooth,ble_simple_peripheral,time
02    from machine import Pin
03    import drone
04
05    ble = bluetooth.BLE()
06    p = ble_simple_peripheral.BLESimplePeripheral(ble,name='pyD_1234')
07
08    d = drone.DRONE(flightmode =1,debug =0)
09
10    key = Pin(0, Pin.IN, Pin.PULL_UP)
11    prev_key = key.value()
12
13    def do_calibration():
14        green_led = Pin(42,Pin.OUT)
15        while True:
16            print(d.read_cal_data())
```

```
17          if d.read_calibrated():
18              print(d.read_cal_data())
19              green_led.off()
20              break
21          green_led.on()
22          time.sleep_ms(50)
23          green_led.off()
24          time.sleep_ms(50)
25
26  def get_key():
27      global  prev_key
28      curr_key = key.value()
29      if curr_key != prev_key:
30          prev_key = curr_key
31          if curr_key ==0:
32              return True
33          time.sleep(0.1)
34
35      return False
36
37  do_calibration()
38
39  try:
40      while True:
41          if get_key() ==True:
42              #버튼을 누르면 3초동안 대기
43              for i in range(3 *20):
44                  d.control(rol =0, pit =0, yaw =0, thr =0)
45                  time.sleep_ms(50)
46
47              #상승 5초동안
48              for i in range(5 *20):
49                  d.control(rol =20, pit =-20, yaw =0, thr =0)
50                  d.take_off(distance =120)
51                  time.sleep_ms(50)
52
53              #앞으로 3초동안
54              for i in range(3 *20):
55                  d.control(rol =20, pit =40, yaw =0, thr =0)
56                  d.take_off(distance =120)
57                  time.sleep_ms(50)
58
59              #뒤로 3초동안
60              for i in range(3 *20):
61                  d.control(rol =20, pit =-40, yaw =0, thr =0)
62                  d.take_off(distance =120)
63                  time.sleep_ms(50)
64
65              #착륙
66              for i in range(5 *20):
67                  d.control(rol =20, pit =-20, yaw =0, thr =0)
68                  time.sleep_ms(50)
69
70  except KeyboardInterrupt:
71      print("코드를 종료합니다.")
```

코드설명

39-70: 무한 루프 내에서 버튼 상태를 확인하고, 버튼이 눌린 경우 정해진 시간 동안 드론을 제어하는 명령을 실행합니다.

41-46: 버튼 입력이 감지되면 3초 동안 드론을 정지시킵니다.

47-51: 드론을 5초간 상승시킵니다.

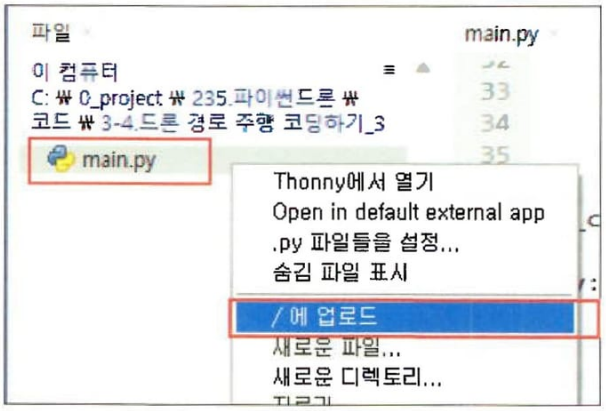
53-57: 드론을 3초간 앞으로 이동시킵니다.

59-63: 드론을 3초간 뒤로 이동시킵니다.

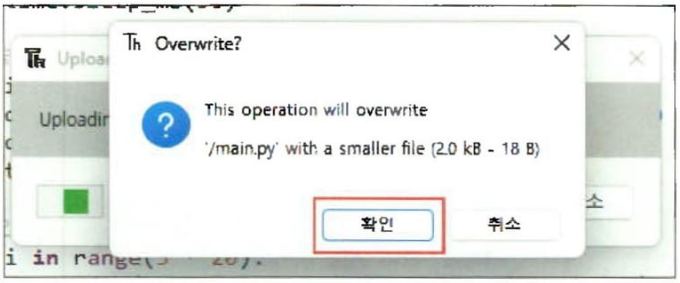
65-68: 드론을 착륙시킵니다. 각 명령은 50밀리초 간격으로 반복 실행됩니다.

main.py파일을 드론에 업로드합니다.

main.py 파일을 마우스 오른쪽을 클릭 후 [/에 업로드]를 클릭하여 코드를 드론에 업로드합니다.



main.py파일이 드론에 이미 있다면 [확인]을 눌러 덮어쓰기 하여 업로드 합니다.



USB를 분리한다음 드론의 기체를 평평한곳에 위치한다음 [RST]버튼을 눌러 기체를 리셋합니다. M4의 녹색LED가 깜빡이면 보정중으로 보정이 완료되면 [KEY]버튼을 눌러 드론의 비행을 시작합니다.

버튼을 누르면 이륙-〉 앞으로 3초동안 이동 -〉 뒤로 2초동안 이동-〉 착륙을 자동으로 하는 드론을 완성하였습니다.

CHAPTER 04

기본기능 다루기

사물인터넷에 기본부품인 RGB LED, 버튼, 부저, 조도센서등을 다루어 보면서 마이크로파이썬으로 GPIO를 이용해 다양한 부품을 다루어 봅니다.

4-1

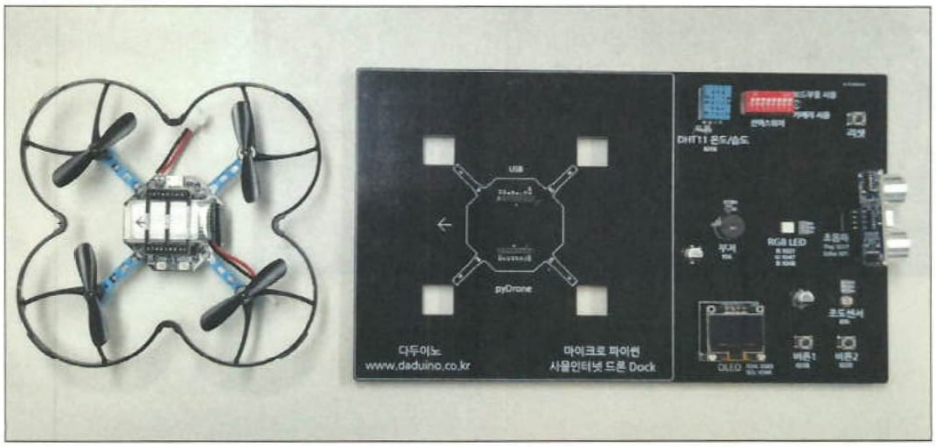
RGB LED 제어하기

GPIO를 활용하면 디지털 신호를 출력하여 다양한 장치를 제어할 수 있습니다. 이를 통해 RGB LED의 각 색상 핀(R, G, B)에 신호를 보내 원하는 색을 구현할 수 있습니다. 예를 들어, GPIO 핀에서 HIGH(1) 신호를 보내면 LED가 켜지고, LOW(0) 신호를 보내면 꺼집니다. 이를 조합하여 다양한 색상을 표현할 수 있습니다.

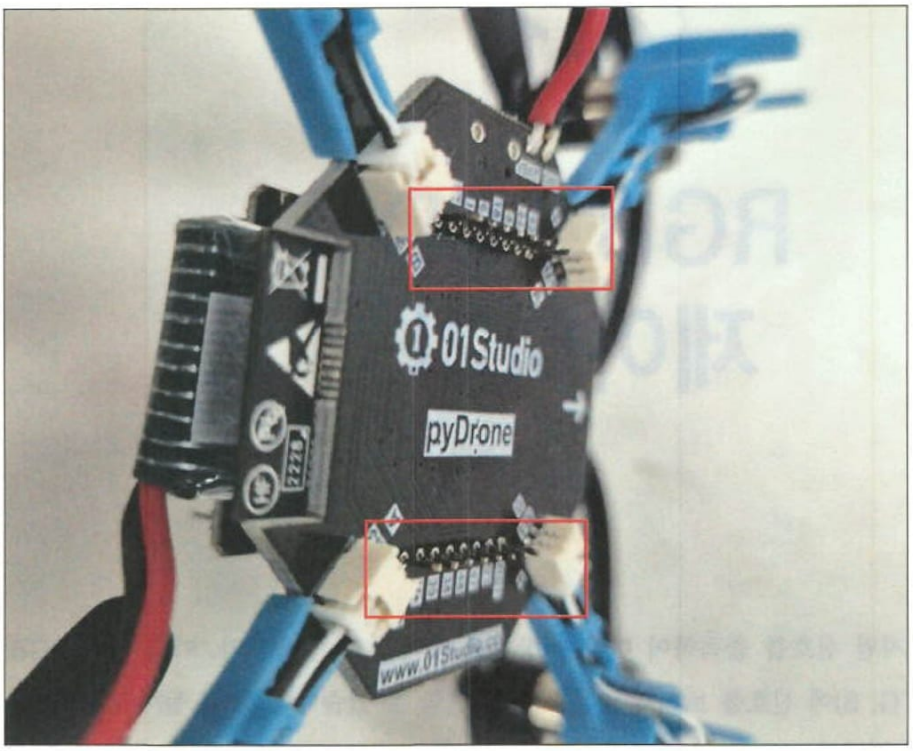
사물인터넷드론 독과 드론을 연결

드론과 사물인터넷 독을 연결하여 사물인터넷독의 다양한 부품을 사용해 사물인터넷을 실습합니다.

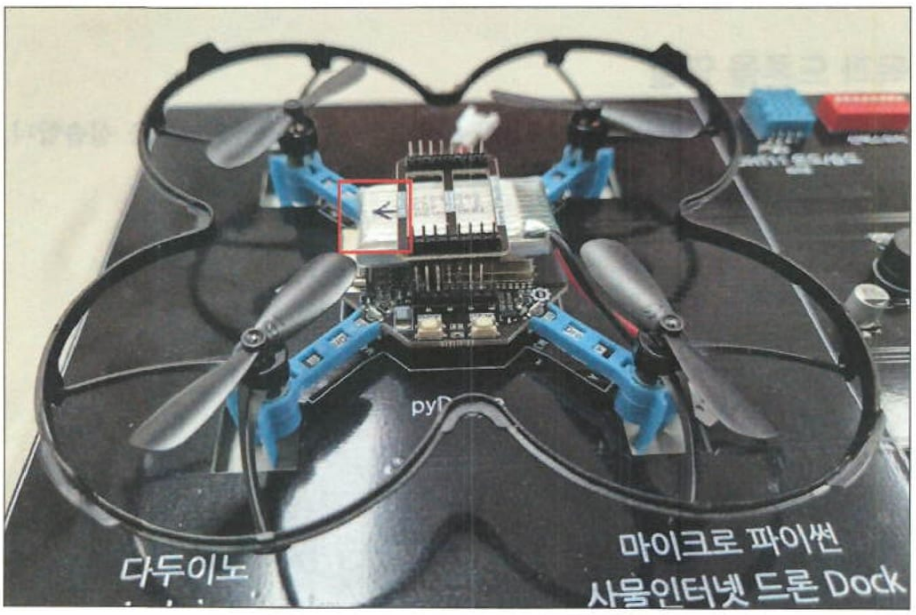
드론과 사물인터넷독을 준비합니다.



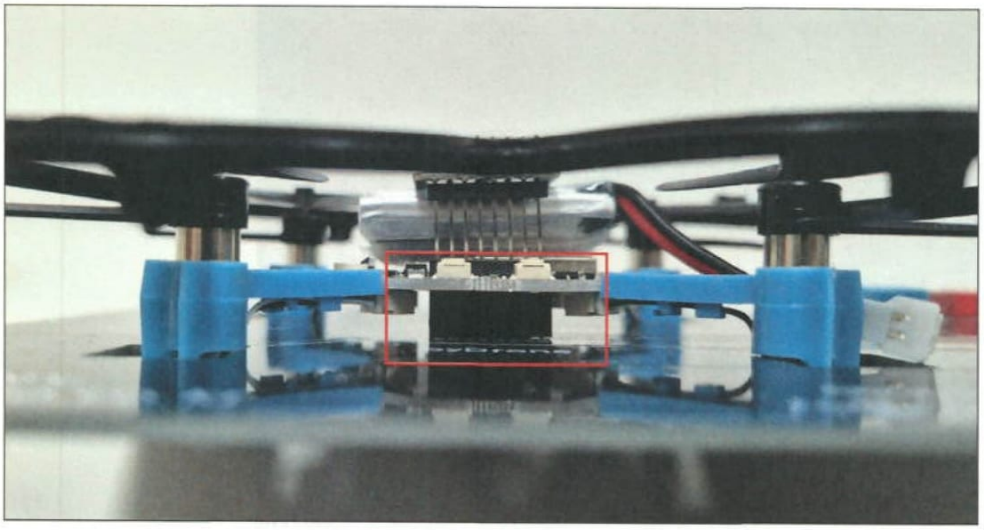
드론의 아래쪽에 핀과 사물인터넷독을 연결합니다.



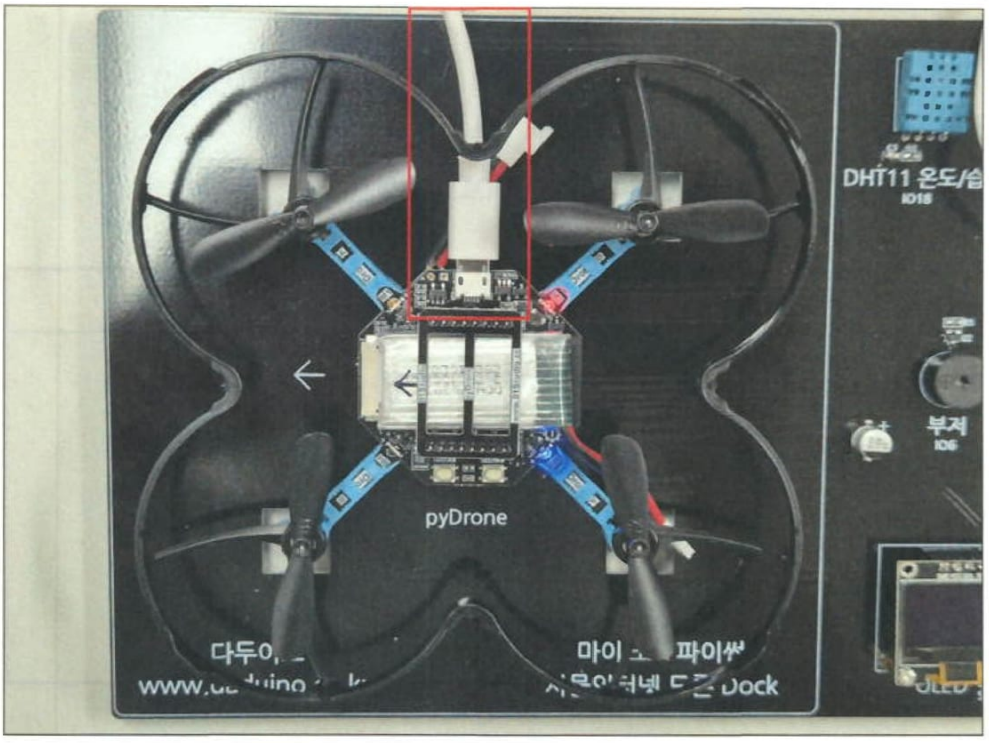
드론의 방향을 잘 확인 후 사물인터넷독과 연결합니다.



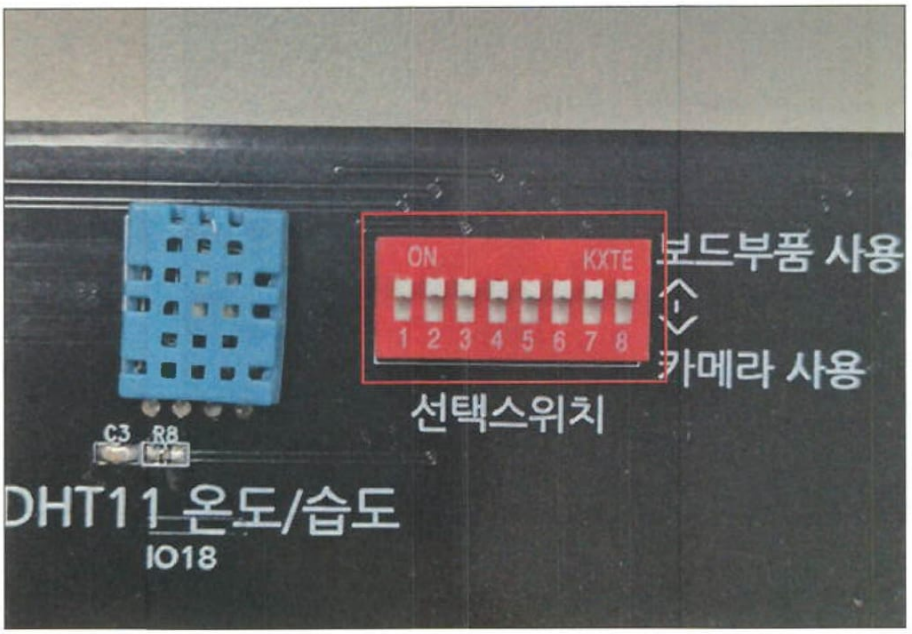
옆을 확인하여 핀이 끝까지 잘 연결되었는지 확인합니다.



USB케이블을 드론에 연결하여 사물인터넷독을 사용할 준비를 합니다.



선택스위치를 [보드부품 사용]으로 모두 설정 후 사용합니다.



빨간색 LED켜기

GPIO를 On/Off 제어하여 RGB LED의 빨간색을 제어해보도록 합니다.

4-1.RGB LED 제어하기_1.py

```
01    from machine import Pin
02    import time
03
04    LED_RED = Pin(21,Pin.OUT)
05
06    while True:
07        LED_RED.value(1)
08        time.sleep(1.0)
09        LED_RED.value(0)
10        time.sleep(1.0)
```

코드설명

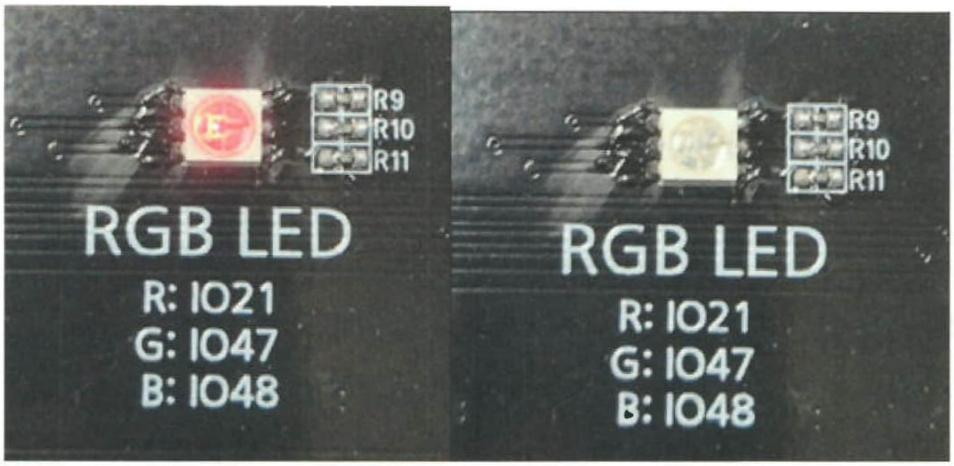
04: 21번핀을 출력핀으로 사용하고 LED_RED에 바인딩합니다.

07: 빨간색 LED를 켭니다.

09: 빨간색 LED를 끕니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

RGB LED의 빨간색 LED를 껐다 켰다를 반복합니다.



빨간색, 녹색, 파란색 LED켜기

GPIO를 제어하여 RGB LED의 빨간색, 녹색, 파란색 LED를 제어해봅니다.

4-1.RGB LED 제어하기_2.py

```
01    from machine import Pin
02    import time
03
04    LED_RED = Pin(21,Pin.OUT)
05    LED_GREEN = Pin(47,Pin.OUT)
06    LED_BLUE = Pin(48,Pin.OUT)
07
08    while True:
09        LED_RED.value(1)
10        LED_GREEN.value(0)
11        LED_BLUE.value(0)
12        time.sleep(1.0)
13
14        LED_RED.value(0)
15        LED_GREEN.value(1)
```

```
16          LED_BLUE.value(0)
17          time.sleep(1.0)
18
19          LED_RED.value(0)
20          LED_GREEN.value(0)
21          LED_BLUE.value(1)
22          time.sleep(1.0)
```

코드설명

09~13: 빨간색 LED만 켜고 1초 기다립니다.

14~17: 녹색 LED만 켜고 1초 기다립니다.

19~22: 파란색 LED만 켜고 1초 기다립니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

RGB LED의 빨간색, 녹색, 파란색 LED가 1초마다 번갈아가면서 켜집니다.

빨간색 LED의 밝기 조정하기

PWM(Pulse Width Modulation, 펄스 폭 변조)은 디지털 신호를 통해 아날로그 효과를 구현하는 기술입니다. GPIO의 PWM 기능을 활용하면 빨간색 LED에 인가되는 신호의 듀티 사이클(펄스의 ON/OFF 비율)을 조정하여 밝기를 조절할 수 있습니다. 듀티 사이클이 높을수록 LED는 더 밝아지고, 낮을수록 어두워집니다. 이를 통해 LED의 밝기를 부드럽게 변화시키는 효과를 구현할 수 있습니다.

4-1.RGB LED 제어하기_3.py

```
01  from machine import Pin,PWM
02  import time
03
04  LED_RED = PWM(Pin(21),freq=1000,duty=0)
05  LED_GREEN = PWM(Pin(47),freq=1000,duty=0)
06  LED_BLUE = PWM(Pin(48),freq=1000,duty=0)
07
08  while True:
09      LED_RED.duty(0)
10      time.sleep(1.0)
11
12      LED_RED.duty(400)
13      time.sleep(1.0)
14
15      LED_RED.duty(800)
16      time.sleep(1.0)
```

```
17
18          LED_RED.duty(1023)
19          time.sleep(1.0)
```

코드설명

01: Pin, PWM 라이브러리를 불러옵니다.

04~06: RGB LED의 핀들을 PWM출력핀으로 설정하여 사용합니다. 주파수는 1000Hz duty(듀티)는 0%로 설정합니다.

09: RGB LED중 빨간색 LED의 듀티비를 0으로 설정합니다. LED가 완전히 꺼집니다.

12: 빨간색 LED의 듀티를 400으로 설정합니다. 0~1023까지로 약 40%의 밝기로 켜집니다.

15: 빨간색 LED의 듀티를 800으로 설정합니다. 0~1023까지로 약 80%의 밝기로 켜집니다.

18: 빨간색 LED의 듀티를 최대인 1023으로 설정합니다. 0~1023까지로 최대 밝기로 켜집니다.

[▶ 현재 스크립트 실행] 아이콘을 클릭하여 코드를 실행합니다.

빨간색 LED가 점점 밝아지면 켜집니다. 최대 밝기가 된 이후에는 꺼진다음 다시 밝아집니다.

무지개 색 표현하기

RGB LED의 밝기를 조절하여 빨주노초파남보 무지개 색상으로 표현해봅니다.

main2-3-4.py

```
01  from machine import Pin, PWM
02  import time
03
04  LED_RED = PWM(Pin(21), freq=1000, duty=0)
05  LED_GREEN = PWM(Pin(47), freq=1000, duty=0)
06  LED_BLUE = PWM(Pin(48), freq=1000, duty=0)
07
08  rainbow_colors = [
09      (1023, 0, 0),
10      (1023, 200, 0),
11      (1023, 400, 0),
12      (0, 1023, 0),
13      (0, 0, 1023),
14      (200, 0, 1023),
15      (400, 0, 1023),
16  ]
17
18  while True:
19      for color in rainbow_colors:
20          LED_RED.duty(color[0])
```

```
21            LED_GREEN.duty(color[1])
22            LED_BLUE.duty(color[2])
23            time.sleep(1.0)
```

코드설명

20. `rainbow_colors` 목록의 각 색상에 대해 반복합니다.

21. `LED_RED.duty(color[0])`를 사용하여 빨간색 LED의 듀티 사이클을 `color` 튜플의 첫 번째 요소로 설정합니다. 이렇게 하면 빨간색 LED의 밝기가 변경됩니다.

22. `LED_GREEN.duty(color[1])`를 사용하여 녹색 LED의 듀티 사이클을 `color` 튜플의 두 번째 요소로 설정합니다. 이렇게 하면 녹색 LED의 밝기가 변경됩니다.

23. `LED_BLUE.duty(color[2])`를 사용하여 파란색 LED의 듀티 사이클을 `color` 튜플의 세 번째 요소로 설정합니다. 이렇게 하면 파란색 LED의 밝기가 변경됩니다.

24. `time.sleep(1.0)`을 사용하여 1초 동안 대기합니다.

[현재 스크립트 실행] 아이콘을 클릭하여 코드를 실행합니다.

무지개 색상 목록을 사용하여 빨간색, 녹색, 파란색 LED를 순서대로 켜고 끄는 것을 반복합니다. 결과적으로 LED는 빨강, 주황, 노랑, 초록, 파랑, 남색, 보라의 순서로 반복하여 색상을 표현하게 됩니다.

4-2

버튼 입력받기

GPIO의 입력 기능을 사용하면 외부 장치나 센서의 상태를 감지할 수 있습니다. 이를 통해 버튼을 연결하고, GPIO 핀이 HIGH(1) 또는 LOW(0) 신호를 감지하여 버튼의 눌림 상태를 확인할 수 있습니다. 예를 들어, 버튼이 눌리면 GPIO 핀에서 LOW 신호를 입력받아 이를 활용한 동작을 구현할 수 있습니다.

버튼입력

디지털 입력 신호를 읽고, 해당 입력 신호에 따라 핀의 상태를 출력하는 코드를 작성합니다.

4-2.버튼입력받기_1.py

```
01  from machine import Pin
02  import time
03
04  key1=Pin(38,Pin.IN,Pin.PULL_UP)
05
06  while True:
07      print(key1.value())
08      time.sleep(0.1)
```

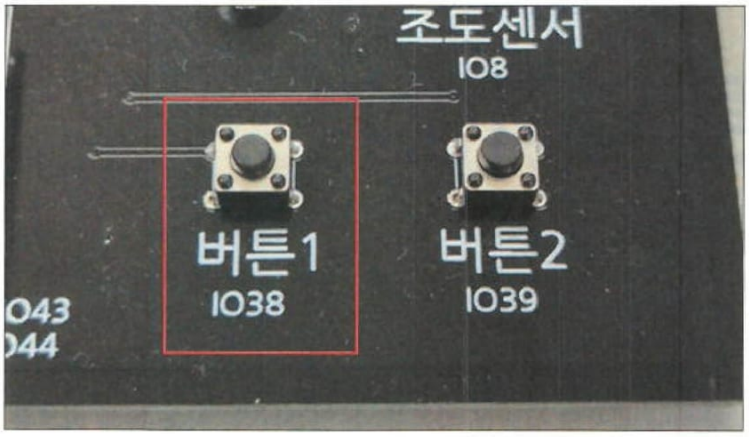
코드설명

4: key1` 변수에 핀 번호 14를 할당하고, `Pin.IN`을 사용하여 이 핀을 디지털 입력 핀으로 설정합니다. 또한 `Pin.PULL_UP`을 사용하여 내장 풀업 저항을 활성화합니다.

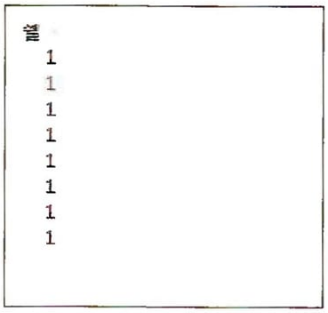
7: `key1` 핀의 상태를 읽고 출력합니다. `key1.value()` 함수를 사용하여 핀의 상태를 읽은 후, 그 값을 화면에 출력합니다.

[현재 스크립트 실행] 아이콘을 클릭하여 코드를 실행합니다.

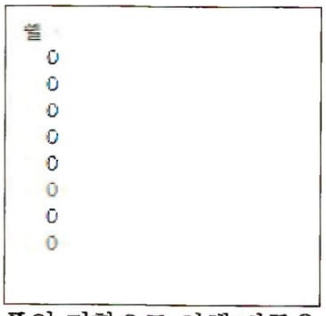
핀 38에 연결된 버튼 1의 디지털 입력 신호를 읽어서 그 값을 출력하고, 0.1초마다 핀의 상태를 감지하고 출력합니다.



버튼을 누르지 않으면 1이 출력됩니다.



버튼을 누르면 0이 출력됩니다.



풀업 저항으로 인해 버튼을 누르지 않을 때 1, 버튼을 누를 때 0의 값이 출력되었습니다.

버튼을 누를때만 출력

입력 신호가 변경될 때 "click" 메시지를 출력하는 프로그램입니다.

4-2.버튼입력받기_2.py

```
01  from machine import Pin
02  import time
03
04  key1 = Pin(38, Pin.IN, Pin.PULL_UP)
05  prev_state = key1.value()
06
07  while True:
08      current_state = key1.value()
09      if current_state != prev_state:
10          prev_state = current_state
11          if current_state ==0:
12              print("click")
13          time.sleep(0.1)
```

코드설명

05: 버튼의 이전상태를 저장하는 prev_state 값에 버튼값을 저장합니다.

08: 버튼의 현재상태를 current_state변수에 저장합니다.

09: 이전버튼의 상태와 현재버튼의 상태가 다르다면 조건에 만족합니다. 즉 상태가 변경되었다면 조건에 만족합니다. 버튼을 누를때와 눌렀다 떼었을 때 조건에 만족합니다.

10: 이전버튼의 상태에 현재버튼의 값을 저장합니다.

11~12: 버튼을 누를때만 조건문에 만족하여 click을 출력합니다.

13: 버튼을 누를 때 의도적으로 0.1초 가량 기다려 채터링을 방지합니다.

[▶ 현재 스크립트 실행] 아이콘을 클릭하여 코드를 실행합니다.

핀 14에 연결된 디지털 입력 신호를 모니터링하고, 버튼이 눌릴 때마다 "click" 메시지를 출력합니다. 이전 상태와 현재 상태를 비교하여 버튼의 상태 변화를 감지하고 처리합니다.

버튼을 누를 때 한번만 click 이라는 글자가 쉘영역에 출력됩니다.

```
쉘
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  click
  click
```

버튼을 확인하는 코드를 함수처리

버튼을 확인하는 코드를 함수로 만들어 재사용이 가능하도록 코드를 변경합니다.

4-2.버튼입력받기_3.py

```
01    from machine import Pin
02    import time
03
04    key1 = Pin(38, Pin.IN, Pin.PULL_UP)
05    prev_state = key1.value()
06
07    def get_key1():
08        global prev_state
09        current_state = key1.value()
10        if current_state != prev_state:
11            prev_state = current_state
12            if current_state ==0:
13                return 1
14        else:
15            return 0
16
17    while True:
18        if get_key1():
19            print("click")
20            time.sleep(0.1)
```

코드설명

07~15: 버튼의 상태를 확인하여 누를 때 1을 반환하는 함수입니다.

08: 이전상태를 저장하는 변수는 global로 전역변수의 값을 가져와 사용합니다.

18~20: 함수를 이용하여 버튼의 상태를 체크하고 버튼이 눌리면 click을 출력합니다.

[현재 스크립트 실행] 아이콘을 클릭하여 코드를 실행합니다.

버튼을 누르면 click이라는 글자가 쉘영역에 출력됩니다. 버튼을 입력받아 처리하는 부분을 함수화 하였습니다.

```
쉘
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  click
```

여러개의 버튼을 함수처리

여러개의 버튼을 함수로 처리하여 코드를 만들어봅니다.

4-2.버튼입력받기_4.py

```
01  from machine import Pin
02  import time
03
04  key1 = Pin(38, Pin.IN, Pin.PULL_UP)
05  prev_state1 = key1.value()
06  key2 = Pin(39, Pin.IN, Pin.PULL_UP)
07  prev_state2 = key2.value()
08
09  def get_key1():
10      global prev_state1
11      current_state = key1.value()
12      if current_state != prev_state1:
13          prev_state1 = current_state
14          if current_state ==0:
15              return 1
16      else:
17          return 0
18
19  def get_key2():
20      global prev_state2
21      current_state = key2.value()
22      if current_state != prev_state2:
23          prev_state2 = current_state
24          if current_state ==0:
25              return 1
26      else:
27          return 0
28
29  while True:
30      if get_key1():
```

```
31            print("key1 click")
32            time.sleep(0.1)
33
34        if get_key2():
35            print("key2 click")
36            time.sleep(0.1)
```

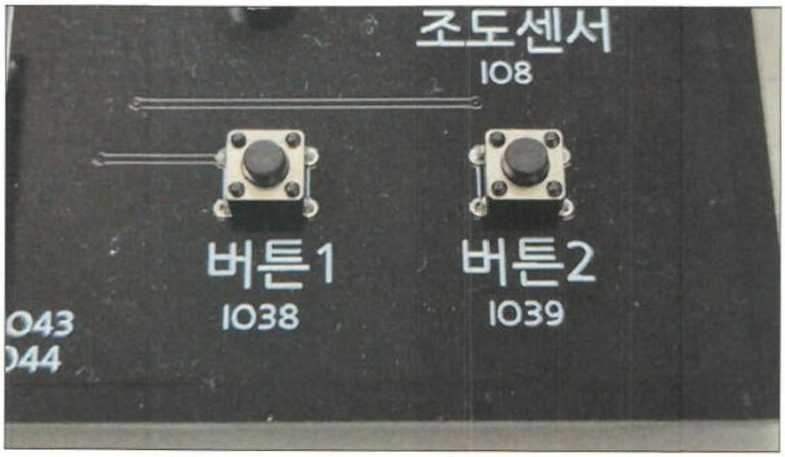
코드설명

09~17: 버튼1의 상태를 확인하는 함수입니다.

19~27: 버튼2의 상태를 확인하는 함수입니다.

[현재 스크립트 실행] 아이콘을 클릭하여 코드를 실행합니다.
1번 버튼, 2번 버튼의 코드를 함수화 하였습니다.

각각의 버튼을 눌러 동작 결과를 확인합니다.



버튼을 누르면 각 버튼에 맞는 값이 한번 출력되었습니다.

```
쉘
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  key1 click
  key2 click
```

버튼을 이용하여 RGB LED의 색상 조절하기

버튼 2개를 이용하여 RGB LED의 색상을 조절해보도록 합니다.

4-2.버튼입력받기_5.py

```
from machine import Pin
import time

key1 = Pin(38, Pin.IN, Pin.PULL_UP)
prev_state1 = key1.value()
key2 = Pin(39, Pin.IN, Pin.PULL_UP)
prev_state2 = key2.value()

LED_RED = Pin(21,Pin.OUT)
LED_GREEN = Pin(47,Pin.OUT)
LED_BLUE = Pin(48,Pin.OUT)

state =0
key_flag =0

LED_RED.value(1)
LED_GREEN.value(0)
LED_BLUE.value(0)

def get_key1():
    global prev_state1
    current_state = key1.value()
    if current_state != prev_state1:
        prev_state1 = current_state
        if current_state ==0:
            return 1
    else:
        return 0

def get_key2():
    global prev_state2
    current_state = key2.value()
    if current_state != prev_state2:
        prev_state2 = current_state
        if current_state ==0:
            return 1
    else:
        return 0

while True:
    if get_key1():
        key_flag =1
        if state >0:
            state = state -1
        print(state)
        time.sleep(0.1)


    if get_key2():
```

```
50            key_flag =1
51            if state <2:
52                state = state +1
53            print(state)
54            time.sleep(0.1)
55
56        if key_flag ==1:
57            key_flag =0
58            if state ==0:
59                LED_RED.value(1)
60                LED_GREEN.value(0)
61                LED_BLUE.value(0)
62            elif state ==1:
63                LED_RED.value(0)
64                LED_GREEN.value(1)
65                LED_BLUE.value(0)
66            elif state ==2:
67                LED_RED.value(0)
68                LED_GREEN.value(0)
69                LED_BLUE.value(1)
```

코드설명

13: state 변수를 초기화하고, 0으로 설정합니다. (LED 상태를 나타내기 위한 변수)

14: key_flag 변수를 초기화하고, 0으로 설정합니다. (키 입력 변화를 감지하기 위한 변수)

16: LED_RED를 켭니다.

17: LED_GREEN을 끕니다.

18: LED_BLUE를 끕니다.

41-46: key1이 눌리면 key_flag를 1로 설정하고, state 변수가 0보다 클 경우 1 감소시킵니다. 현재 state 값을 출력하고, 짧은 지연(0.1초)을 줍니다.

49-54: key2가 눌리면 key_flag를 1로 설정하고, state 변수가 2보다 작을 경우 1 증가시킵니다. 현재 state 값을 출력하고, 짧은 지연을 줍니다.

56-69: key_flag가 1이면 key_flag를 다시 0으로 설정하고, state 값에 따라 LED 색상을 설정합니다.

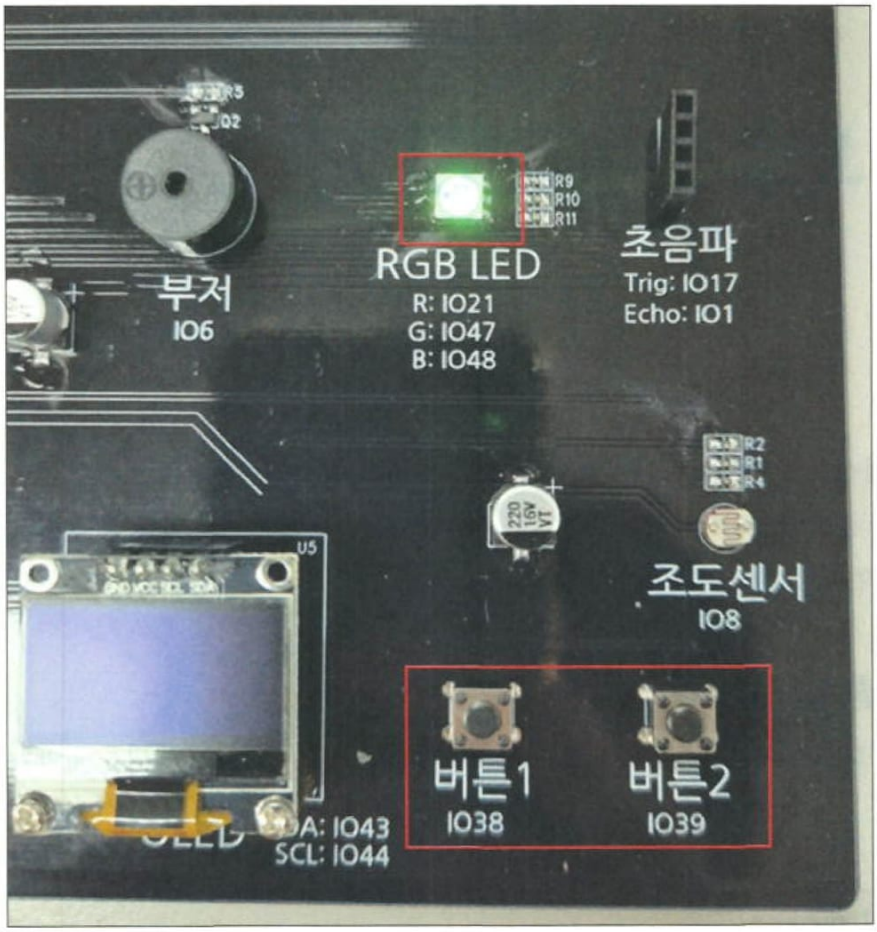
state == 0일 때 빨간색 LED가 켜지고 나머지는 꺼집니다.

state == 1일 때 초록색 LED가 켜지고 나머지는 꺼집니다.

state == 2일 때 파란색 LED가 켜지고 나머지는 꺼집니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

버튼1, 버튼2를 이용하여 RGB LED의 색상을 빨강, 녹색, 파란색으로 조절이 가능합니다.



4-3

부저 출력하기

GPIO를 이용하여 디지털신호의 주파수를 제어할 수 있습니다. 특정 주파수에 따라 부저에서 특정 음이 출력되며, 주파수를 높이면 높은 음, 주파수를 낮추면 낮은 음이 생성됩니다. 이를 통해 간단한 멜로디나 경고음을 구현할 수 있습니다.

도레미파솔라시도 출력하기

피에조부저를 이용하여 "도레미파솔라시도" 소리를 만들어내어 멜로디를 재생하는 방법을 알아봅니다.

4-3.부저 출력하기_1.py

```
01    from machine import Pin, PWM
02    import time
03
04    melody_buzzer = PWM(Pin(6, Pin.OUT), freq=100, duty=0)
05    melody_buzzer.duty(0)
06
07    frequency_list = [261,293,329,349,392,440,493,523]
08
09    try:
10        while True:
11            for frequency in frequency_list:
12                melody_buzzer.duty(512)
13                melody_buzzer.freq(int(frequency))
14                time.sleep_ms(500)
15    except:
16        melody_buzzer.duty(0)
```

코드설명

04: `melody_buzzer` 변수를 생성하고, 핀 6과 연결된 PWM 객체를 만들어 주파수를 1Hz로, 듀티 사이클을 0으로 설정합니다.

05: `melody_buzzer`의 듀티 사이클을 0으로 설정합니다.

07: `frequency_list`라는 리스트를 생성합니다. 이 리스트는 도레미파솔라시도의 주파수를 담고 있습니다.

11~14: 도레미파솔라시도를 반복하며 출력합니다.

[▶ 현재 스크립트 실행] 아이콘을 클릭하여 코드를 실행합니다.

도레미파솔라시도의 음계가 피에조 부저를 통해 출력됩니다. 피에조 부저는 단순한 음계를 출력하는 소자로서 스피커보다는 음질이 부족합니다.

학교종 출력하기

학교종의 음계를 리스트에 담아 출력하는 코드를 작성합니다.

4-3.부저 출력하기_2.py

```
01  from machine import Pin, PWM
02  import time
03
04  melody_buzzer = PWM(Pin(6, Pin.OUT), freq=100, duty=0)
05  melody_buzzer.duty(0)
06
07  MELODY = [392, 392, 440, 440, 392, 392, 330, 392, 392,
08            330, 330, 293, 0, 392, 392, 440, 440, 392,
09            392, 329, 392, 329, 293, 329, 261, 0]
10
11  try:
12      while True:
13          for note_freq in MELODY:
14              if note_freq ==0:
15                  melody_buzzer.duty(0)
16                  time.sleep_ms(250)
17              else:
18                  melody_buzzer.freq(int(note_freq))
19                  melody_buzzer.duty(512)
20                  time.sleep_ms(250)
21                  melody_buzzer.duty(0)
22                  time.sleep_ms(100)
23  except:
24      melody_buzzer.duty(0)
```

코드설명

07: `MELODY`라는 리스트를 생성합니다. 이 리스트는 학교종의 음계를 담고있습니다.

[▶ 현재 스크립트 실행] 아이콘을 클릭하여 코드를 실행합니다.

학교종이 피에조 부저로 출력되었습니다.

버튼2개를 이용하여 도레미 출력하기

버튼2개를 이용하여 버튼의 상태에 따라서 도레미를 출력하는 코드를 작성해봅니다.

4-3.부저 출력하기_3.py

```
01  from machine import Pin, PWM
02  import time
03
04  melody_buzzer = PWM(Pin(6, Pin.OUT), freq=100, duty=0)
05  melody_buzzer.duty(0)
06
07  key1 = Pin(38, Pin.IN, Pin.PULL_UP)
08  key2 = Pin(39, Pin.IN, Pin.PULL_UP)
09
10  try:
11      while True:
12          if key1.value() ==0 and key2.value() ==1:
13              melody_buzzer.duty(512)
14              melody_buzzer.freq(261)
15          elif key1.value() ==1 and key2.value() ==0:
16              melody_buzzer.duty(512)
17              melody_buzzer.freq(293)
18          elif key1.value() ==0 and key2.value() ==0:
19              melody_buzzer.duty(512)
20              melody_buzzer.freq(329)
21          else:
22              melody_buzzer.duty(0)
23  except:
24      melody_buzzer.duty(0)
```

코드설명

12~14: 1번 버튼만 누르면 도를 출력합니다.

15~17: 2번 버튼만 누르면 레를 출력합니다.

18~20: 1,2번 버튼을 동시에 누르면 미를 출력합니다.

21~22: 버튼을 누르지 않을 경우 듀티를 0으로 하여 음을 출력하지 않습니다.

[현재 스크립트 실행] 아이콘을 클릭하여 코드를 실행합니다.

버튼 1만 누르면 도, 버튼 2만 누르면 레, 버튼1,2를 동시에 누르면 미의 음을 출력합니다.

4-4

조도센서 측정

CDS 조도센서는 주변의 밝기를 감지하여 저항값으로 표현합니다. 밝기가 높아질수록 저항값이 낮아지고, 어두워질수록 저항값이 높아지는 특성을 가집니다. 이를 아날로그 입력 핀에 연결하면 ADC(아날로그-디지털 변환)를 통해 밝기 값을 측정할 수 있습니다.

조도센서로 밝기 확인하기

조도센서의 값을 읽어 출력하는 코드를 작성합니다.

4-4.조도센서 측정_1.py

```
01    from machine import Pin
02    from machine import ADC
03    import time
04
05    cds_sensor=ADC(Pin(8))
06    cds_sensor.atten(ADC.ATTN_11DB)
07
08    while True:
09        cds_value = cds_sensor.read()
10        print(cds_value)
11        time.sleep(0.5)
```

코드설명

2: `machine` 라이브러리에서 `ADC` 클래스를 가져옵니다.

4: `cds_sensor` 변수를 만듭니다. 이 변수는 아날로그 디지털 컨버터(ADC)를 나타내며, `Pin(35)`를 사용하여 핀 35를 설정합니다.

5: `cds_sensor`의 감도(atten)를 설정합니다. 11dB 감도로 설정합니다. 이것은 아날로그 입력 신호의 감도를 나타내며, 이 설정에 따라 입력 범위가 조절됩니다.

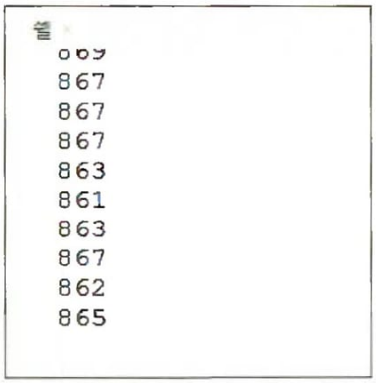
7: 무한 루프를 시작합니다. 이 루프는 계속해서 실행됩니다.

8: `cds_sensor`에서 값을 읽어와 `cds_value` 변수에 저장합니다.

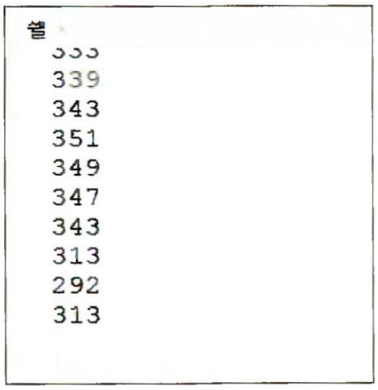
9: `cds_value` 값을 출력합니다.

[▶ 현재 스크립트 실행] 아이콘을 클릭하여 코드를 실행합니다.

조도센서의 값이 쉘 영역에 출력되었습니다.



센서를 손으로 가려 어두워졌을때는 값이 작아졌습니다.



어두워지면 자동으로 켜지는 전등 만들기

조도센서의 값에 따라서 어두워지면 자동으로 켜지는 LED를 만들어봅니다.

4-4.조도센서 측정_2.py

```
01    from machine import Pin
02    from machine import ADC
03    import time
04
05    cds_sensor=ADC(Pin(8))
06    cds_sensor.atten(ADC.ATTN_11DB)
07
08    LED_RED = Pin(21,Pin.OUT)
09    LED_GREEN = Pin(47,Pin.OUT)
10    LED_BLUE = Pin(48,Pin.OUT)
11
12
13    while True:
14        cds_value = cds_sensor.read()
15        print(cds_value)
16
17        if cds_value <500:
18            LED_RED.on()
19            LED_GREEN.on()
20            LED_BLUE.on()
21        else:
22            LED_RED.off()
23            LED_GREEN.off()
24            LED_BLUE.off()
25
26        time.sleep(0.5)
```

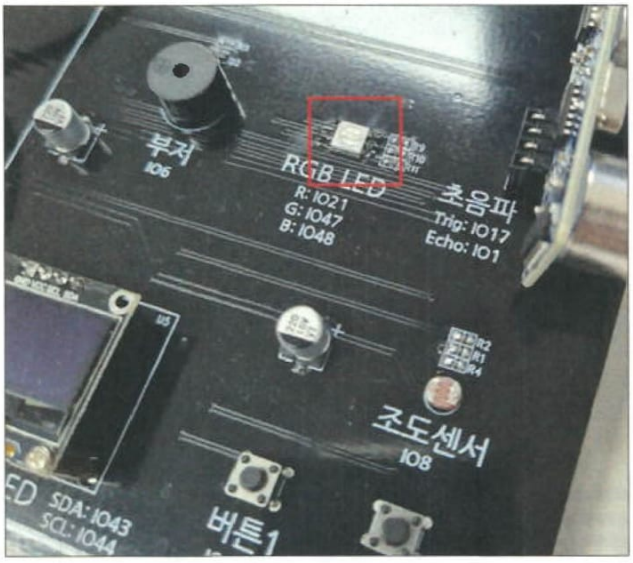
코드설명

17~24: 조도센서의 값이 500보다 작다면 RGB LED를 모두켜서 흰색으로 켜고 그렇지 않다면 500보다 크다면 LED를 끕니다. 즉 어두워지면 LED를 켜고 밝으면 LED를 끕니다.

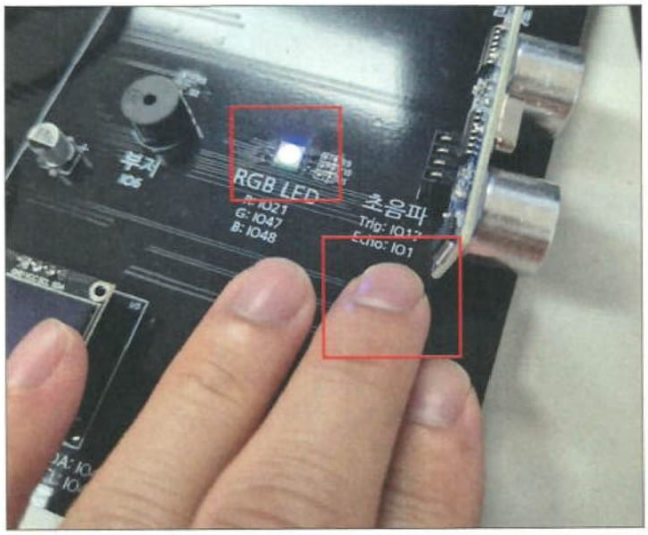
[현재 스크립트 실행] 아이콘을 클릭하여 코드를 실행합니다.

조도센서의 값이 쉘영역에 출력되고 조도센서의 값이 500보다 작으면 LED가 켜집니다.

밝은때는 LED가 꺼져있습니다.



센서를 손으로 가려 어두울때는 LED가 흰색으로 켜집니다.



CHAPTER 05

응용기능 다루기

OLED 디스플레이장치, DHT11 온습도센서 등 라이브러리가 필요로 한 응용부품을 다루어보도록 합니다.

5-1

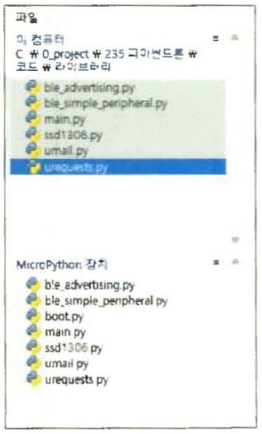
OLED 디스플레이 장치

pyDrone을 활용하여 OLED 디스플레이 장치를 제어해보도록 합니다.

0.96인치 I2C OLED 디스플레이 모듈로, SSD1306 드라이버 칩을 사용하며 128x64 해상도를 제공합니다. 저전력 소모와 높은 시인성을 갖추고 있어 스마트 홈, IoT, 로봇 프로젝트 등에서 간단한 데이터나 그래픽을 표시하는 데 적합합니다. I2C 통신(4핀)을 통해 손쉽게 연결할 수 있습니다.

라이브러리 업로드하기

1장의 시작하기에서 [라이브러리 마이크로파이썬에 업로드하기]를 참고하여 라이브러리를 업로드한다음 진행합니다. OLED를 제어하기 위해서 [ssd1306.py] 라이브러리 파일이 필요로 합니다.



글자출력하기

코드를 작성하여 OLED에 글자를 출력해보도록 합니다.

5-1.OLED 디스플레이 장치_1.py

```
01    from machine import Pin
02    import time
03    from machine import SoftI2C
04    from ssd1306 import SSD1306_I2C
05
06    i2c = SoftI2C(sda=Pin(43), scl=Pin(44))
07    oled = SSD1306_I2C(128, 64, i2c)
08
09    oled.text("hi pyDrone",0,0)
10    oled.show()
```

코드설명

03: SoftI2C: 소프트웨어 방식으로 I2C 통신을 구현하기 위한 클래스입니다.

04: SSD1306_I2C: SSD1306 OLED 디스플레이를 제어하기 위한 라이브러리입니다.

06: SoftI2C 객체 생성: SDA 핀(43)과 SCL 핀(44)을 설정하여 I2C 통신을 초기화합니다.

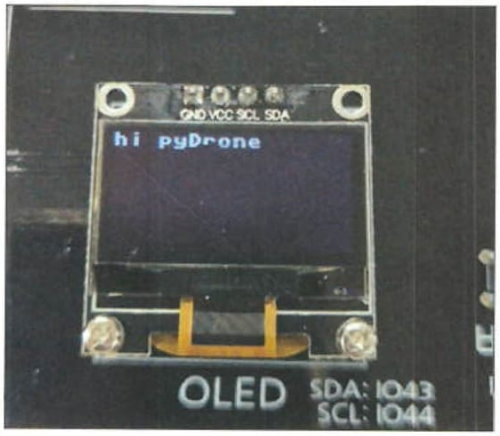
07: SSD1306_I2C 객체 생성: OLED 디스플레이를 제어하기 위해 가로 128, 세로 64 크기로 초기화합니다.

09: 텍스트 출력 설정: "hi pyDrone" 텍스트를 OLED 디스플레이의 (x=0, y=0) 좌표에 출력합니다.

10: 화면 표시: 설정한 내용을 실제 OLED 화면에 표시합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

hi pyDrone 글자가 OLED에 출력되었습니다.



글자 위치변경하여 출력하기

글자의 위치를 변경하여 출력해봅니다. 글자의 시작 좌표를 설정할 수 있습니다.

5-1.OLED 디스플레이 장치_2.py

```
01  from machine import Pin
02  import time
03  from machine import SoftI2C
04  from ssd1306 import SSD1306_I2C
05
06  i2c = SoftI2C(sda=Pin(43), scl=Pin(44))
07  oled = SSD1306_I2C(128, 64, i2c)
08
09  oled.text("hi pyDrone",20,30)
10  oled.show()
```

코드설명

09: 텍스트 출력 설정: "hi pyDrone" 텍스트를 OLED 디스플레이의 (x=20, y=30) 좌표에 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

x좌표로 20픽셀, y좌표로 30픽셀만큼 이동하여 hi pyDrone 글자를 출력하였습니다.

숫자 출력하기

1씩 증가하는 숫자를 출력해보도록 합니다. 숫자를 문자열 형태로 변경하여 출력합니다.

5-1.OLED 디스플레이 장치_3.py

```
01    from machine import Pin
02    import time
03    from machine import SoftI2C
04    from ssd1306 import SSD1306_I2C
05
06    i2c = SoftI2C(sda=Pin(43), scl=Pin(44))
07    oled = SSD1306_I2C(128, 64, i2c)
08
09    cnt =0
10    try:
11        while True:
12            oled.fill(0)
13            oled.text(str(cnt),50,20)
14            oled.show()
15
16            cnt +=1
17            time.sleep(0.2)
18    except:
19        pass
```

코드설명

09: cnt 초기화: 카운터 변수 cnt를 0으로 초기화합니다.

10: try 블록 시작: 프로그램 실행 중 에러가 발생해도 종료되지 않도록 예외 처리를 시작합니다.

11: 무한 루프 시작: 계속 반복적으로 실행되도록 설정합니다.

12: oled.fill(0): OLED 화면을 초기화(검은 화면)합니다.

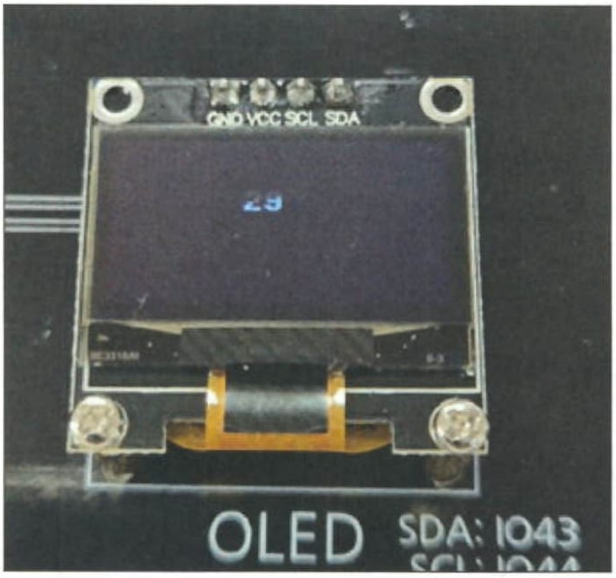
13: oled.text(str(cnt),50,20): cnt 값을 문자열로 변환해 OLED 화면의 (x=50, y=20) 위치에 표시합니다.

18: except 블록 시작: 예외 발생 시 실행됩니다.

19: pass: 예외 처리 시 아무 작업도 하지 않고 루프를 종료합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

숫자를 문자열로 변환후 출력하였습니다. 숫자값이 계속 증가하면서 출력됩니다.



선 및 네모 그리기

oled 라이브러리의 기능인 선, 네모를 그려보도록 합니다.

5-1.OLED 디스플레이 장치_4.py

```
01  from machine import Pin
02  import time
03  from machine import SoftI2C
04  from ssd1306 import SSD1306_I2C
05
06  i2c = SoftI2C(sda=Pin(43), scl=Pin(44))
07  oled = SSD1306_I2C(128, 64, i2c)
08
09  oled.fill(0)
10  oled.hline(0, 32, 127, 1)
11  oled.vline(63, 0, 63, 1)
12  oled.line(0, 0, 127, 63, 1)
13  oled.rect(10, 10, 107, 43, 1)
14  oled.fill_rect(20, 20, 30, 30, 1)
15  oled.show()
```

코드설명

10: oled.hline(0, 32, 127, 1): OLED 화면의 (x=0, y=32) 위치에서 길이 127 픽셀의 수평선을 그립니다.

11: oled.vline(63, 0, 63, 1): OLED 화면의 (x=63, y=0) 위치에서 길이 63 픽셀의 수직선을 그립니다.

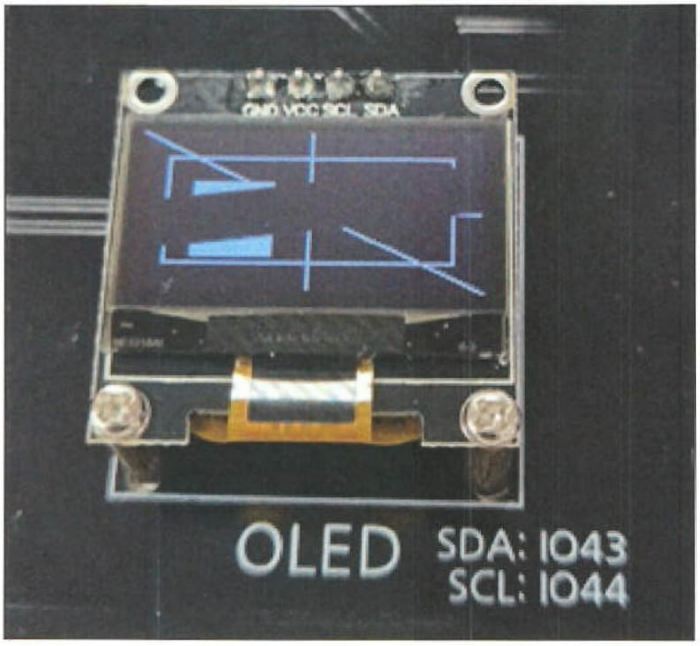
12: oled.line(0, 0, 127, 63, 1): OLED 화면의 (x=0, y=0)에서 (x=127, y=63)까지 대각선을 그립니다.

13: oled.rect(10, 10, 107, 43, 1): OLED 화면의 (x=10, y=10)에서 너비 107, 높이 43의 사각형을 그립니다.

14: oled.fill_rect(20, 20, 30, 30, 1): OLED 화면의 (x=20, y=20)에서 너비 30, 높이 30의 사각형 내부를 채웁니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

가로선, 세로선, 네모, 꽉찬네모가 출력되었습니다.



5-2

DHT11 온도습도 센서값 읽기

DHT11 온습도센서를 이용하여 온도와 습도값을 읽는 방법을 알아봅니다.

DHT11 온습도 센서는, 주변의 온도와 습도를 측정할 수 있는 디지털 센서입니다. 정확도는 적당하지만 저렴하고 사용이 쉬워 스마트 홈, 환경 모니터링 프로젝트 등에 적합합니다.

DHT11온습도센서 값 읽어 출력하기

DHT11온습도 센서값을 읽어 출력하는 코드를 작성합니다.

5-2.DHT11온습도 센서 값 읽기_1.py

```
01    from machine import Pin
02    import time
03    import dht
04
05    dht11=dht.DHT11(Pin(18))
06
07    try :
08        while True:
09            dht11.measure()
```

```
10            temp = dht11.temperature()
11            humi = dht11.humidity()
12            print(f"온도:{temp}°C  습도:{humi}RH")
13            time.sleep(1.0)
14
15    except :
16        pass
```

코드설명

03: dht: DHT11/DHT22 센서를 제어하기 위한 라이브러리입니다.

05: DHT11 객체 생성: 핀 18에 연결된 DHT11 센서를 초기화합니다.

09: dht11.measure(): DHT11 센서로부터 데이터를 측정합니다.

10: 온도 읽기: 측정된 온도 값을 temp 변수에 저장합니다.

11: 습도 읽기: 측정된 습도 값을 humi 변수에 저장합니다.

12: 출력: 온도와 습도를 화면에 출력합니다. 형식은 온도:xx°C 습도:xxRH입니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

DHT11센서에서 온도와 습도값을 읽어 출력하였습니다.

```
셸
'''

MPY: soft reboot
온도:19°C  습도:60RH
온도:19°C  습도:60RH
온도:19°C  습도:60RH
온도:19°C  습도:60RH
온도:19°C  습도:60RH
```

예외처리하기

DHT11센서의 경우 값을 읽을 때 에러가 발생하는 경우가 빈번하여 값을 읽어올 때 에러가 발생하면 예외처리를 하는 코드를 추가합니다.

5-2.DHT11온습도 센서 값 읽기_2.py

```
01    from machine import Pin
02    import time
```

```
03  import dht
04
05  dht11=dht.DHT11(Pin(18))
06
07  try :
08      while True:
09          dht11.measure()
10          temp = dht11.temperature()
11          humi = dht11.humidity()
12          if (temp ==None) or (humi ==None):
13              print("센서 에러")
14          else:
15              print(f"온도:{temp}°C  습도:{humi}RH")
16          time.sleep(1.0)
17
18  except :
19      pass
```

코드설명

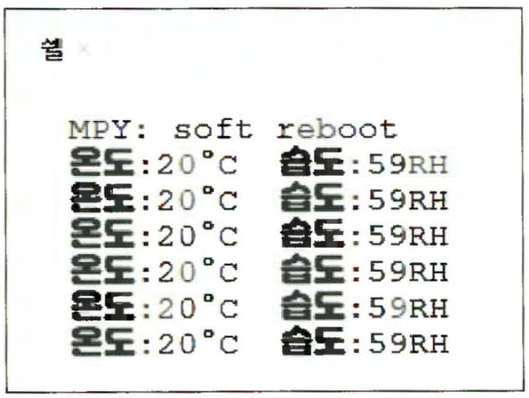
12: 센서 값 검증: 온도(temp) 또는 습도(humi) 값이 None일 경우 센서 에러로 간주합니다.

13: 에러 메시지 출력: 센서 값이 없을 경우 "센서 에러" 메시지를 출력합니다.

15: 정상 데이터 출력: 온도와 습도를 화면에 출력합니다. 형식은 온도:xx°C 습도:xxRH입니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

DHT11에서 읽은 온습도 값이 정상적일때만 출력합니다.



OLED에 온습도 값 표시

온도와 습도를 측정하여 OLED에 값을 표시하는 코드를 작성해봅니다.

5-2.DHT11온습도 센서 값 읽기_3.py

```
01  from machine import Pin
02  import time
03  import dht
04  from machine import SoftI2C
05  from ssd1306 import SSD1306_I2C
06
07  dht11=dht.DHT11(Pin(18))
08
09  i2c = SoftI2C(sda=Pin(43), scl=Pin(44))
10  oled = SSD1306_I2C(128, 64, i2c)
11
12  try :
13      while True:
14          dht11.measure()
15          temp = dht11.temperature()
16          humi = dht11.humidity()
17          if (temp ==None) or (humi ==None):
18              print("센서 에러")
19          else:
20              print(f"온도:{temp}°C  습도:{humi}RH")
21
22              oled.fill(0)
23              oled.text("temp:"+str(temp)+"c",10,10)
24              oled.text("humi:"+str(humi)+"%",10,20)
25              oled.show()
26          time.sleep(1.0)
27
28  except :
29      pass
```

코드설명

04: SoftI2C: 소프트웨어 방식으로 I2C 통신을 구현하기 위한 클래스입니다.

05: SSD1306_I2C: SSD1306 OLED 디스플레이를 제어하기 위한 라이브러리입니다.

09: SoftI2C 객체 생성: SDA 핀(43)과 SCL 핀(44)을 설정하여 I2C 통신을 초기화합니다.

10: SSD1306_I2C 객체 생성: OLED 디스플레이를 제어하기 위해 가로 128, 세로 64 크기로 초기화합니다.

22: oled.fill(0): OLED 화면을 초기화(검은 화면)합니다.

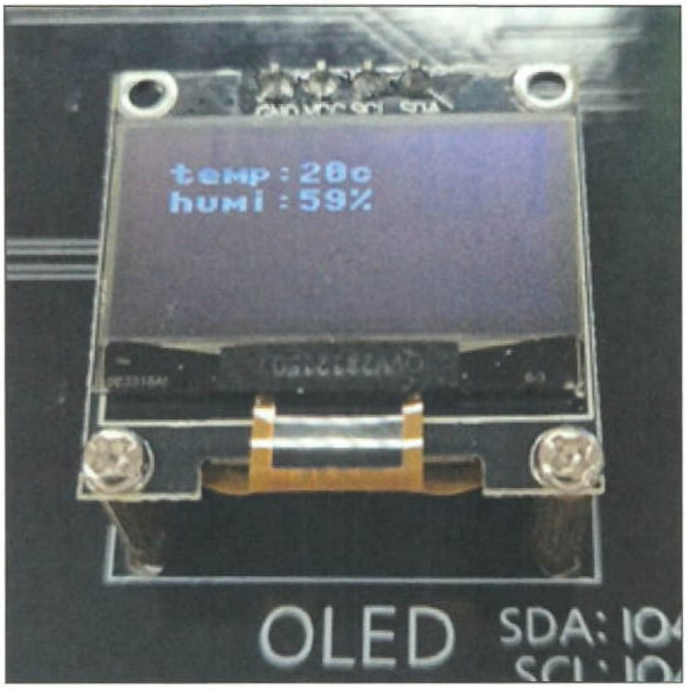
23: oled.text("temp:"+str(temp)+"c",10,10): OLED 화면의 (x=10, y=10) 위치에 온도 값을 출력합니다.

24: oled.text("humi:"+str(humi)+"%",10,20): OLED 화면의 (x=10, y=20) 위치에 습도 값을 출력합니다.

25: oled.show(): 설정된 내용을 실제 OLED 화면에 표시합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

온도와 습도값이 OLED에 출력되었습니다.



5-3

초음파센서 값 읽기

HC-SR04는 초음파를 이용해 물체와의 거리를 측정하는 센서입니다. Trig 핀에서 초음파 신호를 발사하고, 물체에 반사된 초음파가 Echo 핀으로 돌아오는 시간을 측정하여 거리를 계산합니다. 주로 로봇 공학, 장애물 감지, 자동문 제어 등 다양한 IoT 및 자동화 시스템에서 사용됩니다.

초음파센서로 거리값 측정하기

초음파센서를 이용하여 거리값을 측정하여 cm단위로 출력하는 코드를 작성합니다.

5-3.초음파센서 값 읽기_1.py

```
01    import machine
02    import time
03    from machine import Pin
04
05    trigger = Pin(17, mode=Pin.OUT, pull=None)
06    echo = Pin(1, mode=Pin.IN, pull=None)
07    trigger.value(0)
08
09    try:
10        while True:
```

```
11            trigger.value(0)
12            time.sleep_us(5)
13            trigger.value(1)
14            time.sleep_us(10)
15            trigger.value(0)
16
17            pulse_time = machine.time_pulse_us(echo, 1, 30000)
18            distance_cm = (pulse_time /2) /29.1
19            if 2 <= distance_cm <=200:
20                print(distance_cm,"cm")
21
22            time.sleep(0.1)
23
24    except KeyboardInterrupt:
25        pass
```

코드설명

05: trigger 핀 초기화: 핀 17을 출력 모드로 설정하여 초음파 센서의 트리거 신호를 제어합니다.

06: echo 핀 초기화: 핀 1을 입력 모드로 설정하여 초음파 센서의 에코 신호를 읽습니다.

07: trigger.value(0): 트리거 핀의 초기 상태를 LOW로 설정합니다.

11: trigger.value(0): 트리거 핀을 LOW로 설정하여 안정화합니다.

12: time.sleep_us(5): 5마이크로초 동안 대기합니다.

13: trigger.value(1): 트리거 핀을 HIGH로 설정하여 초음파 신호를 발생시킵니다.

14: time.sleep_us(10): 트리거 핀을 HIGH 상태로 유지하며 10마이크로초 동안 대기합니다.

15: trigger.value(0): 트리거 핀을 다시 LOW로 설정합니다.

17: machine.time_pulse_us(echo, 1 , 30000): 에코 핀에서 HIGH 신호가 유지된 시간을 마이크로초 단위로 측정합니다. 최대 대기 시간은 30,000마이크로초입니다.

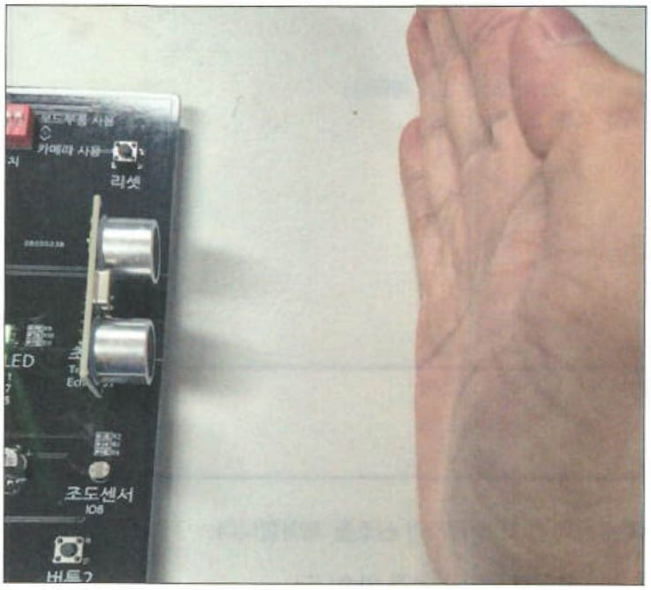
18: 거리 계산: 초음파 신호의 왕복 시간을 기반으로 거리를 계산합니다. 거리는 (pulse_time / 2) / 29.1로 계산됩니다.

19: 거리 검증: 계산된 거리가 2cm 이상 200cm 이하인 경우 유효한 거리로 간주합니다.

20: 거리 출력: 계산된 거리를 cm 단위로 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

초음파센서를 손이나 물건으로 가려 거리를 측정합니다.



초음파센서에서 거리를 측정하여 cm 단위로 출력하였습니다.

```
쉘
  30.65292 cm
  18.78007 cm
  17.52577 cm
  16.35739 cm
  17.68041 cm
  19.17526 cm
  18.40206 cm
  16.97595 cm
```

함수로 만들어서 편리하게 사용하기

초음파센서를 이용하여 거리를 측정하고 측정된 거리값을 반환하는 함수를 만들어 코드에서 거리값이 필요할 때 호출하여 편리하게 사용하도록 합니다.

5-3.초음파센서 값 읽기_2.py

```
01  import machine
02  import time
03  from machine import Pin
04
05  trigger = Pin(17, mode=Pin.OUT, pull=None)
06  echo = Pin(1, mode=Pin.IN, pull=None)
07  trigger.value(0)
08
09  def get_distance_cm():
10      trigger.value(0)
11      time.sleep_us(5)
12      trigger.value(1)
13      time.sleep_us(10)
14      trigger.value(0)
15
16      pulse_time = machine.time_pulse_us(echo, 1, 30000)
17      distance_cm = (pulse_time /2) /29.1
18      if 2 <= distance_cm <=200:
19          return distance_cm
20      else:
21          return 0
22
23  try:
24      while True:
25          distance_cm = get_distance_cm()
26          if(distance_cm !=0):
27              print(distance_cm,"cm")
28          time.sleep(0.1)
29
30  except KeyboardInterrupt:
31      pass
```

코드설명

09~21: 거리 측정 함수 정의: 초음파 센서를 통해 거리를 측정하고 결과를 반환하는 함수입니다.

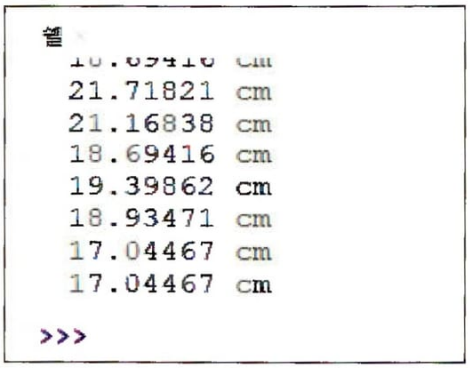
25: get_distance_cm() 함수를 호출하여 거리를 측정합니다.

26~27: 측정된 거리가 0이 아니면 거리를 cm 단위로 출력합니다.

28: 0.1초 동안 대기하여 측정 주기를 조절합니다.

[▶ 현재 스크립트 실행] 아이콘을 클릭하여 코드를 실행합니다.

함수를 이용하여 코드의 재사용성을 높혀 거리를 측정하였습니다.



가까이 다가오면 부저 출력

초음파센서에서 측정한 거리가 설정한 거리보다 가까우면 부저를 울리도록 합니다.

5-3.초음파센서 값 읽기_3.py

```
01  import machine
02  import time
03  from machine import Pin, PWM
04
05  trigger = Pin(17, mode=Pin.OUT, pull=None)
06  echo = Pin(1, mode=Pin.IN, pull=None)
07  trigger.value(0)
08
09  melody_buzzer = PWM(Pin(6, Pin.OUT), freq=300, duty=0)
10  melody_buzzer.duty(0)
11
12  def get_distance_cm():
13      trigger.value(0)
14      time.sleep_us(5)
15      trigger.value(1)
16      time.sleep_us(10)
17      trigger.value(0)
18
19      pulse_time = machine.time_pulse_us(echo, 1, 30000)
20      distance_cm = (pulse_time /2) /29.1
21      if 2 <= distance_cm <=200:
22          return distance_cm
23      else:
24          return 0
25
26  try:
27      while True:
28          distance_cm = get_distance_cm()
29          if 2 <= distance_cm <=10:
30              print(distance_cm,"cm")
31              melody_buzzer.duty(512)
32              melody_buzzer.freq(300)
33              time.sleep_ms(500)
34          else:
35              melody_buzzer.duty(0)
36
37          time.sleep(0.1)
38
39
40  except KeyboardInterrupt:
41      melody_buzzer.duty(0)
```

코드설명

03: Pin, PWM: 핀 제어 및 PWM(Pulse Width Modulation) 출력을 위한 클래스입니다.

05: trigger 핀 초기화: 핀 17을 출력 모드로 설정하여 초음파 센서의 트리거 신호를 제어합니다.

09: melody_buzzer 초기화: 핀 6을 PWM 출력으로 설정하고 초기 주파수를 300Hz로 설정합니다.

10: melody_buzzer.duty(0): 부저의 초기 출력 강도를 0으로 설정하여 소리를 끕니다.

29~33: 측정된 거리가 2cm에서 10cm 사이일 경우, 거리를 출력하고 부저를 활성화합니다.

31: 부저의 출력 강도를 512로 설정하여 소리를 켭니다.

32: 부저의 주파수를 300Hz로 설정합니다.

33: 500ms 동안 부저를 울립니다.

35: 거리가 범위를 벗어나면 부저를 끕니다(duty=0).

40: except KeyboardInterrupt: 키보드 인터럽트(CTRL+C)가 발생했을 때 프로그램을 안전하게 종료합니다.

41: melody_buzzer.duty(0): 부저 출력을 종료합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

초음파센서에 10cm 이하의 값이 들어오면 그 값을 출력하고 부저에 300Hz의 주파수음인 삐~~를 출력합니다.

```
셸 ×

>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  4.5189 cm
  5.773196 cm
```

초음파센서 피아노 만들기

거리에따라서 부저의 주파수를 변경하여 동작하는 피아노를 만들어봅니다.

5-3.초음파센서 값 읽기_4.py

```
import machine
import time
from machine import Pin, PWM

trigger = Pin(17, mode=Pin.OUT, pull=None)
echo = Pin(1, mode=Pin.IN, pull=None)
trigger.value(0)

melody_buzzer = PWM(Pin(6, Pin.OUT), freq=300, duty=0)
melody_buzzer.duty(0)

def get_distance_cm():
    trigger.value(0)
    time.sleep_us(5)
    trigger.value(1)
    time.sleep_us(10)
    trigger.value(0)

    pulse_time = machine.time_pulse_us(echo, 1, 30000)
    distance_cm = (pulse_time /2) /29.1
    if 2 <= distance_cm <=200:
        return distance_cm
    else:
        return 0

try:
    while True:
        distance_cm = get_distance_cm()
        if 2 <= distance_cm <4:
            melody_buzzer.freq(261)  # 도
            melody_buzzer.duty(512)
            time.sleep_ms(200)
        elif 4 <= distance_cm <6:
            melody_buzzer.freq(293)  # 레
            melody_buzzer.duty(512)
            time.sleep_ms(200)
        elif 6 <= distance_cm <8:
            melody_buzzer.freq(329)  # 미
            melody_buzzer.duty(512)
            time.sleep_ms(200)
        elif 8 <= distance_cm <10:
            melody_buzzer.freq(349)  # 파
            melody_buzzer.duty(512)
            time.sleep_ms(200)
        elif 10 <= distance_cm <12:
            melody_buzzer.freq(392)  # 솔
            melody_buzzer.duty(512)
            time.sleep_ms(200)
        elif 12 <= distance_cm <14:
```

```
50                melody_buzzer.freq(440)  # 라
51                melody_buzzer.duty(512)
52                time.sleep_ms(200)
53            elif 14 <= distance_cm <16:
54                melody_buzzer.freq(493)  # 시
55                melody_buzzer.duty(512)
56                time.sleep_ms(200)
57            elif 16 <= distance_cm <=18:
58                melody_buzzer.freq(523)  # 높은 도
59                melody_buzzer.duty(512)
60                time.sleep_ms(200)
61            else:
62                melody_buzzer.duty(0)
63
64            time.sleep(0.1)
65
66
67    except KeyboardInterrupt:
68        melody_buzzer.duty(0)
```

코드설명

28: get_distance_cm() 함수를 호출하여 거리를 측정합니다.

29~60: 측정된 거리를 기반으로 각 구간에 대해 부저 음계를 설정합니다.

29~32: 2cm ~ 4cm: 도(261Hz)

33~36: 4cm ~ 6cm: 레(293Hz)

37~40: 6cm ~ 8cm: 미(329Hz)

41~44: 8cm ~ 10cm: 파(349Hz)

45~48: 10cm ~ 12cm: 솔(392Hz)

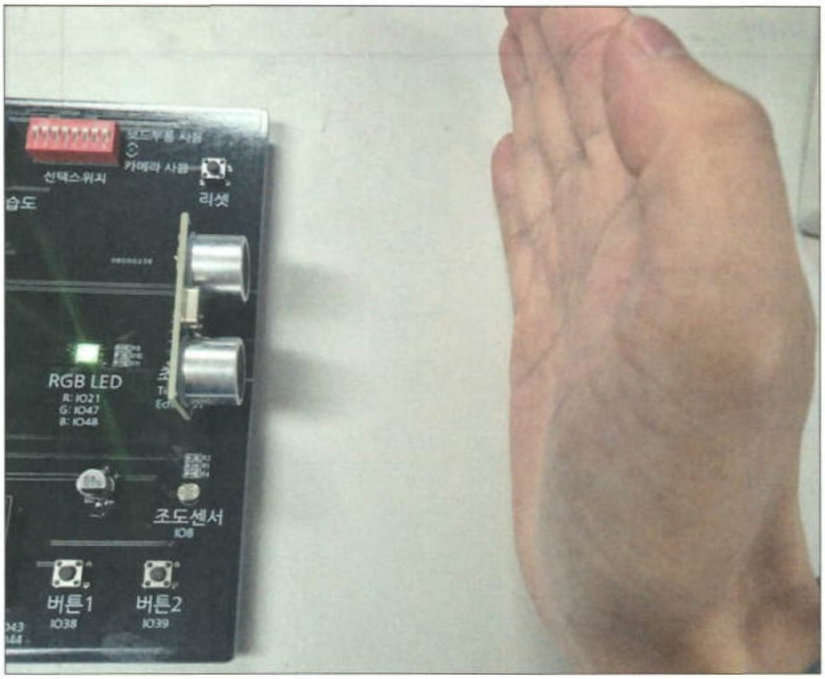
49~52: 12cm ~ 14cm: 라(440Hz)

53~56: 14cm ~ 16cm: 시(493Hz)57~60: 16cm ~ 18cm: 높은 도(523Hz)

62: 거리가 범위를 벗어나면 부저를 끕니다(duty=0).

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

초음파센서를 손으로 가려 거리별로 음계를 다르게 출력하는 초음파센서 피아노를 완성하였습니다.



초음파센서값을 OLED에 출력하기

초음파센서로 측정한 거리값을 OLED에 출력해보록 합니다.

5-3.초음파센서 값 읽기_5.py

```
01  import machine
02  import time
03  from machine import Pin
04  from machine import SoftI2C
05  from ssd1306 import SSD1306_I2C
06
07  trigger = Pin(17, mode=Pin.OUT, pull=None)
08  echo = Pin(1, mode=Pin.IN, pull=None)
09  trigger.value(0)
10
11  i2c = SoftI2C(sda=Pin(43), scl=Pin(44))
12  oled = SSD1306_I2C(128, 64, i2c)
13
14  def get_distance_cm():
15      trigger.value(0)
16      time.sleep_us(5)
17      trigger.value(1)
18      time.sleep_us(10)
19      trigger.value(0)
20
21      pulse_time = machine.time_pulse_us(echo, 1, 30000)
22      distance_cm = (pulse_time /2) /29.1
23      if 2 <= distance_cm <=200:
24          return distance_cm
25      else:
26          return 0
27
28  try:
29      while True:
30          distance_cm = get_distance_cm()
31          if distance_cm!=0:
32              oled.fill(0)
33              oled.text("distance:"+str(round(distance_cm)),10,30)
34              oled.show()
35          time.sleep(0.1)
36
37
38  except KeyboardInterrupt:
39      pass
```

코드설명

29~35: 거리 측정 및 OLED 표시 루프:

30: get_distance_cm() 함수를 호출하여 거리를 측정합니다.

31~34: 측정된 거리가 0이 아니면 OLED 화면에 표시합니다.

32: oled.fill(0): OLED 화면을 초기화합니다.

33: oled.text: 측정된 거리를 "distance" 형식으로 (x=10, y=30) 위치에 출력합니다.

34: oled.show(): OLED 화면에 출력 내용을 표시합니다.

35: time.sleep(0.1): 0.1초 동안 대기합니다.

38: except KeyboardInterrupt: 키보드 인터럽트(CTRL+C)가 발생했을 때 프로그램을 안전하게 종료합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

초음파센서로 측정한 거리값을 OLED에 표시하였습니다.

CHAPTER 06

블루투스 사물인터넷

pyDrone의 블루투스 통신을 활용하여 스마트폰과 블루투스 통신을 통한 다양한 사물인터넷 장치를 만들어봅니다.

6-1

블루투스 통신으로 데이터 주고받기

pyDrone은 ESP32-S3 모델의 CPU를 사용하고 있습니다. 2.4GHz의 통신을 지원하여 블루투스와 WiFI통신을 지원합니다. 이 챕터에서는 블루투스 통신을 이용하여 무선으로 데이터를 주고 받는 방법에 대해서 알아봅니다.

라이브러리 설치

1장의 시작하기에서 [라이브러리 마이크로파이썬에 업로드하기]를 참고하여 라이브러리를 업로드 한다음 진행합니다. 블루투스 통신을 위해서 [ble_advertising.py],[ble_simple_peripheral.py] 라이브러리 파일이 필요로 합니다.

블루투스 통신으로 데이터 주고받기

블루투스 통신으로 1초마다 증가되는 숫자값을 전송하고 데이터를 수신하여 출력하는 코드를 작성해봅니다.

6-1.블루투스 통신_1.py

```
01    import bluetooth,ble_simple_peripheral
02    import time
03
04    ble = bluetooth.BLE()
05    p = ble_simple_peripheral.BLESimplePeripheral(ble,name='pyD_1234')
06
07    def on_rx(read):
08        print("RX:", read)
09
10    p.on_write(on_rx)
11
12    cnt =0
13    try:
14        while True:
15            if p.is_connected():
16                p.send(str(cnt)+"\r\n")
17                cnt +=1
18            time.sleep(1.0)
19    except :
20        pass
```

코드설명

01: bluetooth, ble_simple_peripheral: 블루투스 통신과 BLE 주변 장치 기능을 위한 라이브러리를 가져옵니다.

02: time: 시간 지연을 제공하는 모듈입니다.

04: BLE 객체 생성: 블루투스 LE(BLE)를 초기화하기 위해 BLE() 객체를 생성합니다.

05: BLESimplePeripheral 객체 생성: BLE 주변 장치 객체를 생성합니다. 이름은 'pyD_1234'로 설정됩니다.

07~08: on_rx 함수 정의:

07: 데이터를 수신했을 때 호출되는 콜백 함수입니다.

08: 수신된 데이터를 출력합니다.

10: p.on_write(on_rx): 데이터를 수신했을 때 실행할 콜백 함수를 on_rx로 설정합니다.

12: cnt 초기화: 전송 횟수를 기록하는 카운터 변수를 0으로 초기화합니다.

13~19: try-except 블록: 예외 발생 시 프로그램이 종료되지 않도록 설정합니다.

1418: BLE 연결 상태 확인 및 데이터 전송 루프:

15: p.is_connected()를 사용하여 BLE 장치가 연결 상태인지 확인합니다.

16: BLE로 카운터 값을 문자열 형태로 전송하며, 전송 후 줄 바꿈을 추가합니다.

17: cnt 값을 1씩 증가시킵니다.

18: time.sleep(1.0): 1초 동안 대기합니다.

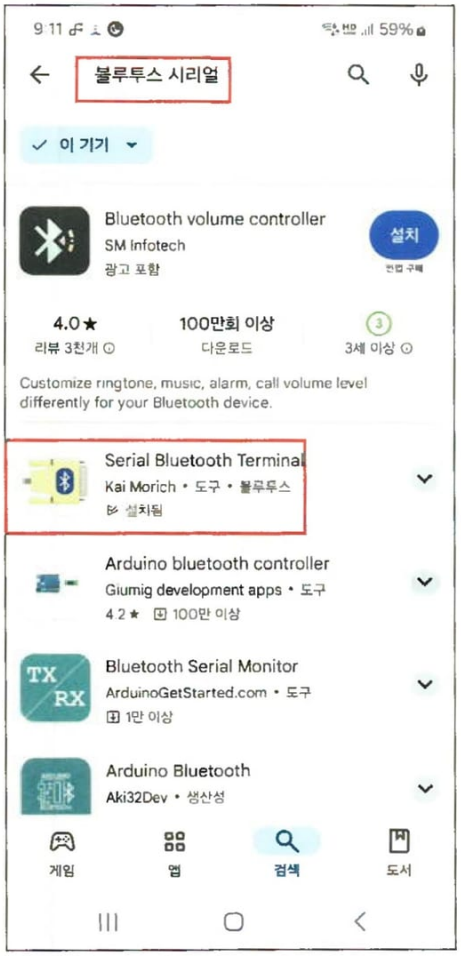
19~20: 예외 처리:

19: except: 예외가 발생했을 경우 실행됩니다.

20: pass: 예외 발생 시 아무 작업도 하지 않고 프로그램을 종료합니다.

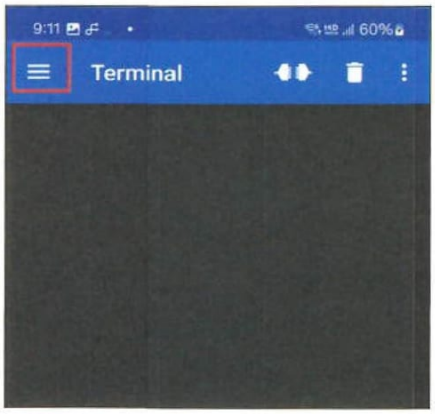
[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

블루투스로 데이터를 보내거나 받기위해서는 스마트폰의 앱이 필요합니다. 안드로이드 스마트폰의 플레이스토어에서 "블루투스 시리얼"을 검색 후 [Serial Blutooth Terminal]앱을 설치합니다. 블루투스 통신을 이용한 시리얼 통신 앱입니다.

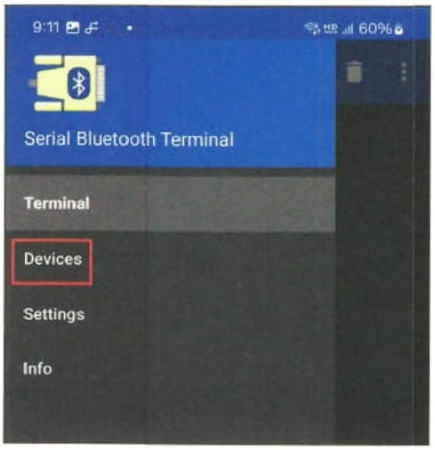


설치 후 앱을 실행합니다. 스마트폰의 블루투스와 권한은 설정되어야 합니다.

왼쪽위의 [설정] 부분을 클릭합니다.



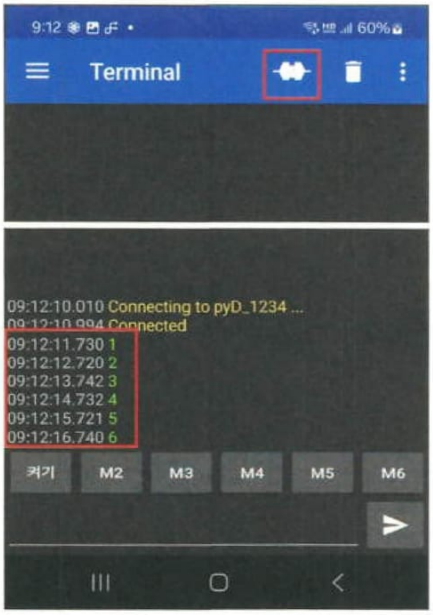
[Devices] 탭을 선택합니다.



[Bluetooth LE]탭을 선택 후 [SCAN]을 클릭한다음 "pyD_1234"를 선택합니다. pyD_1234는 코드에서 설정한 이름으로 여러대가 같은환경에 있다면 코드에서 이름을 변경 후 변경된 이름으로 선택합니다.

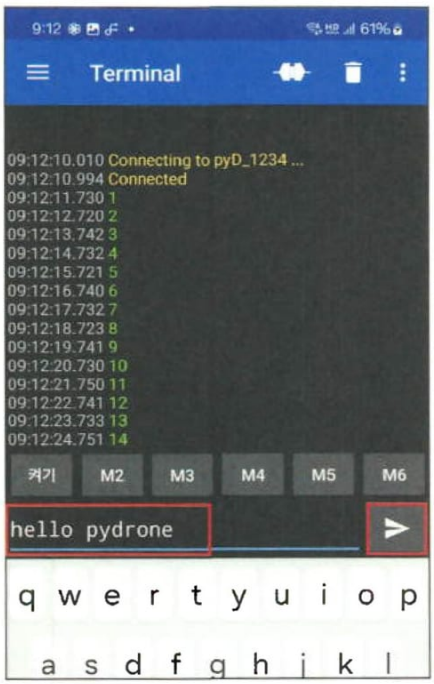
연결 아이콘이 연결된 상태면 정상적으로 연결되었습니다. 1초마다 증가된 숫자를 스마트폰에서 수신받았습니다.

한번 연결하면 매번 디바이스를 찾아 연결하지 않아도 연결 아이콘을 클릭하면 마지막에 연결된 블루투스 기기와 자동 연결됩니다.



데이터를 pyDrone으로 보내기 위해서 보낼데이터를 입력 후 전송 버튼을 클릭합니다.

* 단 영어가 아닌 글자를 보낼 경우 pyDrone에서 처리하지 못해 에러가 발생 합니다.



Thonny IDE를 확인합니다.

pyDrone에서 수신받은 데이터를 출력하였습니다.

```
셸
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  MAC:  f4:12:fa:e2:f5:42
  Starting advertising
  New connection 1
  RX: b'hello pydrone\r\n'
```

온도습도 데이터 측정하여 블루투스 통신으로 데이터 전송하기

DHT11 센서를 이용하여 온도와 습도를 측정하여

6-1.블루투스 통신_2.py

```
01  import bluetooth,ble_simple_peripheral
02  import time
03  from machine import Pin
04  import dht
05
06  ble = bluetooth.BLE()
07  p = ble_simple_peripheral.BLESimplePeripheral(ble,name='pyD_1234')
08
09  dht11=dht.DHT11(Pin(18))
10
11  def on_rx(read):
12      print("RX:", read)
13
14  p.on_write(on_rx)
15
16  try:
17      while True:
18          if p.is_connected():
19              dht11.measure()
20              temp = dht11.temperature()
21              humi = dht11.humidity()
22              if (temp ==None) or (humi ==None):
23                  print("센서 에러")
24              else:
25                  send_str =str(temp) +","+str(humi) +"\r\n"
26                  p.send(send_str)
27          time.sleep(2.0)
28
29  except :
30      pass
```

코드설명

04: dht: DHT11 온습도 센서를 제어하기 위한 라이브러리입니다.

09: DHT11 객체 생성: 핀 18에 연결된 DHT11 센서를 초기화합니다.

18: p.is_connected()를 사용하여 BLE 장치가 연결 상태인지 확인합니다.

19: DHT11 센서를 측정합니다.

20: 측정된 온도 값을 temp 변수에 저장합니다.

21: 측정된 습도 값을 humi 변수에 저장합니다.

22~23: 온도(temp) 또는 습도(humi) 값이 None이면 "센서 에러" 메시지를 출력합니다.

25: 온도와 습도 값을 문자열로 변환하여 send_str 변수에 저장합니다.

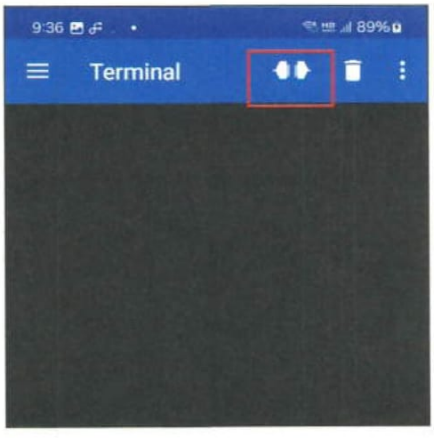
데이터 형식: "온도,습도\r\n"

26: send_str 데이터를 BLE를 통해 전송합니다.

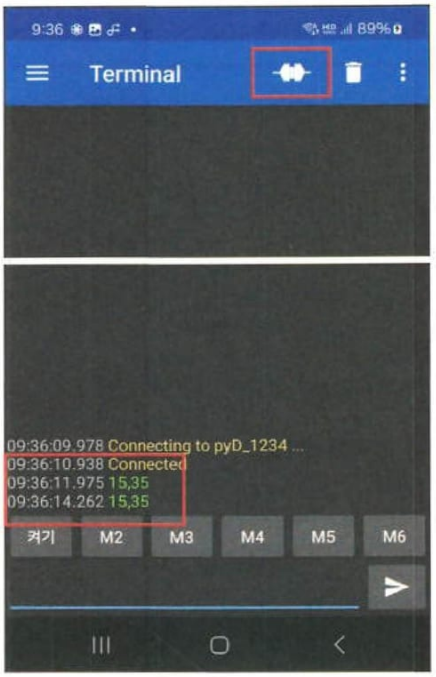
27: 2초 동안 대기하여 측정 주기를 조절합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

[연결] 아이콘을 클릭하여 마지막에 연결된 블루투스와 연결합니다. 이전에 연결되었기 때문에 아이콘만 클릭하여 재 연결이 가능합니다.



연결된 후 온도와 습도값이 전송되어 출력되었습니다.



time.sleep를 사용하지 않고 2초마다 데이터 전송하기

time.sleep,을 사용하면 코드에 지연이 생겨 다양한 작업을 위한 코드를 추가하기 어렵습니다. time.sleep 대신에 time.time() 함수를 이용하여 일정시간 마다 동작하도록 코드를 수정해봅니다.

6-1.블루투스 통신_3.py

```
01    import bluetooth, ble_simple_peripheral
02    import time
03    from machine import Pin
04    import dht
05
06    ble = bluetooth.BLE()
07    p = ble_simple_peripheral.BLESimplePeripheral(ble, name='pyD_1234')
08
09    dht11 = dht.DHT11(Pin(18))
10
11    def on_rx(read):
12        print("RX:", read)
13
14    p.on_write(on_rx)
15
16    previous_time = time.time()
17    send_interval =2.0
18
```

```
19  try:
20      while True:
21          current_time = time.time()
22          if current_time - previous_time >= send_interval:
23              previous_time = current_time
24              if p.is_connected():
25                  dht11.measure()
26                  temp = dht11.temperature()
27                  humi = dht11.humidity()
28                  if (temp is None) or (humi is None):
29                      print("센서 에러")
30                  else:
31                      send_str =str(temp) +","+str(humi) +"\r\n"
32                      p.send(send_str)
33  except :
34      pass
```

코드설명

16: previous_time 초기화: 이전 데이터 전송 시간을 기록하는 변수를 현재 시간으로 초기화합니다.

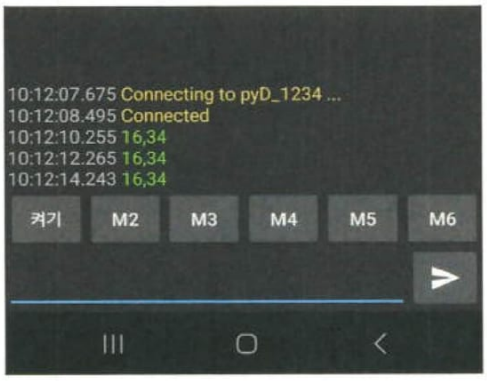
17: send_interval 설정: 데이터 전송 주기를 2.0초로 설정합니다.

21: 현재 시간을 current_time 변수에 저장합니다.

22~23: current_time과 previous_time 차이가 send_interval 이상이면 데이터를 전송합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

time.slepp을 사용하지 않고 코드의 지연없이 일정시간마다 온도와 습도값을 전송하였습니다. 이러한 방식으로 코드를 작성하면 많은 작업을 코드에 추가하여도 다른 기능들과 영향이 적어 다양한 작업이 가능합니다.



버튼의 상태 블루투스 통신으로 데이터 전송하기

온도와 습도를 전송하는 코드에서 버튼이 눌리면 눌린값을 전송하는 기능을 추가해보록 합니다.

6-1.블루투스 통신_4.py

```
01  import bluetooth, ble_simple_peripheral
02  import time
03  from machine import Pin
04  import dht
05
06  ble = bluetooth.BLE()
07  p = ble_simple_peripheral.BLESimplePeripheral(ble, name='pyD_1234')
08
09  dht11 = dht.DHT11(Pin(18))
10
11  key1 = Pin(38, Pin.IN, Pin.PULL_UP)
12  prev_state1 = key1.value()
13  key2 = Pin(39, Pin.IN, Pin.PULL_UP)
14  prev_state2 = key2.value()
15
16  def on_rx(read):
17      print("RX:", read)
18
19  p.on_write(on_rx)
20
21  def get_key1():
22      global prev_state1
23      current_state = key1.value()
24      if current_state != prev_state1:
25          prev_state1 = current_state
26          if current_state ==0:
27              return 1
28      else:
29          return 0
30
31  def get_key2():
32      global prev_state2
33      current_state = key2.value()
34      if current_state != prev_state2:
35          prev_state2 = current_state
36          if current_state ==0:
37              return 1
38      else:
39          return 0
40
41  previous_time = time.time()
42  send_interval =2.0
43
44  try:
45      while True:
46          current_time = time.time()
47          if current_time - previous_time >= send_interval:
48              previous_time = current_time
49              if p.is_connected():
```

```
50                  dht11.measure()
51                  temp = dht11.temperature()
52                  humi = dht11.humidity()
53                  if (temp is None) or (humi is None):
54                      print("센서 에러")
55                  else:
56                      send_str =str(temp) +","+str(humi) +"\r\n"
57                      p.send(send_str)
58
59          if get_key1():
60              p.send("key1 click\r\n")
61              time.sleep(0.1)
62
63          if get_key2():
64              p.send("key2 click\r\n")
65              time.sleep(0.1)
66  except :
67      pass
```

코드설명

03: Pin: 하드웨어 핀을 제어하기 위한 클래스입니다.

11: key1 핀 초기화: 핀 38을 입력 모드로 설정하고 풀업(PULL_UP)으로 초기화합니다.

12: prev_state1 초기화: 버튼 1의 이전 상태를 저장하는 변수를 초기화합니다.

13: key2 핀 초기화: 핀 39를 입력 모드로 설정하고 풀업(PULL_UP)으로 초기화합니다.

14: prev_state2 초기화: 버튼 2의 이전 상태를 저장하는 변수를 초기화합니다.

21~29: get_key1 함수 정의:

22: 버튼 1의 이전 상태를 전역 변수로 사용합니다.

23~27: 버튼 1의 상태 변화를 확인하고, 눌린 상태(LOW)가 감지되면 1을 반환합니다.

28~29: 버튼 상태에 변화가 없으면 0을 반환합니다.

31~39: get_key2 함수 정의:

32: 버튼 2의 이전 상태를 전역 변수로 사용합니다.

33~37: 버튼 2의 상태 변화를 확인하고, 눌린 상태(LOW)가 감지되면 1을 반환합니다.

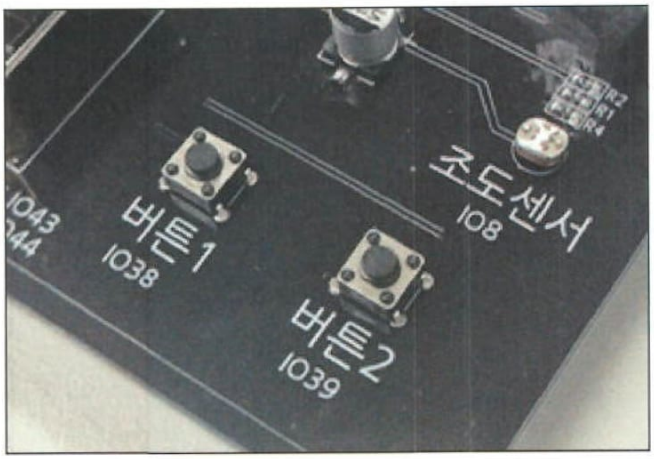
38~39: 버튼 상태에 변화가 없으면 0을 반환합니다.

59~61: 버튼 1이 눌렸을 경우, "key1 click" 메시지를 BLE를 통해 전송하고 0.1초 대기합니다.

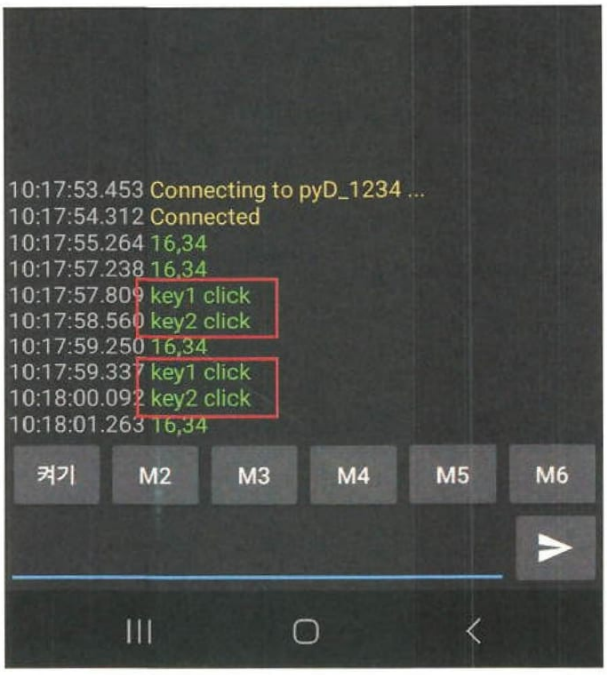
63~65: 버튼 2가 눌렸을 경우, "key2 click" 메시지를 BLE를 통해 전송하고 0.1초 대기합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

앱에서 연결 후 버튼1, 버튼2를 눌러 데이터를 전송합니다.



온도와 습도값을 출력하는 도중에도 버튼을 누르면 즉시 값을 전송합니다.



6-2

블루투스 통신을 활용한 RGB LED제어

블루투스 통신으로 수신받은 데이터에서 원하는 값을 분리하여 RGB LED를 제어해보도록 합니다.

아래표와 같이 RGB명령어를 입력받아 RGB의 값을 , 콤마로 분리하는 명령어를 만들어 데이터를 수신받아봅니다.

RGB	RED	GREEN	BLUE
RGB	0~1023	0~1023	0~1023

예를 들어 아래와 같은 명령어를 받아 RGB LED를 제어합니다.

RGB,100,300,,500

빨간색 100, 녹색 300, 파란색 500으로 LED를 켭니다.

프로토콜 만들어 데이터 수신받기

데이터를 수신받아 값을 분리하는 코드를 작성해봅니다.

6-2.블루투스 통신을 활용한 RGB LED제어_1.py

```
01    import bluetooth,ble_simple_peripheral
02    import time
03
04    ble = bluetooth.BLE()
05    p = ble_simple_peripheral.BLESimplePeripheral(ble,name='pyD_1234')
06
```

```
07    def on_rx(read):
08        print("RX:", read)
09
10        decoded_data = read.decode('utf-8').strip()
11        print("Decoded:", decoded_data)
12
13        split_data = decoded_data.split(",")
14        print("Split Data:", split_data)
15
16    p.on_write(on_rx)
17
18    try:
19        while True:
20            time.sleep(1.0)
21    except :
22        pass
```

코드설명

07: 데이터를 수신했을 때 호출되는 콜백 함수입니다.

10: 수신된 데이터를 UTF-8로 디코딩하고 양쪽 공백을 제거하여 decoded_data에 저장합니다.

11: 디코딩된 데이터를 콘솔에 출력합니다.

13: 디코딩된 데이터를 쉼표(,)로 분리하여 리스트로 변환하고 split_data에 저장합니다.

14: 분리된 데이터를 콘솔에 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

RGB,100,300,500 을 입력 후 전송합니다.

수신받은 데이터를 분리하였고 그 값을 출력하였습니다.

```
셸

MPY: soft reboot
MAC:  f4:12:fa:e2:f5:42
Starting advertising
New connection 1
RX: b'RGB,100,300,500\r\n'
Decoded: RGB,100,300,500
Split Data: ['RGB', '100', '300', '500']
```

RGB 데이터 적용하기

데이터 형식이 맞는지 확인하고 r,g,b에 값을 분리하는 코드를 작성해봅니다.

6-2.블루투스 통신을 활용한 RGB LED제어_2.py

```
01  import bluetooth,ble_simple_peripheral
02  import time
03
04  ble = bluetooth.BLE()
05  p = ble_simple_peripheral.BLESimplePeripheral(ble,name='pyD_1234')
06
07  def on_rx(read):
08      print("RX:", read)
09
10      decoded_data = read.decode('utf-8').strip()
11      print("Decoded:", decoded_data)
12
13      split_data = decoded_data.split(",")
14
15      if len(split_data) ==4 and split_data[0] =="RGB":
16          r =int(split_data[1])
17          g =int(split_data[2])
18          b =int(split_data[3])
19          print(f"Setting RGB LED to Red: {r}, Green: {g}, Blue: {b}")
20
21      else:
22          print("Error: Invalid command. Expected 'RGB,R,G,B'")
23
24  p.on_write(on_rx)
25
26  try:
27      while True:
28          time.sleep(1.0)
29  except :
30      pass
```

코드설명

08: 수신된 원본 데이터를 콘솔에 출력합니다.

10: 수신된 데이터를 UTF-8로 디코딩하고 양쪽 공백을 제거하여 decoded_data에 저장합니다.

11: 디코딩된 데이터를 콘솔에 출력합니다.

13: 디코딩된 데이터를 쉼표(,)로 분리하여 리스트로 변환하고 split_data에 저장합니다.

15~19:

15: split_data의 길이가 4이고 첫 번째 요소가 "RGB"인 경우를 확인합니다.

16~18: 리스트에서 두 번째, 세 번째, 네 번째 값을 각각 r, g, b로 변환합니다.

19: RGB 값을 설정하는 메시지를 콘솔에 출력합니다.

21~22: 유효하지 않은 명령어인 경우 오류 메시지를 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

RGB,100,300,500 을 입력 후 전송합니다.

분리된값을 r,g,b 에 각각 값을 대입하였습니다.

```
MPY: soft reboot
MAC:  f4:12:fa:e2:f5:42
Starting advertising
New connection 1
RX: b'RGB,100,300,500\r\n'
Decoded: RGB,100,300,500
Setting RGB LED to Red: 100, Green: 300, Blue: 500
```

RGB LED제어하기

블루투스 통신으로 수신받은 데이터를 실제 RGB LED에 적용하여 켜보도록 합니다.

6-2.블루투스 통신을 활용한 RGB LED제어_3.py

```
01  import bluetooth,ble_simple_peripheral
02  import time
03  from machine import Pin, PWM
04
05  ble = bluetooth.BLE()
06  p = ble_simple_peripheral.BLESimplePeripheral(ble,name='pyD_1234')
07
08  LED_RED = PWM(Pin(21), freq=1000, duty=0)
09  LED_GREEN = PWM(Pin(47), freq=1000, duty=0)
10  LED_BLUE = PWM(Pin(48), freq=1000, duty=0)
11
12  def on_rx(read):
13      print("RX:", read)
14
15      decoded_data = read.decode('utf-8').strip()
16      print("Decoded:", decoded_data)
17
18      split_data = decoded_data.split(",")
19
20      if len(split_data) ==4 and split_data[0] =="RGB":
21          r =int(split_data[1])
22          g =int(split_data[2])
23          b =int(split_data[3])
24          LED_RED.duty(r)
25          LED_GREEN.duty(g)
26          LED_BLUE.duty(b)
27          print(f"Setting RGB LED to Red: {r}, Green: {g}, Blue: {b}")
28
29      else:
30          print("Error: Invalid command. Expected 'RGB,R,G,B'")
31
32  p.on_write(on_rx)
33
34  try:
```

```
35          while True:
36              time.sleep(1.0)
37      except :
38          pass
```

코드설명

03: Pin, PWM: 핀 제어 및 PWM(Pulse Width Modulation) 출력을 위한 클래스입니다.

08~10: PWM 객체 생성:

08: 핀 21을 PWM으로 설정하여 빨간색 LED를 제어합니다. 초기 주파수는 1000Hz, 듀티는 0으로 설정합니다.

09: 핀 47을 PWM으로 설정하여 녹색 LED를 제어합니다.

10: 핀 48을 PWM으로 설정하여 파란색 LED를 제어합니다.

21~23: 두 번째, 세 번째, 네 번째 값을 각각 r, g, b로 변환합니다.

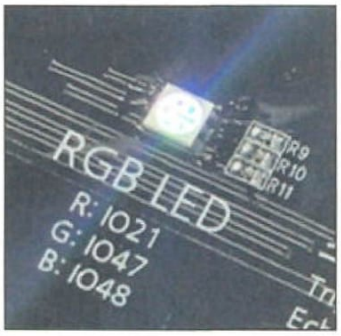
24~26: r, g, b 값을 PWM 듀티 사이클로 설정하여 LED의 밝기를 제어합니다.

27: 설정된 RGB 값을 콘솔에 출력합니다.

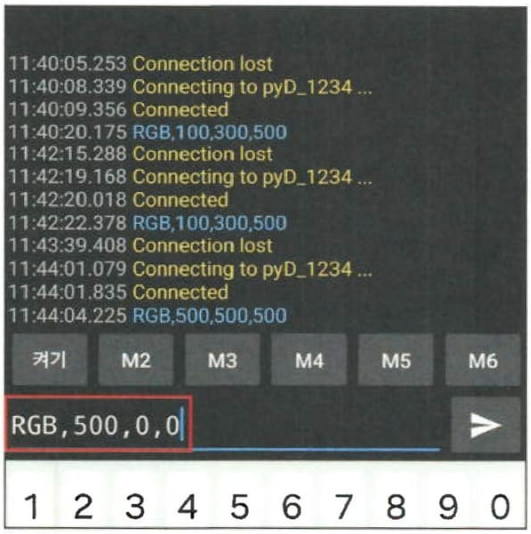
[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

RGB,100,300,500 을 입력 후 전송합니다.

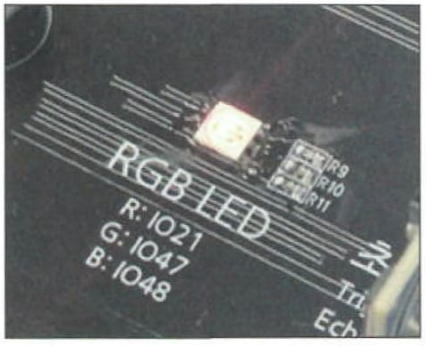
RGB값이 동일하게 500 으로 켜졌습니다. 0~1023의 약 중간값으로 빨강, 녹색, 파랑이 동일하게 켜져서 흰색으로 표시되었습니다. 약 절반 가량의 밝기로 켜졌습니다.



RGB,500,0,0 으로 빨간색만 약 50%의 값을 전송합니다.



RGB LED의 빨간색 LED가 약 절반가량의 밝기로 켜졌습니다.



통신 프로토콜을 만들어 RGB LED를 제어하였습니다. 이와 같은 방식으로 프로토콜을 만들면 다양한 부품들을 제어 할 수 있습니다.

CHAPTER 07

WIFI 사물인터넷

WiFi 통신을 활용하여 주변WiFi를 검색, 기상청 날씨 등 다양한 사물 인터넷 서비스를 직접 만들어봅니다.

7-1

WiFi 통신

Wi-Fi (무선 로컬 영역 네트워크) 통신은 무선 레이디오 파장을 사용하여 디지털 기기 간에 데이터를 무선으로 전송하는 기술입니다. Wi-Fi는 일반적으로 인터넷에 연결하거나 로컬 네트워크에서 데이터를 공유하는 데 사용됩니다.

1. 작동 원리: Wi-Fi는 라우터 또는 액세스 포인트를 통해 무선 신호를 발생시키고, 이 신호를 Wi-Fi 지원 장치 (예: 스마트폰, 노트북, 태블릿, 스마트 홈 기기)가 수신하여 데이터를 주고받습니다. Wi-Fi 신호는 2.4GHz 또는 5GHz 주파수 대역에서 작동하며, 고속 데이터 전송을 지원합니다.
2. 응용 분야: Wi-Fi는 주로 인터넷에 연결하거나 로컬 네트워크에서 데이터를 공유하는 데 사용됩니다. 집, 회사, 학교, 카페, 공공 장소 등 다양한 장소에서 무선 인터넷 접속을 제공하고, 스마트폰, 노트북, 스마트 TV, 스마트 홈 기기 등 다양한 디바이스를 연결하는 데 사용됩니다.
3. 특징: Wi-Fi는 무선 통신을 통해 빠르고 편리한 인터넷 접속을 제공합니다. 또한 다수의 장치를 동시에 연결하고, 무선으로 데이터를 전송하고 받을 수 있습니다.
4. 보안: Wi-Fi 네트워크는 보안을 위해 암호화 및 액세스 제어를 사용할 수 있습니다. WPA2 또는 WPA3와 같은 보안 프로토콜을 사용하여 네트워크를 보호할 수 있습니다.

Wi-Fi는 현대적인 컴퓨팅과 무선 통신의 핵심 기술 중 하나이며, 무선 인터넷 접속을 가능하게 하여 다양한 디지털 기기 간에 데이터를 손쉽게 공유하고 웹 브라우징, 스트리밍, 온라인 게임 및 다른 온라인 활동을 지원합니다.

WIFI 스캔을 통해 주변 WIFI 찾기

ESP32에는 2.4GHz 의 WIFI 기능이 내장되어 있습니다. wifi를 스캔하여 주변 wifi를 찾는 방법을 알아봅니다.

7-1.wifi 통신_1.py

```
01  print("Scanning for WiFi networks, please wait...")
02  print("")
03
04  import network
05  sta_if = network.WLAN(network.STA_IF)
06  sta_if.active(True)
07
08  authmodes = ['Open', 'WEP', 'WPA-PSK', 'WPA2-PSK', 'WPA/WPA2-PSK']
09
10  for (ssid, bssid, channel, RSSI, authmode, hidden) in sta_if.scan():
11      print("* {:s}".format(ssid))
12      if 0 <= authmode <len(authmodes):
13          auth_desc = authmodes[authmode]
14      else:
15          auth_desc ="Unknown"
16
17      print("   - Auth: {} {}".format(auth_desc, '(hidden)'if hidden else ''))
18      print("   - Channel: {}".format(channel))
19      print("   - RSSI: {}".format(RSSI))
20      print("   - BSSID: {:02x}:{:02x}:{:02x}:{:02x}:{:02x}:{:02x}".format(*bssid))
21      print()
```

코드설명

04: network 모듈 가져오기

04: WiFi 네트워크 관리에 필요한 network 모듈을 가져옵니다.

05~06: WiFi 인터페이스 활성화

05: network.STA_IF를 사용해 WiFi 스테이션 인터페이스 객체를 생성하여 무선 네트워크 연결을 설정합니다.

06: sta_if.active(True)로 인터페이스를 활성화합니다.

08: WiFi 인증 모드를 설명하는 문자열 리스트를 정의합니다.

'Open': 비보안 네트워크.

'WEP': WEP 보안.

'WPA-PSK': WPA-PSK 보안.

'WPA2-PSK': WPA2-PSK 보안.

'WPA/WPA2-PSK': WPA 및 WPA2 혼합 보안.

10~20: WiFi 네트워크 스캔 및 정보 출력

10: sta_if.scan()을 사용해 주변 WiFi 네트워크를 스캔하고 각 네트워크의 정보를 가져옵니다.

ssid: 네트워크 이름.

bssid: 네트워크의 MAC 주소.

channel: 네트워크 채널.

RSSI: 신호 강도.

authmode: 인증 모드.

hidden: 네트워크가 숨김 상태인지 여부.

11: WiFi 네트워크 이름(ssid)을 출력합니다.

12~15: 인증 모드 확인 및 설명 출력

12: authmode 값이 authmodes 리스트의 범위 내에 있는지 확인합니다.

13: 인증 모드의 설명(auth_desc)을 설정합니다.

15: 범위를 벗어난 경우 "Unknown"으로 설정합니다.

17: 인증 모드와 네트워크가 숨김 상태인 경우 (hidden)을 표시합니다.

18: 네트워크가 사용하는 채널 번호를 출력합니다.

19: 신호 강도 출력

19: RSSI 값을 출력하여 WiFi 신호 강도를 표시합니다.

20: BSSID를 MAC 주소 형식(xx:xx:xx:xx:xx:xx)으로 출력합니다.

21: 다음 네트워크 정보를 구분하기 위해 빈 줄을 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

주변 Wi-Fi 네트워크를 스캔하여 정보를 확인하는 데 사용됩니다.

검색된 주변의 WIFI의 정보가 출력됩니다.

```
MPY: soft reboot
Scanning for WiFi networks, please wait...

* daduino
   - Auth: WPA2-PSK
   - Channel: 8
   - RSSI: -51
   - BSSID: a8:42:a1:12:87:23

*
   - Auth: WPA2-PSK
   - Channel: 8
   - RSSI: -52
   - BSSID: ca:42:a1:12:87:23
```

WIFI 연결하기

ESP32를 이용하여 wifi에 연결하는 방법을 알아봅니다.

ssid와 password는 자신의 wifi 환경의 값을 입력합니다. 2.4GHz 대역의 wifi만 연결이 가능합니다.

7-1.wifi 통신_2.py

```
01    import network
02    import time
03
04    ssid ="daduino"
05    password ="daduino12345"
06
07    def wifi_connect():
08        wlan = network.WLAN(network.STA_IF)
09        wlan.active(True)
10
11        if not wlan.isconnected():
12            print("Connecting to network...")
13            wlan.connect(ssid, password)
14
15            while not wlan.isconnected():
16                time.sleep_ms(300)
17
18            print("Connected to network:", wlan.ifconfig())
19            return True
20        else:
21            print("Already connected to network:", wlan.ifconfig())
22            return True
23
24    if wifi_connect():
25        pass
```

코드설명

01: WiFi 연결을 관리하기 위해 network 모듈을 가져옵니다.

02: 시간 지연 기능을 제공하기 위해 time 모듈을 가져옵니다.

04: 연결하려는 WiFi 네트워크 이름을 ssid 변수에 저장합니다.

05: WiFi 네트워크 비밀번호를 password 변수에 저장합니다.

07~22: WiFi 연결 함수 정의

07: wifi_connect() 함수는 WiFi 네트워크에 연결하는 기능을 제공합니다.

08: WiFi 스테이션(STA) 모드를 활성화하기 위해 network.WLAN(network.STA_IF) 객체를 생성합니다.

09: wlan.active(True)로 WiFi 스테이션 모드를 활성화합니다.

11~22: WiFi 연결 상태 확인 및 연결 설정

11: 현재 WiFi가 연결되어 있는지 wlan.isconnected()로 확인합니다.

12~13: 연결되어 있지 않으면 네트워크에 연결을 시도합니다.

15~16: WiFi가 연결될 때까지 300ms 간격으로 대기합니다.

18: 연결 완료 후, 네트워크 설정 정보를 출력합니다.

19: 연결 성공 여부를 True로 반환합니다.

20~21: 이미 연결되어 있는 경우, 네트워크 정보를 출력하고 True를 반환합니다.

24~25: WiFi 연결 확인 및 후속 작업

24: wifi_connect()를 호출하여 WiFi 연결을 확인합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

SSID와 비밀번호로 Wi-Fi 네트워크에 연결하였습니다. 연결 완료 후 연결된 정보가 출력됩니다.

```
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  Connecting to network...
  Connected to network: ('192.168.0.199', '255.255.255.0', '192.168.0.1', '192.168.0.1')
>>>
```

WIFI 고정 IP 설정하기

wifi를 연결시에 고정 IP를 설정하는 방법에 대해서 알아봅니다.

7-1.wifi 통신_3.py

```
01  import network
02  import time
03
04  ssid ="daduino"
05  password ="daduino12345"
06  static_ip ="192.168.0.150"  # 원하는 고정 IP 주소 설정
07  subnet_mask ="255.255.255.0"
08  gateway ="192.168.0.1"
09  dns_server ="8.8.8.8"
10
11  def wifi_connect():
12      wlan = network.WLAN(network.STA_IF)
13      wlan.active(True)
14
15      if not wlan.isconnected():
16          print("Connecting to network...")
17          wlan.active(True)
```

```
            wlan.config(dhcp_hostname="my-esp32")
            wlan.ifconfig((static_ip, subnet_mask, gateway, dns_server))
            wlan.connect(ssid, password)

            while not wlan.isconnected():
                time.sleep_ms(300)

            print("Connected to network:", wlan.ifconfig())
            return True
        else:
            print("Already connected to network:", wlan.ifconfig())
            return True

if wifi_connect():
    pass
```

코드설명

06: static_ip: ESP32의 고정 IP 주소를 설정합니다.

07: subnet_mask: 서브넷 마스크를 설정합니다.

08: gateway: 기본 게이트웨이 주소를 설정합니다.

09: dns_server: DNS 서버 주소를 설정합니다.

19: wlan.ifconfig()를 사용하여 고정 IP 설정을 적용합니다. (IP 주소, 서브넷 마스크, 게이트웨이, DNS 서버)

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

wifi에 다시 연결하기 위해서 [리셋] 버튼을 눌러 드론을 리셋한 다음 진행합니다. 리셋 후 Thonny에서 [STOP] 아이콘을 클릭하면 pyDrone은 micropython 코드를 입력 할 수 있는 상태로 진입합니다.

주어진 SSID와 비밀번호로 Wi-Fi 네트워크에 연결하고 고정 IP 주소를 할당하는 함수를 제공합니다. 이렇게 하면 MicroPython 기반 장치에서 특정 IP 주소를 사용하여 네트워크에 연결할 수 있습니다.

코드에서 설정한 192.168.0.150 으로 고정 IP로 설정하였습니다.

```
>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Connecting to network...
Connected to network: ('192.168.0.150', '255.255.255.0', '192.168.0.1', '8.8.8.8')
>>
```

WIFI AP모드로 변경하기

wifi를 AP모드로 사용하는 방법에 대해서 알아봅니다.

7-1.wifi 통신_4.py

```
01  import network
02  import time
03
04  ap_ssid ="pyDrone1234"
05
06  def wifi_ap_mode():
07      ap = network.WLAN(network.AP_IF)
08      ap.config(essid=ap_ssid)
09      ap.active(True)
10
11  wifi_ap_mode()
```

코드설명

04: 액세스 포인트(AP) 모드에서 사용할 WiFi 네트워크 이름(SSID)을 ap_ssid 변수에 저장합니다. 여기서는 'pyDrone1234'로 설정되어 있습니다.

06~10: WiFi AP 모드 설정 함수 정의

06: wifi_ap_mode() 함수는 WiFi를 AP(Access Point) 모드로 설정합니다.

07: network.AP_IF를 사용해 AP 모드 인터페이스 객체를 생성합니다.

08: ap.config(essid=ap_ssid)로 AP의 SSID를 설정합니다.

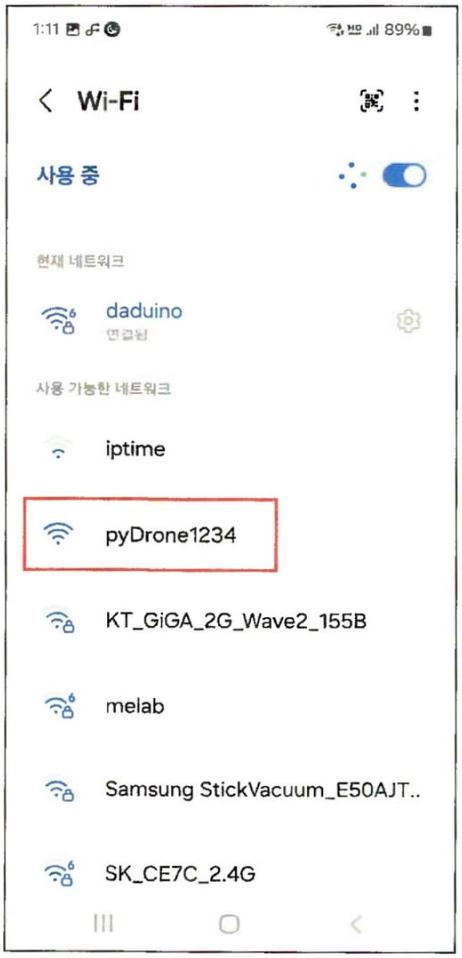
09: ap.active(True)로 AP 모드를 활성화합니다.

11: wifi_ap_mode() 함수를 호출하여 AP 모드를 설정하고 활성화합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

ESP32를 액세스 포인트(AP)로 설정하여 외부 장치가 해당 네트워크에 연결할 수 있게 합니다. 설정한 SSID와 비밀번호를 사용하여 액세스 포인트에 연결할 수 있습니다.

ESP32장치가 공유기에 연결하는 형태가 아닌 ESP32가 AP가되어 다른 장치에서 ESP32로 접속이 가능합니다.



7-2

OLED 인터넷시간 표시하기

NTP (Network Time Protocol)는 컴퓨터 네트워크에서 정확한 시간을 동기화하기 위한 표준 프로토콜입니다. NTP를 사용하여 인터넷 시간을 동기화할 때, 컴퓨터나 네트워크 디바이스는 인터넷을 통해 시간 서버로부터 정확한 시간 정보를 받아와 자체 시계를 조절합니다. 간단하게 설명하면:

1. 작동 원리: NTP는 전 세계의 다양한 시간 서버와 통신하여 정확한 시간 정보를 받아옵니다. 이 정보를 사용하여 컴퓨터나 네트워크 디바이스의 시계를 조정하고 정확한 시간을 유지합니다.

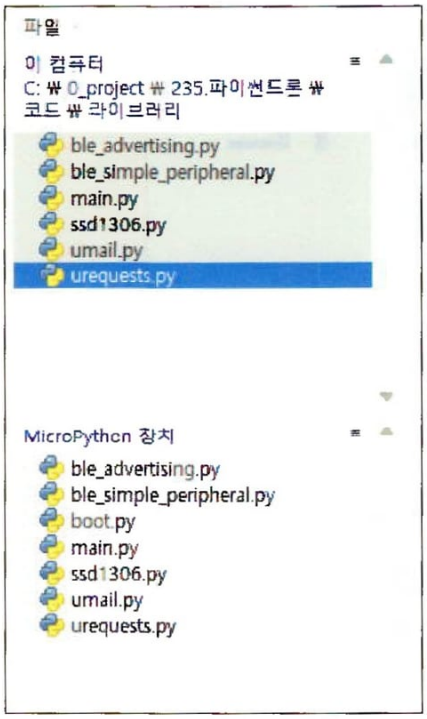
2. 응용 분야: NTP는 네트워크에서 정확한 시간을 필요로 하는 다양한 분야에서 사용됩니다. 예를 들어, 웹 서버, 데이터베이스 서버, 보안 시스템, 로그 기록 및 타임 스탬프 등에서 정확한 시간 동기화가 중요합니다.

3. 특징: NTP는 정밀한 시간 동기화를 지원하며, 미세한 시간 오차를 최소화합니다. 이를 통해 네트워크에서 정확한 시간을 유지하고 다양한 디바이스 간의 시간 동기화를 가능하게 합니다.

4. 시간 서버: NTP 시간 서버는 정확한 시간 정보를 제공하는 역할을 합니다. 전 세계에는 공개적으로 사용 가능한 NTP 서버와 기업 및 정부 기관 등이 운영하는 NTP 서버가 있습니다.

라이브러리 업로드하기

1장의 시작하기에서 [라이브러리 마이크로파이썬에 업로드하기]를 참고하여 라이브러리를 업로드 한다음 진행합니다. OLED를 제어하기 위해서 [ssd1306.py] 라이브러리 파일이 필요로 합니다.



NTP 서버에 접속하여 시간받기

ESP32에서 인터넷에 접속하여 NTP 서버에서 시간을 받아 출력하는 방법에 대해서 알아봅니다.

7-2.OLED 인터넷 시간표시하기_1.py

```
01    import network
02    import time
03    import ntptime
04
05    ssid ="daduino"
06    password ="daduino12345"
07
08    def wifi_connect():
09        wlan = network.WLAN(network.STA_IF)
10        wlan.active(True)
11
12        if not wlan.isconnected():
13            print("Connecting to network...")
14            wlan.connect(ssid, password)
15
16            while not wlan.isconnected():
```

```
17              time.sleep_ms(300)
18
19          print("Connected to network:", wlan.ifconfig())
20          return True
21      else:
22          print("Already connected to network:", wlan.ifconfig())
23          return True
24
25  def set_time_from_ntp():
26      try:
27          ntptime.settime()
28          print("Time synchronized with NTP")
29      except Exception as e:
30          print("Failed to synchronize time with NTP:", str(e))
31
32  if wifi_connect():
33      set_time_from_ntp()
34      print(time.localtime())
```

코드설명

25: set_time_from_ntp() 함수는 NTP 서버에서 시간을 가져와 시스템 시간을 설정합니다.

27: ntptime.settime()를 호출하여 NTP 서버와 시간을 동기화합니다.

28: 동기화 성공 시 성공 메시지를 출력합니다.

29~30: 동기화 실패 시 예외를 처리하고 에러 메시지를 출력합니다.

32~34: WiFi 연결 및 시간 동기화

32: wifi_connect()를 호출하여 WiFi 네트워크에 연결합니다.

33: WiFi 연결에 성공한 경우 set_time_from_ntp()를 호출하여 시간을 동기화합니다.

34: 현재 로컬 시간을 time.localtime()으로 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

ESP32 장치를 사용하여 Wi-Fi 네트워크에 연결하고 NTP를 통해 시간을 동기화하는 예제입니다.

```
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Connecting to network...
Connected to network: ('192.168.0.199'
Time synchronized with NTP
(2024, 11, 18, 4, 33, 11, 0, 323)
```

시간을 표시하기

시간을 표시하는 방법에 대해서 알아봅니다.

7-2.OLED 인터넷 시간표시하기_2.py

```
01  import network
02  import time
03  import ntptime
04
05  ssid ="daduino"
06  password ="daduino12345"
07
08  def wifi_connect():
09      wlan = network.WLAN(network.STA_IF)
10      wlan.active(True)
11
12      if not wlan.isconnected():
13          print("Connecting to network...")
14          wlan.connect(ssid, password)
15
16          while not wlan.isconnected():
17              time.sleep_ms(300)
18
19          print("Connected to network:", wlan.ifconfig())
20          return True
21      else:
22          print("Already connected to network:", wlan.ifconfig())
23          return True
24
25  def set_time_from_ntp():
26      try:
27          ntptime.settime()
```

```
28            print("Time synchronized with NTP")
29        except Exception as e:
30            print("Failed to synchronize time with NTP:", str(e))
31
32    if wifi_connect():
33        set_time_from_ntp()
34
35    while True:
36        rtc_time = time.localtime()
37        formatted_time ="{}-{:02d}-{:02d} {:02d}:{:02d}:{:02d}".format(
38            rtc_time[0], rtc_time[1], rtc_time[2], rtc_time[3], rtc_time[4], rtc_time[5]
39        )
40        print("Current Time:", formatted_time)
41        time.sleep(1)
```

코드설명

35-41: 무한 루프에서 현재 시간을 주기적으로 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

NTP를 통해 시간을 동기화한 다음 현재 시간을 주기적으로 출력합니다.

```
셸

>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  Already connected to network: ('192
  Time synchronized with NTP
  Current Time: 2024-11-18 04:36:37
  Current Time: 2024-11-18 04:36:38
  Current Time: 2024-11-18 04:36:39
```

+9시간을 더해 한국시간으로 표시하기

시간이 9시간 작습니다. +9시간을 더해 한국시간으로 표시하는 방법에 대해서 알아봅니다.

7-2.OLED 인터넷 시간표시하기_3.py

```
import network
import time
import ntptime

ssid ="daduino"
password ="daduino12345"

def wifi_connect():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)

    if not wlan.isconnected():
        print("Connecting to network...")
        wlan.connect(ssid, password)

        while not wlan.isconnected():
            time.sleep_ms(300)

        print("Connected to network:", wlan.ifconfig())
        return True
    else:
        print("Already connected to network:", wlan.ifconfig())
        return True

def set_time_from_ntp():
    try:
        ntptime.settime()
        print("Time synchronized with NTP")
    except Exception as e:
        print("Failed to synchronize time with NTP:", str(e))

def get_kst_time():
    utc_time = time.time()
    kst_time = time.localtime(utc_time +9 *3600)  # 9시간 추가
    return kst_time

if wifi_connect():
    set_time_from_ntp()

while True:
    rtc_time = get_kst_time()
    formatted_time ="{}-{:02d}-{:02d} {:02d}:{:02d}:{:02d}".format(
        rtc_time[0], rtc_time[1], rtc_time[2], rtc_time[3], rtc_time[4], rtc_time[5]
    )
    print("Current Time (KST):", formatted_time)
    time.sleep(1)
```

코드설명

32~35: 한국 표준시(KST) 변환

32: get_kst_time() 함수는 UTC 시간을 KST로 변환합니다.

33: time.time()를 호출하여 현재 UTC 시간을 가져옵니다.

34: UTC 시간에 9시간(초 단위)을 추가하여 KST 시간을 계산합니다.

35: KST 시간을 time.localtime() 형식으로 반환합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

NTP를 통해 시간을 동기화한 다음 현재 시간에서 +9시간을 더해 한국시간으로 표시하고 있습니다.

```
셸
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  Already connected to network: ('192.168.0
  Time synchronized with NTP
  Current Time (KST): 2024-11-18 13:37:41
  Current Time (KST): 2024-11-18 13:37:42
  Current Time (KST): 2024-11-18 13:37:43
```

OLED에 NTP 시간 표시하기

NTP 시간을 OLED에 표시합니다. OLED의 경우 라이브러리가 업로드 되어야 합니다. [5-1. OLED 디스플레이 장치] 부분을 참고하여 라이브러리 및 OLED의 사용방법을 숙지합니다.

7-2.OLED 인터넷 시간표시하기_4.py

```
01  import network
02  import time
03  import ntptime
04  from machine import Pin
05  from machine import SoftI2C
06  from ssd1306 import SSD1306_I2C
07
08  ssid ="daduino"
09  password ="daduino12345"
10
11  i2c = SoftI2C(sda=Pin(43), scl=Pin(44))
12  oled = SSD1306_I2C(128, 64, i2c)
13
14  def wifi_connect():
15      wlan = network.WLAN(network.STA_IF)
16      wlan.active(True)
17
18      if not wlan.isconnected():
19          print("Connecting to network...")
20          wlan.connect(ssid, password)
21
22          while not wlan.isconnected():
23              time.sleep_ms(300)
24
25          print("Connected to network:", wlan.ifconfig())
26          return True
27      else:
28          print("Already connected to network:", wlan.ifconfig())
29          return True
30
31  def set_time_from_ntp():
32      try:
33          ntptime.settime()
34          print("Time synchronized with NTP")
35      except Exception as e:
36          print("Failed to synchronize time with NTP:", str(e))
37
38  def get_kst_time():
39      utc_time = time.time()
40      kst_time = time.localtime(utc_time +9 *3600)  # 9시간 추가
41      return kst_time
42
43  if wifi_connect():
44      set_time_from_ntp()
45
46  while True:
47      rtc_time = get_kst_time()
48      formatted_time ="{}-{:02d}-{:02d} {:02d}:{:02d}:{:02d}".format(
```

```
49              rtc_time[0], rtc_time[1], rtc_time[2], rtc_time[3], rtc_time[4], rtc_time[5]
50          )
51          print("Current Time (KST):", formatted_time)
52
53          yyyy_mm_dd ="{}-{:02d}-{:02d}".format(rtc_time[0], rtc_time[1], rtc_time[2])
54          hh_mm_ss ="{:02d}:{:02d}:{:02d}".format(rtc_time[3], rtc_time[4], rtc_time[5])
55
56          oled.fill(0)
57          oled.text(yyyy_mm_dd, 20, 20)
58          oled.text(hh_mm_ss, 20, 30)
59          oled.show()
60
61          time.sleep(1)
```

코드설명

11: SoftI2C 객체 생성: SDA 핀(43)과 SCL 핀(44)을 설정하여 I2C 통신을 초기화합니다.

12: SSD1306_I2C 객체 생성: OLED 디스플레이를 제어하기 위해 가로 128, 세로 64 크기로 초기화합니다.

44: 연결에 성공한 경우 NTP로 시간을 동기화합니다.

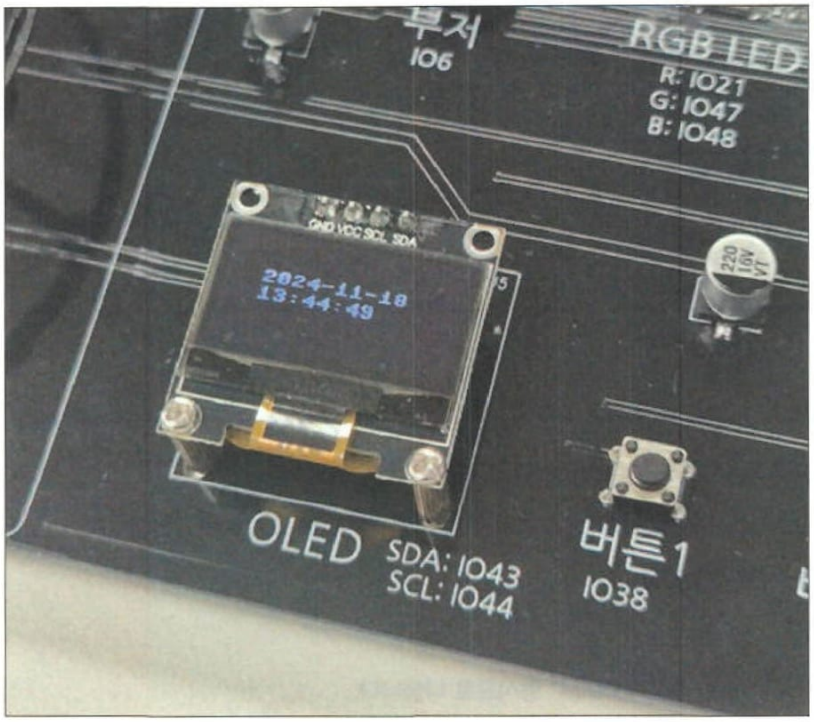
46~59: 현재 시간 OLED 출력 루프

53~54: 날짜와 시간을 각각 "YYYY-MM-DD"와 "HH:MM" 형식으로 나눕니다.

56~59: OLED 화면을 초기화하고, 날짜와 시간을 각각 출력한 후 화면에 표시합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

인터넷으로 시간을 받아 OLED에 출력하였습니다.



7-3

OLED 기상청 날씨 출력하기

기상청 RSS를 이용하여 날씨 정보를 읽어와 출력하는 방법에 대해서 알아봅니다.

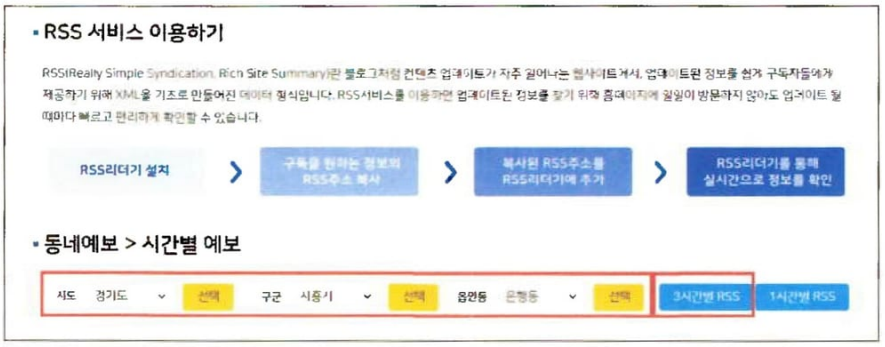
라이브러리 업로드하기

1장의 시작하기에서 [라이브러리 마이크로파이썬에 업로드하기]를 참고하여 라이브러리를 업로드 한다음 진행합니다. OLED와 웹에접속하기 위해서 [ssd1306.py],[urequests.py] 라이브러리 파일이 필요로 합니다.

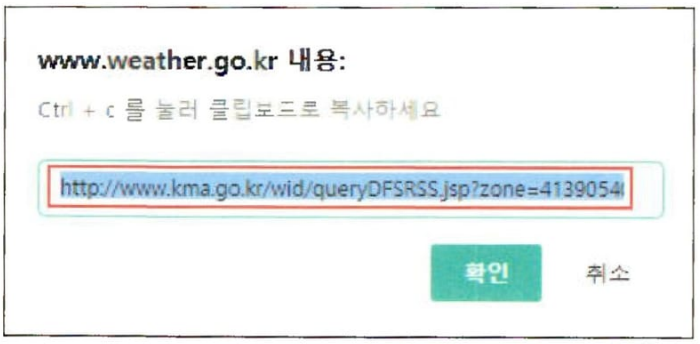
기상청 RSS 접속

구글에서 "기강청 RSS"를 검색 후 아래 사이트에 접속합니다.

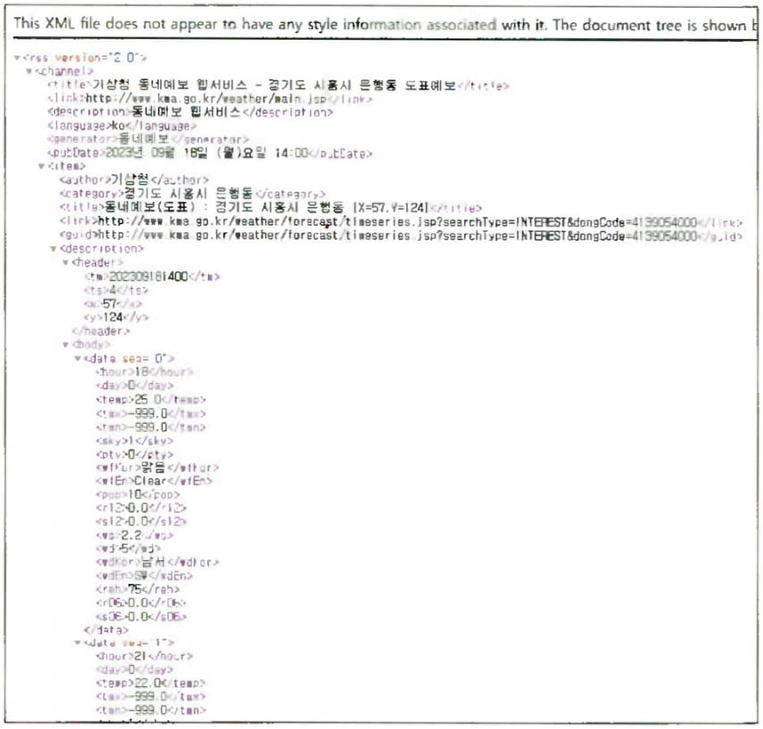
동네예보 〉 시간별 예보에서 원하는 지역을 선택한 다음 [3시간별 RSS]를 클릭합니다.



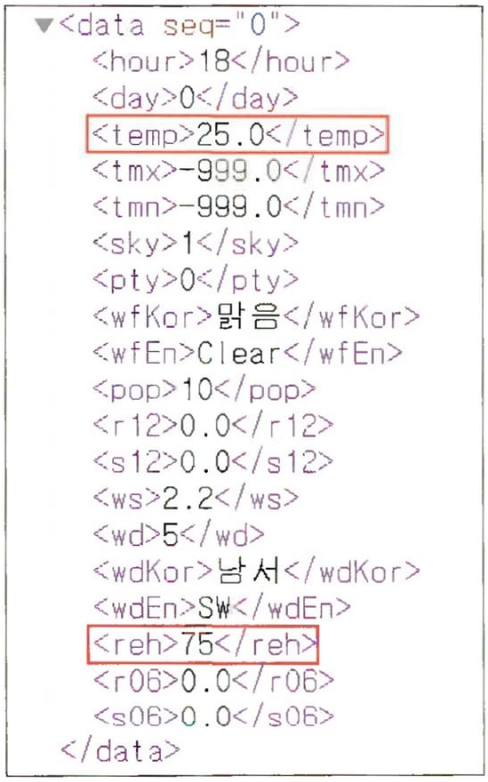
팝업으로 주소가 나옵니다. [컨트롤 +C]를 눌러 복사해둡니다.



주소에 접속하면 예보를 확인 할 수 있습니다. XML 형태의 데이터입니다.



〈temp〉온도〈/temp〉 로 온도를 확인 할 수 있고 〈reh〉습도〈/reh〉로 습도를 확인 할 수 있습니다.



기상청 RSS 접속하여 RSS데이터 받기

복사해둔 주소로 접속하여 RSS데이터를 출력하는 방법에 대해서 알아봅니다.

7-3.OLED 기상청 날씨 출력하기_1.py

```
01  import network
02  import time
03  import urequests
04
05  ssid ="daduino"
06  password ="daduino12345"
07  url ="http://www.kma.go.kr/wid/queryDFSRSS.jsp?zone=4139054000"
08
09  def wifi_connect():
10      wlan = network.WLAN(network.STA_IF)
11      wlan.active(True)
12
13      if not wlan.isconnected():
14          print("Connecting to network...")
15          wlan.connect(ssid, password)
16
17          while not wlan.isconnected():
18              time.sleep_ms(300)
19
20          print("Connected to network:", wlan.ifconfig())
21          return True
22      else:
23          print("Already connected to network:", wlan.ifconfig())
24          return True
25
26  def main():
27      if wifi_connect():
28          try:
29              response = urequests.get(url)
30              if response.status_code ==200:
31                  print("HTTP GET 요청 성공!")
32                  print("Response:")
33                  print(response.text)
34              else:
35                  print("HTTP GET 요청 실패. 상태 코드:", response.status_code)
36          except Exception as e:
37              print("HTTP GET 요청 중 오류 발생:", e)
38
39  if __name__ =="__main__":
40      main()
```

코드설명

26~38: HTTP GET 요청 처리

27: Wi-Fi 연결을 확인하고 연결에 성공하면 실행합니다.

29: 설정된 URL로 HTTP GET 요청을 보냅니다.

30~33: 요청이 성공(상태 코드 200)한 경우, 응답 데이터를 출력합니다.

34~35: 요청 실패 시 상태 코드를 출력합니다.

36~37: 요청 중 오류가 발생하면 예외 메시지를 출력합니다.

39~40: 프로그램 시작점

39: __name__ 변수가 "__main__"일 경우, 프로그램이 직접 실행된 것으로 판단합니다.

40: main() 함수를 호출하여 Wi-Fi 연결과 HTTP 요청을 수행합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

기상청 RSS에 접속하여 데이터를 출력하였습니다.

```
쉘 ×
      <wd>1</wd>
      <wdKor>북동</wdKor>
      <wdEn>NE</wdEn>
      <reh>90</reh>
      <r06>0.0</r06>
      <s06>0.0</s06>
    </data>
   </body>
  </description>
  </item>
  </channel>
  </rss>

>>>
```

main()함수를 만들어 동작하는 코드

```
1    def main():
2        print("메인함수실행")
3
4    if __name__ =="__main__":
5        main()
```

이 코드는 Python 스크립트에서 주로 사용되는 관용적인 코드 구조 중 하나입니다. 간단한 설명은 다음과 같습니다:

1. `main` 함수가 정의되었습니다. 이 함수는 "메인함수실행"이라는 메시지를 출력합니다.

2. 코드의 맨 아래에 있는 `if __name__ == "__main__":` 구문은 스크립트를 직접 실행할 때만 `main` 함수를 호출하라는 지시입니다. 즉, 이 스크립트가 다른 스크립트에서 모듈로 임포트될 때는 `main` 함수가 실행되지 않습니다.

이 코드는 스크립트를 실행하면 `main` 함수가 호출되고 "메인함수실행" 메시지가 출력됩니다. 이것은 Python 스크립트를 구조화하고 모듈화하는 데 일반적으로 사용되는 방법 중 하나입니다. `main` 함수에는 스크립트의 주요 로직을 배치할 수 있으며, 스크립트를 모듈로 사용할 때는 해당 로직이 실행되지 않습니다.

온도 습도 데이터만 출력하기

온도와 습도데이터만을 분리하여 출력하는 방법을 알아봅니다.

7-3.OLED 기상청 날씨 출력하기_2.py

```
01   import network
02   import time
03   import urequests
04
05   ssid ="daduino"
06   password ="daduino12345"
07   url ="http://www.kma.go.kr/wid/queryDFSRSS.jsp?zone=4139054000"
08
09   def wifi_connect():
10       wlan = network.WLAN(network.STA_IF)
11       wlan.active(True)
12
13       if not wlan.isconnected():
14           print("Connecting to network...")
15           wlan.connect(ssid, password)
16
17           while not wlan.isconnected():
```

```
            time.sleep_ms(300)

        print("Connected to network:", wlan.ifconfig())
        return True
    else:
        print("Already connected to network:", wlan.ifconfig())
        return True

def extract_temp_and_reh(xml_text):
    try:
        temp_start = xml_text.find("<temp>") +len("<temp>")
        temp_end = xml_text.find("</temp>", temp_start)
        temp = xml_text[temp_start:temp_end]

        reh_start = xml_text.find("<reh>") +len("<reh>")
        reh_end = xml_text.find("</reh>", reh_start)
        reh = xml_text[reh_start:reh_end]

        return temp, reh
    except Exception as e:
        print("Error extracting data from XML:", e)
        return None, None

def main():
    if wifi_connect():
        try:
            response = urequests.get(url)
            if response.status_code ==200:
                print("HTTP GET 요청 성공!")

                temp, reh = extract_temp_and_reh(response.text)
                if temp is not None and reh is not None:
                    print("온도:", temp)
                    print("습도:", reh)
            else:
                print("HTTP GET 요청 실패. 상태 코드:", response.status_code)
        except Exception as e:
            print("HTTP GET 요청 중 오류 발생:", e)

if __name__ =="__main__":
    main()
```

코드설명

26~39: XML 데이터에서 온도와 습도 추출 함수

28~30: XML 텍스트에서 〈temp〉 태그 안의 온도 데이터를 추출합니다.

32~34: XML 텍스트에서 〈reh〉 태그 안의 습도 데이터를 추출합니다.

36: 추출한 온도와 습도를 반환합니다.

37~39: 추출 중 오류가 발생하면 예외를 출력하고 None, None을 반환합니다.

26-40. `extract_temp_and_reh` 함수를 정의합니다. 이 함수는 HTTP 응답의 XML 텍스트에서 온도와 습도 정보를 추출합니다.

48. `extract_temp_and_reh` 함수를 사용하여 온도와 습도 정보를 추출합니다.

49-51. 추출된 온도와 습도 정보를 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

기상청 RSS에 접속하여 온도,습도데이터만 분리하여 출력하였습니다.

```
쉘

>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  Already connected to network: ('192.16
  HTTP GET 요청 성공!
  온도: 8.0
  습도: 50

>>>
```

주기적으로 접속하여 온도 습도 데이터 출력하기

주기적으로 접속하여 온도와 습도데이터를 출력하는 방법에 대해서 알아봅니다.

7-3.OLED 기상청 날씨 출력하기_3.py

```
01    import network
02    import time
```

```
import urequests

ssid ="daduino"
password ="123456789"
polling_interval =10
url ="http://www.kma.go.kr/wid/queryDFSRSS.jsp?zone=4139054000"

def wifi_connect():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)

    if not wlan.isconnected():
        print("Connecting to network...")
        wlan.connect(ssid, password)

        while not wlan.isconnected():
            time.sleep_ms(300)

        print("Connected to network:", wlan.ifconfig())
        return True
    else:
        print("Already connected to network:", wlan.ifconfig())
        return True

def extract_temp_and_reh(xml_text):
    try:
        temp_start = xml_text.find("<temp>") +len("<temp>")
        temp_end = xml_text.find("</temp>", temp_start)
        temp = xml_text[temp_start:temp_end]

        reh_start = xml_text.find("<reh>") +len("<reh>")
        reh_end = xml_text.find("</reh>", reh_start)
        reh = xml_text[reh_start:reh_end]

        return temp, reh
    except Exception as e:
        print("Error extracting data from XML:", e)
        return None, None

def main():
    while True:
        if wifi_connect():
            try:
                response = urequests.get(url)
                if response.status_code ==200:
                    print("HTTP GET 요청 성공!")

                    temp, reh = extract_temp_and_reh(response.text)
                    if temp is not None and reh is not None:
                        print("온도:", temp)
                        print("습도:", reh)
                else:
                    print("HTTP GET 요청 실패. 상태 코드:", response.status_code)
            except Exception as e:
                print("HTTP GET 요청 중 오류 발생:", e)

        time.sleep(polling_interval)

```

```
61    if __name__ =="__main__":
62        main()
```

코드설명

43. 무한 루프를 시작하여 작업을 주기적으로 수행합니다. 기상청 RSS 접속하여 온도와 습도데이터를 가져와 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

주기적으로 Wi-Fi 네트워크에 연결하여 날씨 정보를 가져와 출력하는 예제로, 지정된 시간 간격으로 데이터를 업데이트할 수 있습니다.

10초마다 기상청 RSS에 접속하여 데이터를 출력합니다.

```
쉘

>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  Already connected to network:
  HTTP GET 요청 성공!
  온도: 8.0
  습도: 50
  Already connected to network:
  HTTP GET 요청 성공!
  온도: 8.0
  습도: 50
```

OLED에 온도와 습도 표시하기

주기적으로 접속하여 온도와 습도데이터를 출력하는 방법에 대해서 알아봅니다.

7-3.OLED 기상청 날씨 출력하기_4.py

```
import network
import time
import urequests
from machine import Pin
from machine import SoftI2C
from ssd1306 import SSD1306_I2C

ssid ="daduino"
password ="daduino12345"
polling_interval =10
url ="http://www.kma.go.kr/wid/queryDFSRSS.jsp?zone=4139054000"

i2c = SoftI2C(sda=Pin(43), scl=Pin(44))
oled = SSD1306_I2C(128, 64, i2c)

def wifi_connect():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)

    if not wlan.isconnected():
        print("Connecting to network...")
        wlan.connect(ssid, password)

        while not wlan.isconnected():
            time.sleep_ms(300)

        print("Connected to network:", wlan.ifconfig())
        return True
    else:
        print("Already connected to network:", wlan.ifconfig())
        return True

def extract_temp_and_reh(xml_text):
    try:
        temp_start = xml_text.find("<temp>") +len("<temp>")
        temp_end = xml_text.find("</temp>", temp_start)
        temp = xml_text[temp_start:temp_end]

        reh_start = xml_text.find("<reh>") +len("<reh>")
        reh_end = xml_text.find("</reh>", reh_start)
        reh = xml_text[reh_start:reh_end]

        return temp, reh
    except Exception as e:
        print("Error extracting data from XML:", e)
        return None, None

def main():
    while True:
        if wifi_connect():
```

```
51              try:
52                  response = urequests.get(url)
53                  if response.status_code ==200:
54                      print("HTTP GET 요청 성공!")
55
56                      temp, reh = extract_temp_and_reh(response.text)
57                      if temp is not None and reh is not None:
58                          print("온도:", temp)
59                          print("습도:", reh)
60                          oled.fill(0)
61                          temp_str ="temp: "+str(temp)
62                          reh_str ="humi: "+str(reh)
63                          oled.text(temp_str, 20, 20)
64                          oled.text(reh_str, 20, 30)
65                          oled.show()
66                  else:
67                      print("HTTP GET 요청 실패. 상태 코드:", response.status_code)
68              except Exception as e:
69                  print("HTTP GET 요청 중 오류 발생:", e)
70
71          time.sleep(polling_interval)
72
73  if __name__ =="__main__":
74      main()
```

코드설명

05: SoftI2C: 소프트웨어 방식으로 I2C 통신을 구현하기 위한 클래스입니다.

06: SSD1306_I2C: SSD1306 OLED 디스플레이를 제어하기 위한 라이브러리입니다.

13: SoftI2C 객체 생성: SDA 핀(43)과 SCL 핀(44)을 설정하여 I2C 통신을 초기화합니다.

14: SSD1306_I2C 객체 생성: OLED 디스플레이를 제어하기 위해 가로 128, 세로 64 크기로 초기화합니다.

60: OLED 화면을 초기화합니다.

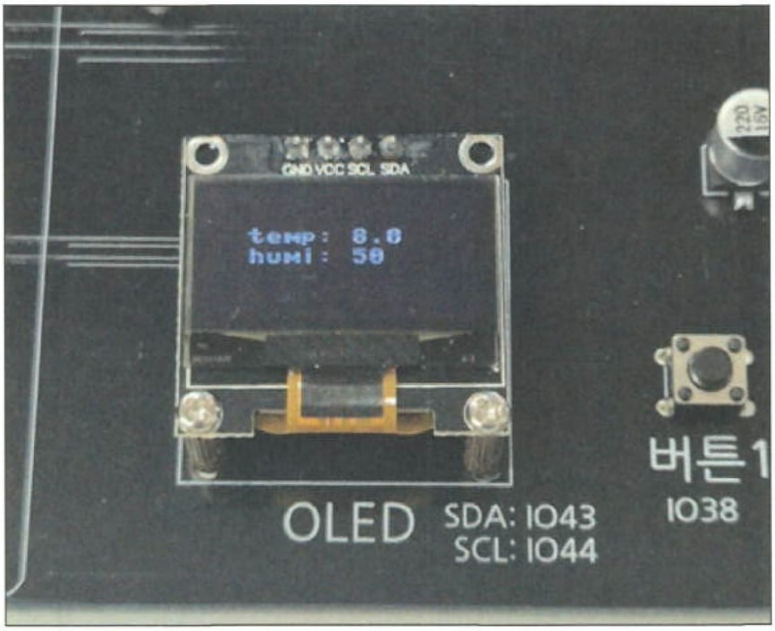
61~62: 온도와 습도 데이터를 문자열로 변환합니다.

63~64: 데이터를 OLED 디스플레이에 출력합니다.

65: OLED 화면에 출력한 내용을 표시합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

OLED에 기상청 RSS로부터 받은 온도와 습도 데이터를 주기적으로 출력하였습니다.



7-4

OLED 공공데이터 미세먼지 출력하기

공공데이터포털의 정보를 이용하여 미세먼지값을 출력하는 방법에 대해서 알아봅니다.

공공데이터 포탈에서 활용신청 및 데이터 확인하기

공공데이터 포탈 가입 및 활용신청을 해봅니다.

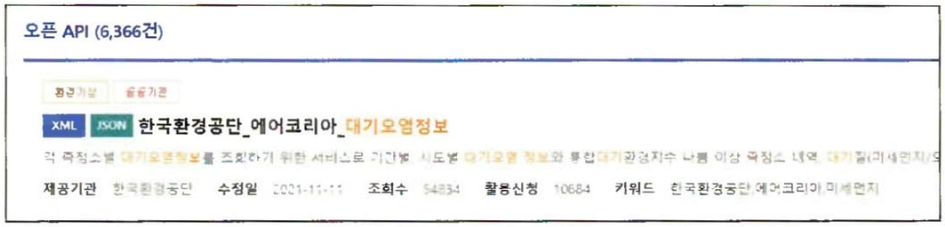
[구글]에서 "공공데이터포털"을 검색하여 공공데이터포털 사이트에 접속합니다.

또는: https://www.data.go.kr/ 바로 접속합니다.

정부에서 운영하는 사이트로 여러 공공데이터를 제공합니다.

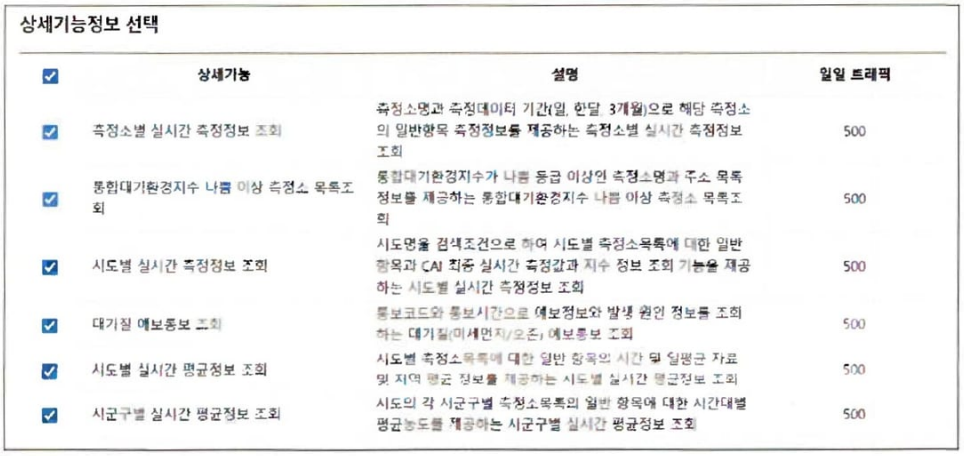
회원가입 후 진행합니다. "대기오염정보" 로 검색합니다.

스크롤을 아래로 내려 [한국환경공단_에어코리아_대기오염정보]에서 [활용신청]을 클릭합니다.

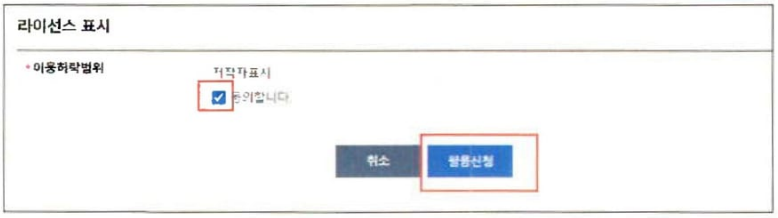


활용목적을 적어줍니다.

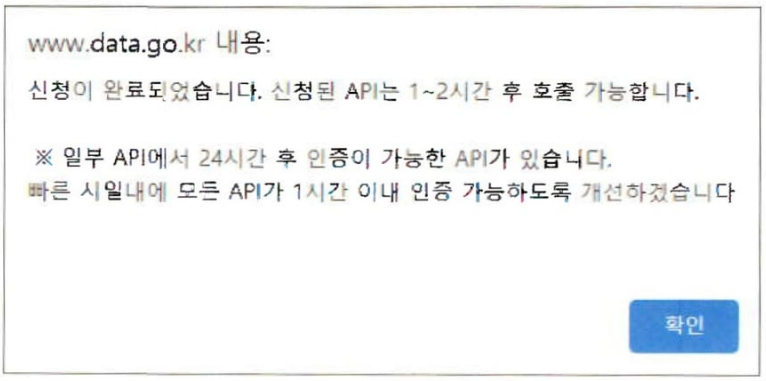
일일 트래픽의 제한이 있어 너무 자주 읽으면 접속이 되지 않습니다.



스크롤을 아래로 내려 동의에 체크한 후 [활용신청]버튼을 클릭합니다.



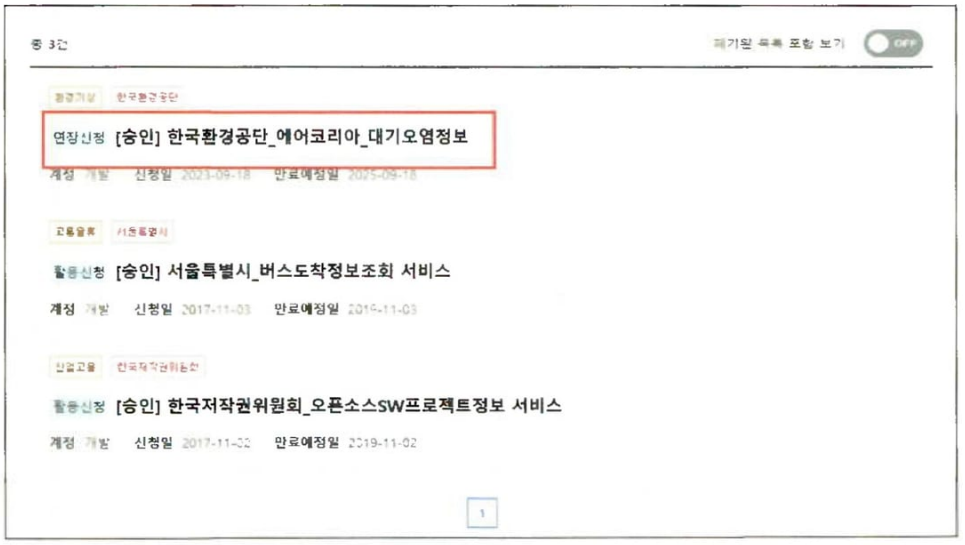
신청이 완료되었다. 바로 호출되지 않고 1~2시간 후 호출이 가능하다고 출력됩니다. 1~2시간 기다렸다가 아래를 진행하도록 합니다.



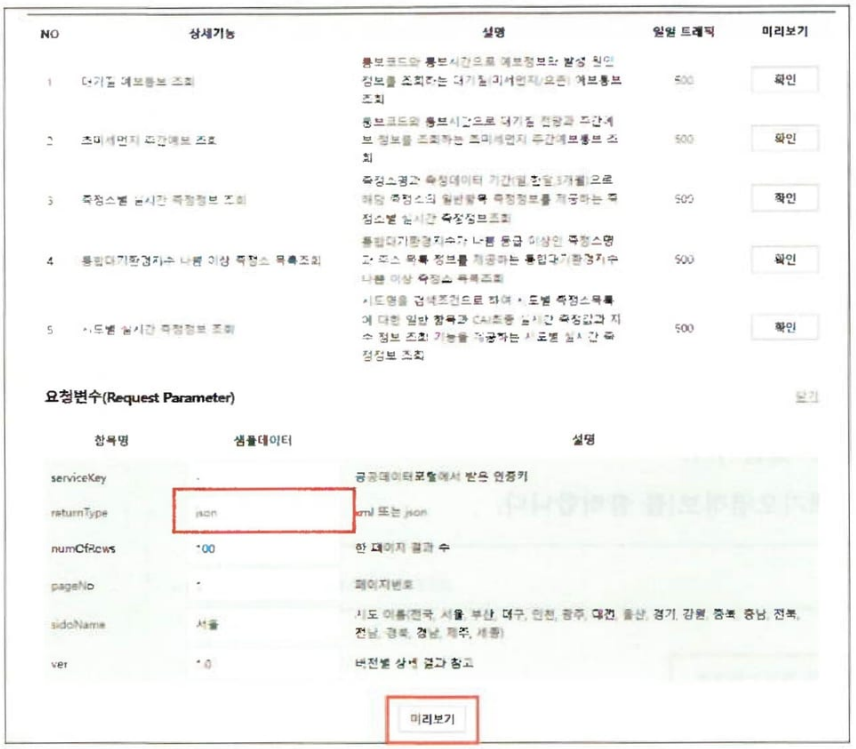
[마이페이지]에서 [활용]을 클릭 합니다.

내가 활용한 목록을 확인 할 수 있습니다.

[한국환경공단_에어코리아_대기오염정보]를 클릭합니다.



[5.시도별 실시간 측정정보 조회]에서 출력타입을 json으로 변경 후 [미리보기]를 눌러 데이터를 확인합니다.



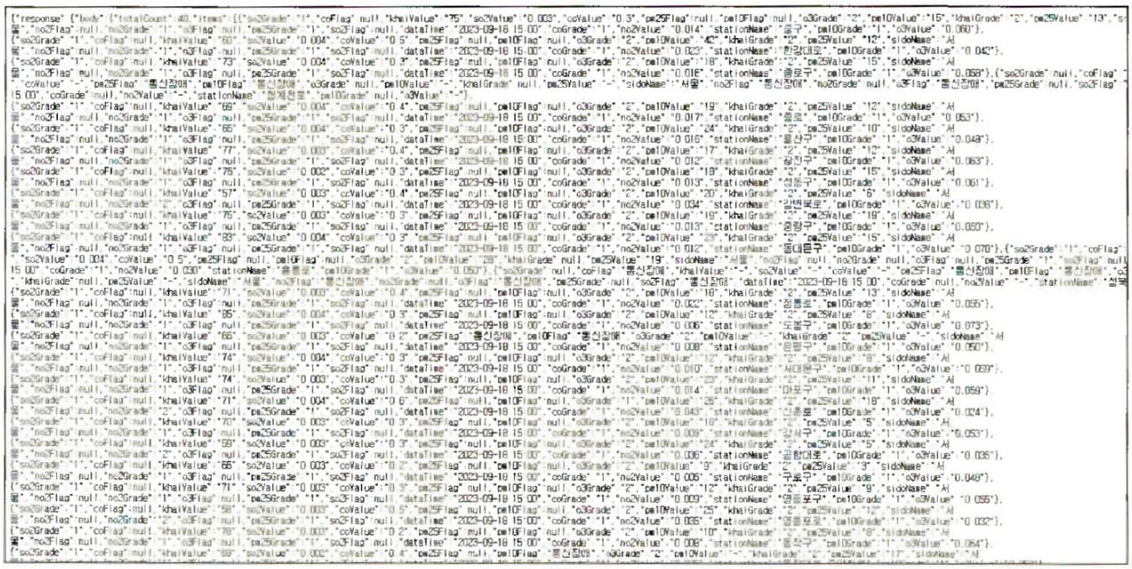
NO	상세기능	설명	일일 트래픽	미리보기
1	대기질 예보통보 조회	통보코드와 통보시간으로 예보정보와 발생 원인 정보를 조회하는 대기질(미세먼지/오존) 예보통보 조회	500	확인
2	초미세먼지 주간예보 조회	통보코드와 통보시간으로 대기질 전망과 주간예보 정보를 조회하는 초미세먼지 주간예보통보 조회	500	확인
3	측정소별 실시간 측정정보 조회	측정소명과 측정데이터 기간(일,3개월)으로 해당 측정소의 일반항목 측정정보를 제공하는 측정소별 실시간 측정정보조회	500	확인
4	통합대기환경지수 나쁨 이상 측정소 목록조회	통합대기환경지수가 나쁨 등급 이상인 측정소명과 주소 목록 정보를 제공하는 통합대기환경지수 나쁨 이상 측정소 목록조회	500	확인
5	시도별 실시간 측정정보 조회	시도명을 검색조건으로 하여 시도별 측정소목록에 대한 일반 항목과 CAI등급 실시간 측정값과 지수 정보 조회 기능을 제공하는 시도별 실시간 측정정보 조회	500	확인

요청변수(Request Parameter) 보기

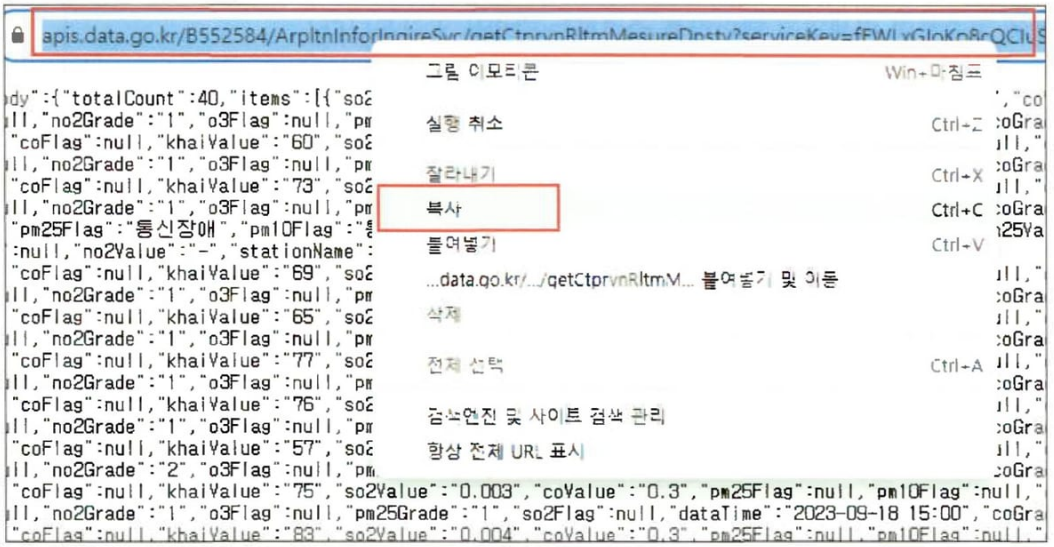
항목명	샘플데이터	설명
serviceKey	-	공공데이터포털에서 받은 인증키
returnType	json	xml 또는 json
numOfRows	*00	한 페이지 결과 수
pageNo	*	페이지번호
sidoName	서울	시도 이름(전국, 서울, 부산, 대구, 인천, 광주, 대전, 울산, 경기, 강원, 충북, 충남, 전북, 전남, 경북, 경남, 제주, 세종)
ver	*.0	버전별 상세 결과 참고

미리보기

json형식으로 데이터를 확인하였습니다.

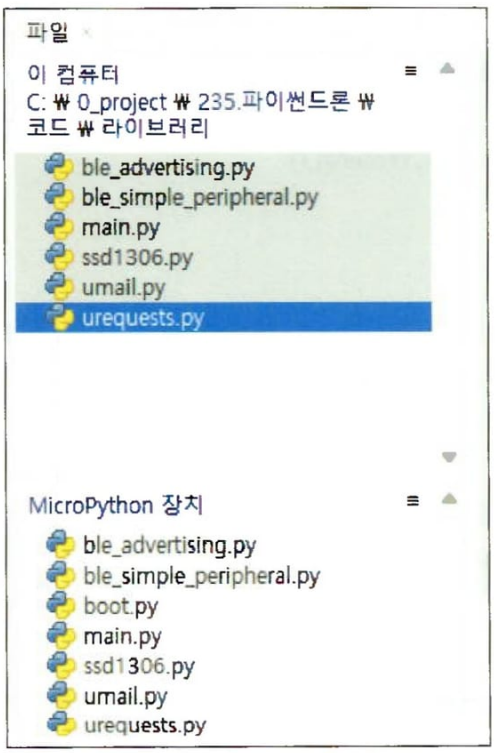


접속 주소를 복사합니다.



라이브러리 업로드하기

1장의 시작하기에서 [라이브러리 마이크로파이썬에 업로드하기]를 참고하여 라이브러리를 업로드 한다음 진행합니다. OLED와 웹에접속하기 위해서 [ssd1306.py],[urequests.py] 라이브러리 파일이 필요로 합니다.



공공데이터 미세먼지 데이터 출력하기

공공데이터에 접속하여 미세먼지 데이터를 출력하는 방법을 알아봅니다.

7-4.OLED 공공데이터 미세먼지 출력하기_1.py

```
01    import network
02    import time
03    import urequests
04
05    ssid ="daduino"
06    password ="daduino12345"
07     url ="https://apis.data.go.kr/B552584/ArpltnInforInqireSvc/getCtprvnRltmMesureDnst
y?serviceKey=fFWLxGIoKo8cQCIuS5Is1fVoiKXkdls%2FU5DSGRwzmbiwIBI0nlz5V6jllexlrGLKR9y8wV3E3
i0SMPTLtAhyvw%3D%3D&returnType=json&numOfRows=100&pageNo=1&sidoName=%EC%84%9C%EC%9A%B8&v
er=1.0"
08    if url.startswith("https"):
09        url = url.replace("https", "http", 1)
10
11
12    def wifi_connect():
13        wlan = network.WLAN(network.STA_IF)
14        wlan.active(True)
15
16        if not wlan.isconnected():
17            print("Connecting to network...")
18            wlan.connect(ssid, password)
19
20            while not wlan.isconnected():
21                time.sleep_ms(300)
22
23            print("Connected to network:", wlan.ifconfig())
24            return True
25        else:
26            print("Already connected to network:", wlan.ifconfig())
27            return True
28
29    def main():
30        if wifi_connect():
31            try:
32                response = urequests.get(url)
33                if response.status_code ==200:
34                    print("HTTP GET 요청 성공!")
35                    print("Response:")
36                    print(response.text)
37                else:
38                    print("HTTP GET 요청 실패. 상태 코드:", response.status_code)
39            except Exception as e:
40                print("HTTP GET 요청 중 오류 발생:", e)
41
42    if __name__ =="__main__":
43        main()
```

코드설명

07: url: 공공데이터 API에서 대기질 정보를 가져올 요청 URL을 설정합니다.

08: url.startswith("https"): URL이 https로 시작하면 http로 변경합니다. (urequests는 HTTPS를 지원하지 않으므로 HTTP로 변경 필요)

09: url.replace: HTTPS를 HTTP로 대체합니다.

32: 설정된 URL로 HTTP GET 요청을 보냅니다.

33~36: 요청이 성공(상태 코드 200)한 경우 응답 데이터를 출력합니다.

37~38: 요청 실패 시 상태 코드를 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

Wi-Fi 네트워크에 연결하고, 공공 데이터 포털 API를 통해 대기 오염 정보를 가져옵니다.

json형태의 미세먼지 데이터를 가져왔습니다.

너무 많은 데이터가 출력되어 Thonny에서 모두 표시하지는 않고 클릭시 더 많은 데이터를 보여줍니다.

```
셸

MPY: soft reboot
Connecting to network...
Connected to network: ('192.168.0.199', '255.255.255.0', '192.168.0.1', '192.168.0.1')
HTTP GET 요청 성공!
Response:
{"response":{"body":{"totalCount":40,"items":[{"so2Grade":"1","coFlag":null,"khaiValue":"27","s
o2Value":"0.003","coValue":"0.4","pm25Flag":null,"pm10Flag":null,"o3Grade":"1","pm10Value":"17"
,"khaiGrade":"1","pm25Value":"9","sidoName":"서울","no2Flag":null,"no2Grade":"1","o3Flag":null,"
pm25Grade":"1","so2Flag":null,"dataTime":"2024-11-19 10:00","coGrade":"1","no2Value":"0.016","s
tationName":"중구","pm10Grade":"1","o3Value":"0.007"},{"so2Grade":"1","coFlag":null,"khaiValue":
"59","so2Value":"0.002","coValue":"0.5","pm25Flag":null,"pm10Flag":null,"o3Grade":"1","pm10Valu
e":"32","khaiGrade":"2","pm25Value":"11","sidoName":"서울","no2Flag":nul ...
```

json 데이터 분리하여 출력하기

json형식의 데이터에서 원하는 데이터만 분리하여 출력하는 방법을 알아봅니다. 미세먼지 부분을 분리합니다.

7-4.OLED 공공데이터 미세먼지 출력하기_2.py

```
01    import network
02    import time
03    import urequests
04    import json
05
06    ssid ="daduino"
07    password ="daduino12345"
08     url ="https://apis.data.go.kr/B552584/ArpltnInforInqireSvc/getCtprvnRltmMesureDnst
y?serviceKey=fFWLxGIoKo8cQCIuS5Is1fVoiKXkdls%2FU5DSGRwzmbiwIBI0nlz5V6jllexlrGLKR9y8wV3E3
i0SMPTLtAhyvw%3D%3D&returnType=json&numOfRows=100&pageNo=1&sidoName=%EC%84%9C%EC%9A%B8&v
er=1.0"
09    if url.startswith("https"):
10        url = url.replace("https", "http", 1)
11
12
13    def wifi_connect():
14        wlan = network.WLAN(network.STA_IF)
15        wlan.active(True)
16
17        if not wlan.isconnected():
18            print("Connecting to network...")
19            wlan.connect(ssid, password)
20
21            while not wlan.isconnected():
22                time.sleep_ms(300)
23
24            print("Connected to network:", wlan.ifconfig())
25            return True
26        else:
27            print("Already connected to network:", wlan.ifconfig())
28            return True
29
30    def main():
31        if wifi_connect():
32            try:
33                response = urequests.get(url)
34                if response.status_code ==200:
35                    print("HTTP GET 요청 성공!")
36                    data = json.loads(response.text)
37
38                    items = data["response"]["body"]["items"]
39
40                    # 각 항목을 출력
41                    for item in items:
42                        print("시도명:", item["sidoName"])
43                        print("측정소명:", item["stationName"])
44                        print("측정일시:", item["dataTime"])
45                        print("미세먼지(PM10) 농도:", item["pm10Value"])
```

```
                    print("초미세먼지(PM2.5) 농도:", item["pm25Value"])
                    print("이산화질소(NO2) 농도:", item["no2Value"])
                    print("오존(O3) 농도:", item["o3Value"])
                    print("일산화탄소(CO) 농도:", item["coValue"])
                    print("아황산가스(SO2) 농도:", item["so2Value"])
                    print("통합대기환경수치(Khai) 농도:", item["khaiValue"])
                    print("미세먼지(PM10) 등급:", item["pm10Grade"])
                    print("초미세먼지(PM2.5) 등급:", item["pm25Grade"])
                    print("이산화질소(NO2) 등급:", item["no2Grade"])
                    print("오존(O3) 등급:", item["o3Grade"])
                    print("일산화탄소(CO) 등급:", item["coGrade"])
                    print("아황산가스(SO2) 등급:", item["so2Grade"])
                    print("통합대기환경수치(Khai) 등급:", item["khaiGrade"])
                    print("\n")

            else:
                print("HTTP GET 요청 실패. 상태 코드:", response.status_code)
        except Exception as e:
            print("HTTP GET 요청 중 오류 발생:", e)

if __name__ =="__main__":
    main()
```

코드설명

36~59: json에서 원하는 데이터를 분리하여 출력하였습니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

json형식에서 아래와 같이 데이터를 분리하여 출력하였습니다.

```
셸
시도명: 서울
측정소명: 영등포구
측정일시: 2024-11-19 10:00
미세먼지(PM10) 농도: 20
초미세먼지(PM2.5) 농도: 10
이산화질소(NO2) 농도: 0.030
오존(O3) 농도: 0.013
일산화탄소(CO) 농도: 0.4
아황산가스(SO2) 농도: 0.002
통합대기환경수치(Khai) 농도: 51
미세먼지(PM10) 등급: 1
초미세먼지(PM2.5) 등급: 1
이산화질소(NO2) 등급: 2
오존(O3) 등급: 1
일산화탄소(CO) 등급: 1
```

.

데이터를 읽어올 때 에러가 발생한다면 다시 코드를 실행합니다. 또는 ESP32를 리셋버튼을 눌러 리셋합니다.

강남대로의 미세먼지, 초미세먼지만 출력하기

json에서 원하는 값을 분리하여 강남대로의 미세먼지, 초미세먼지의 값만 출력하는 방법에 대해서 알아봅니다.

main5-5-3.py

```
01    import network
02    import time
03    import urequests
04    import json
05
06    ssid ="daduino"
07    password ="123456789"
08     url ="https://apis.data.go.kr/B552584/ArpltnInforInqireSvc/getCtprvnRltmMesureDnst
y?serviceKey=fFWLxGIoKo8cQCIuS5Is1fVoiKXkdls%2FU5DSGRwzmbiwIBI0nlz5V6jllexlrGLKR9y8wV3E3
i0SMPTLtAhyvw%3D%3D&returnType=json&numOfRows=100&pageNo=1&sidoName=%EC%84%9C%EC%9A%B8&v
er=1.0"
09    if url.startswith("https"):
10        url = url.replace("https", "http", 1)
11
```

```
12
13    def wifi_connect():
14        wlan = network.WLAN(network.STA_IF)
15        wlan.active(True)
16
17        if not wlan.isconnected():
18            print("Connecting to network...")
19            wlan.connect(ssid, password)
20
21            while not wlan.isconnected():
22                time.sleep_ms(300)
23
24            print("Connected to network:", wlan.ifconfig())
25            return True
26        else:
27            print("Already connected to network:", wlan.ifconfig())
28            return True
29
30    def main():
31        if wifi_connect():
32            try:
33                response = urequests.get(url)
34                if response.status_code ==200:
35                    print("HTTP GET 요청 성공!")
36                    data = json.loads(response.text)
37
38                    items = data["response"]["body"]["items"]
39
40                     # 각 항목을 출력
41                    for item in items:
42                        if item["stationName"] =="강남대로":  # 측정소명이 "강남대로"인
경우에만 출력
43                            print("시도명:", item["sidoName"])
44                            print("측정소명:", item["stationName"])
45                            print("측정일시:", item["dataTime"])
46                            print("미세먼지(PM10) 농도:", item["pm10Value"])
47                            print("초미세먼지(PM2.5) 농도:", item["pm25Value"])
48                            print("\n")
49
50                else:
51                    print("HTTP GET 요청 실패. 상태 코드:", response.status_code)
52            except Exception as e:
53                print("HTTP GET 요청 중 오류 발생:", e)
54
55    if __name__ =="__main__":
56        main()
```

코드설명

30-56: `main` 함수를 정의합니다. 이 함수는 주요 로직을 수행합니다. HTTP GET 요청이 성공하면 응답 데이터를 JSON 형식으로 파싱하고, 측정소명이 "강남대로"인 경우에만 해당 정보를 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

측정소명이 "강남대로"인 경우에만 해당 정보를 출력하였습니다.

```
셸 ×

>>> %Run -c $EDITOR_CONTENT


  MPY: soft reboot
  Already connected to network: ('192.168.0.199',
  HTTP GET 요청 성공!
  시도명: 서울
  측정소명: 강남대로
  측정일시: 2024-11-19 10:00
  미세먼지(PM10) 농도: 37
  초미세먼지(PM2.5) 농도: 12
```

OLED에 미세먼지 표시하기

분리된 미세먼지 값을 OLED에 표시해보도록 합니다.

7-4.OLED 공공데이터 미세먼지 출력하기_4.py

```
01    import network
02    import time
03    import urequests
04    import json
05    from machine import Pin
06    from machine import SoftI2C
07    from ssd1306 import SSD1306_I2C
08
09    ssid ="daduino"
10    password ="123456789"
11      url ="https://apis.data.go.kr/B552584/ArpltnInforInqireSvc/getCtprvnRltmMesureDnst
y?serviceKey=fFWLxGIoKo8cQCIuS5Is1fVoiKXkdls%2FU5DSGRwzmbiwIBI0nlz5V6jllexlrGLKR9y8wV3E3
i0SMPTLtAhyvw%3D%3D&returnType=json&numOfRows=100&pageNo=1&sidoName=%EC%84%9C%EC%9A%B8&v
er=1.0"
12    if url.startswith("https"):
13        url = url.replace("https", "http", 1)
14
```

```
i2c = SoftI2C(sda=Pin(43), scl=Pin(44))
oled = SSD1306_I2C(128, 64, i2c)

def wifi_connect():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)

    if not wlan.isconnected():
        print("Connecting to network...")
        wlan.connect(ssid, password)

        while not wlan.isconnected():
            time.sleep_ms(300)

        print("Connected to network:", wlan.ifconfig())
        return True
    else:
        print("Already connected to network:", wlan.ifconfig())
        return True

def main():
    while True :
        if wifi_connect():
            try:
                response = urequests.get(url)
                if response.status_code ==200:
                    print("HTTP GET 요청 성공!")
                    data = json.loads(response.text)

                    items = data["response"]["body"]["items"]

                     # 각 항목을 출력
                    for item in items:
                         if item["stationName"] =="강남대로":  # 측정소명이 "강남대로"인 경우에만 출력
                            print("시도명:", item["sidoName"])
                            print("측정소명:", item["stationName"])
                            print("측정일시:", item["dataTime"])
                            print("미세먼지(PM10) 농도:", item["pm10Value"])
                            print("초미세먼지(PM2.5) 농도:", item["pm25Value"])
                            print("\n")

                            time_str =str(item["dataTime"])
                            pm10Value_str ="pm10: "+str(item["pm10Value"])
                            pm25Value ="pm25: "+str(item["pm25Value"])

                            oled.fill(0)
                            oled.text(time_str, 0, 10)
                            oled.text(pm10Value_str, 20, 20)
                            oled.text(pm25Value, 20, 30)
                            oled.show()

                else:
                    print("HTTP GET 요청 실패. 상태 코드:", response.status_code)
            except Exception as e:
                print("HTTP GET 요청 중 오류 발생:", e)

        time.sleep(10.0)
```

```
72
73    if __name__ =="__main__":
74        main()
```

코드설명

56: 측정일시를 문자열로 저장합니다.

57~58: PM10, PM2.5 농도를 문자열로 저장합니다.

60: OLED 화면을 초기화합니다.

61~63: 시간, PM10, PM2.5 데이터를 각각의 줄에 출력합니다.

64: 출력한 내용을 OLED 화면에 표시합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

OLED에 측정시간, pm10, pm25 값이 출력되었습니다. 10초마다 접속하여 값을 출력합니다.

7-5

OLED 주식가격 출력하기

인터넷으로 주식 시세를 알려주는 사이트에 접속하여 원하는 주식시세를 출력하는 방법에 대해서 알아봅니다.

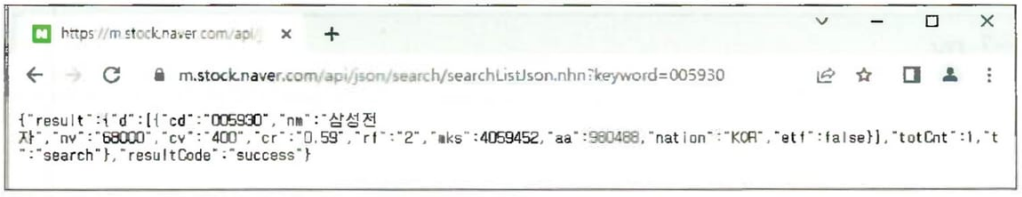
네이버 주식데이터 접속하여 값확인하기

네이버의 주식데이터 접속하여 값을 확인하여 봅니다.

다음의 주소로 접속합니다. 005930은 삼성전자의 주식번호 입니다.

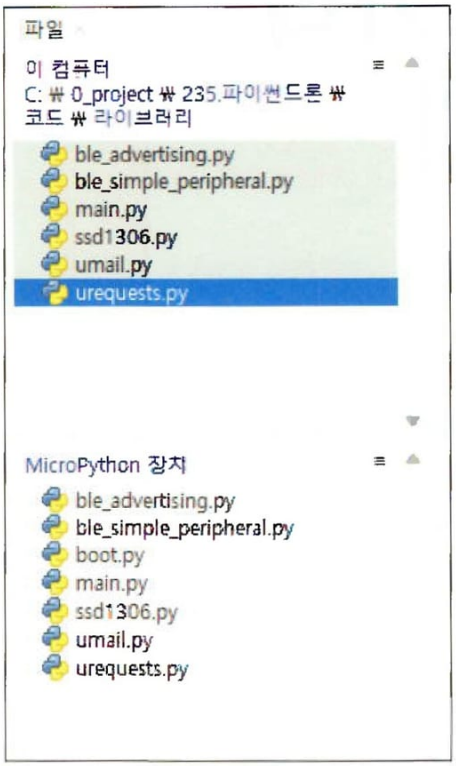
https://m.stock.naver.com/api/json/search/searchListJson.nhn?keyword=005930

다음과 같이 네이버에서 제공하는 주식시세로 값을 보여줍니다. json타입으로 값을 보여줍니다. 여러단계로 나누어져있는 json타입으로 요번작품에서는 json라이브러리를 사용하지 않고 문자열을 찾아 분리를 해보도록 합니다.



라이브러리 업로드하기

1장의 시작하기에서 [라이브러리 마이크로파이썬에 업로드하기]를 참고하여 라이브러리를 업로드 한다음 진행합니다. OLED와 웹에접속하기 위해서 [ssd1306.py],[urequests.py] 라이브러리 파일이 필요로 합니다.



삼성전자 주식 시세 확인하기

네이버의 주식시세 데이터에 접속하여 삼성전자의 주식시세를 출력하는 방법을 알아봅니다.

main5-7-1.py

```
01    import network
02    import time
03    import urequests
04    import json
05
06    ssid ="daduino"
07    password ="daduino12345"
08    stock_number ="005930"  # 삼성전자 종목 코드
09    url = f"https://m.stock.naver.com/api/stock/{stock_number}/integration"
10
11    def wifi_connect():
```

```
        wlan = network.WLAN(network.STA_IF)
        wlan.active(True)

        if not wlan.isconnected():
            print("Connecting to network...")
            wlan.connect(ssid, password)

            while not wlan.isconnected():
                time.sleep_ms(300)

            print("Connected to network:", wlan.ifconfig())
            return True
        else:
            print("Already connected to network:", wlan.ifconfig())
            return True

    def main():
        if wifi_connect():
            try:
                response = urequests.get(url)
                if response.status_code ==200:
                    print("HTTP GET 요청 성공!")
                    data = json.loads(response.text)

                    # 주요 주식 데이터 추출
                    stock_name = data["stockName"]
                     last_close_price =next(item["value"] for item in data["totalInfos"]
if item["code"] =="lastClosePrice")
                      open_price =next(item["value"] for item in data["totalInfos"] if
item["code"] =="openPrice")
                      high_price =next(item["value"] for item in data["totalInfos"] if
item["code"] =="highPrice")
                       low_price =next(item["value"] for item in data["totalInfos"] if
item["code"] =="lowPrice")
                     market_value =next(item["value"] for item in data["totalInfos"] if
item["code"] =="marketValue")
                          per =next(item["value"] for item in data["totalInfos"] if
item["code"] =="per")

                    print("주식 이름:", stock_name)
                    print("전일 종가:", last_close_price)
                    print("시가:", open_price)
                    print("고가:", high_price)
                    print("저가:", low_price)
                    print("시가총액:", market_value)
                    print("PER:", per)

                else:
                    print("HTTP GET 요청 실패. 상태 코드:", response.status_code)
            except Exception as e:
                print("HTTP GET 요청 중 오류 발생:", e)

    if __name__ =="__main__":
        main()
```

코드설명

34: 응답 데이터를 JSON 형식으로 파싱합니다.

37~43: JSON 데이터에서 주요 주식 정보를 추출합니다.

37: 주식 이름을 추출합니다.

38~43: totalInfos 리스트에서 종가, 시가, 고가, 저가, 시가총액, PER 데이터를 추출합니다. 각각의 코드(lastClosePrice, openPrice, highPrice, lowPrice, marketValue, per)로 항목을 찾습니다.

45~51: 추출한 데이터를 출력합니다.

45: 주식 이름 출력.

46~51: 전일 종가, 시가, 고가, 저가, 시가총액, PER 값을 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

네이버 금융에서 삼성전자 주식 정보를 가져와서 해당 정보를 출력하였습니다.

```
셸
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  Already connected to network: ('192.168.
  HTTP GET 요청 성공!
  주식 이름: 삼성전자
  전일 종가: 56,700
  시가: 56,500
  고가: 57,500
  저가: 55,900
  시가총액: 334조 3,078억
  PER: 11.86배
```

OLED에 표시하기

OLED에 삼성전자의 주식 가격을 10초마다 표시하는 코드를 작성합니다.

7-5.OLED 주식가격 출력하기_2.py

```
import network
import time
import urequests
import json
from machine import Pin
from machine import SoftI2C
from ssd1306 import SSD1306_I2C

# Wi-Fi 및 주식 데이터 URL 설정
ssid ="daduino"
password ="123456789"
stock_number ="005930"  # 삼성전자 종목 코드
url = f"https://m.stock.naver.com/api/stock/{stock_number}/integration"

# I2C 및 OLED 초기화
i2c = SoftI2C(sda=Pin(43), scl=Pin(44))
oled = SSD1306_I2C(128, 64, i2c)

def wifi_connect():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)

    if not wlan.isconnected():
        print("Connecting to network...")
        wlan.connect(ssid, password)

        while not wlan.isconnected():
            time.sleep_ms(300)

        print("Connected to network:", wlan.ifconfig())
        return True
    else:
        print("Already connected to network:", wlan.ifconfig())
        return True

def main():
    while True:
        if wifi_connect():
            try:
                # 주식 데이터 요청
                response = urequests.get(url)
                if response.status_code ==200:
                    print("HTTP GET 요청 성공!")
                    data = json.loads(response.text)

                    # 주식 이름을 하드코딩
                    stock_name ="SAMSUNG"
                    current_price =next(item["value"] for item in data["totalInfos"]
if item["code"] =="lastClosePrice")

```

```
50                      # OLED에 출력
51                      oled.fill(0)
52                      oled.text("Stock Name:", 0, 10)
53                      oled.text(stock_name, 0, 20)
54                      oled.text("Price:", 0, 40)
55                      oled.text(current_price, 0, 50)
56                      oled.show()
57
58                  else:
59                      print("HTTP GET 요청 실패. 상태 코드:", response.status_code)
60              except Exception as e:
61                  print("HTTP GET 요청 중 오류 발생:", e)
62
63          time.sleep(10.0)  # 1초마다 업데이트
64
65  if __name__ =="__main__":
66      main()
```

코드설명

51~56: OLED 디스플레이에 데이터를 출력합니다.

51: OLED 화면을 초기화합니다.

52: "Stock Name:"을 출력합니다.

53: 주식 이름을 출력합니다.

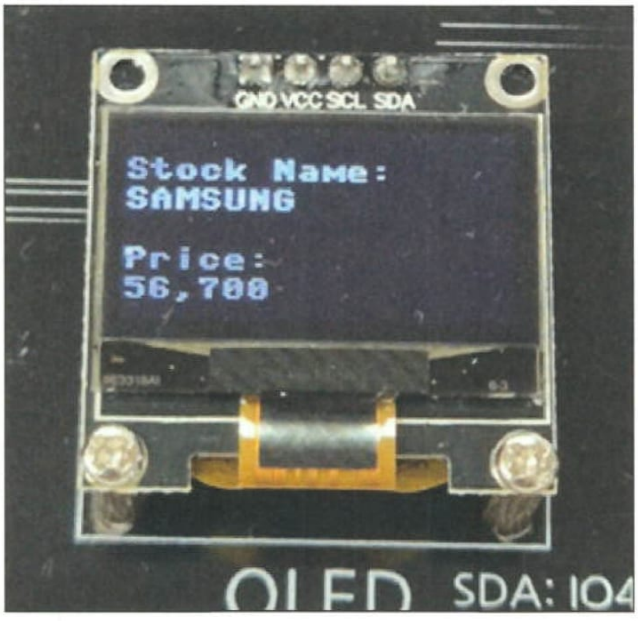
54: "Price:"를 출력합니다.

55: 주식 가격을 출력합니다.

56: OLED 화면에 출력한 내용을 표시합니다.

[현재 스크립트 실행] 아이콘을 클릭하여 코드를 실행합니다.

네이버 금융에서 여러 주식 정보를 가져와서 해당 정보를 출력하였습니다.



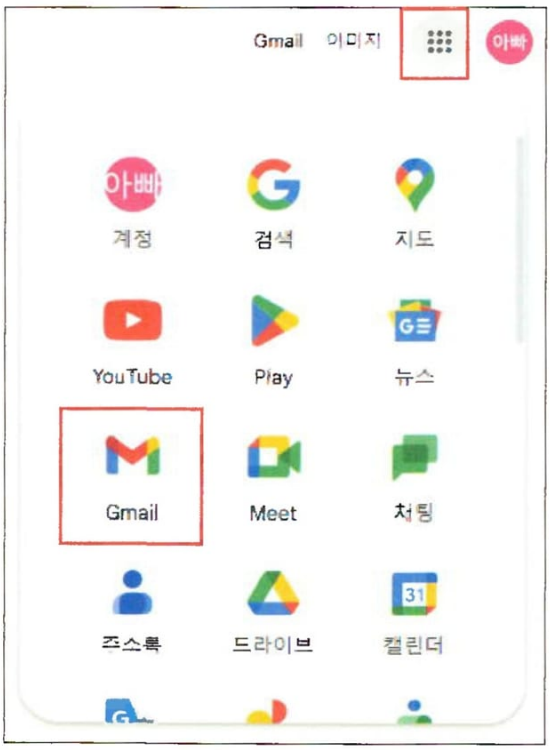
7-6

온도,습도,조도 측정하여 이메일 보내기

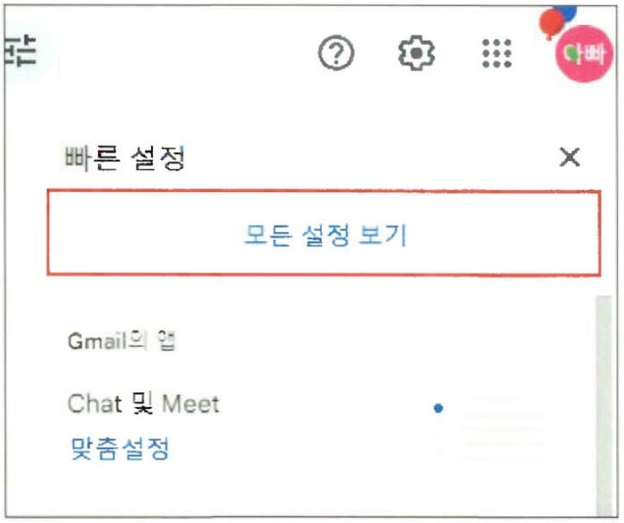
이메일을 보내는 방법에 대해서 알아봅니다.

구글이메일 설정하기

구글 설정을 통해 ESP32 장치에서 이메일을 보내도록 합니다. 구글에서 Gamil에 접속합니다.



[톱니바퀴] 아이콘을 클릭 후 [모든 설정 보기]를 클릭합니다.



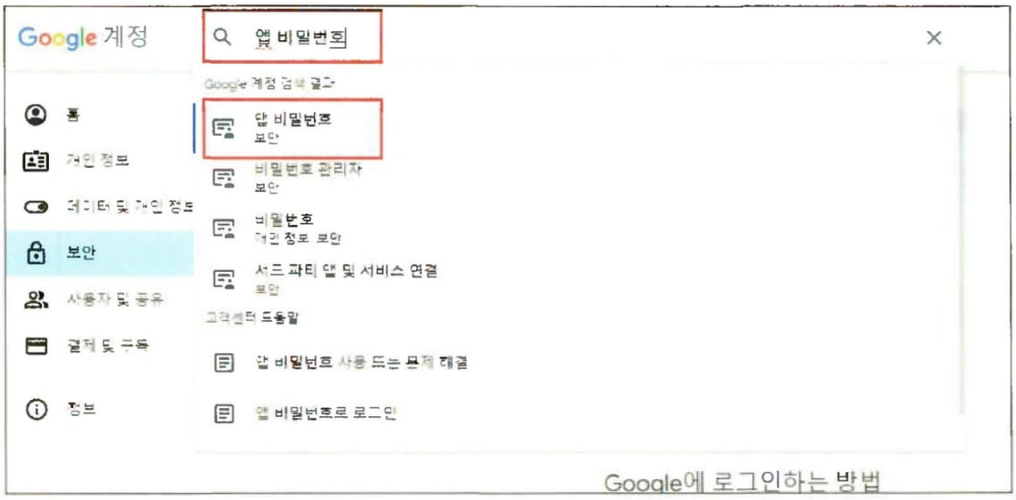
[전달 및 POP/IMAP] 탭에서 [IMAP 사용]을 체크함으로 한다음 [변경사항 저장]을 눌러 설정을 저장합니다.

외부장치에서 나의 구글 계정을 통해 이메일일 보내기 위해서는 2단계 설정이 되어야 합니다.

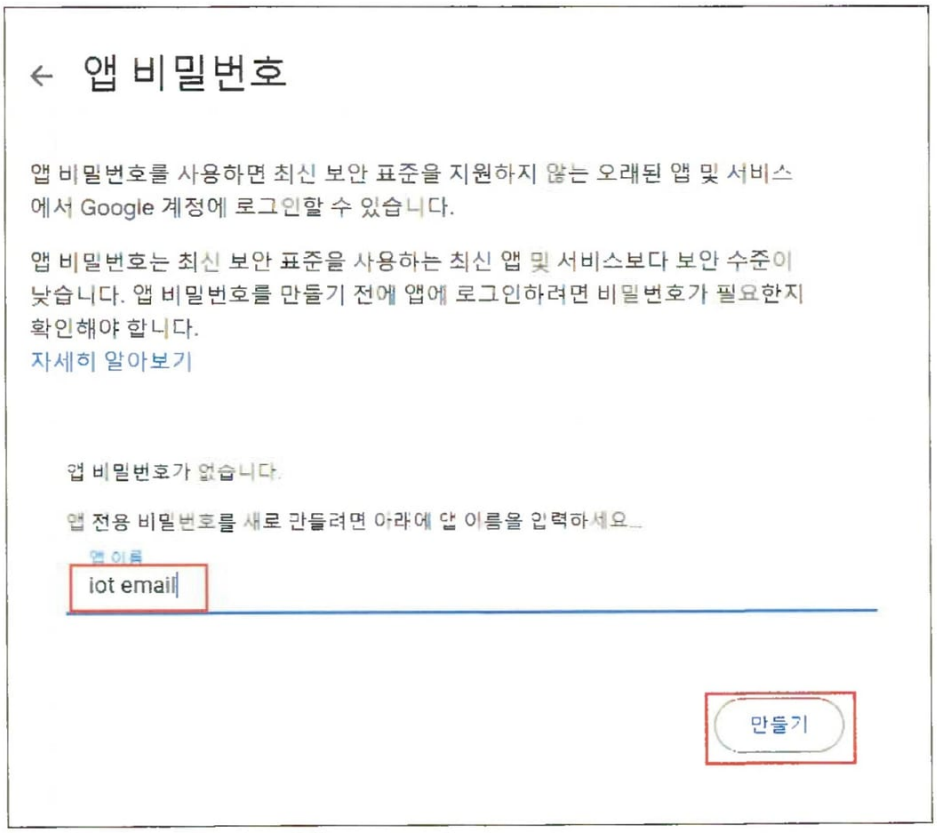
[Google 계정 관리] 로 이동합니다.

[보안] 탭으로 이동하여 [2단계 인증]을 클릭하여 2단계 인증을 사용함으로 설정합니다.

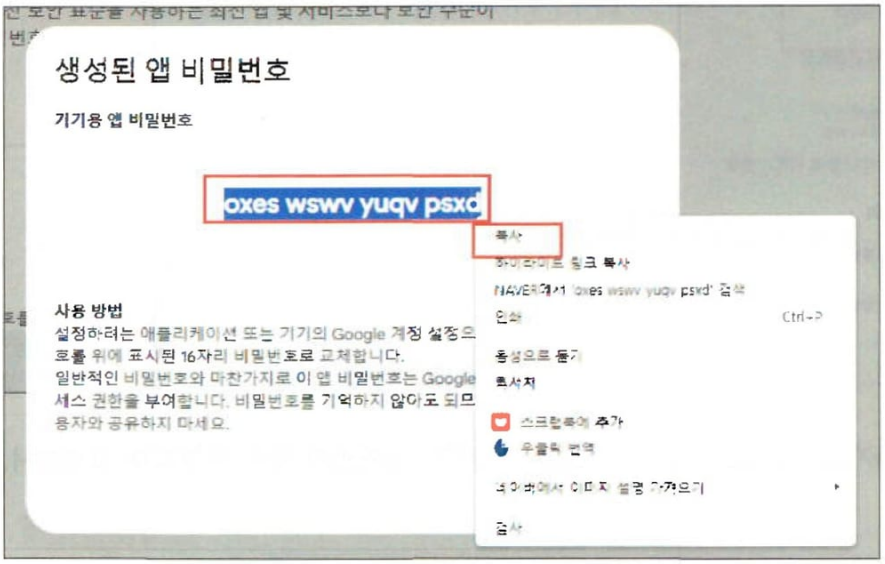
검색에서 "앱 비밀번호"를 검색 후 [앱 비밀번호]를 클릭하여 접속합니다.



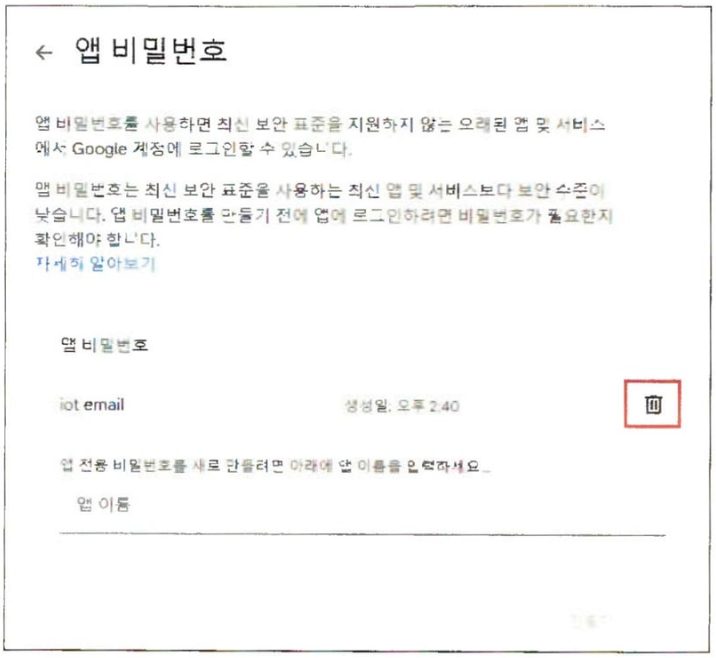
앱 이름을 입력 후 [만들기]를 클릭합니다. 앱 이름은 사용자가 알아보기 쉬운 이름으로 입력합니다.



앱 비밀번호가 생성되었습니다. 비밀번호를 복사 후 메모장이나 개인적으로 확인 할 수 있는곳에 잘 저장해둡니다. 앱 비밀번호는 다시는 확인이 불가능합니다. 앱 비밀번호를 분실하였다면 설정한 앱을 지운다음 다시 앱을 생성합니다.

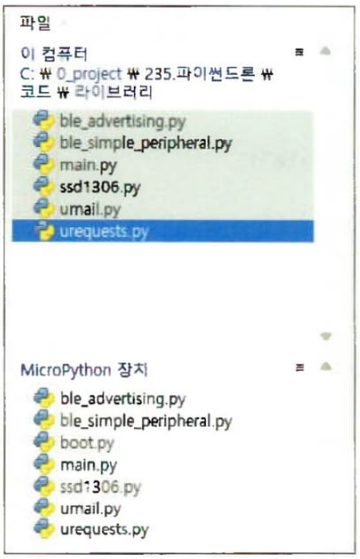


앱은 쓰레기통 아이콘을 눌러 지울 수 있습니다. 비밀번호를 분실하였다면 지운다음 다시 앱을 생성 후 진행합니다.



라이브러리 설치

1장의 시작하기에서 [라이브러리 마이크로파이썬에 업로드하기]를 참고하여 라이브러리를 업로드 한다음 진행합니다. 이메일을 보내기 위해서 [umail.py] 라이브러리 파일이 필요로 합니다.



내 구글 계정으로 이메일보내기

구글계정와 앱비밀번호를 이용하여 이메일을 보내는 방법을 알아봅니다.

7-6.온도,습도,조도측정하여 이메일 보내기_1.py

```
01    import umail
02    import network
03
04    ssid ='daduino'
05    password ='123456789'
06
07    sender_email ='munjjac@gmail.com'#구글 ID test@gmail.com 형식
08    sender_name ='ESP32'#보내는 사람 이름
09    sender_app_password ='nwrr dlak kvdx wfju'#구글 앱 비밀번호
10    recipient_email ='munjjac@hanmail.net'#받는 사람 이메일주소
11    email_subject ='Test Email'#이메일 제목
12
13    def wifi_connect():
14        wlan = network.WLAN(network.STA_IF)
15        wlan.active(True)
16
17        if not wlan.isconnected():
18            print("Connecting to network...")
19            wlan.connect(ssid, password)
20
21            while not wlan.isconnected():
22                time.sleep_ms(300)
23
```

```
24            print("Connected to network:", wlan.ifconfig())
25            return True
26        else:
27            print("Already connected to network:", wlan.ifconfig())
28            return True
29
30    def main():
31        if wifi_connect():
32            smtp = umail.SMTP('smtp.gmail.com', 587, ssl=False)
33            smtp.login(sender_email, sender_app_password)
34            smtp.to(recipient_email)
35            smtp.write("From:"+ sender_name +"<"+ sender_email+">\n")
36            smtp.write("Subject:"+ email_subject +"\n")
37            smtp.write("Hello from ESP32")
38            smtp.send()
39            smtp.quit()
40            print("메일을 성공적으로 보냈습니다.")
41
42    if __name__ =="__main__":
43        main()
```

코드설명

07: sender_email: 보내는 사람의 이메일 주소를 설정합니다.

08: sender_name: 보내는 사람의 이름을 설정합니다.

09: sender_app_password: 구글 앱 비밀번호를 설정합니다. (2단계 인증이 활성화된 계정에서 생성된 앱 비밀번호)

10: recipient_email: 수신자 이메일 주소를 설정합니다.

11: email_subject: 이메일 제목을 설정합니다.

30~40: 메일 전송 함수

31: Wi-Fi 연결을 확인한 후 실행합니다.

32: Gmail SMTP 서버(smtp.gmail.com)에 연결하고 포트 587을 사용합니다. SSL은 비활성화합니다.

33: sender_email과 sender_app_password를 사용해 SMTP 서버에 로그인합니다.

34: 수신자 이메일 주소를 설정합니다.

35: 이메일의 발신자 정보를 작성합니다.

36: 이메일 제목을 작성합니다.

37: 이메일 본문 내용을 작성합니다.

38: 이메일을 전송합니다.

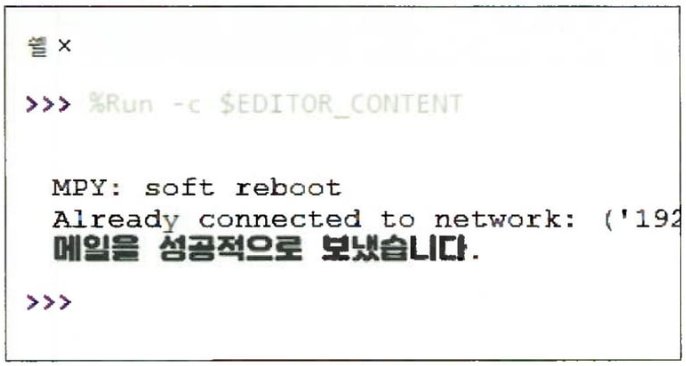
39: SMTP 세션을 종료합니다.

40: 이메일 전송 성공 메시지를 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

Gmail 서버를 사용하여 이메일을 보내는 마이크로파이썬 스크립트입니다. SMTP를 사용하여 Gmail 계정으로 로그인하고 이메일을 작성하고 전송합니다. Gmail 앱 비밀번호는 Gmail 계정의 보안 설정에서 생성해야 합니다.

메일을 성공적으로 전송하였습니다.



pyDrone를 이용한 메일이 정상적으로 보내졌습니다.

이메일보내는 부분 함수화 하기

이메일을 보내는 부분의 코드를 함수화하여 코드를 간편하게 사용하도록 수정합니다.

7-6.온도,습도,조도측정하여 이메일 보내기_2.py

```
01    import umail
02    import network
03    import time
04
05    ssid ='daduino'
06    password ='daduino12345'
07
08    sender_email ='munjjac@gmail.com'#구글 ID test@gmail.com 형식
09    sender_name ='prDrone'#보내는 사람 이름
10    sender_app_password ='oxes wswv yuqv psxd'#구글 앱 비밀번호
11    recipient_email ='munjjac@hanmail.net'#받는 사람 이메일주소
12    email_subject ='Test Email'#이메일 제목
13
14    def wifi_connect():
15        wlan = network.WLAN(network.STA_IF)
16        wlan.active(True)
17
18        if not wlan.isconnected():
19            print("Connecting to network...")
20            wlan.connect(ssid, password)
21
22            while not wlan.isconnected():
23                time.sleep_ms(300)
24
25            print("Connected to network:", wlan.ifconfig())
26            return True
27        else:
28            print("Already connected to network:", wlan.ifconfig())
29            return True
30
31     def send_email(sender_email,sender_app_password,sender_name,recipient_email,email_
subject):
32        smtp = umail.SMTP('smtp.gmail.com', 587, ssl=False)
33        smtp.login(sender_email, sender_app_password)
34        smtp.to(recipient_email)
35        smtp.write("From:"+ sender_name +"<"+ sender_email+">\n")
36        smtp.write("Subject:"+ email_subject +"\n")
37        smtp.write("Hello from ESP32")
38        smtp.send()
39        smtp.quit()
40        print("메일을 성공적으로 보냈습니다.")
41
42    def main():
43        if wifi_connect():
44                send_email(sender_email,sender_app_password,sender_name,recipient_
email,email_subject)
45
46    if __name__ =="__main__":
47        main()
```

코드설명

31~40: 이메일 전송 함수

31: 함수 정의. 이메일 발신 정보를 매개변수로 받습니다.

32: Gmail SMTP 서버(smtp.gmail.com)에 연결하고 포트 587을 사용합니다. SSL은 비활성화합니다.

33: sender_email과 sender_app_password를 사용해 SMTP 서버에 로그인합니다.

34: 수신자 이메일 주소를 설정합니다.

35: 이메일의 발신자 정보를 작성합니다.

36: 이메일 제목을 작성합니다.

37: 이메일 본문 내용을 작성합니다.

38: 이메일을 전송합니다.

39: SMTP 세션을 종료합니다.

40: 이메일 전송 성공 메시지를 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

코드를 함수화 하였고 이메일을 전송하였습니다.

pyDrone를 이용한 메일이 정상적으로 보내졌습니다.

온도, 습도, 조도를 측정하여 이메일 보내기

온도와 습도, 조도를 측정하여 값을 이메일로 보내는 코드를 만들어봅니다.

7-6.온도,습도,조도측정하여 이메일 보내기_3.py

```
import umail
import network
import time
from machine import Pin, ADC
import dht

# Wi-Fi 설정
ssid ='daduino'
password ='daduino12345'

# 이메일 설정
sender_email ='munjjac@gmail.com'  # 보내는 사람 이메일
sender_name ='prDrone'  # 보내는 사람 이름
sender_app_password ='oxes wswv yuqv psxd'  # 구글 앱 비밀번호
recipient_email ='munjjac@hanmail.net'  # 받는 사람 이메일
email_subject ='Sensor Data Report'  # 이메일 제목
```

```
17
18    # 온습도 센서(DHT11) 설정
19    dht11 = dht.DHT11(Pin(18))
20
21    # 조도 센서 설정
22    cds_sensor = ADC(Pin(8))
23    cds_sensor.atten(ADC.ATTN_11DB)
24
25    def wifi_connect():
26        wlan = network.WLAN(network.STA_IF)
27        wlan.active(True)
28
29        if not wlan.isconnected():
30            print("Connecting to network...")
31            wlan.connect(ssid, password)
32
33            while not wlan.isconnected():
34                time.sleep_ms(300)
35
36            print("Connected to network:", wlan.ifconfig())
37            return True
38        else:
39            print("Already connected to network:", wlan.ifconfig())
40            return True
41
42    def get_sensor_data():
43        """온도, 습도, 조도 데이터를 측정합니다."""
44        try:
45            # 온도와 습도 측정
46            dht11.measure()
47            temp = dht11.temperature()
48            humi = dht11.humidity()
49
50            # 조도 측정
51            cds_value = cds_sensor.read()
52
53            return temp, humi, cds_value
54        except Exception as e:
55            print(f"센서 데이터를 가져오는 중 오류 발생: {e}")
56            return None, None, None
57
58     def send_email(sender_email, sender_app_password, sender_name, recipient_email,
email_subject, message):
59        """이메일 전송"""
60        smtp = umail.SMTP('smtp.gmail.com', 587, ssl=False)
61        smtp.login(sender_email, sender_app_password)
62        smtp.to(recipient_email)
63        smtp.write(f"From:{sender_name} <{sender_email}>\n")
64        smtp.write(f"Subject:{email_subject}\n")
65        smtp.write(message)
66        smtp.send()
67        smtp.quit()
68        print("메일을 성공적으로 보냈습니다.")
69
70    def main():
71        if wifi_connect():
72            # 센서 데이터 가져오기
73            temp, humi, cds_value = get_sensor_data()
```

```
74
75            if temp is not None and humi is not None and cds_value is not None:
76                # 이메일에 포함할 메시지 작성
77                message = f"Hello from ESP32\n\nTemperature: {temp}C\nHumidity: {humi}%\
nLight Intensity: {cds_value}\n"
78
79                print("보낼 이메일 내용:")
80                print(message)
81
82                # 이메일 전송
83                  send_email(sender_email, sender_app_password, sender_name, recipient_
email, email_subject, message)
84            else:
85                print("센서 데이터를 가져올 수 없어 이메일을 보내지 못했습니다.")
86
87    if __name__ =="__main__":
88        main()
```

코드설명

73: 온도, 습도, 조도 데이터를 측정합니다.

75~80: 데이터를 성공적으로 가져온 경우 이메일에 포함할 메시지를 작성하고 출력합니다.

83: 작성한 메시지를 포함하여 이메일을 전송합니다.

85: 센서 데이터가 없는 경우 이메일을 보내지 않고 오류 메시지를 출력합니다.

[▶] 현재 스크립트 실행(F5) 아이콘을 클릭하여 코드를 실행합니다.

여러개의 이메일을 반복하면서 전송하였습니다.

```
셸
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  Already connected to network: ('192.168.0.199
  보낼 이메일 내용:
  Hello from ESP32

  Temperature: 19°C
  Humidity: 35%
  Light Intensity: 469

  메일을 성공적으로 보냈습니다.
```

측정된 온도, 습도, 조도 값이 이메일로 잘 보내졌습니다.

Sensor Data Report

^ 보낸사람 prDrone<munjjac@gmail.com> 주소추가 수신차단

Hello from ESP32

Temperature: 19C
Humidity: 35%
Light Intensity: 470